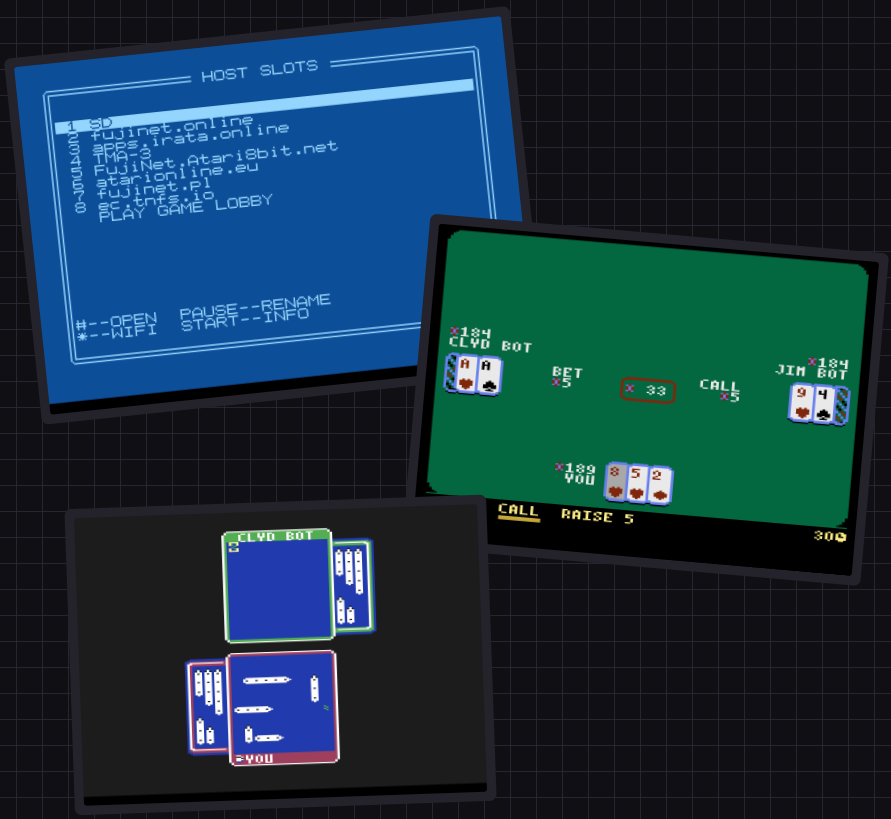
FUJINET 5200

Programmer's Handbook



FUJINET

A FujiNet Project Publication



FUJINET 5200

FujiNet 5200 Programmer's Handbook. First edition, 2026.

Typeset after the 1982 *ATARI 5200 Owner's Manual* as a tribute. FujiNet is an independent project and is not affiliated with, endorsed by, or sponsored by Atari. "Atari" and "5200" are trademarks of their owner and are used here to identify the console this handbook is about.

Every address, register, command and code fragment in this handbook is transcribed from the live project sources and verified in the FujiNet MAME cartridge model against a running FujiNet adapter; the sources and their commits are listed in the Parts List at the back.

Text and diagrams © 2026 the FujiNet Project, CC BY-SA 4.0. Program listings carry the licence of their repositories. cc65 is © its authors and is distributed under its own licence.

Table of Contents

1	Your FujiNet Cartridge	1
2	Install the Tools	3
3	The Memory Map	6
4	How the Mailbox Works	10
5	The Text Display	16
6	Living in 16K	21
7	The 6502 Library	24
8	C and fujinet-lib	29
9	Controllers, Keypad and Sound	33
10	First Contact	38
11	The Network Device	41
12	netcat	46
13	Directories, Mount and Boot	51
14	App Keys, Clock and Utilities	59
15	Banks and the Super Cart	64
16	Display Lists, Fonts and Players	68
17	Command Reference: The Fuji Device	71
18	Command Reference: The Network Device	88
19	Command Reference: Disk, Printer, Clock, Modem	97
20	Command Reference: The Cartridge Itself	102
21	Battleship	106
22	5 Card Stud	110
23	CONFIG	113

24 Things That Cost Real Time 117
25 Troubleshooting Checklist 120
26 Parts List 125
A Mailbox Quick Reference 127
B Error and Status Codes 129
C Program Listings 131
D Game and CONFIG Listings 170

1. Your FujiNet Cartridge

The FujiNet cartridge for the ATARI 5200 is a network adapter that plugs into the cartridge slot. Inside the shell are two computers. On the cartridge edge sits an RP2354B microcontroller. It answers every read the console makes of the cartridge, as a ROM chip would, and puts its byte on the bus through a 74HCT541 buffer that only the console's own cartridge enables can open.

Behind the RP2354B, over a USB cable, an ESP32-S3 runs the FujiNet firmware. It carries the WiFi radio, the SD card, the network protocols and everything else a FujiNet does on every other computer it has been built for.

The console never sees the ESP32-S3. It sees CONFIG when the power comes on, a mailbox from `$B000` to `$B7FF`, and up to 128K of program, laid out the way a plain ROM, a two-chip board, Bounty Bob Strikes Back or a Super Cart would lay it out. This handbook is about all of them.

What you should have

When you set up to work through this handbook, you will have:

- A FujiNet cartridge, or the FujiNet cartridge model grafted into MAME, which is what every program in this handbook was verified on.
- A FujiNet adapter to talk to: an ESP32-S3 running the `fujiversal-atari5200` firmware, or `fujinet-pc` on the same machine as MAME, listening on bus-over-IP.
- cc65 2.19: `ca65` and `ld65` for the assembly programs, `cc65` for the C ones.
- fujinet-lib, built for its `atari5200` target, and MekkoGX, which carries the start-up code and the linker configuration, for the C programs.
- This handbook.

NOTE: The cartridge edge carries no reset line and no read/write line. The RESET key on the controller is a keypad code that your program reads like any other; it does not reach the cartridge. Only a power cycle, or a program that asks for it, sends the cartridge back to CONFIG.

IMPORTANT: The FujiNet 5200 board is a Rev0 design that has not yet been built. Everything in this handbook ran in host tests and in the MAME cartridge model. The first things a real board must prove are its timing on a console with no clock on the edge, and that reads the console makes twice in a row are told apart.

How this handbook is organised

Sections 3 through 9 are the machine: the memory map, the mailbox, the display, the memory you have to work in, the two languages, and the controllers. Sections 10 through 16 are programs, each one written twice: in 6502 assembly and in C. Sections 17 through 20 are the command reference: every command the adapter answers, and everything the cartridge itself does. Sections 21 through 23 walk through Battleship and 5 Card Stud, two complete networked games, and CONFIG. The rest is what went wrong on the way, and how to tell when it is going wrong for you.



Figure 1. The first program in Section 5, running from the cartridge in the MAME model.

2. Install the Tools

Every program in this handbook is built two ways, and every build runs on the same bench: the FujiNet cartridge grafted into MAME's ATARI 5200, talking over bus-over-IP to a FujiNet adapter running on the same computer. Install the pieces in this order.

cc65

The assembler, linker and C compiler are all cc65's: `ca65`, `ld65` and `cc65`, with `cl65` to drive them. Everything here was built with version 2.19. cc65's own `atari5200` target supplies the C library: the console, the joystick driver, the clock. The start-up code and the linker configuration come from MekkoGX instead, because cc65's own start-up leans on the BIOS, and a FujiNet program tells the BIOS to step aside (Section 3).

```
git clone https://github.com/cc65/cc65.git
cd cc65 && make && sudo make install PREFIX=/usr/local
cc65 --version          # cc65 V2.19
```

fujinet-lib

The C programs call fujinet-lib, the same library every FujiNet C program on every platform uses. Its `atari5200` target adds the mailbox transport: the hotspot reads, the transaction, and the calls that hand the console to a new image. Until it is merged, build it from the branch that adds it:

```
git clone -b add-atari5200 https://github.com/tschak909/fujinet-lib.git
cd fujinet-lib && make atari5200
ls r2r/atari5200        # fujinet.atari5200.lib
```

The library is the `.lib`; its headers are `include/` and `bus/atari5200/`. The build scripts in this handbook find it through `FNLIB`.

MekkoGX

MekkoGX is the build template FujiNet's cross-platform programs use. Its `atari5200` platform carries the three files every 5200 FujiNet program in C needs, and that the library leaves out:

File	What it does
`atari5200-crt0.s`	The start-up code. It does what the BIOS would have done, keeps the BIOS's zero page and vectors so cc65's library works, and takes the keypad's interrupt (Section 8).
`atari5200-fujinet.cfg`	The linker configuration: 32K of ROM around the mailbox, and the console's 16K of RAM for data, the heap and the C stack.
`atari5200-romstamp.py`	The checker: the claim, the BIOS header, and the mailbox's rules (Section 4).

```
git clone -b add-atari5200 https://github.com/tschak909/MekkoGX.git
ls MekkoGX/mekkogx/atari5200*      # crt0, cfg, romstamp
```

A program built the MekkoGX way, as the games in this handbook are, uses `platforms/atari5200.mk`, which names all three. This handbook's `listings/c/build.sh` calls `cl65` with them directly, so that every step shows; it finds them through `MEKKO`.

MAME with the FujiNet cartridge

The cartridge model lives in the firmware's `pico/atari-5200/emu/`. Its `apply.sh` grafts it into a MAME source tree; a build of the 5200 driver alone is enough.

```
git clone https://github.com/mamedev/mame.git
fujinet-firmware/pico/atari-5200/emu/apply.sh mame
cd mame
make SUBTARGET=a5200 SOURCES=src/mame/atari/atari400.cpp REGENIE=1 -j$(nproc)
./a5200 -listslots | grep fujinet
```

The last command should print `a5200  cartslot  fujinet  Atari 5200 FujiNet Cartridge`, and the same for `a5200a`, the two-port console. The 5200 BIOS image, `a5200.zip`, goes where MAME's `-rompath` finds it. The cartridge connects to a FujiNet adapter at `127.0.0.1:9995` unless the environment variable `FUJINET_TCP` names another address.

IMPORTANT: MAME must run from its own tree, or it ignores `-autoboot_script` and every headless check in this handbook does nothing. With no display, add `-video none -sound none`, as `emu/run.sh` does.

A FujiNet adapter

The adapter is `fujinet-pc`, the FujiNet firmware built to run on a computer, in its RS-232 flavour, which speaks bus-over-IP. Build it from the firmware:

```
cd fujinet-firmware
./build.sh -p RS232
cd build/dist && ./run-fujinet
```

In its `fnconfig.ini`, the `[BOIP]` section names the port it listens on; make it the port the cartridge connects to. Its `SD/` directory is the SD card, host slot 0. Only one MAME can talk to one `fujinet-pc` at a time.

CAUTION: A `fujinet-pc` older than October 2026 does not know the `.a52` extension. It mounts a 5200 image as a disk, says yes to MOUNT_IMAGE, and pushes nothing, so the cartridge waits for an image that never comes. Build it from current firmware.

Running a program

Each language has a build script, `listings/asm/build.sh` and `listings/c/build.sh`. Each writes a 32K image (64K for a Super Cart), checks it, and leaves it in its `build/` directory. MAME's `-cart` takes the image as it is, and the cartridge model serves a claimed image as if CONFIG had just booted it.

```
listings/c/build.sh acfg                          # listings/c/build/acfg.bin
emu/run.sh listings/c/build/acfg.bin              # in a window
SHOT_AT=6 WANT="Dummy Cafe" \
  emu/run.sh listings/c/build/acfg.bin drive      # headless, and checked
emu/verify.sh                                     # all 20 programs
```

`emu/drive.lua` presses the controller's keys and moves its stick on a timetable, then reads the screen as text, saves a snapshot, and with `WANT` passes only if every piece appears. The text is read straight out of the screen memory both languages' programs use.

NOTE: There is no FujiNet Go for the 5200. To try a program without building MAME, you need a cartridge.

3. The Memory Map

The ATARI 5200 is an Atari 400 rearranged: the same 6502C at 1.79 MHz, the same ANTIC, GTIA and POKEY, 16K of RAM, and a 2K BIOS in place of the 400's operating system. The chips sit at new addresses, and the cartridge has 32K of the address space to itself.

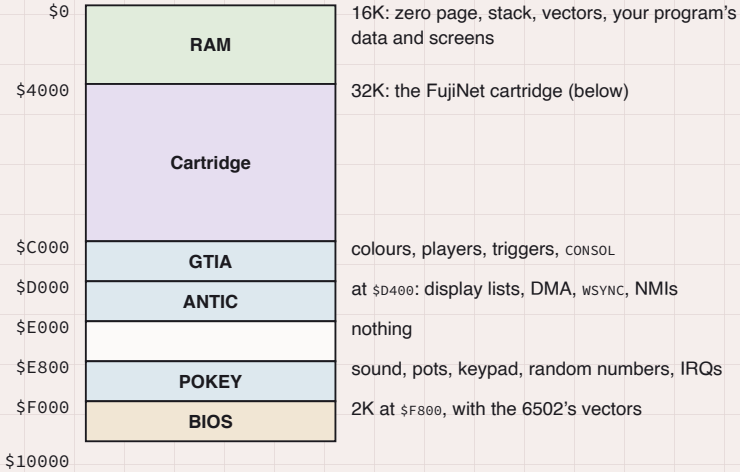


Figure 2. The console's address space.

The chip registers repeat through their pages: GTIA every 32 bytes, POKEY every 16. Use the base addresses in `fujinet.inc`; there is nothing to gain from the mirrors.

The cartridge window

The cartridge answers every read from `$4000` to `$BFFF`. What it answers with depends on what it is serving, which it reports in `MODE` at `$B412`:

MODE	Name	What the console sees
0	BOOT	CONFIG, from the cartridge's own flash. This is what a power-on gives you.
2	GAME	An image without the FujiNet claim: a game, served exactly as its own cartridge board would serve it. The mailbox is gone.

MODE	Name	What the console sees
3	APP	An image with the claim: your program, with the mailbox at `$B000`–`$B7FF` in every bank.

Every program in this handbook is an APP. Its 32K are laid out like this:

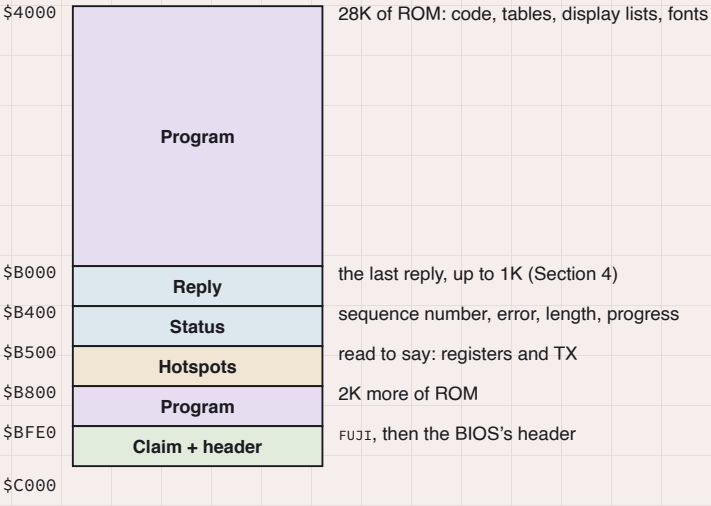


Figure 3. An APP in the cartridge window.

Your image is 32K, and the build fills `$B000`–`$B7FF` with `$FF`; the cartridge lays the mailbox over that part as it serves the image. Nothing you put there is ever seen.

The claim and the header

The BIOS reads the top 24 bytes of the window at power-on. The FujiNet claim sits just below them:

Address	Bytes	What it is
`$BFE0`	4	`"FUJI"`: the claim. Without it, the image runs as a GAME and the mailbox is switched off.
`$BFE4`	1	The claim's version: 1.
`$BFE5`	1	The board to serve the image as, plus one; 0 lets the cartridge decide from the size. A Super Cart app may say 4.

Address	Bytes	What it is
`$BFE6`	1	Flags: 0.
`$BFE7`	1	`$02`: the two-port console's BIOS reads this on a PAL machine and refuses a cartridge that does not say "PAL-compatible". Not part of the claim.
`$BFE8`	20	The title the BIOS would show. A FujiNet program never lets it.
`$BFFC`	1	The year the BIOS would show.
`$BFFD`	1	`$FF`: skip the BIOS's title screen and start the program at once.
`$BFFE`	2	The program's start address.

In assembly, `header.inc` lays all of this out for you; in C, MekkoGX's start-up code does. Both checkers refuse an image without it.

How a program starts

At power-on the cartridge serves CONFIG. When you pick a program, CONFIG has the FujiNet push it into the cartridge, quiets the console as a power-on leaves it, and jumps to a three-byte stub the cartridge serves at `$B5F0`: `JMP ($FFFC)`. The cartridge swaps your image in on the read of the stub's last byte, so the jump takes the console into the BIOS's own reset code with your program in the window. The BIOS sets up the chips, sees `$FF` at `$BFFD`, and jumps through `$BFFE`. Section 13 shows a program doing the same for another one.

Your program starts with interrupts off, display DMA off, and nothing set up. The start-up code of either language does what the BIOS would have done, and keeps the BIOS's zero page and vectors as the BIOS lays them out (Section 6).

No reset line, no read/write line

The 5200's cartridge edge carries the address lines, the data lines and two enables, one for each 16K half of the window, and that is all. Two things follow.

- **The cartridge cannot see a reset.** The RESET key is a key on the keypad, read like any other. A power cycle restarts the cartridge, and it comes back in CONFIG; nothing else does, except a program that asks (Section 13).

- **The cartridge cannot tell a read from a write.** Everything a program says to it is said with reads (Section 4), and a store anywhere in `$4000`–`$BFFF` is a mistake: the cartridge drives the data bus as if for a read, against the CPU.

The four-port console has no clock on the edge either. The cartridge decides that an access has happened when the address has held still long enough, so two reads of the same address on two cycles in a row look like one.

What every program must not do

`tools/checkrom.py` and MekkoGX's `atari5200-romstamp.py` check most of these on every build. The rest is up to you.

- Store anywhere in `$4000`–`$BFFF`.
- Use a read-modify-write instruction (`INC`, `DEC`, `ASL`, `LSR`, `ROL`, `ROR`) on the cartridge window.
- Read the hotspot pages `$B500`–`$B7FF` with anything but `LDA`, `LDX`, `LDY` or `BIT`, absolute or absolute-indexed, from a page-aligned base. An index that carries into the next page makes the 6502 read the wrong page first, and the cartridge counts that read.
- Read one hotspot address on two cycles in a row.
- Use `JMP ($xxFF)`, which fetches its high byte from the wrong page.
- Point ANTIC at the mailbox. A display list, a screen or a font in `$B000`–`$B7FF` would be read by DMA, and every DMA read of a hotspot is an event.
- Leave out the claim and the header.

4. How the Mailbox Works

The 5200's cartridge edge has no read/write line. The cartridge cannot tell a store from a load, and it never sees the byte a store puts on the bus. So everything the console sends to the FujiNet travels in the **address** of a read: the console reads `$B600`+*v* to say "*v*", and the cartridge, which sees every address, takes the byte from the low half of it. Everything the console learns, it reads from memory the cartridge paints. Five pages do all the work.

Address	Read it to	What it is
`$B000`–`$B3FF`	learn	The reply window: up to 1024 bytes of the last reply.
`$B400`–`$B4FB`	learn	The status page: sequence number, error, reply length, boot progress, the cartridge's mode.
`$B500`–`$B57F`	say	Register select. A read of `$B500`+*n* picks register *n*.
`$B600`–`$B6FF`	say	Register data. A read of `$B600`+*v* sets the picked register to *v*.
`$B700`–`$B7FF`	say	The TX stream. A read of `$B700`+*b* appends the byte *b*.

The value a read of `$B500`–`$B7FF` returns means nothing: the cartridge answers `$FF`. The top half of the select page, `$B580`–`$B5FF`, is the cartridge's own; Section 20 lists what is there.

A register is two reads

To set register *n* to *v*, read `$B500`+*n*, then read `$B600`+*v*. With the value in `Y`, the second read is one indexed load. To set the device, the command and the parameter count, and to rewind the TX stream:

6502 ASSEMBLY (ca65)

```
        ldx     #FN_REG_DEVICE
        lda     FN_REGSEL,x     ; pick register $00
        ldy     #$70            ; the FUJI device
        lda     FN_REGDATA,y    ; ...and set it
        ldx     #FN_REG_CMD
        lda     FN_REGSEL,x
        ldy     #$C4            ; GET_ADAPTERCONFIG_EXTENDED
        lda     FN_REGDATA,y
        ldx     #FN_REG_NPARAM
        lda     FN_REGSEL,x
        ldy     #0              ; no parameters
        lda     FN_REGDATA,y
        ldx     #FN_REG_DATA_RST
        lda     FN_REGSEL,x
        lda     FN_REGDATA,y    ; any value rewinds
```

C (cc65 + fujinet-lib)

```
fn_regwr(FNR_DEVICE, 0x70);      /* the FUJI device */
fn_regwr(FNR_CMD, 0xC4);         /* GET_ADAPTERCONFIG_EXTENDED */
fn_regwr(FNR_NPARAM, 0);         /* no parameters */
fn_regwr(FNR_DATA_RST, 0);
```

The assembly library wraps the pair as `fn_rw`, register `X` = `A`. In C, `fn_regwr()` is fujinet-lib's, written in assembly so that each half is exactly one `LDA` from a page-aligned base. A data read with no select read just before it does nothing at all.

Register	Select	Meaning
`$00` DEVICE	`$B500`	The FujiBus device: `$70` the FujiNet itself, `$71`–`$78` the network devices, `$45` the clock.
`$01` CMD	`$B501`	The command.
`$02` NPARAM	`$B502`	How many parameters the TX stream holds.
`$05` DATA_RST	`$B505`	Any value: rewind the TX stream to its start.
`$06` RXSLICE	`$B506`	Which 1K slice the reply window shows. There is one slice; leave it 0.
`$10` SEQ	`$B510`	Launch the transaction (below).
`$11` BOOTLOCK	`$B511`	`$B5`: arm a pushed image for the hand-over (Section 13).
`$12`, `$13`	`$B512`–`$B513`	`$B5` then `$4A`: reboot the cartridge's RP2354B for a firmware update.
`$14` CONFIG	`$B514`	`$C5`: stage CONFIG and arm the hand-over back to it.

CAUTION: Read `$B500`–`$B7FF` only with `LDA`, `LDX`, `LDY` or `BIT`, absolute or absolute-indexed, from a page-aligned base. Never store there: with no read/write line, the cartridge counts a store as a read, and its buffer drives the bus against the CPU. Never use `INC`, `DEC`, `ASL`, `LSR`, `ROL`, `ROR` or `(zp),Y` on these pages: each makes extra accesses that the cartridge counts. Never read one hotspot address on two bus cycles in a row: on the 4-port console, which has no clock on the edge, the cartridge sees them as one. In C, never touch these pages except through `fn_regwr()` and `fn_tx()`.

The build tools hold you to this. `checkrom.py` reads the assembler's map and refuses an image that stores into the cartridge window, uses a read-modify-write instruction or `JMP ($xxFF)` there, or reads a hotspot page from a base that is not page-aligned. MekkoGX's `atari5200-romstamp.py` makes the same checks on a C program.

The TX stream

Parameters and payload go into the TX stream one read per byte. Each parameter is its size (1, 2 or 4) and then its value, low byte first. The payload, if the command takes one, follows the last parameter.

size	value…	size	value…	…	payload…

The stream holds 320 bytes and at most eight parameters. A parameter of any other size fails the transaction with error 3 before anything is sent; bytes past 320 are dropped. `DATA_RST` rewinds the stream, so write it before the first byte of every transaction.

6502 ASSEMBLY (ca65)

```
        lda     #1              ; a one-byte parameter...
        tay
        lda     FN_TXDATA,y     ; ...its size
        ldy     #$02            ; ...and its value
        lda     FN_TXDATA,y
```

C (cc65 + fujinet-lib)

```
fn_tx(1);                       /* a one-byte parameter: its size */
fn_tx(0x02);                    /* ...and its value */
```

Launching, and the sequence number

A transaction is launched by one more register write: the next sequence number into register `$10`. The cartridge sends the request to the FujiNet,

waits for the answer, paints the reply into $B000, the error and the reply length into the status page, and then, **last**, copies your sequence number into ACKSEQ at $B400. The moment ACKSEQ equals your number, everything else is in place.

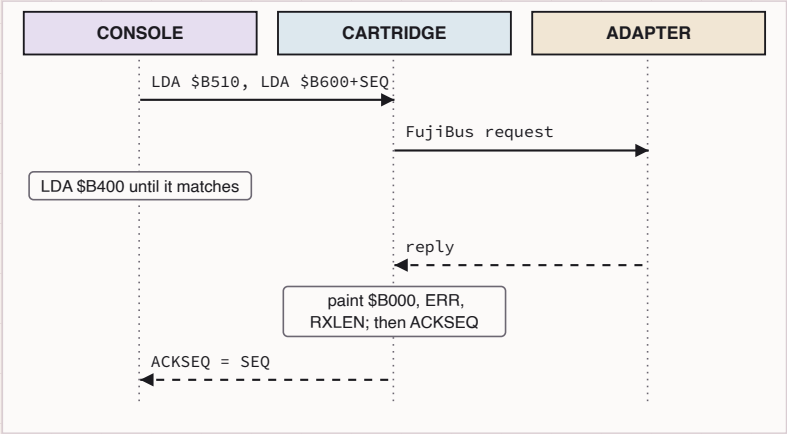


Figure 4. A transaction. The console only ever reads.

The next number is always the cartridge's own ACKSEQ plus one, wrapping from 255 to 1. Zero is reserved, and a number equal to the last one does nothing at all, so a stray repeat cannot send a request twice. Take the number from the cartridge every time, never from a counter of your own. The cartridge starts its count over whenever a program starts, at power-on and at every hand-over (Section 13), and only the cartridge knows where it has got to.

The status page

The cartridge paints the status page at $B400 after every transaction and whenever its state changes. Read it in place; it is ordinary memory to the console.

Address	Name	Meaning
$B400	ACKSEQ	Your sequence number, copied here last, when the reply is in place.
$B401	STAT	Bit 0: the link to the FujiNet is up.

Address	Name	Meaning
$B402	ERR	The transaction's error: 0 none, 1 no link, 2 the FujiNet did not answer in time, 3 a bad frame (a parameter of the wrong size), 4 too big.
$B403	REPLYCMD	The FujiNet's answer: $06 ACK, $15 NAK.
$B404	RXLEN	The reply's length in bytes, low byte first.
$B406	BOOTSTATE	Booting another image: 0 idle, 1 arriving, 2 ready, $80 failed (Section 13).
$B407	BOOTPCT	How much of it has arrived, 0–100.
$B409	MAGIC	'F', 'N': a FujiNet cartridge is answering.
$B412	MODE	0 BOOT, 2 GAME, 3 APP (Section 3).

Section 20 lists the rest. A program checks MAGIC once, at the start, before it trusts anything else: on a console with any other cartridge, or with none, the bytes there are something else.

The reply window

The reply is painted at $B000, up to 1024 bytes, and RXLEN says how many are real. It stays until the next transaction replaces it, so a program can read it at leisure, but only until then. Copy anything you want to keep into RAM before you launch the next one.

How long to wait

The cartridge gives every transaction a budget: five seconds for most commands, sixty for MOUNT_IMAGE and COPY_FILE, and ninety for a network device's OPEN, CLOSE, STATUS and READ. When the budget runs out it reports error 2 and copies your sequence number anyway, so a program waiting on ACKSEQ always hears back from a cartridge that is there.

Wait a little longer than that budget yourself, for the case where nothing answers at all. The assembly library counts down FN_TMO seconds (12 unless you raise it after fn_beg) and returns FNEWAIT ($FF) when they run out. fujinet-lib waits at least as long as the cartridge would for each command, and reports FN_EWAIT the same way.

The recipe

Every transaction is the same six steps:

1. Set DEVICE, CMD and NPARAM, and rewind the TX stream.
2. Append the parameters and the payload.
3. Read `ACKSEQ`, add one (255 wraps to 1), and write it to SEQ.
4. Wait for `ACKSEQ` to equal it.
5. Check `ERR` for 0, then `REPLYCMD` for `$06`.
6. Read the reply.

In assembly, `fn_beg` does step 1 from `FN_DEV`, `FN_CMD` and `FN_NPR`; `fn_pb` and `fn_pw` do step 2; `fn_go` does steps 3 to 5 and returns `ERR`, or `FNEWAIT`; `fn_ack` checks `REPLYCMD`. In C, one fujinet-lib call does all six.

6502 ASSEMBLY (ca65)

```
        lda     #FNDEVF         ; 1. the FUJI device...
        sta     FN_DEV
        lda     #FNCMHST        ;    ...MOUNT_HOST...
        sta     FN_CMD
        lda     #1              ;    ...one parameter
        sta     FN_NPR
        jsr     fn_beg
        lda     #0              ; 2. host slot 0
        jsr     fn_pb
        jsr     fn_go           ; 3-5. launch, wait, ERR
        bne     failed
        jsr     fn_ack          ;    REPLYCMD
        bne     refused
```

C (cc65 + fujinet-lib)

```
if (!fuji_mount_host_slot(0))    /* all six steps */
    failed();
```

5. The Text Display

The 5200 draws its picture the way every Atari computer does. ANTIC reads a **display list**, a short program in memory that says, line by line, what kind of picture to make and where its data is; GTIA colours what ANTIC sends it. For text, the display list asks for ANTIC mode 2: 40 characters a row, each 8 × 8 pixels from a font of 128 characters, with bit 7 of a character code meaning reverse video. Twenty-four rows fill the screen.

Every program in this handbook writes its text the same way, into a 40 × 24 screen in RAM. There are three ways to get there.

Road	Language	Screen	What it is
`fujidisp.s`	assembly	`$1000`	The bring-up's: mode 2, a display list and a font in ROM, routines to place, print and show hex and decimal.
`antic.c`	C	`$3C00`	CONFIG's: the same mode, a cursor, strings, fills, reverse video.
cc65 conio	C	`$3E20`	cc65's own console: 20 × 24 in ANTIC mode 6, four colours, bigger letters.

The first two give the same screen, from the same font, so the programs in this handbook look alike in both languages.

The display list

`fujidisp.s` keeps its display list in ROM. Three instructions of eight blank lines push the picture down from the top of the television; the fourth says "mode 2, and load the memory counter" with the screen's address; twenty-three more say "mode 2" and carry on from there; the last jumps back to the start and waits for the next frame.

6502 ASSEMBLY · fujidisp.s

```
dlist:  .byte   $70, $70, $70
        .byte   $42, <SCREEN, >SCREEN
        .repeat 23
        .byte   $02
        .endrepeat
        .byte   $41, <dlist, >dlist
```

The font is in ROM too, 1K-aligned, because ANTIC finds it from the page number in CHBASE ($D409). Neither may sit in the mailbox at $B000–$B7FF: ANTIC would read the hotspots (Section 3). The screen may go anywhere in RAM that does not cross a 4K boundary, where ANTIC's memory counter wraps.

The BIOS keeps a shadow of each display register in zero page, and its vertical-blank interrupt copies them to the chips every frame. Both languages' start-up code keeps that arrangement, so a program sets the shadows, not the chips:

Shadow	Copied to	In mode 2
SDLSTL/H $05/$06	DLISTL/H $D402	The display list's address.
SDMCTL $07	DMACTL $D400	$22: a normal-width playfield and display-list DMA; 0 turns the picture off.
COLOR1 $0D	COLPF1 $C017	The text's brightness (its colour is the background's).
COLOR2 $0E	COLPF2 $C018	The background.
COLOR4 $10	COLBK $C01A	The border.

Character codes are ANTIC's, not ASCII: $20–$5F become $00–$3F, $00–$1F become $40–$5F, and $60–$7F stay as they are. Both libraries translate as they print, so programs pass ASCII.

hello

The first program shows four lines: a title, whether a FujiNet cartridge is answering (MAGIC at $B409), the frame counter the VBI keeps, and the last key pressed on the keypad. It makes no transactions, so it runs with no adapter at all.

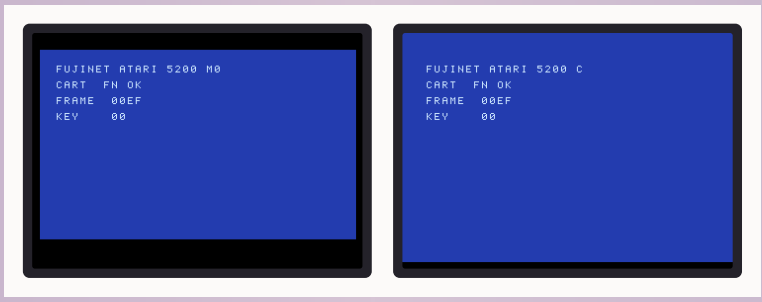


Figure 5. hello in assembly and in C.

In assembly, `hello.s` is the bring-up's own first program, unchanged. `disp_init` clears the screen and points ANTIC at it with the display still off; `disp_on` turns it on once the text is there, so nothing half-drawn is ever seen.

6502 ASSEMBLY (ca65) · hello.s

```
.proc main
        jsr     disp_init
        PUTS    2, 2, title
        PUTS    4, 2, tcart
        jsr     fn_chk
        bne     nocart
        PUTS    4, 8, tyes
        jmp     on
nocart: PUTS    4, 8, tno
on:     PUTS    6, 2, tframe
        PUTS    8, 2, tkey
        jsr     disp_on
loop:   lda     #6
        ldx     #9
        jsr     disp_at
        lda     RTCLOKH
        jsr     disp_hex
        lda     RTCLOKL
        jsr     disp_hex
        lda     #8
        ldx     #9
        jsr     disp_at
        lda     KEY
        jsr     disp_hex
        jmp     loop
```

In C, `mt_init` does what `disp_init` and `disp_on` do together, and `mt_colours` sets the background and the text's brightness:

C (cc65 + fujinet-lib) · hello.c

```
void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    puts_at(2, 2, "FUJINET ATARI 5200 C");
    puts_at(4, 2, "CART");
    puts_at(4, 8, fuji_a5200_present() ? "FN OK" : "NO MAILBOX");
    puts_at(6, 2, "FRAME");
    puts_at(8, 2, "KEY");
```

The third road: cc65's console

cc65's `atari5200` library has a console of its own: `clrscr`, `gotoxy`, `cputs` and the rest of `conio.h`, on a 20 × 24 screen of big four-colour letters. It works under MekkoGX's start-up code, because that code keeps the BIOS's zero page and vectors where cc65 expects them. It has no keyboard input; read the keypad yourself (Section 9).

C (cc65 + fujinet-lib) · conio.c

```
void main(void)
{
    clrscr();
    gotoxy(2, 2);
    cputs("HELLO FROM CONIO");
    gotoxy(2, 4);
    cputs(fuji_a5200_present() ? "CART FN OK" : "NO MAILBOX");
    for (;;)
        ;
```

CAUTION: cc65 keeps the console's screen at `$3E20`–`$3FFF`, at the very top of RAM, where MekkoGX's linker configuration puts the C stack unless you reserve the room. Link with `-Wl -D,__RESERVED_MEMORY__=0x1E0` or more. With nothing reserved, the stack is built on the screen, and every call writes on it.



Figure 6. The console with `0x1E0` reserved, and with nothing reserved: the two marks low on the right are the stack.

6. Living in 16K

The 5200 has 16K of RAM, from `$0000` to `$3FFF`, and a FujiNet program has all of it: the BIOS steps aside before the program starts, and the cartridge never needs the console's RAM for anything. What the BIOS would have used is still laid out the BIOS's way, because both languages' start-up code keeps that layout, and cc65's library depends on it.

Zero page

Address	Name	What it is
`$00`	POKMSK	The POKEY interrupts the program wants on; the IRQ handler restores `IRQEN` from it.
`$01–$02`	RTCLOK	The frame counter, high byte first: the VBI adds one to `$02` every frame.
`$03`	CRITIC	Non-zero: the VBI skips the shadow copies this frame.
`$04`	ATRACT	The attract-mode counter. A key press clears it.
`$05–$07`	SDLST, SDMCTL	Display shadows (Section 5).
`$08–$10`	PCOLR0–3, COLOR0–4	Colour shadows for the players and the playfield.
`$11–$18`	PADDL0–7	The eight pots, copied by the VBI: each controller's stick X and Y (Section 9).
`$19`	KEY	Assembly: the last keypad key + 1 (`a52init.s`).
`$19–$1C`		C: cc65's console cursor and colour.
`$1D–$FF`		C: cc65's own zero page, and yours.
`$E0–$ED`	`FN_*`	Assembly: the 6502 library's (Section 7).
`$F0–$F5`	PTR1, PTR2, ZTMP	Assembly: this handbook's programs' pointers (`book.inc`).

CAUTION: The bring-up's `a52.cfg` starts the linker's `ZEROPAGE` segment at `$19`, which is also `KEY`, the byte the keypad interrupt writes. A program that puts a variable in `ZEROPAGE` and reads the keypad loses one or the other. This handbook's programs keep their variables in `BSS` and put the few pointers they need at fixed addresses above the library's.

The vectors

The BIOS's interrupt handlers jump through RAM vectors at `$0200`–`$021B`, with nothing saved: a handler saves what it uses. Both start-up files fill the table with handlers that do what the BIOS's would.

Address	Vector	Taken on
`$0200`	VIMIRQ	Every IRQ: the handler looks at `IRQST` and goes on through one of the rest.
`$0202`	VVBLKI	The vertical blank, first: the frame counter and the shadow copies.
`$0204`	VVBLKD	The vertical blank, deferred: put your own per-frame work here.
`$0206`	VDSLST	A display-list interrupt (Section 16).
`$0208`	VKYBDI	A keypad key: reads `KBCODE` and translates it.
`$020A`	VKYBDF	The keypad, continued: here the translated key is counted.
`$020C`–`$021A`		POKEY's other interrupts: break, `BRK`, serial, timers 1, 2 and 4. Unused here.

RAM

	Assembly (`a52.cfg`)	C (MekkoGX)
`$0100`–`$01FF`	the 6502 stack	the 6502 stack
`$0200`–`$021B`	vectors	vectors
`$021C`–`$02FF`	free	data, BSS, heap…
`$0300`–`$0FFF`	`BSS`	…
`$1000`–`$13BF`	the screen	…
`$13C0`–`$37FF`	free	…
`$3800`–`$3BFF`	free	the C stack (1K)
`$3C00`–`$3FFF`	free	reserved: `antic.c`'s screen

In C, `__RESERVED_MEMORY__` takes room off the top of RAM, and the 1K C stack goes below it. This handbook's C programs reserve 1K for the `antic.c` screen at `$3C00`; cc65's console needs `$3E20`–`$3FFF`, so 1K covers it too.

What lasts

Nothing in RAM outlives the program. Handing the console to another image clears all 16K as a power-on does. A power cycle restarts everything, the cartridge included: it watches the console's power, and when the power comes back it serves CONFIG with its count started over. A program that must remember something between runs keeps it on the FujiNet, in an appkey (Section 14).

The reply is not yours

The reply window at `$B000` is cartridge memory. A program may read it as often as it likes, but only until its next transaction, and ANTIC must never read it. Text from a reply goes on the screen by being copied into screen RAM, as `disp_rpl` does in assembly and `mt_text` in C; data a program wants to keep goes into its own RAM before the next transaction.

7. The 6502 Library

The assembly programs in this handbook are built on the bring-up's own client code, copied unchanged from the cartridge's `testrom/` directory into `listings/common/`. Four files do the work, and three more of the handbook's own name the commands the bring-up never needed.

File	What it is
`fujinet.inc`	The mailbox's addresses and registers, the 5200's chip registers, the BIOS's zero page and vectors, the library's zero page and error codes. Mirrored by hand from the cartridge's `fuji_mailbox.h`, and checked against it by the bring-up's `checkdefs.py`.
`fujilib.s`	The transport: begin, append, launch and wait, check, hand over.
`fujidisp.s`	The 40 × 24 text screen (Section 5).
`a52init.s`	Start-up: the chips, the vectors, the VBI and the keypad.
`header.inc`	The claim and the BIOS's header (Section 3).
`devdefs.inc`	The handbook's: the other devices, every FUJI command by name, and the `FNBEGIN` macro.
`netdefs.inc`	The handbook's: the network device's commands, open modes and status layout.
`book.inc`	The handbook's: its programs' zero page, `PUTS` and `WAITFRAME`.

The transport

Routine	In / out	What it does
`fn_chk`	Z set if present	Looks for `'F'`, `'N'` at `$B409`.
`fn_beg`	`FN_DEV`, `FN_CMD`, `FN_NPR`	Writes the device, command and parameter count, then rewinds the TX stream. Sets `FN_TMO` to 12 seconds.
`fn_pb`	A	Appends a one-byte parameter.
`fn_pw`	A low, X high	Appends a two-byte parameter.
`fn_txb`	A	Appends one byte of payload. Uses Y.
`fn_path`	`FN_PTR`	Appends the NUL-terminated string at `FN_PTR`, padded with NULs to exactly 256 bytes, which is what OPEN_DIRECTORY and SET_DEVICE_FULLPATH read.

Routine	In / out	What it does
`fn_go`	A = result	Launches the transaction with `ACKSEQ`+1 and waits up to `FN_TMO` seconds. Returns the cartridge's error (0 is success), or `$FF` if it never answered.
`fn_ack`	A = result	After `fn_go` returned 0: 0 if the FujiNet said yes, `$EE` if it said no.
`fn_blk`	—	Arms a pushed image (BOOTLOCK).
`fn_boot`	A = `$FC`	Arms the pushed image and hands the console to it (Section 13). Returns only if the cartridge never armed it.
`fn_config`	A = `$FC`	Hands the console back to CONFIG. Returns only if the cartridge never armed it.
`fn_rw`	X register, A value	One register write: two reads. The others are built on it.
`fn_frames`, `fn_wait`	A seconds; C	A countdown in frames: `fn_frames` starts it, `fn_wait` returns C clear when it has run out.

WARNING: `fn_txb` carries its byte to the cartridge in the address of `LDA FN_TXDATA,Y`, so it changes Y. A loop that sends a string must count in X. `tools/snipcheck.py` rejects an example that counts in Y around it, because such a loop assembles cleanly and sends the wrong bytes.

Every transaction in the handbook's programs has the same shape:

6502 ASSEMBLY (ca65) · clock.s

```
; ASK cmd, row -- a clock command with no parameters, its text on the row.
.macro  ASK     cmd, row
        lda     #CLKDEV
        sta     FN_DEV
        lda     #cmd
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
```

and every failure goes to one place, through a `go_ack` that each program carries:

6502 ASSEMBLY (ca65) · netget.s

```
; go_ack -- fn_go, then fn_ack; on any failure the code goes in aperr and
; the failure screen follows, so callers need no branch.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla                     ; drop our caller's return
        pla
        jmp     fail
.endproc
```

`go_ack` drops its caller's return address on failure and jumps to the program's `fail`, so a caller can treat each transaction as one line that either works or does not come back.

The wait

`fn_go` is the recipe of Section 4 with a timeout. The timeout is counted in frames, by the frame counter the VBI keeps at `RTCLOKL` (`$02`): `fn_frames` turns seconds into frames (64 a second, near enough), and `fn_wait` counts one off each time the counter moves.

6502 ASSEMBLY (ca65) · fujilib.s

```
.proc fn_go
        lda     FN_ACKSEQ
        clc
        adc     #1
        bne     have
        lda     #1              ; 0 is reserved as "never used"
have:   sta     FN_SEQ
        ldx     #FN_REG_SEQ
        jsr     fn_rw           ; launching it is this one pair
        lda     FN_TMO
        jsr     fn_frames
wait:   lda     FN_ACKSEQ       ; published LAST, after the whole reply
        cmp     FN_SEQ
        beq     got
        jsr     fn_wait
        bcs     wait
        lda     #FNEWAIT
        rts
got:    lda     FN_ERR
        rts
.endproc
```

Because the count comes from the VBI, the wait is the same whatever the program is doing, and stops if the VBI does: never call `fn_go` with interrupts

off. Raise `FN_TMO` after `fn_beg` for a command the cartridge allows longer: 66 for MOUNT_IMAGE and COPY_FILE, 96 for the network device's OPEN, STATUS, READ and CLOSE.

The zero page

Address	Name	What
`$E0`	`FN_SEQ`	The sequence number being waited for.
`$E1`–`$E2`	`FN_CNT`	Frames left.
`$E3`–`$E5`	`FN_DEV`, `FN_CMD`, `FN_NPR`	The transaction `fn_beg` begins.
`$E6`	`FN_TMO`	The timeout, in seconds.
`$E7`	`FN_LAST`	`RTCLOKL` as `fn_wait` last saw it.
`$E8`–`$E9`	`FN_PTR`	A string pointer, for `fn_path` and `disp_str`.
`$EA`–`$ED`	`DISP_PTR`, `DISP_TMP`	The text cursor and scratch.

The screen

Routine	In	What it does
`disp_init`	—	Clears the screen at `$1000`, points ANTIC at it, sets the colours, cursor home. The display stays off.
`disp_on`	—	Turns the display on (`SDMCTL` = `$22`).
`disp_at`	A row, X column	Moves the cursor.
`disp_putc`	A	Prints one ASCII character and advances. Keeps Y.
`disp_str`	`FN_PTR`	Prints a NUL-terminated string.
`disp_hex`	A	Prints two hex digits.
`disp_dec`	A	Prints up to three decimal digits.
`disp_rpl`	X offset, Y most	Prints from the reply window at `$B000`+X, up to a NUL or Y characters: a reply field straight onto the screen.

Start-up

The header's `$FF` at `$BFFD` sends the BIOS straight to `start`, with nothing set up. `a52init.s` does what the BIOS would have done, without depending on which of the two BIOS revisions it is:

6502 ASSEMBLY (ca65) · a52init.s

```
.proc start
        sei
        cld
        ldx     #$FF
        txs
        lda     #0
        sta     NMIEN
        sta     DMACTL
        sta     IRQEN
        tax
clr:    sta     POKEY,x         ; every chip register to 0, as the BIOS does
        sta     GTIA,x
        sta     ANTIC,x
        sta     $00,x           ; and the zero page
        sta     $0200,x
        inx
        bne     clr
        ldx     #vecs_end - vecs - 1
vec:    lda     vecs,x
        sta     VIMIRQ,x
        dex
        bpl     vec
        lda     #0              ; no key held
        sta     hold
        lda     #$02            ; keypad scan on
        sta     SKCTL
        lda     #$40            ; its IRQ, alone
        sta     POKMSK
        sta     IRQEN
        sta     NMIEN           ; the VBI, no DLIs
        cli
        jmp     main
```

It clears every chip register, the zero page and page 2, fills the vectors with its own handlers, turns the keypad's scan and interrupt on, and enables the VBI. The VBI is the BIOS's, register for register: it counts frames, copies the display and colour shadows and the pots, and jumps through `VVBLKD`. The keypad handler is in Section 9.

Building

`a52.cfg` lays out a 32K program: code from `$4000`, the arena at `$B000` filled with `$FF`, a second ROM area at `$B800`, the claim at `$BFE0` and the header at `$BFE8`; the zero page from `$19`; `BSS` from `$0300`. `listings/asm/build.sh` assembles a program with the three shared files, links it, and checks it with `checkrom.py`:

```
cd listings/asm
./build.sh netget             # build/netget.bin
```

The script finds a program in `listings/asm/` first and in `listings/common/` second, so the bring-up's own `hello`, `fujitest`, `fujiboot` and `fujibank` build the same way.

8. C and fujinet-lib

A C program for the FujiNet 5200 is an ordinary cc65 program with three things added: MekkoGX's start-up code and linker configuration in place of cc65's, fujinet-lib for the FujiNet, and a check after the link. The C library is cc65's own `atari5200` one, unchanged.

The start-up code

cc65's own start-up for the 5200 expects the BIOS to have run first, and a FujiNet program tells the BIOS not to (`$BFFD` = `$FF`). MekkoGX's `atari5200-crt0.s` does the BIOS's work itself:

1. Interrupts and DMA off, the chip registers cleared, the stack set.
2. The vectors at `$0200`–`$021B` filled with handlers that do what the BIOS's do: a VBI that counts frames in `RTCLOK` and copies the display, colour and pot shadows; a keypad handler (Section 9).
3. A blank display list, so the screen is quiet until the program sets its own.
4. cc65's own initialisation: the data copied from ROM, the BSS cleared, the constructors run.
5. `main()`.

It also lays down the claim and the BIOS header, so a C program never writes them. Because the zero page and vectors are the BIOS's layout, cc65's console, joystick driver and clock work as they would under the BIOS.

The linker configuration

Area	Where	What goes there
`ZP`	`$1D`–`$FF`	cc65's zero page and yours.
`RAM`	`$021C` up	`DATA` (copied from ROM at start-up), `BSS`, the heap.
`STACK`	1K under the reserve	The C stack.
`ROM`	`$4000`–`$AFFF`	Code, read-only data, display lists (`DLIST`, 1K-aligned).
`ARENA`	`$B000`–`$B7FF`	Nothing: filled with `$FF`; the cartridge serves the mailbox here.

Area	Where	What goes there
ROM2	$B800–$BFDF	Fonts (FONT, 1K-aligned) and anything else that fits.
CLAIM, HDR	$BFE0–$BFFF	The claim and the header, from the start-up code.

__RESERVED_MEMORY__ takes room off the top of RAM for screens. This handbook's C programs reserve 1K for the screen at $3C00.

The checker

atari5200-romstamp.py reads the image and the linker's map, and refuses an image that is the wrong size, lacks the claim, $02 at $BFE7, $FF at $BFFD or a start address, stores into the cartridge window, uses a read-modify-write instruction there, reads a hotspot page with an index from a base that is not page-aligned, or uses JMP ($xxFF). This handbook's build runs checkrom.py as well, which checks the same rules from the bring-up's side.

What the library gives you

Everything in fujinet-fuji.h, fujinet-network.h and fujinet-clock.h works as it does on every FujiNet computer: fuji_get_adapter_config_extended(), network_open(), fuji_read_appkey() and the rest. fujinet-atari5200.h adds what only a 5200 needs:

Call	What it does
fuji_a5200_present()	True if a FujiNet cartridge answers, with a protocol the library knows.
fuji_a5200_boot_state(), _percent(), _error(), _got(), _total()	The progress of an image being pushed (Section 13).
fuji_a5200_boot()	Hands the console to the pushed image. Returns false only if the cartridge never armed it.
fuji_a5200_config()	Hands the console back to CONFIG.
fuji_a5200_mode(), fuji_a5200_staged()	What the cartridge is serving, and what it has been pushed.
fuji_a5200_key	The last keypad key + 1, 0 for none (Section 9).

Call	What it does
`fn_default_timeout`	Seconds to wait for an ordinary command; the library waits longer for the commands the cartridge gives longer.
`fn_device_error`	Why the last call failed: the cartridge's error, `$FF` if it never answered, 144 for a NAK.

Below the library

For a command with no wrapper, `fuji_bus_call()` takes the device, the command, which of four one-byte parameters and a payload or reply buffer to use, and does the whole transaction. GET_ISO_LOCAL on the clock device, for example, takes nothing and answers with text:

C (cc65 + fujinet-lib)

```
if (!fuji_bus_call(0x45, 0x49, FUJI_FIELD_REPLY,
                   0, 0, 0, 0, when, sizeof when))
    failed();
mt_text((char *)when, 25);
```

Below that is the bus layer itself, `fujinet-bus-atari5200.h`: `fn_regwr()` writes a register, `fn_tx()` appends a byte, and `fn_commit()` launches and waits, returning the cartridge's error or `FN_EWAIT`. The reply is `FN_REPLY` and the status page has a name for every field. This is the recipe of Section 4, a step at a time:

C (cc65 + fujinet-lib)

```
fn_regwr(FNR_DATA_RST, 0);
fn_regwr(FNR_DEVICE, 0x70);      /* FUJI */
fn_regwr(FNR_CMD, 0xC4);         /* GET_ADAPTERCONFIG_EXTENDED */
fn_regwr(FNR_NPARAM, 0);
if (fn_commit() != FN_OK || FN_REPLYCMD != 0x06)
    failed();
for (n = 0; n < 16 && FN_REPLY[140 + n]; ++n)
    mt_put(FN_REPLY[140 + n]);   /* the IP address, as text */
```

Rules for C

- **Touch the hotspot pages only through `fn_regwr()` and `fn_tx()`.** They are assembly, so each byte is exactly one `LDA` from a page-aligned base. A C expression that reads `$B700+`*b* may compile to `(zp),Y`, or read twice.

- **Never assign to an address in `$4000–$BFFF`.** There is nothing there to write, and the cartridge drives the bus against you.
- **Switch Super Cart banks in assembly.** A bank is chosen by reading a hotspot, and cc65's optimiser drops a read whose value is unused, even an inline `lda`. Use a `bit` in an assembly file (Section 15).
- **Spell out CR and LF.** cc65's Atari targets compile `'\n'` as `$9B`, the Atari's end-of-line, not 10. Text from the network uses `0x0D` and `0x0A`.
- **Reserve the screens you use**, with `__RESERVED_MEMORY__` (Section 5).

Building

`listings/c/build.sh` compiles each program, links it with MekkoGX's start-up code first on the line (so it, not cc65's, is the start-up), `antic.c`, and the library, then runs both checkers:

```
cd listings/c
FNLIB=~/fujinet-lib MEKKO=~/MekkoGX/mekkogx ./build.sh netget
```

A project built the MekkoGX way gets the same from `make atari5200`, which reads `platforms/atari5200.mk`. That is how the games are built (Section 21).

9. Controllers, Keypad and Sound

The 5200 controller is three inputs in one case: an analog stick, two buttons on each side, and a twelve-key keypad with START, PAUSE and RESET above it. The original console has four controller jacks; the later two-port console, two. Each part of the controller reaches the 6502 through a different chip.

Part	Read from	How
The stick	POKEY's pots, through `PADDL0`–`7`	Two pots per controller. Controller 1's X and Y are `PADDL0` and `PADDL1`, controller 2's the next two, and so on. Near 0 at one end, 228 at the other, 114 at rest.
The bottom buttons	GTIA: `TRIG0`–`TRIG3`, `$C010`–`$C013`	0 while held. The Owner's Manual calls these the fire buttons for every game.
The top buttons	POKEY: `SKSTAT` bit 3, `$E80F`	Clear while the top button of the selected controller is held.
The keypad and START, PAUSE, RESET	POKEY: `KBCODE`, `$E809`, by interrupt	The selected controller's keypad only.

"Selected" means picked by `CONSOL` at `$C01F`, which on the 5200 is an output: bits 0 and 1 choose the controller whose keypad and top button POKEY scans, and bit 2 turns the pots on.

IMPORTANT: Write `CONSOL` before you read the stick. Neither the BIOS nor either language's start-up code sets bit 2, and with it clear the pots read nothing on a real console. MAME's 5200 ignores the bit, so a program that forgets runs perfectly in the emulator. `$04` selects controller 1 with the pots on.

The stick

The VBI copies the eight pots into `PADDL0`–`PADDL7` (`$11`–`$18`) every frame and starts the next reading, so a program reads the shadows, never the pots. The stick springs back to the middle, and its middle is not exact: treat 64 to 164 as "not pushed", as the games do.

The keypad

POKEY scans the selected keypad on its own and raises an interrupt when it finds a key down; the handler reads `KBCODE`, turns its four key bits into a number through a table, and counts the press.

Key	Code	Key	Code	Key	Code
0–9	`$00`–`$09`	*	`$0A`	#	`$0B`
START	`$0C`	PAUSE	`$0D`	RESET	`$0E`

The library stores the code plus one, so that 0 means no key: `KEY` (`$19`) in assembly, `fuji_a5200_key` in C. Nothing clears it; a program takes a key by reading it and storing 0.

CAUTION: The BIOS runs POKEY's keypad scan with debouncing off. That way POKEY interrupts on every pass over a key that is held down, and the key-down bit in `SKSTAT` drops between passes, so a handler that counts interrupts counts one press many times. Both start-up files count a key once, then ignore it until four frames have passed without it.

6502 ASSEMBLY (ca65) · a52init.s

```
.proc kbd_done
        ldx     #HOLD_FRAMES
        cmp     held
        bne     @new
        ldy     hold
        stx     hold
        cpy     #0
        bne     @same
@new:   stx     hold
        sta     held
        clc
        adc     #1
        sta     KEY
        lda     #0
        sta     ATRACT
@same:  jmp     vbi_exit
```

Sound

POKEY has four voices. Each has a frequency register (`AUDF1`–`AUDF4`, `$E800`, `$E802`, `$E804`, `$E806`; higher is lower) and a control register (`AUDC1`–`AUDC4`, the odd addresses), whose top three bits choose the distortion and bottom four the volume: `$A8` is a pure tone at volume 8, 0 is silence. `AUDCTL` at `$E808` joins voices and changes clocks; 0 is the ordinary setting. POKEY makes no

sound while `SKCTL` is 0, its reset state; both start-up files leave it at `$02`, with the keypad scan running.

pads

`pads` shows everything controller 1 is doing: the stick's two readings, both buttons, the last key on the keypad, and a tone from voice 1 while the bottom button is held.

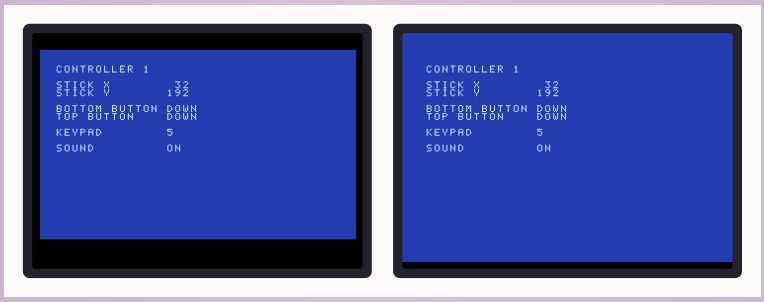


Figure 7. pads in assembly and in C, the stick pushed down and to the left, both buttons held, 5 the last key.

In assembly, the pots go on first, then every frame the stick, the buttons, the key and the voice:

6502 ASSEMBLY (ca65) · pads.s

```
        lda     #$04            ; the pots on; controller 1's keypad and top button
        sta     CONSOL
```

6502 ASSEMBLY (ca65) · pads.s

```
loop:   WAITFRAME
        lda     #4              ; the stick, as the VBI last read it
        ldx     #16
        jsr     disp_at
        lda     PADDL0
        jsr     dec3
        lda     #5
        ldx     #16
        jsr     disp_at
        lda     PADDL0+1
        jsr     dec3

        lda     #7              ; the bottom button: TRIG0 is 0 while held
        ldx     #16
        jsr     disp_at
        lda     TRIG0
        jsr     updown
        lda     #8              ; the top button: SKSTAT bit 3 clear while held
        ldx     #16
        jsr     disp_at
        lda     SKSTAT
        and     #$08
        jsr     updown
```

The key's name comes from a table of string addresses indexed by KEY itself, so the "no key" entry is entry 0:

6502 ASSEMBLY (ca65) · pads.s

```
        lda     #10             ; the last key
        ldx     #16
        jsr     disp_at
        ldx     KEY
        lda     knlo,x
        sta     FN_PTR
        lda     knhi,x
        sta     FN_PTR+1
        jsr     disp_str
```

In C the same reads are named registers, and the voice is two assignments:

C (cc65 + fujinet-lib) · pads.c

```
    for (;;) {
        mt_sync();
        mt_at(16, 4);
        dec3(PADDL0);
        mt_at(16, 5);
        dec3(PADDL1);
        mt_at(16, 7);
        mt_puts(TRIG0 ? "UP  " : "DOWN");
        mt_at(16, 8);
        mt_puts(SKSTAT & 0x08 ? "UP  " : "DOWN");
        mt_at(16, 10);
        mt_puts(names[fuji_a5200_key & 15]);
        mt_at(16, 12);
        if (TRIG0 == 0) {
            AUDF1 = TONE;
            AUDC1 = VOICE;
```

NOTE: The stick, buttons and keys in the figures were pressed by `emu/drive.lua`, which sets MAME's controller inputs on a timetable: `PRESS="1=x=20+y=C0+fire+top+k5;2=-k5"` pushes the stick to X `$20`, Y `$C0`, holds both buttons, and taps 5.

10. First Contact

The first program that talks to the FujiNet asks it one question, GET_ADAPTERCONFIG_EXTENDED, and shows three parts of the answer: the WiFi network it is on, its IP address, and its firmware version. It is the smallest complete transaction there is: no parameters, no payload, a 240-byte reply.

Build and run

1. Start the FujiNet adapter (`fujinet-pc`), listening on bus-over-IP.
2. Build the program: `listings/asm/build.sh fujitest` or `listings/c/build.sh acfg`.
3. Run it: `emu/run.sh listings/c/build/acfg.bin`. The cartridge sees the claim, serves the program as an APP, and its first transaction comes back in well under a second.
4. The SSID, IP address and firmware version appear. On the bench in this handbook the SSID is `Dummy Cafe`, fujinet-pc's stand-in for a real network, and the address is `127.0.0.1`.



Figure 8. First Contact in assembly and in C. The assembly one also shows `ACKSEQ` and `ERR`.

The reply

The reply is the adapter's configuration, both as raw bytes and as text ready to print. Everything is at an offset under 256, so one index register reaches all of it.

Offset	Size	Field
0	33	SSID, NUL-terminated
33	64	hostname
97, 101, 105, 109	4 each	IP address, gateway, netmask, DNS server: raw bytes
113, 119	6 each	MAC address, BSSID: raw bytes
125	15	firmware version
140, 156, 172, 188	16 each	IP address, gateway, netmask, DNS server: as text
204, 222	18 each	MAC address, BSSID: as text

In assembly

`fujitest.s` is the bring-up's own first contact, unchanged. It checks for the cartridge, makes the one transaction through the library, and puts the three fields on the screen straight from the reply window with `disp_rpl`, which copies as it draws.

6502 ASSEMBLY (ca65) · fujitest.s

```
have:   lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        bne     show
        jsr     fn_ack
        sta     aperr
```

6502 ASSEMBLY (ca65) · fujitest.s

```
ok:     PUTS    5, 2, tssid
        lda     #5
        ldx     #8
        jsr     disp_at
        ldx     #AC_SSID
        ldy     #30
        jsr     disp_rpl

        PUTS    7, 2, tip
        lda     #7
        ldx     #8
        jsr     disp_at
        ldx     #AC_SIP
        ldy     #16
        jsr     disp_rpl
```

It also shows `ACKSEQ` and `ERR` from the status page. Power the console off and on and run it again: `ACKSEQ` reads 01 again, because the cartridge saw the power go and started its count over with the console (Section 4).

In C

fujinet-lib fills a struct with the reply, so the C version reads like any FujiNet C program on any computer. Only the screen is the 5200's.

C (cc65 + fujinet-lib) · acfg.c

```
    if (!fuji_a5200_present()) {
        mt_at(2, 5);
        mt_puts("NO FUJINET CARTRIDGE");
    } else if (!fuji_get_adapter_config_extended(&ac)) {
        mt_at(2, 5);
        mt_puts("FAILED: E");
        hex(fn_device_error);
    } else {
        field(5, "SSID", ac.ssid, 24);
        field(7, "IP", ac.sLocalIP, 16);
        field(9, "FW", ac.fn_version, 15);
    }
```

`fuji_get_adapter_config_extended()` is a macro over `fuji_bus_call()`: no parameters, a reply the size of `AdapterConfigExtended`. `mt_text` copies each field onto the screen and stops at its NUL.

11. The Network Device

The FujiNet's network device is how a 5200 reaches the rest of the world: HTTP and HTTPS, TCP and UDP, FTP, TNFS, and more, all behind one small set of commands. There are eight of them, N1: to N8:, devices `$71` to `$78`, each a connection of its own. A program names the far end with a **devicespec** such as `N:http://example.com/file.txt` and then opens, reads, writes and closes it like a file.

The lifecycle

Command	Code	Parameters	Payload and reply
OPEN	`$4F 'O'`	mode, translation (one byte each)	Payload: the devicespec. Reply: none.
STATUS	`$53 'S'`	two one-byte zeros	Reply: bytes waiting (two bytes, low first), connected (1 byte), status (1 byte; 1 is good).
READ	`$52 'R'`	the count (two bytes)	Reply: the bytes.
WRITE	`$57 'W'`	the count (two bytes)	Payload: the bytes.
CLOSE	`$43 'C'`	none	—

The modes are the protocol's: 4 to read (an HTTP GET), 8 to write (PUT), 12 to read and write (a GET that may also set headers), 13 for an HTTP POST. Translation 0 leaves the bytes alone; 1, 2 and 3 turn the far end's line endings into CR, LF or CR LF.

Five rules make the lifecycle behave:

1. **STATUS until it settles.** The adapter answers STATUS as soon as some of the response has arrived, so one reading can be a fraction of it. Read STATUS until two readings agree and are not zero.
2. **READ at most what fits.** The reply window holds 1024 bytes, and a READ's answer must fit in it. Ask for no more than STATUS says is waiting, and no more than 1024.

3. **Wait long enough.** The cartridge gives OPEN, STATUS, READ and CLOSE 90 seconds each. An HTTP request is not even sent until the first STATUS or READ, so that is where a slow server shows.
4. **Close what you open**, every time, including on the way out of a failure. The adapter keeps a connection open until it is told otherwise, even across a power cycle of the console.
5. **Use the devicespec on every call** in C: fujinet-lib takes the unit number from it.

netget

`netget` fetches `http://127.0.0.1:8768/hello.txt`, on the bench a two-line file served by `python3 -m http.server` from `listings/fixtures`, and shows it a line at a time: LF ends a row, CR is skipped, a row stops at the edge.



Figure 9. netget in assembly and in C.

In assembly

OPEN takes its two parameters, then the devicespec, a byte at a time. The loop counts in X, because `fn_txb` sends its byte in Y:

6502 ASSEMBLY (ca65) · netget.s

```
open:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCOPEN
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NMHTTPG
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldx     #0              ; X: fn_txb takes its byte in Y
@url:   lda     url,x
        beq     @urle
        jsr     fn_txb
        inx
        bne     @url
@urle:  jsr     go_ack
```

STATUS until two readings agree, three frames apart:

6502 ASSEMBLY (ca65) · netget.s

```
stat:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCSTAT
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY+NSAVLO
        sta     navail
        lda     FN_REPLY+NSAVHI
        sta     navail+1
        cmp     nprev+1
        bne     again
        lda     navail
        cmp     nprev
        bne     again
        ora     navail+1
        bne     read
again:  lda     navail
        sta     nprev
        lda     navail+1
        sta     nprev+1
        inc     ntry
        lda     ntry
        cmp     #NTRIES
        bcc     @wait
        lda     #FNEWAIT
        sta     aperr
        jmp     fail
```

```
@wait:  ldx     #3              ; three frames between readings
@w:     lda     RTCLOKL
@f:     cmp     RTCLOKL
        beq     @f
        dex
        bne     @w
        jmp     stat
```

READ takes one two-byte parameter. The text is then walked straight out of the reply window with a pointer, since it can be longer than 256 bytes:

6502 ASSEMBLY (ca65) · netget.s

```
@len:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCREAD
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     navail
        ldx     navail+1
        jsr     fn_pw
        jsr     go_ack
```

NOTE: The library waits 12 seconds unless told otherwise. A real server can take longer than that to answer the first STATUS; set `FN_TMO` to 96 after `fn_beg` for each network command, and the cartridge's 90 seconds will run out first.

In C

fujinet-lib does the STATUS loop inside `network_read()`, which returns how many bytes it read, or a negative number on failure:

C (cc65 + fujinet-lib) · netget.c

```
    network_init();
    err = network_open(URL, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE);
    if (err)
        fail("OPEN", err);
    n = network_read(URL, buf, sizeof buf);
    if (n < 0)
        fail("READ", fn_device_error);
```

`network_init()` does nothing on the 5200, but some platforms need it, so a portable program calls it before the first `network_open()`. The library waits at least 91 seconds for each of the long network calls. The line endings are

spelled out as `0x0D` and `0x0A`, because cc65's Atari targets read `'\n'` as the Atari's own end-of-line, `$9B`.

JSON

An open connection can parse what it fetched as JSON, so a program never has to. Switch the channel to JSON mode (`$FC`, mode 1 as the second of two parameters), send PARSE (`$50 'P'`), then for each value send QUERY (`$51 'Q'`) with the path, `/0/name` say, as the payload, and STATUS and READ the value's text. In C that is `network_json_parse()` and `network_json_query()`. The command cards in Section 18 have the details. FujiNet's game servers also answer in plain binary when asked (`?bin=1`), which is what the games do.

12. netcat

`netcat` holds a conversation with a TCP server: what the server sends scrolls up the top of the screen, and what you type goes back a line at a time. It is the network device used both ways at once, and it needs something no other program in this handbook does: text typed on a console with no keyboard.

Typing without a keyboard

The keypad types digits by itself. Everything else comes from an on-screen keyboard of three rows, letters, then punctuation, with the stick moving a reverse-video cursor round it and the bottom button typing the key under the cursor. _ types a space.

Control	Does
The stick	Moves the cursor one key per push. It must come back to the middle before it moves again, so a held stick does not race across the keyboard.
The bottom button	Types the key under the cursor, once per press.
0–9	Type themselves.
*	Rubs out the last character.
#	Sends the line, with CR LF on the end.
START	Closes the connection.



Figure 10. netcat in assembly and in C, after the server's greeting and one line typed: A and B from the keyboard, 1 and 2 from the keypad, then #.

On the bench the far end is `listings/fixtures/ncserver.py`, which greets each connection with `NETCAT SAYS HI` and answers every line with `ECHO` and the line in capitals.

The loop

`netcat` opens its devicespec, `N:TCP://127.0.0.1:8769/`, read and write (mode 12), and then never waits for anything. Every frame it looks at the stick, the button and the keypad; every eighth frame it asks STATUS whether anything has arrived, and READs it if so.

6502 ASSEMBLY (ca65) · netcat.s

```
loop:   WAITFRAME
        jsr     stick
        jsr     button
        jsr     keypad
        lda     RTCLOKL         ; every eighth frame, see what has come
        and     #7
        bne     loop
        jsr     receive
        jmp     loop
```

C (cc65 + fujinet-lib) · netcat.c

```
    for (;;) {
        mt_sync();
        stick();
        button();
        k = fuji_a5200_key;             /* the last key + 1, 0 = none */
        if (k) {
            fuji_a5200_key = 0;
            --k;
            if (k < 10)
                type('0' + k);
            else if (k == 0x0A)         /* * */
                rub();
            else if (k == 0x0B)         /* # */
                send();
            else if (k == 0x0C) {       /* START */
                network_close(URL);
                mt_at(2, LROW);
                mt_puts("CLOSED");
                mt_fill(' ', 30);
                for (;;)
                    mt_sync();
            }
        }
```

A STATUS takes the cartridge about as long as any small transaction, a few milliseconds on the bench; once every eight frames leaves the stick and keypad smooth. What the server sends while nobody asks waits in the FujiNet.

Receiving

STATUS, then a READ of what is waiting, at most 128 bytes, then each byte onto the screen: CR is dropped, LF starts a new row, and a row that reaches the edge wraps. When the text reaches the bottom of its area, the rows move up one.

6502 ASSEMBLY (ca65) · netcat.s

```
.proc receive
        FNBEGIN NETDEV, NCSTAT, 2
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY+NSAVLO
        sta     navail
        lda     FN_REPLY+NSAVHI
        sta     navail+1
        ora     navail
        beq     done
        lda     navail+1        ; at most NMAX
        bne     cap
        lda     navail
```

```
        cmp     #NMAX+1
        bcc     read
cap:    lda     #NMAX
        sta     navail
        lda     #0
        sta     navail+1
read:   FNBEGIN NETDEV, NCREAD, 1
        lda     navail
        ldx     navail+1
        jsr     fn_pw
        jsr     go_ack
        lda     FN_RXLEN        ; the reply is at most NMAX: one byte
        sta     nleft
        lda     #<FN_REPLY
        sta     PTR1
        lda     #>FN_REPLY
        sta     PTR1+1
```

The C version lets the library count:

C (cc65 + fujinet-lib) · netcat.c

```
static void receive(void)
{
    uint16_t bw;
    uint8_t connected, err;
    int16_t n, i;

    if (network_status(URL, &bw, &connected, &err))
        fail("STATUS", fn_device_error);
    if (bw == 0)
        return;
    n = network_read(URL, buf, bw > NMAX ? NMAX : bw);
    if (n < 0)
        fail("READ", fn_device_error);
    for (i = 0; i < n; ++i)
        show(buf[i]);
}
```

Sending

A WRITE has one two-byte parameter, the count, and the bytes as its payload. `netcat` puts CR and LF on the end of the typed line, sends it, and clears the line.

6502 ASSEMBLY (ca65) · netcat.s

```
.proc send
        ldx     llen
        lda     #13
        sta     line,x
        lda     #10
        sta     line+1,x
        inx
        inx
        stx     ZTMP
        FNBEGIN NETDEV, NCWRITE, 1
        lda     ZTMP
        ldx     #0
        jsr     fn_pw
        ldx     #0
@b:     lda     line,x
        jsr     fn_txb
        inx
        cpx     ZTMP
        bne     @b
```

C (cc65 + fujinet-lib) · netcat.c

```
static void send(void)
{
    line[llen] = CR;
    line[llen + 1] = LF;
    if (network_write(URL, line, llen + 2))
        fail("WRITE", fn_device_error);
    llen = 0;
    mt_at(4, LROW);
    mt_fill(' ', LMAX);
}
```

NOTE: The keyboard's cursor is the cell's own character in reverse video: bit 7 set in screen memory, which `CHACTL` = 2 tells ANTIC to show inverted. In assembly that is an `EOR #$80` on the cell; in C, `mt_setpal(1, 1)` at the cell.

13. Directories, Mount and Boot

A FujiNet keeps up to eight **host slots**: places it can find files, such as its own SD card or a TNFS server on the network. This section lists a directory on one of them, and then does what CONFIG does with the file you pick: has the FujiNet push it into the cartridge, and hands the console to it.

Hosts and directories

Command	Code	Parameters, payload, reply
MOUNT_HOST	$F9	The host slot (one byte). Connects to the host.
OPEN_DIRECTORY	$F7	The host slot; then exactly 256 bytes of payload: the path, a NUL, and optionally a filter pattern after the NUL, padded with NULs.
READ_DIR_ENTRY	$F6	The longest name wanted, and flags (one byte each). Reply: the next name, or $7F,$7F when there are no more.
CLOSE_DIRECTORY	$F5	None.
SET_DIRECTORY_POSITION	$E4	A position (two bytes): where the next READ_DIR_ENTRY starts, for paging back and forth.
READ_HOST_SLOTS	$F4	None. Reply: the eight hosts' names, 32 bytes each.

CAUTION: Do not ask READ_DIR_ENTRY for names of exactly 31 bytes. At that length the firmware puts icon bytes in front of the name. Every program here asks for 36.



Figure 11. The directory of `SD:/a5200book/`, in assembly and in C. The font has no lower case, so `hello.a52` shows in capitals.

In assembly

`fn_path` streams the path and pads it to 256 bytes; READ_DIR_ENTRY's reply is drawn straight from the reply window.

6502 ASSEMBLY (ca65) · fujidir.s

```
        ; OPEN_DIRECTORY: the host slot, then exactly 256 bytes of path
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCODIR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #HOSTSL
        jsr     fn_pb
        lda     #<path
        sta     FN_PTR
        lda     #>path
        sta     FN_PTR+1
        jsr     fn_path
        jsr     go_ack

        ; READ_DIR_ENTRY: two one-byte parameters, the longest name wanted
        ; and the flags; one name a call
        lda     #ROW0
        sta     row
entry:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCRDIR
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NAMELN
        jsr     fn_pb
```

```
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY        ; $7F,$7F: the end of the directory
        cmp     #$7F
        bne     @show
        lda     FN_REPLY+1
        cmp     #$7F
        beq     close
@show:  lda     row
        ldx     #2
        jsr     disp_at
        ldx     #0
        ldy     #NAMELN
        jsr     disp_rpl
        inc     row
        lda     row
        cmp     #ROW0+NROWS
        bne     entry
```

In C

`fuji_open_directory()` sends 256 bytes from its pointer, so the path lives in a 256-byte buffer:

C (cc65 + fujinet-lib) · dir.c

```
    strcpy(path, "/a5200book/");
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_open_directory(HOSTSL, path))
        fail("OPEN_DIRECTORY");
    for (row = ROW0; row < ROW0 + ROWS; ++row) {
        if (!fuji_read_directory(NAMELN, 0, name))
            fail("READ_DIR_ENTRY");
        if (name[0] == 0x7F && name[1] == 0x7F)
            break;                          /* the end of the directory */
        mt_at(2, row);
        mt_text(name, NAMELN);
    }
```

Mounting and pushing

To run a file, a program asks the FujiNet to **mount** it on one of its device slots. On the 5200 cartridge, mounting means more: the FujiNet reads the whole file and pushes it, while the MOUNT_IMAGE request is still outstanding, into the cartridge, which keeps it until the program says go.

Command	Code	Parameters
MOUNT_HOST	$F9	The host slot.
SET_DEVICE_FULLPATH	$E2	Device slot, host slot, mode (1 for read), one byte each; then 256 bytes of path.

Command	Code	Parameters
MOUNT_IMAGE	$F8	Device slot, mode. The cartridge allows 60 seconds.

The FujiNet treats a file as a cartridge image by its extension: `.a52` or `.bin` for the 5200. If a file of the same name ending in `.cfg` sits beside it, it is pushed first, and a `mapper=` line in it tells the cartridge which board to serve the image as. While the push goes on, the status page reports it:

Address	Name	Meaning
$B406	BOOTSTATE	0 idle, 1 arriving, 2 ready, $80 failed.
$B407	BOOTPCT	0–100, for a progress bar.
$B408	BOOTERR	Why it failed: 1 too big, 2 cut short, 3 no board fits it, 4 the only room that fits it is in use.
$B416, $B419	BOOTGOT, BOOTTOT	Bytes received, and the size (24 bits each, low first). Read each twice and keep a matching pair: the bytes can change between reads.
$B41D	STAGED	What arrived: bit 0 ready, bit 1 it carries the FujiNet claim, bit 2 it is CONFIG.

The boards

A 5200 cartridge is one of four boards, and the cartridge serves each image the way its own board would. It decides which from, in order: a database of known images by checksum; a `.cfg` file's `mapper=` line; the claim's kind byte (Section 3); and the size.

Board	Size	How it is laid out
ROM	up to 32K	Straight in the window, repeated to fill it. 16K images are told apart from two-chip ones by tracing their code.
Two chips	16K	Each 8K appears twice.
Bounty Bob Strikes Back	40K	Two 4K windows that switch on reads near their top, and a fixed 8K.
Super Cart	64K or 128K	32K banks, switched by reads of $BFC0–$BFFF (Section 15).

Only a ROM image or a Super Cart can be a FujiNet APP: the other two boards have nowhere the mailbox could live. Anything bigger than 128K is refused.

Handing over

The push can finish before MOUNT_IMAGE's answer comes back, or after it. Wait for BOOTSTATE to read 2 (or `$80`, and then BOOTERR says why), then hand the console over:

1. Write `$B5` to BOOTLOCK (register `$11`), and wait for ARMED at `$B41C` to read 1.
2. Quiet the machine as a power-on leaves it: interrupts off, `NMIEN`, `DMACTL`, `GRACTL` and `IRQEN` to 0, wait for the last line of the frame, every chip register and all of RAM to 0.
3. `JMP $B5F0`. The cartridge serves `JMP ($FFFC)` there; on the read of its last byte it swaps the new image in, and the jump lands in the BIOS's reset code, which starts the new image as if the console had just been switched on.

`fn_boot` in assembly and `fuji_a5200_boot()` in C do all three. Here is the middle of it:

6502 ASSEMBLY (ca65) · fujilib.s

```
armed:  sei
        lda     #0
        sta     NMIEN           ; first: a pending NMI would run through $0202
        sta     NMIRES
        sta     DMACTL          ; no display-list fetch may reach the new image
        sta     GRACTL
        sta     IRQEN
vbl:    lda     VCOUNT          ; let the frame's DMA finish
        cmp     #124
        bcc     vbl
        lda     #0
        tax
chips:  sta     POKEY,x
        sta     GTIA,x
        sta     ANTIC,x
        inx
        bne     chips
```

If the new image carries the claim, it finds the mailbox waiting, with the count started over. If not, it is a GAME: the mailbox is gone, and only a power cycle brings CONFIG back.

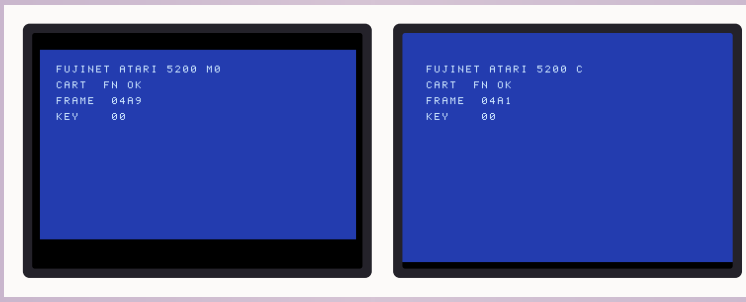


Figure 12. After `fujiboot` and `boot.c`: the pushed programs, the two `hello`s, running. Each found the mailbox still there.

In assembly

`fujiboot.s` is the bring-up's, unchanged; the handbook's build points it at `/a5200book/hello.a52`.

6502 ASSEMBLY (ca65) · fujiboot.s

```
; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ----
; Three one-byte parameters and then a FULL 256-byte NUL-padded path.
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCSDFP
        sta     FN_CMD
        lda     #3
        sta     FN_NPR
        jsr     fn_beg
        lda     #DEVSLOT
        jsr     fn_pb
        lda     #BOOTHST
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     fn_path
```

6502 ASSEMBLY (ca65) · fujiboot.s

```
; ---- wait for the staged image ----
        lda     #4
        sta     apstep
        lda     #10
        jsr     fn_frames
wait:   lda     FN_BOOTSTATE
        cmp     #FN_BOOT_READY
        beq     boot
        cmp     #FN_BOOT_FAILED
        beq     bfail
        jsr     fn_wait
        bcs     wait
        lda     #FNEWAIT
        sta     aperr
        jmp     show
bfail:  lda     FN_BOOTERR
        sta     aperr
        lda     #5
        sta     apstep
        jmp     show

; ---- boot it: returns only if the cart never armed the swap ----
boot:   jsr     fn_boot
        sta     aperr
        lda     #6
        sta     apstep
```

In C

C (cc65 + fujinet-lib) · boot.c

```
    strcpy(path, BOOTPATH);
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_set_device_filename(FMREAD, HOSTSL, DEVSL, path))
        fail("SET_DEVICE_FULLPATH");
    if (!fuji_mount_disk_image(DEVSL, FMREAD))
        fail("MOUNT_IMAGE");

    /* the push may finish before or after MOUNT_IMAGE returns */
    mt_at(2, 4);
    for (;;) {
        state = fuji_a5200_boot_state();
        pct = fuji_a5200_boot_percent();
        while (shown < pct / 4) {       /* 25 cells for 100 per cent */
            mt_put('#');
            ++shown;
```

Back to CONFIG

A FujiNet APP can go back to CONFIG without a power cycle: write `$C5` to register `$14`, and the cartridge stages CONFIG and arms the swap; then hand over exactly as above. `fn_config` and `fuji_a5200_config()` do it, and the games call it when you quit.

6502 ASSEMBLY (ca65)

```
        jsr     fn_config       ; does not come back...
        jmp     failed          ; ...unless the swap was never armed
```

C (cc65 + fujinet-lib)

```
fuji_a5200_config();            /* does not come back... */
failed();                       /* ...unless the swap was never armed */
```

14. App Keys, Clock and Utilities

A FujiNet can remember things for a program, tell it the time, and do the arithmetic a 6502 should not have to. This section covers the three a program reaches for first: appkeys, the clock, and the utilities.

Appkeys

An **appkey** is a small file on the FujiNet's SD card, named by three numbers: a **creator** (two bytes), an **application** and a **key** (one byte each). A program reads and writes its own keys to remember settings, names and scores between sessions; every FujiNet game reads the same key, creator 1, app 1, key 0, for the player's name. On a console whose RAM forgets everything at every hand-over and every power cycle (Section 6), appkeys are the only memory a 5200 program has.

Command	Code	Parameters, payload, reply
OPEN_APPKEY	`$DC`	No parameters. Payload: six bytes: creator (low, high), app, key, mode (0 read, 1 write), and a reserved 0 that must be sent.
READ_APPKEY	`$DD`	None. Reply: the length (two bytes, low first), then the value.
WRITE_APPKEY	`$DE`	The length (one byte); then the value as the payload.
CLOSE_APPKEY	`$DB`	None.

A key holds up to 64 bytes. Register a creator id of your own on the FujiNet wiki's AppKey Registry before you ship a program that writes keys. This handbook's programs use creator `$5200`, app `$FF`, as scratch.



Figure 13. The shared player name, then a key of our own written and read back.

In assembly

6502 ASSEMBLY (ca65) · appkey.s

```
; akopen -- OPEN_APPKEY for the key in akcre/akapp/akkey, mode akmode: no
; parameters, the six-byte struct as the payload.
.proc akopen
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKOP
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        lda     akcre
        jsr     fn_txb
        lda     akcre+1
        jsr     fn_txb
        lda     akapp
        jsr     fn_txb
        lda     akkey
        jsr     fn_txb
        lda     akmode
        jsr     fn_txb
        lda     #0              ; reserved, and not optional
        jsr     fn_txb
        jmp     go_ack
.endproc
```

The write sends its value with `fn_txb`, counted in X:

6502 ASSEMBLY (ca65) · appkey.s

```
; akwrite -- WRITE_APPKEY: one parameter, the length, then the value.
.proc akwrite
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKWR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #valend-val
        jsr     fn_pb
        ldx     #0              ; X: fn_txb takes its byte in Y
@w:     lda     val,x
        jsr     fn_txb
        inx
        cpx     #valend-val
        bne     @w
        jmp     go_ack
.endproc
```

In C

fujinet-lib keeps the creator and app in `fuji_set_appkey_details()`, and its read and write each do the OPEN themselves:

C (cc65 + fujinet-lib) · appkey.c

```
static void show(uint8_t row)
{
    uint16_t n;

    mt_at(4, row);
    if (!fuji_read_appkey(0, &n, value))
        mt_puts("(READ FAILED)");
    else if (n == 0)
        mt_puts("(EMPTY)");
    else
        mt_text((char *)value, n);
}
```

C (cc65 + fujinet-lib) · appkey.c

```
    fuji_set_appkey_details(0x5200, 0xFF, DEFAULT);
    if (!fuji_write_appkey(0, sizeof ours - 1, (uint8_t *)ours)) {
        mt_at(4, 10);
        mt_puts("(WRITE FAILED)");
    } else
        show(10);
```

The read's buffer must be two bytes bigger than the key: the library reads the length into it too.

The clock

The clock is a device of its own, `$45`. Its commands take no parameters and answer with the time in one of several forms:

Command	Code	Reply
GET_ISO_LOCAL	`$49 'I'`	`2026-10-30T18:42:08-0400` and a NUL, in the adapter's time zone.
GET_ISO_UTC	`$5A 'Z'`	The same in UTC.
GETTIME	`$93`	Six bytes: day, month, year (two digits), hour, minute, second.
(simple time)	`$54 'T'`	Seven bytes: century, year, month, day, hour, minute, second.
SETTZ_ALT	`$74 't'`	Payload: a POSIX time-zone string, such as `EST5EDT`: the zone the local reads use.



Figure 14. The clock, local and UTC. The bench's fujinet-pc keeps `EST5EDT`.

6502 ASSEMBLY (ca65) · clock.s

```
.proc main
        jsr     disp_init
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     @go
        lda     #FNENOC
        sta     aperr
        jmp     fail
@go:    PUTS    5, 2, tlocal
        ASK     CKISOL, 5
        PUTS    7, 2, tutc
        ASK     CKISOU, 7
loop:   jmp     loop
```

C (cc65 + fujinet-lib) · clock.c

```
static void line(uint8_t row, const char *label, TimeFormat f)
{
    mt_at(2, row);
    mt_puts(label);
    mt_at(10, row);
    if (clock_get_time(when, f))
        mt_puts("FAILED");
    else
        mt_text((char *)when, 25);
}
```

Entropy, hashing, encoding

The FujiNet does the work a 6502 would take seconds over:

- **GENERATE_GUID** (`$BB`) answers with a fresh 36-character GUID.
- **HASH_INPUT** (`$C8`), **HASH_COMPUTE** (`$C7`), **HASH_LENGTH** (`$C6`) and **HASH_OUTPUT** (`$C5`) hash data: feed it in pieces, compute with an algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512), ask the length, read the result. In C, `fuji_hash_data()` does all four.
- The **Base64** commands (`$D0`–`$C9`, encode and decode) work the same way.
- The **QR code** commands (`$BC`–`$BF`) turn text into a QR code's modules.

The cards in Section 17 show each one in both languages.

NOTE: The 5200 has no random-number problem to solve: POKEY's `RANDOM` register at `$E80A` reads a fresh byte from a 17-bit noise generator at any moment.

15. Banks and the Super Cart

A program that outgrows 32K can be a **Super Cart**: 64K or 128K, seen through the cartridge window 32K at a time. The cartridge serves a FujiNet app on a Super Cart with the mailbox over every bank, so a program can make transactions from any of them.

Switching

The Super Cart switches the whole window at once, on a read of one of its top 64 bytes:

Read	Selects
`$BFE0`–`$BFFF`	The last bank: bank 1 of a 64K cart, bank 3 of a 128K one. This is also the bank it powers on in.
`$BFD0`, `$BFD4`, `$BFD8`, `$BFDC`	Bank 0, 1, 2 or 3: address bits 2 and 3 are the bank number.
`$BFC0`–`$BFCF`	The high bits of the bank number, for carts bigger than 128K. Nothing, on the cartridge.

Use `BIT` to make the read: it leaves A and X alone, so a far call keeps its arguments and its result. Two consequences of switching by reading are easy to miss:

- **The claim is in the switch.** `$BFE0` is both the start of the FujiNet claim and a hotspot, so anything that reads the claim selects the last bank.
- **ANTIC reads the window too.** A display list or a font in banked ROM is read from whichever bank is selected when ANTIC gets to it. Keep them where every bank has them.

Laying out the banks

Because the whole window switches, code that runs during a switch must be at the same address in every bank, with the same bytes. Both of this handbook's banked programs do it the same simple way: every bank is a copy of the last one, except for 4K at `$A000`–`$AFFF` that each bank has to itself. The common part holds the start-up code, the libraries, the interrupt handlers, the display list and the font, and the code that switches; the window at `$A000` holds what each bank adds.

	Bank 0	Bank 1 (last)
`$4000`–`$9FFF`	common	common
`$A000`–`$AFFF`	bank 0's own	bank 1's own
`$B000`–`$B7FF`	the mailbox	the mailbox
`$B800`–`$BFFF`	common, claim, header	common, claim, header

The build links bank 1 as an ordinary 32K program, links bank 0's 4K separately against bank 1's addresses, makes bank 0 a copy of bank 1 with its own 4K laid in, and writes bank 0 first. The checker accepts 64K and 128K images.

A far call

`far0` switches to bank 0, calls its entry at `$A000`, and switches back. It is in the common part, so the instruction after each `BIT` is fetched from the bank just selected, and is the same there:

6502 ASSEMBLY (ca65) · bank.s

```
; far0 -- call bank 0's entry and come back to bank 1. Common code: the
; same bytes are at this address in both banks.
.proc far0
        bit     $BFD0           ; bank 0 from the next fetch on
        jsr     BANK0_ENTRY
        bit     $BFE0           ; the last bank again; A is bank 0's answer
        rts
.endproc
```

In C the switch is in assembly too, in `supercart.s`, because cc65's optimiser drops a read whose value is unused, even an inline `lda`. Bank 0's code is ordinary C, put in its own segment with `#pragma code-name`:

6502 ASSEMBLY (ca65) · supercart.s

```
.proc _far_bank0
        bit     $BFD0           ; bank 0 from the next fetch on
        jsr     _bank0_main
        bit     $BFE0           ; the last bank again
        rts
```

C (cc65 + fujinet-lib) · bank0.c

```
#pragma code-name (push, "BANK0")
#pragma rodata-name (push, "BANK0")

static AdapterConfigExtended ac;

uint8_t bank0_main(void)
{
    mt_at(2, 6);
    mt_puts("BANK ");
    mt_put(*(const char *) 0xA000);
    if (!fuji_get_adapter_config_extended(&ac))
        return 0;
    mt_at(12, 6);
    mt_text(ac.ssid, 20);
    return 1;
}
```

`bank.cfg`, the C program's linker configuration, is MekkoGX's with `$A000`–`$AFFF` cut out of the ROM twice: once for bank 1, in the image, and once for bank 0, in a file of its own that the build lays over the copy.

bank

`bank` shows a byte that differs between the banks, makes a transaction from bank 1, calls bank 0 (which shows its own byte and makes a transaction of its own), and shows bank 1's byte again on the way back.



Figure 15. bank in assembly and in C: `1`, then bank 0's `0` and the SSID it fetched, then `1` again.

6502 ASSEMBLY (ca65) · bank0.s

```
        jmp     entry           ; $A000: far0 calls here
marker: .byte "0"               ; $A003: bank 1 has '1' here

; Returns A = FNE*.
entry:  lda     #6
        ldx     #2
        jsr     disp_at
        lda     #<tb0
        sta     FN_PTR
        lda     #>tb0
        sta     FN_PTR+1
        jsr     disp_str
        lda     marker
        jsr     disp_putc
        jsr     adapter         ; a transaction from banked code
        bne     done
        lda     #6
        ldx     #12
        jsr     disp_at
        ldx     #0              ; the SSID
        ldy     #20
        jsr     disp_rpl
        lda     #FNEOK
done:   rts
```

NOTE: `bank.s` is the bring-up's `fujibank.s` without its last step, which boots a game. `bank0.s` is the bring-up's, unchanged.

16. Display Lists, Fonts and Players

The text screen of Section 5 is the simplest thing ANTIC can do. A game wants more: characters of its own design in four colours, a few rows of bigger letters, a cursor that moves smoothly over the board. All of it is the display list, the font and GTIA's players, and all of it is the same as on the Atari 400 and 800, at the 5200's addresses.

The display list

A display list is a string of one-byte instructions, some followed by an address. ANTIC reads it with DMA, one instruction per **mode line**, every frame.

Byte	Instruction
`$00`–`$70`	Blank lines: `$00` is one, `$10` two, up to `$70`, eight.
`$01`	Jump: the next two bytes are where the list continues. Used to cross a 1K boundary, which a display list may not otherwise do.
`$41`	Jump and wait for the vertical blank: the end of the list, pointing back at its start.
`$02`–`$0F`	A mode line of that mode (below).
1. `$40`	Load the memory counter: the next two bytes are the address of this line's data. The first mode line needs it; lines after it carry on from where the last one stopped.
1. `$80`	Display-list interrupt at the end of this line (below).
1. `$20`, + `$10`	Fine scrolling, vertical and horizontal.

Mode	Bytes a line	Scan lines	What it is
2	40	8	Text: 40 characters, two shades of one colour. The handbook's screen.
3	40	10	Text with descenders.
4	40	8	Text in four colours: each character is 4 × 8 coloured pixels. Battleship's screen.
5	40	16	Mode 4, twice as tall.
6	20	8	Big text: 20 characters in five colours. cc65's console.
7	20	16	Mode 6, twice as tall.

Mode	Bytes a line	Scan lines	What it is
8–$F	10–40	1–8	Bitmaps, from 4 colours at 40 × 24 down to one colour at 320 × 192.

A display list must not cross a 1K boundary, and a mode line's data must not cross a 4K boundary, without a jump or a fresh load. And none of it may be in the mailbox at $B000–$B7FF (Section 3).

Fonts

CHBASE ($D409) is the page number of the font, which must start on a 1K boundary for modes 2 and 3 and a 512-byte one for modes 6 and 7. A font in ROM is the cheapest, and is what the handbook's programs use. A game that redefines its characters while it runs copies its font into RAM first: Battleship copies its own to $3000 and draws its ships by changing characters.

Players and missiles

GTIA has four **players**, each a strip 8 pixels wide and the height of the screen, and four **missiles**, 2 pixels wide, all in front of (or behind) the playfield and positioned to the pixel. ANTIC reads their shapes from **player/missile memory** by DMA, one byte per line for each:

Register	Where	What
PMBASE	$D407	The page number of player/missile memory: 2K-aligned for single-line players. Missiles at +$300, player 0 at +$400, then every $100.
SDMCTL	$07	$3E: the normal playfield and display list, players and missiles, single-line.
GRACTL	$C01D	3: GTIA shows players and missiles.
HPOSP0–3	$C000–$C003	Each player's horizontal position. 48 is the left edge of the normal playfield.
PCOLR0–3	$08–$0B	Their colours (shadows).

6502 ASSEMBLY (ca65)

```
        lda     #$38            ; player/missile memory at $3800
        sta     PMBASE
        lda     #$3E            ; playfield, players, missiles, single-line
        sta     SDMCTL
        lda     #3              ; GTIA: show them
        sta     GRACTL
        lda     #120            ; player 0 across the screen...
        sta     GTIA            ; ...HPOSP0
        lda     #$46            ; ...in red
        sta     PCOLR0
```

C (cc65 + fujinet-lib)

```
ANTIC.pmbase = 0x38;            /* player/missile memory at $3800 */
OS.sdmctl = 0x3E;               /* playfield, players, missiles, single-line */
GTIA_WRITE.gractl = 3;          /* GTIA: show them */
GTIA_WRITE.hposp0 = 120;        /* player 0 across the screen... */
OS.pcolr0 = 0x46;               /* ...in red */
```

Then draw the player's shape into its strip, at the lines where it should appear, and move it by changing its horizontal position and where in the strip the shape is.

Display-list interrupts

A display list instruction with bit 7 set raises an NMI at the end of its line. The BIOS's NMI handler sends it through VDSLST ($0206); enable it with $C0 in NMIEN ($D40E), the VBI and display-list interrupts together. A handler that changes a colour or CHBASE for the next rows stores to WSYNC ($D40A) first, which holds the 6502 until the end of the scan line, so the change lands in the margin.

CAUTION: MAME's ANTIC reads CHBASE once per mode line, when it fetches the line's instruction, not on every scan line. A handler that does work before its WSYNC can miss that moment on a busy line, and the next row comes out in the old font, in MAME only. Have every value ready in zero page before the interrupt, and store it immediately after WSYNC.

Colours

A colour is a byte: the top four bits are one of 16 hues, the next three one of 8 luminances. $84 is the handbook's blue, $0E its white, $46 a red. Set the shadows (PCOLR0–3, COLOR0–4), and the VBI copies them to GTIA every frame. After several minutes without a key press, the VBI's attract mode cycles them to save the television's screen; a key press stops it.

17. Command Reference: The Fuji Device

Every command device `$70` answers, checked against the firmware the cartridge's ESP32-S3 runs: the RS-232 build's `rs232Fuji.cpp` and the four mixins every FujiNet shares (appkeys, Base64, hashing, QR codes). A command not on a card is answered with a NAK; the last section names them.

On the cards, *nparam* is the value for the NPARAM register, and each parameter goes into the TX stream as its size and then its value: a *byte* is 1 and the byte, a *word* 2 and two bytes, low first. The reply is in the reply window at `$B000`. The assembly examples use the library of Section 7 and its `FNBEGIN` macro (`devdefs.inc`), which sets `FN_DEV`, `FN_CMD` and `FN_NPR` and calls `fn_beg`. The C examples use fujinet-lib where it has a call, and its bus layer where it does not. Every example is built by `tools/snipcheck.py`.

WiFi and the adapter

GET_ADAPTERCONFIG_EXTENDED / GET_ADAPTERCONFIG — $C4 / $E8

DEVICE	$70	**NPARAM**	0
PARAMS	none		
PAYLOAD	none		
REPLY	`$C4`: 240 bytes, laid out in Section 10. `$E8`: the first 140, addresses in binary only		

The first-contact command, and the one every status screen wants. It works before the WiFi is up, and its text fields save writing a routine to print an IP address.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCADPX, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+140    ; the IP address, as text
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_adapter_config_extended(&ac))
    failed();
mt_text(ac.sLocalIP, 16);
```

SCAN_NETWORKS $FD

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: how many networks were found		

Starts a fresh scan and answers when it is done, which takes seconds; well inside the cartridge's five. Follow it with GET_SCAN_RESULT once for each network.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSCAN, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY        ; how many
        sta     cnt
```

C (cc65 + fujinet-lib)

```
if (!fuji_scan_for_networks(&count))
    failed();
```

GET_SCAN_RESULT $FC

DEVICE	$70	NPARAM	1
PARAMS	byte: which result, from 0		
PAYLOAD	none		
REPLY	34 bytes: the SSID (33, NUL-terminated), then the signal strength, a signed dBm		

The strength is negative: closer to zero is stronger.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSCNR, 1
        lda     idx             ; which network
        jsr     fn_pb
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+33     ; signal strength, negative dBm
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_scan_result(idx, &si))
    failed();
mt_text(si.ssid, 24);           /* si.rssi: negative dBm */
```

SET_SSID $FB

DEVICE	$70	NPARAM	1
PARAMS	byte: any value; **required but ignored**		
PAYLOAD	**exactly 97 bytes**: the SSID (33) then the password (64), each NUL-padded		
REPLY	ACK		

Joins a network and saves it as the default. Two sharp edges: the parameter must be sent although nothing reads it, and the payload must be all 97 bytes; a short one fails the transaction. The BASIC example joins `Dummy Cafe` with no password: ten bytes of name, then 87 NULs.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSSSD, 1
        lda     #0
        jsr     fn_pb           ; the parameter that must be there
        ldx     #0
@pw:    lda     ssidpw,x        ; ssid[33] + password[64]
        jsr     fn_txb
        inx
        cpx     #97
        bne     @pw
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
strcpy(nc.ssid, ac.ssid);       /* nc: a NetConfig, 33 + 64 */
nc.password[0] = 0;
if (!fuji_set_ssid(&nc))
    failed();
```

GET_SSID $FE

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	97 bytes: the SSID (33) then the password (64)		

The network the adapter is set to join, password in clear text.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCGSSD, 0
        jsr     fn_go
        bne     failed          ; SSID at FN_REPLY, password at +33
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_ssid(&nc))
    failed();
```

GET_WIFISTATUS / GET_WIFI_ENABLED $FA / $EA

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	$FA: 1 byte, 3 when connected; $EA: 1 byte, 1 if the radio is on		

What to poll while a join is in progress.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCWIFI, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY
        cmp     #3              ; connected?
        beq     done
```

C (cc65 + fujinet-lib)

```
if (fuji_get_wifi_status(&err) && err == 3)
    linked();
```

RESET $FF

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK, then the adapter restarts		

Restarts the ESP32-S3. The cartridge, its mailbox and your program carry on; the link bit at `$B401` drops and comes back, and the cartridge waits up to three seconds for it before the next transaction.

STATUS / DEVICE_READY $53 / $00

DEVICE	$70	NPARAM	1 / 0
PARAMS	STATUS: byte, the kind of status wanted (1: mount times)		
PAYLOAD	none		
REPLY	STATUS 1: four bytes per device slot, a time, 0 if nothing is mounted; DEVICE_READY: the bus test's answer		

STATUS needs its parameter on this build. DEVICE_READY is a round trip with no other effect: it proves the mailbox, the cartridge, the USB link and the adapter at once.

Hosts, slots and mounting

READ_HOST_SLOTS $F4

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	256 bytes: eight host names of 32 bytes		

Slot 0 is the SD card on a standard `fujinet-pc`.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCRHST, 0
        jsr     fn_go
        bne     failed
        lda     #4              ; host 0's name on row 4
        ldx     #2
        jsr     disp_at
        ldx     #0              ; FN_REPLY + slot * 32
        ldy     #32
        jsr     disp_rpl
```

C (cc65 + fujinet-lib)

```
if (!fuji_get_host_slots(hosts, 8))
    failed();
mt_text((char *)hosts[0], 32);
```

WRITE_HOST_SLOTS $F3

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	256 bytes: all eight slots, 32 bytes each		
REPLY	ACK		

All or nothing: there is no command to write one slot. Read the table with READ_HOST_SLOTS and stream it back with the one slot changed; the reply window holds still until the next launch, so the stream can be copied straight out of it.

MOUNT_HOST $F9

DEVICE	$70	NPARAM	1
PARAMS	byte: the host slot, from 0		
PAYLOAD	none		
REPLY	ACK; NAK if the host cannot be reached		

Connects the host: SD, TNFS, SMB and the rest. Needed before OPEN_DIRECTORY or a mount. UNMOUNT_HOST (`$E6`) is not answered on this build.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCMHST, 1
        lda     #0              ; host slot 0
        jsr     fn_pb
        jsr     fn_go
        bne     failed
        jsr     fn_ack
        bne     refused
```

C (cc65 + fujinet-lib)

```
if (!fuji_mount_host_slot(0))
    failed();
```

READ_DEVICE_SLOTS / WRITE_DEVICE_SLOTS $F2 / $F1

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	$F2: the device-slot table: host slot, mode and file name for each slot; $F1 takes the same table as its payload		

What is set up in each of the FujiNet's virtual drives. A 5200 boots images rather than reading drives, so CONFIG does not show them, but they answer.

SET_DEVICE_FULLPATH $E2

DEVICE	$70	NPARAM	3
PARAMS	bytes: device slot, host slot, mode (1 to read)		
PAYLOAD	**256 bytes**: the path, NUL-padded		
REPLY	ACK		

Names the file for a device slot: the first half of booting. fujinet-lib's call takes the mode first, the device slot last; the wire has the device slot first.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSDFP, 3
        lda     #0              ; device slot
        jsr     fn_pb
        lda     #0              ; host slot
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<file
        sta     FN_PTR
        lda     #>file
        sta     FN_PTR+1
        jsr     fn_path         ; padded to 256
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
strcpy(path, "/a5200book/hello.a52");
if (!fuji_set_device_filename(1, 0, 0, path))  /* mode, host, device */
    failed();
```

MOUNT_IMAGE / UNMOUNT_IMAGE — $F8 / $E9

DEVICE	$70	**NPARAM**	2 / 1
PARAMS	$F8: device slot, mode (1 to read); $E9: device slot		
PAYLOAD	none		
REPLY	ACK when the mount completes		

The second half of booting. While it is outstanding the FujiNet pushes the file into the cartridge, and BOOT_STATE and BOOT_PCT report the progress; the cartridge allows sixty seconds. Section 13 has the rest.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCMIMG, 2
        lda     #0              ; device slot
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #66             ; seconds: past the cartridge's 60
        sta     FN_TMO
        jsr     fn_go
        bne     failed
@push:  lda     FN_BOOTSTATE    ; then watch the push
        cmp     #FN_BOOT_READY
        bne     @push
```

C (cc65 + fujinet-lib)

```
if (!fuji_mount_disk_image(0, 1))
    failed();
while (fuji_a5200_boot_state() != FUJI_A5200_BOOT_READY)
    mt_sync();
```

MOUNT_ALL / SET_BOOT_MODE / CONFIG_BOOT — $D7 / $D6 / $D9

DEVICE	$70	NPARAM	0 / 1 / 1
PARAMS	SET_BOOT_MODE: byte, the mode; CONFIG_BOOT: byte, on or off		
PAYLOAD	none		
REPLY	ACK		

MOUNT_ALL mounts every slot that has a file; the other two choose what the adapter serves at power-on on computers that boot from its drives. On the 5200 the cartridge decides what boots, and CONFIG does not send them.

Directories

OPEN_DIRECTORY — $F7

DEVICE	$70	NPARAM	1
PARAMS	byte: the host slot		
PAYLOAD	**256 bytes**: the path, a NUL, then an optional filter such as `*.A78`, the rest NULs		
REPLY	ACK		

Opens a directory on a mounted host. The adapter applies the filter, so a program never sees what it does not want. `fuji_open_directory2()` builds the path-and-filter payload for C.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCODIR, 1
        lda     #0              ; host slot
        jsr     fn_pb
        lda     #<path
        sta     FN_PTR
        lda     #>path
        sta     FN_PTR+1
        jsr     fn_path
        jsr     fn_go
        bne     failed
```

C (cc65 + fujinet-lib)

```
strcpy(path, "/a5200book/");
if (!fuji_open_directory(0, path))
    failed();
```

READ_DIR_ENTRY $F6

DEVICE	$70	NPARAM	2
PARAMS	bytes: the longest name wanted (**never 31**), flags (0)		
PAYLOAD	none		
REPLY	one name, NUL-terminated, a `/` at the end of a directory's; `$7F`, `$7F` **when there are no more**		

One name a call; the adapter moves on by itself. At a length of exactly 31 the firmware puts icon bytes in front of the name. Stop at the end marker: an SD host keeps answering `..` past it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCRDIR, 2
        lda     #36
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     fn_go
        bne     failed
        lda     FN_REPLY
        cmp     #$7F            ; the end?
        beq     done
```

C (cc65 + fujinet-lib)

```
if (!fuji_read_directory(36, 0, buf))
    failed();
if (buf[0] == 0x7F && buf[1] == 0x7F)
    linked();                   /* the end */
```

SET_DIRECTORY_POSITION / GET_DIRECTORY_POSITION — $E4 / $E5

DEVICE	$70	NPARAM	1 / 0
PARAMS	$E4: **one word**, the entry number		
PAYLOAD	none		
REPLY	$E5: a word, the current entry number; $E4: ACK		

Seek and tell for the open directory: paging in one command. The position is one two-byte parameter, not two one-byte ones.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCSDPS, 1
        lda     #20             ; entry 20: the second page
        ldx     #0
        jsr     fn_pw
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (!fuji_set_directory_position(20))
    failed();
```

CLOSE_DIRECTORY — $F5

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Close what you open: the adapter has room for few open directories.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCCDIR, 0
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
fuji_close_directory();
```

Files, prefixes and paths

COPY_FILE — $D8

DEVICE	$70	NPARAM	2
PARAMS	bytes: source host, destination host, **counting from 1**		
PAYLOAD	`source|destination`; at its own length, or NUL-padded		
REPLY	ACK when the copy is done; the cartridge allows sixty seconds		

Host to host, without a byte passing through the console. Its hosts count from 1, unlike every other slot number; 0 is refused. When the destination ends in `/`, the adapter adds the source's file name. fujinet-lib pads the payload to 256 bytes and CONFIG sends it at its own length; the firmware takes either.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCCOPY, 2
        lda     #1              ; host slot 0, counted from 1
        jsr     fn_pb
        lda     #1
        jsr     fn_pb
        ldx     #0              ; X: fn_txb takes its byte in Y
@spec:  lda     spec,x          ; source|destination
        beq     @go
        jsr     fn_txb
        inx
        bne     @spec
@go:    lda     #66             ; seconds: past the cartridge's 60
        sta     FN_TMO
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
strcpy(path, "/a5200book/readme.txt|/a5200book/copy.txt");
if (!fuji_copy_file(1, 1, path))  /* hosts from 1 */
    failed();
```

SET_HOST_PREFIX / GET_HOST_PREFIX — $E1 / $E0

DEVICE	$70	NPARAM	1
PARAMS	byte: the host slot; SET carries the prefix as its payload		
PAYLOAD	none		
REPLY	GET: the prefix		

A working directory per host, put in front of relative paths.

GET_DEVICE_FULLPATH $DA

DEVICE	$70	NPARAM	1
PARAMS	byte: the device slot		
PAYLOAD	none		
REPLY	the path SET_DEVICE_FULLPATH stored		

In C: `fuji_get_device_filename()`.

Appkeys

OPEN_APPKEY $DC

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	6 bytes: creator (word), app, key, mode (0 read, 1 write), and a reserved 0		
REPLY	ACK; NAK with no SD card, a creator of 0 or a bad mode		

Chooses the key the next read or write uses. The payload is one packed structure; leave off the reserved byte and the adapter waits for it until the transaction times out. On the SD card the key is the file `/FujiNet/CCCCAAKK.key`; it holds 64 bytes at most.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCAKOP, 0
        lda     #$00            ; creator $5200, low first
        jsr     fn_txb
        lda     #$52
        jsr     fn_txb
        lda     #$FF            ; app
        jsr     fn_txb
        lda     #0              ; key
        jsr     fn_txb
        lda     #AKMRD          ; mode
        jsr     fn_txb
        lda     #0              ; reserved, and required
        jsr     fn_txb
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
fuji_set_appkey_details(0x5200, 0xFF, DEFAULT);
/* fuji_read_appkey() and fuji_write_appkey() each open the key */
```

READ_APPKEY / WRITE_APPKEY / CLOSE_APPKEY — $DD / $DE / $DB

DEVICE	$70	NPARAM	0 / 1 / 0
PARAMS	WRITE: byte, the length		
PAYLOAD	WRITE: the value, at most 64 bytes		
REPLY	READ: a word, the length, then the value; the others: ACK		

A write ends the selection, so each write needs its own OPEN in write mode. Section 14 has both directions in both languages.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCAKRD, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY        ; the length; the value from FN_REPLY+2
        beq     done            ; empty
```

C (cc65 + fujinet-lib)

```
if (!fuji_read_appkey(0, &n, value))  /* value: the key + 2 */
    failed();
```

Utilities

GENERATE_GUID — $BB

DEVICE	$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	36 characters and a NUL		

A fresh, printable, unique id. RANDOM_NUMBER (`$D3`) is not answered on this build; the adapter's randomness reaches a program through a GUID or a hash.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCGUID, 0
        jsr     fn_go
        bne     failed
        lda     #6
        ldx     #2
        jsr     disp_at
        ldx     #0
        ldy     #36
        jsr     disp_rpl
```

C (cc65 + fujinet-lib)

```
if (!fuji_generate_guid(guid))
    failed();
```

HASH: INPUT, COMPUTE, LENGTH, OUTPUT

$C8 / $C7 / $C6 / $C5

DEVICE	$70	**NPARAM**	1
PARAMS	INPUT: word, the byte count; COMPUTE: byte, the algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512); LENGTH and OUTPUT: byte, 1 for hex text, 0 for binary		
PAYLOAD	INPUT: the bytes		
REPLY	LENGTH: the size; OUTPUT: the hash		

Four steps on the adapter: feed bytes with INPUT, as often as needed; COMPUTE; ask the LENGTH; read the OUTPUT.
HASH_COMPUTE_NO_CLEAR (`$C3`) keeps the input for another algorithm, and HASH_CLEAR (`$C2`) empties it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN FNDEVF, FNCHSHI, 1   ; feed it
        lda     #dataend-data
        ldx     #0
        jsr     fn_pw
        ldx     #0              ; X: fn_txb takes its byte in Y
@in:    lda     data,x
        jsr     fn_txb
        inx
        cpx     #dataend-data
        bne     @in
        jsr     fn_go
        FNBEGIN FNDEVF, FNCHSHC, 1   ; SHA-256
        lda     #HASHA256
        jsr     fn_pb
        jsr     fn_go
        FNBEGIN FNDEVF, FNCHSHO, 1   ; read it, as hex
        lda     #1
        jsr     fn_pb
        jsr     fn_go           ; 64 characters at FN_REPLY
```

C (cc65 + fujinet-lib)

```
if (!fuji_hash_data(SHA256, (uint8_t *)data, 5, true, buf))
    failed();                    /* 64 hex characters */
```

BASE64: ENCODE and DECODE

$D0--$CD / $CC--$C9

DEVICE	$70	NPARAM	varies
PARAMS	INPUT: word, the byte count; OUTPUT: word, how many bytes to read		
PAYLOAD	INPUT: the bytes		
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the result		

The same four steps as the hash. Encoding is INPUT `$D0`, COMPUTE `$CF`, LENGTH `$CE`, OUTPUT `$CD`; decoding is `$CC`, `$CB`, `$CA`, `$C9`. In C, `fuji_base64_encode_input()` and its seven siblings.

QR CODE: INPUT, ENCODE, LENGTH, OUTPUT

$BC / $BD / $BE / $BF

DEVICE	$70	NPARAM	varies
PARAMS	INPUT: word, the count; ENCODE: version (0 to choose), error correction (0–3), shorten; LENGTH: the output mode (0 bytes, 2 bitmap); OUTPUT: word, the count		
PAYLOAD	INPUT: the text		
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the modules		

The adapter draws a QR code's matrix; a program only has to plot it. A version 1 code is 21 × 21 modules.

Answered with a NAK on this build

These are in `fujiCommandID.h` but fall through every table the RS-232 build dispatches, so they earn a NAK: `ENABLE_UDPSTREAM` (`$F0`), `SET_BAUDRATE` (`$EB`), `NEW_DISK` (`$E7`), `UNMOUNT_HOST` (`$E6`), `SET_HSIO_INDEX` (`$E3`), `SET_SIO_EXTERNAL_CLOCK` (`$DF`), `ENABLE_DEVICE` and `DISABLE_DEVICE` (`$D5`, `$D4`), `RANDOM_NUMBER` (`$D3`), `GET_TIME` (`$D2`; use the clock device), `DEVICE_ENABLE_STATUS` (`$D1`), `GET_HEAP` (`$C1`), `QRCODE_CLEAR` (`$BA`), `GET_DEVICE1..10_FULLPATH` (`$A0`–`$A9`), `UPDATE_FIRMWARE` (`$90`) and `HSIO_INDEX` (`$3F`). Most tune other computers' buses.

18. Command Reference: The Network Device

Every command devices `$71`–`$78` answer, checked against the network device every FujiNet shares, `NDevice.cpp`. Most of the codes are the ASCII letter of their name — `'O'`pen, `'R'`ead, `'W'`rite — a habit from the Atari 8-bits. What a devicespec can say, and how each protocol behaves, belongs to *The FujiNet Network Protocol Handbook*; these are the cards. A READ's answer must fit the 1K reply window, and a WRITE's bytes the 320-byte TX stream with the parameter in front of them.

The assembly examples use `netdefs.inc`'s names; the C examples, fujinet-lib's network calls, which take the devicespec every time. Each network command may take the cartridge 90 seconds, so the examples raise the timeout.

The lifecycle

NET_OPEN — $4F 'O'

DEVICE	$71	NPARAM	2
PARAMS	bytes: mode (4 read, 8 write, 12 read and write, 13 POST), translation (0 none, 1 CR, 2 LF, 3 CR LF)		
PAYLOAD	the devicespec, `N:` and the URL		
REPLY	ACK once the connection is set up		

On HTTP, modes 4 and 12 are both GET; 12 also lets a program set headers (NET_SET_HTTP_MODE). The request itself waits for the first STATUS or READ. Other HTTP modes: 14 PUT with headers, 5 and 9 DELETE. Opening a channel that is already open closes the old connection first.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCOPEN, 2
        lda     #NMHTTPG
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldx     #0
@url:   lda     url,x           ; the devicespec, as long as it is
        beq     @go
        jsr     fn_txb
        inx
        bne     @url
@go:    lda     #96             ; seconds: past the cartridge's 90
        sta     FN_TMO
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK)
    failed();
```

NET_STATUS

$53 'S'

DEVICE	$71	**NPARAM**	2
PARAMS	two bytes of 0		
PAYLOAD	none		
REPLY	4 bytes: bytes waiting (word), connected, status (1 good, 136 end of file)		

The heartbeat. A terminal watches *connected*; a program that fetches and shows watches the status for end of file; every program reads the count, until two readings agree (Section 11).

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCSTAT, 2
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        lda     #96             ; seconds: past the cartridge's 90
        sta     FN_TMO
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+NSAVLO ; bytes waiting
        ora     FN_REPLY+NSAVHI
        beq     done            ; nothing yet
```

C (cc65 + fujinet-lib)

```
if (network_status(url, &n, &count, &err) != FN_ERR_OK)
    failed();                    /* n waiting, count connected */
```

NET_READ $52 'R'

DEVICE	$71	NPARAM	1
PARAMS	**one word**: how many bytes, no more than are waiting and no more than 1024		
PAYLOAD	none		
REPLY	that many bytes		

A short or failed read is a NAK. Use the bytes, or copy them, before the next transaction paints over the window. fujinet-lib's `network_read()` does the STATUS loop itself and returns how many it read; `network_read_nb()` reads only what is waiting.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCREAD, 1
        lda     #<256           ; 256 bytes
        ldx     #>256
        jsr     fn_pw
        lda     #96             ; seconds: past the cartridge's 90
        sta     FN_TMO
        jsr     fn_go           ; the bytes at FN_REPLY, FN_RXLEN of them
```

C (cc65 + fujinet-lib)

```
got = network_read(url, buf, 256);
if (got < 0)
    failed();
```

NET_WRITE $57 'W'

DEVICE	$71	NPARAM	1
PARAMS	word: how many bytes		
PAYLOAD	that many bytes		
REPLY	ACK		

The count goes twice: as the parameter, and as the length of the payload. The two parameter bytes and the count leave 317 bytes for the data in one transaction.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCWRITE, 1
        lda     #dataend-data
        ldx     #0
        jsr     fn_pw
        ldx     #0
@w:     lda     data,x
        jsr     fn_txb
        inx
        cpx     #dataend-data
        bne     @w
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (network_write(url, data, 5) != FN_ERR_OK)
    failed();
```

NET_CLOSE $43 'C'

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Close every connection you open, including on the way out of a failure; the adapter keeps it open otherwise, even across a power cycle of the console.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCCLOSE, 0
        lda     #96             ; seconds: past the cartridge's 90
        sta     FN_TMO
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
network_close(url);
```

Parsers, parameters and line endings

NET_SET_PARSER / NET_PARSE / NET_QUERY

`$FC / $50 'P' / $51 'Q'`

DEVICE	$71
NPARAM	2 / 0 / 0
PARAMS	SET_PARSER: byte (any), then the parser: 0 none, 1 JSON, 2 HTML, 3 XML
PAYLOAD	QUERY: the path, such as `/0/name`
REPLY	QUERY: ACK; then STATUS and READ the value as text

Switch an open connection to one of the adapter's parsers (the parser is the **second** parameter), parse the body, then ask for one value by its path. fujinet-lib's `network_json_parse()` sets the parser and parses in one call, and `network_json_query()` does the query, STATUS and READ.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCPRSR, 2   ; JSON
        lda     #0
        jsr     fn_pb
        lda     #NPJSON
        jsr     fn_pb
        jsr     fn_go
        FNBEGIN NETDEV, NCPARSE, 0  ; parse what was fetched
        jsr     fn_go
        FNBEGIN NETDEV, NCQUERY, 0  ; ask for one value
        ldx     #0
@q:     lda     query,x
        beq     @go
        jsr     fn_txb
        inx
        bne     @q
@go:    jsr     fn_go           ; then STATUS and READ it
```

C (cc65 + fujinet-lib)

```
if (network_json_parse(url) != FN_ERR_OK)
    failed();
got = network_json_query(url, query, (char *)buf);
if (got < 0)
    failed();
```

NET_SET_PARAMETER $FB

DEVICE	$71	NPARAM	2
PARAMS	bytes: which (0 the query flags, 1 the line ending), the value		
PAYLOAD	none		
REPLY	ACK		

How the parser answers a query: flags remap characters and choose the output form; the line ending is the byte that ends each value. In C: `network_set_query_param()` and `network_set_line_ending()`.

NET_SET_HTTP_MODE $4D 'M'

DEVICE	$71	NPARAM	2
PARAMS	byte (any), then the mode: 0 body, 1 collect headers, 2 get headers, 3 set headers, 4 POST data		
PAYLOAD	none		
REPLY	ACK; NAK on a connection that is not HTTP		

HTTP only. In mode 3 each WRITE sets a request header; in mode 4 a WRITE supplies the body of a POST; in mode 2 a READ returns the headers collected in mode 1. Mode 0 is the page itself. Open with mode 12 to use it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN NETDEV, NCHTTPM, 2  ; from now, WRITE sets headers
        lda     #0
        jsr     fn_pb
        lda     #3
        jsr     fn_pb
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
network_http_start_add_headers(url);
network_http_add_header(url, "Accept: text/plain");
network_http_end_add_headers(url);
```

NET_SEEK / NET_TELL — $25 '%' / $26 '&'

DEVICE	$71	NPARAM	1 / 0
PARAMS	SEEK: **one four-byte parameter**, the offset		
PAYLOAD	none		
REPLY	TELL: 4 bytes, the position		

Random access where the protocol has it: TNFS and other files, HTTP ranges. In C: `network_seek()` and `network_tell()`.

NET_TRANSLATION / NET_SET_EOL / NET_SET_INT_RATE — $54 'T' / $4C 'L' / $5A 'Z'

DEVICE	$71	NPARAM	2 / 0 / 1
PARAMS	TRANSLATION: byte (any), then the code; SET_INT_RATE: the rate		
PAYLOAD	SET_EOL: the line ending, as a string; empty restores the default		
REPLY	ACK		

Line-ending translation and terminators after OPEN. SET_INT_RATE paces an interrupt line on computers that have one; the cartridge has no such line, so the command is accepted and does nothing. Polling `ACKSEQ` is the only way a 5200 program learns a transaction is done.

Files, sockets and credentials

RENAME / DELETE / LOCK / UNLOCK / MKDIR / RMDIR — $20 / $21 / $23 / $24 / $2A / $2B

DEVICE	$71	NPARAM	2
PARAMS	bytes: mode (4), and 0		
PAYLOAD	the devicespec; RENAME: `N:…/old,new`		
REPLY	ACK		

File operations on the protocols that have files: TNFS, FTP, SMB and the SD card. Each one opens a connection just for itself and closes it again. In C: `network_fs_rename()` and the rest.

NET_CHDIR / NET_GETCWD — $2C / $30

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	CHDIR: the path: absolute, relative, . . , or a whole new `SCHEME://host/`		
REPLY	GETCWD: the prefix		

A working directory for the channel, put in front of every devicespec that does not name its own host.

NET_ACCEPT / NET_CLOSE_CLIENT — $41 'A' / $63 'c'

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

For a TCP server: a channel opened on `N:TCP://:port/` listens; ACCEPT takes the waiting client, and CLOSE_CLIENT hangs up on that client while the channel keeps listening. A 5200 can be the bulletin board.

NET_GET_REMOTE / NET_SET_DESTINATION / NET_USERNAME / NET_PASSWORD — $72 'r' / $44 'D' / $FD / $FE

DEVICE	$71	NPARAM	0
PARAMS	none		
PAYLOAD	SET_DESTINATION: `host:port`; USERNAME and PASSWORD: the credential		
REPLY	GET_REMOTE: who sent the last UDP datagram		

UDP's "who said that" and "talk to them instead", and credentials for protocols that log in (FTP, SMB and others). Send the credentials before the OPEN.

Answered with a NAK on this build

In `fujiCommandID.h` but not in the network device's table:
`NET_GET_DSTATS_VALUE` (`$FF`), `NET_SET_UNIT` (`$FA`), `NET_SET_HSIO_INDEX` (`$E3`),

NET_QUERY_ALT and NET_PARSE_ALT ($81, $80), NET_GET_ERROR ($45; STATUS's fourth byte says what went wrong), and NET_HSIO_INDEX ($3F).

19. Command Reference: Disk, Printer, Clock, Modem

The mailbox is a transport, not a translator: whatever byte goes into the DEVICE register goes onto the wire. Beyond the FUJI and network devices, the FujiNet answers four more for a 5200: disk drives, a printer, a clock and a Hayes-style modem. The clock is exercised by a program in this handbook (Section 14), and its replies below were measured on the bench. The others are reachable, transcribed from the firmware's handlers, and offered as such.

Disk: devices $31–$38

Eight virtual drives, $31 plus the drive number, each serving the image mounted in its device slot. The sector size is the image's. On the 5200 a mount also pushes the image into the cartridge (Section 13); reading it as a disk as well is possible, and rarely useful.

DISK_READ / DISK_WRITE / DISK_PUT $52 / $57 / $50

DEVICE	$31	NPARAM	1
PARAMS	the sector number, as one parameter (two or four bytes)		
PAYLOAD	WRITE and PUT: one sector		
REPLY	READ: one sector; WRITE and PUT: ACK		

With nothing mounted, READ answers NAK. WRITE verifies what it wrote; PUT does not.

6502 ASSEMBLY (ca65)

```
        FNBEGIN DSKDEV, $52, 1  ; DISK_READ, drive 1
        lda     #1              ; sector 1
        ldx     #0
        jsr     fn_pw
        jsr     fn_go           ; the sector at FN_REPLY
```

C (cc65 + fujinet-lib)

```
if (!fuji_bus_call(0x31, 0x52, FUJI_FIELD_B12 | FUJI_FIELD_REPLY,
                   1, 0, 0, 0, buf, 512))
    failed();
```

DISK_STATUS `$53`

DEVICE	$31	NPARAM	0 or 1
PARAMS	optional byte: the kind of status		
PAYLOAD	none		
REPLY	4 bytes: drive status, controller status, format timeout, 0		

The Atari-style status block, from whatever image is mounted.

DISK_FORMAT / FORMAT_MEDIUM / PERCOM_READ / PERCOM_WRITE `$21 / $22 / $4E / $4F`

DEVICE	$31	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	FORMAT: a map of bad sectors; PERCOM_READ: a 12-byte geometry block		

Atari-drive commands the RS-232 build keeps; on an image they do little.

Printer: device `$40`

PRINTER_WRITE / PRINTER_PUT / PRINTER_STATUS `$57 / $50 / $53`

DEVICE	$40	NPARAM	0 / 0 / 1
PARAMS	STATUS: byte, required but ignored		
PAYLOAD	WRITE and PUT: the bytes to print, fewer than 255		
REPLY	WRITE and PUT: ACK; STATUS: 4 bytes		

The page goes to a file on the adapter, in the printer it is set to imitate, and comes out through its web interface. A printer switched off in the adapter's settings does not answer at all, and the cartridge reports a timeout.

6502 ASSEMBLY (ca65)

```
        FNBEGIN PRNDEV, $57, 0  ; PRINTER_WRITE
        ldx     #0
@p:     lda     data,x
        jsr     fn_txb
        inx
        cpx     #dataend-data
        bne     @p
        lda     #13             ; a carriage return
        jsr     fn_txb
        jsr     fn_go
```

C (cc65 + fujinet-lib)

```
if (!fuji_bus_call(0x40, 0x57, FUJI_FIELD_DATA,
                   0, 0, 0, 0, data, 5))
    failed();
```

Clock: device $45

On by default. A read with a parameter of 1 uses the alternate time zone instead of the adapter's own. The lengths below are what the bench's `fujinet-pc` answered.

GETTIME / GET_ISO_LOCAL / GET_ISO_UTC — $93 / $49 'I' / $5A 'Z'

DEVICE	$45	NPARAM	0 or 1
PARAMS	optional byte: 1 for the alternate zone		
PAYLOAD	none		
REPLY	GETTIME: 6 bytes: day, month, year (two digits), hour, minute, second. The ISO forms: `2026-10-20T16:14:12+0000` and a NUL, 25 bytes		

Section 14 has the ISO forms in both languages. GET_ISO_UTC ignores the alternate zone.

6502 ASSEMBLY (ca65)

```
        FNBEGIN CLKDEV, CKGTIME, 0
        jsr     fn_go
        bne     failed
        lda     FN_REPLY+3      ; the hour
        sta     cnt
```

C (cc65 + fujinet-lib)

```
if (clock_get_time(when, APETIME_BINARY))
    failed();                    /* when[3]: the hour */
```

The other formats — $54 $50 $53 $4D $47 $4C $9A

DEVICE	$45	NPARAM	0 or 1
PARAMS	none		
PAYLOAD	none		
REPLY	$54 'T': 7 bytes, century, year, month, day, hour, minute, second; $50 'P': 4 bytes, ProDOS; $53 'S': 19 characters, Apple III SOS; $4D 'M': the 7 bytes and hundredths; $47 'G': the zone's name, NUL-terminated; $4C 'L': one byte, its length; $9A: GETTIME in the alternate zone		

Every format fujinet-lib's `clock_get_time()` offers, plus the zone. `$54` doubles as a write command on buses that have one; on this one it only reads.

SETTZ_ALT / SETTZ — $74 't' / $99

DEVICE	$45	NPARAM	0
PARAMS	none		
PAYLOAD	a POSIX time-zone string, such as CST6CDT		
REPLY	ACK		

`$74` sets the zone the adapter uses for local time; `$99` sets the **alternate** zone that a read with a parameter of 1 uses. In C, `clock_set_tz()` sets the adapter's, and `clock_get_time_tz()` reads in another zone without changing it.

6502 ASSEMBLY (ca65)

```
        FNBEGIN CLKDEV, $74, 0  ; the adapter's own zone
        ldx     #0
@z:     lda     tz,x
        beq     @go
        jsr     fn_txb
        inx
        bne     @z
@go:    jsr     fn_go
        jmp     done
tz:     .byte   "CST6CDT", 0
```

C (cc65 + fujinet-lib)

```
if (clock_set_tz("CST6CDT"))
    failed();
```

Modem: device $50

A Hayes-style modem that dials TCP instead of telephone lines: `ATDT host:port` opens a connection, and `ATNET1` runs it through telnet negotiation. Ten packet commands set it up and poll it; the characters themselves flow through WRITE and STATUS.

MODEM_STATUS / MODEM_WRITE / MODEM_STREAM — $53 / $57 / $58

DEVICE	$50	**NPARAM**	1 / 0 / 0
PARAMS	STATUS: byte, required but ignored		
PAYLOAD	WRITE: an AT command line, or data once connected		
REPLY	STATUS: 2 bytes, the second the handshake lines (`$CD` connected with data waiting, `$CC` connected, `$00` idle); WRITE: ACK; STREAM: a 9-byte baud table		

The other seven set the modem up: CONTROL `$41`, CONFIGURE `$42`, SET_DUMP `$44`, LISTEN `$4C`, UNLISTEN `$4D`, BAUDRATELOCK `$4E`, AUTOANSWER `$4F`. Switched off in the adapter's settings, the modem does not answer.

20. Command Reference: The Cartridge Itself

Some of what a FujiNet program can ask for never leaves the cartridge: its registers, the stub that hands the console over, the status page, the boards it serves. This section is all of it, from the cartridge's own `fuji_mailbox.h` and the shared `fuji_mailbox_proto.h`.

The hotspot pages

Read	What it does
`$B500`+*n*, *n* < `$80`	Picks register *n* for the next read of `$B600`.
`$B580`–`$B5FF`	Not registers. Nothing happens, except at the stub (below).
`$B600`+*v*	Sets the picked register to *v*. With no register picked, nothing happens.
`$B700`+*b*	Appends the byte *b* to the TX stream.

Every hotspot read returns `$FF`, except the three bytes of the stub.

Registers

Registers `$00`–`$13` are the protocol's, shared by every FujiNet cartridge; `$14` is the 5200's.

Register	Select	Meaning
`$00` DEVICE	`$B500`	FujiBus device of the next transaction.
`$01` CMD	`$B501`	Its command.
`$02` NPARAM	`$B502`	How many parameters the TX stream holds, at most 8.
`$05` DATA_RST	`$B505`	Any value: rewind the TX stream.
`$06` RXSLICE	`$B506`	The reply slice shown; the 5200 has one, so 0.
`$10` SEQ	`$B510`	Non-zero and not the last one: launch.
`$11` BOOTLOCK	`$B511`	`$B5`: arm the staged image for the hand-over. Ignored until BOOTSTATE says READY.
`$12`, `$13` BOOTSEL	`$B512`, `$B513`	`$B5` then `$4A`: restart the RP2354B in its USB boot mode, to load new firmware. Two writes, so no single stray read can do it.
`$14` CONFIG	`$B514`	`$C5`: stage CONFIG and arm the hand-over to it.

The stub

`$B5F0`–`$B5F2` serve `$6C $FC $FF`: `JMP ($FFFC)`. The read of `$B5F2`, the instruction's last byte, swaps in the armed image, so the jump reads the BIOS's reset vector with the new image already in the window. Unarmed, the jump just restarts the BIOS with what is already there.

The status page

Everything the cartridge paints, `$B400`–`$B4FB`. Section 4 introduced the first ones.

Address	Name	Meaning
`$B400`	ACKSEQ	The last sequence number answered. Painted last. 0 after every power-on and hand-over.
`$B401`	STAT	Bit 0: the link to the FujiNet is up.
`$B402`	ERR	The last transaction's error (below).
`$B403`	REPLYCMD	`$06` ACK, `$15` NAK; 0 if the transaction never reached the FujiNet.
`$B404`–`$B405`	RXLEN	The reply's length, low byte first; at most 1024.
`$B406`	BOOTSTATE	0 idle, 1 arriving, 2 ready, `$80` failed.
`$B407`	BOOTPCT	0–100.
`$B408`	BOOTERR	Why the push failed (below).
`$B409`–`$B40A`	MAGIC	`'F', 'N'`.
`$B40B`	PROTOVER	The mailbox protocol's version: 1.
`$B40C`	SLICE_ECHO	The reply slice shown.
`$B412`	MODE	0 BOOT, 2 GAME, 3 APP.
`$B414`	MAPPER	The board the live image is served as (below).
`$B415`	LINK	1 once the FujiNet has been seen.
`$B416`–`$B418`	BOOTGOT	Bytes of the push received so far, 24 bits, low first.
`$B419`–`$B41B`	BOOTTOT	The size the push declared, 24 bits; 0 until it starts.
`$B41C`	ARMED	1 once a swap is armed: jump to the stub.
`$B41D`	STAGED	The pushed image: bit 0 ready, bit 1 it carries the claim, bit 2 it is CONFIG.
`$B41E`	STAGEDKIND	The board it will be served as.
`$B41F`–`$B422`	STAGEDCRC	Its CRC-32, low byte first.
`$B423`	SWAPS	Hand-overs since power-on, modulo 256.

Codes

Field	Value	Meaning
ERR	0	None.
	1	No link to the FujiNet.
	2	The FujiNet did not answer within the cartridge's budget.
	3	A bad frame: a parameter whose size is not 1, 2 or 4.
	4	Too big.
BOOTERR	1	The image is bigger than 128K.
	2	The push ended before the declared size.
	3	No board fits the image.
	4	The only room that fits it is the one being served.
MAPPER, STAGEDKIND	0	ROM: up to 32K, repeated to fill the window.
	1	Two chips: 16K, each 8K twice.
	2	Bounty Bob Strikes Back: 40K.
	3	Super Cart: 64K or 128K.

The claim's kind byte at `$BFE5` is the board plus one, or 0 for "decide from the size".

The budgets

Seconds	For
5	Every command not below.
60	MOUNT_IMAGE and COPY_FILE.
90	OPEN, CLOSE, STATUS and READ on the network devices `$71`–`$78`.
3	Waiting for the link before the console's first transaction.

The push

While MOUNT_IMAGE is outstanding, the FujiNet sends the image to the cartridge as device `$FF`, on the cartridge's own link: an OPEN (`$4F`) with the stream number and the 32-bit size, WRITEs (`$57`) of the bytes in order, and a CLOSE (`$43`), whose payload `$01` means the push was abandoned. Stream 1, the image's `.cfg` file, if there is one, comes first; stream 0 is the

image. The console sees none of it, only BOOTSTATE, BOOTPCT, BOOTGOT and BOOTTOT moving.

21. Battleship

FujiNet Battleship is a sea battle for up to four players, on any mix of computers and consoles a FujiNet serves. The game itself lives on a server. Each client sends the player's moves as HTTP requests and draws the state the server sends back, so a 5200 can sink an Apple II's fleet. The 5200 client is the same program as every other client with a thin layer for the console under it.

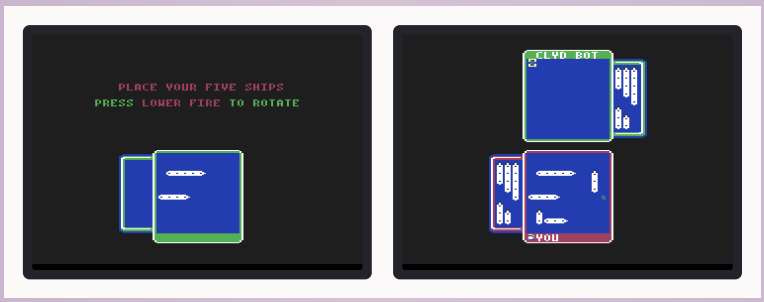


Figure 16. Placing the fleet, and the battle against a bot on the bench's own server. The board at the top is the bot's, the one below is yours.

How the code is laid out

Where	What
`src/*.c`	The game, shared by every client: `main.c` (the loop), `stateclient.c` (the server), `gamelogic.c` and `screens.c` (what to draw and when).
`src/platform-specific/`	The interface each client implements: graphics, input, sound, appkeys.
`src/atari5200/`	The 5200's implementation, guarded by `BUILD_ATARI5200`.
`mekkogx/`	The build: `make PLATFORMS=atari5200 atari5200` makes `r2r/atari5200/fbs.bin`.

The 5200 layer is about 1,400 lines:

File	What it does
graphics.c, charset.s	The Atari 8-bit client's screen: ANTIC mode 4, 40 × 26, in the 8-bit client's character set, with players for the target cursors (Section 16).
input.c	The stick, the buttons and the keypad, turned into the key codes the shared code expects.
osk.c	An on-screen keyboard for the player's name.
sound.c	POKEY effects for shots, hits and turns.
fujinet.c	Every call into fujinet-lib: the network, the appkeys, and the lobby.

Talking to the server

The shared code builds a URL from the server's address and a query, and the 5200 layer fetches it: one OPEN, one READ of everything that arrives, one CLOSE. The server answers in a packed binary form when asked (`bin=1`), which the shared code reads straight into a struct.

C (cc65 + fujinet-lib) · fujinet.c

```
int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    int16_t read;

    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}
```

`fujinet.c` is the only file that includes fujinet-lib's headers, because on the 5200 most of the library's FUJI calls are macros over its bus call; the shared code reaches it through hooks (`CUSTOM_FUJINET_CALLS`).

Which server

The address comes from an appkey: creator 1, app 1, key 5, which the FujiNet Lobby writes before it boots the game, holding `server?query`. With no key, the game uses `https://battleship.carr-designs.com/`. The bench in this handbook writes the key itself, pointing at a copy of the server running locally:

```
printf 'http://127.0.0.1:8781/?table=ai1' > SD/FujiNet/00010105.key
```

`ai1` is the server's table with one bot, so the game starts at once.

The screen in RAM

The 8-bit client draws in ANTIC mode 4 with a character set it rewrites as the boards change. On the 5200 nothing in the cartridge window can be rewritten, so all of it lives in the top 4K of RAM, which the build reserves with `__RESERVED_MEMORY__=0x1000`:

Address	What
`$3000`	The character set, copied from ROM at start-up (1K-aligned for `CHBASE`).
`$3400`	The screen, 40 × 26 = 1040 bytes.
`$3800`	Player/missile memory, single-line (2K-aligned for `PMBASE`).

The controls

The stick moves the cursor; one button fires or places a ship, the other rotates a ship or goes back; the keypad answers the screens' questions. `input.c` folds them all into key codes behind the `kbhit()` and `cgetc()` the shared code calls.

C (cc65 + fujinet-lib) · input.c

```
/*
  Controller 1: the analog stick from the crt0 VBI's POT shadows, the top
  fire on TRIG0, the lower fire on SKSTAT, and the keypad from the lib's
  keypad IRQ (fuji_a5200_key, one event per press).

  Only the stick travels in readJoystick(). The buttons and the keypad are
  synthesized as key codes behind kbhit()/cgetc(), as on the 7800; vars.h
  lists the mapping.
*/
```

NOTE: The game calls the button it reads through `SKSTAT` the "lower fire" and `TRIG0` the top one. MAME, the Atari800 emulator and the 1982 Owner's Manual put `TRIG0`, the fire button for every game, at the bottom (Section 9). On a real controller, believe the controller.

Building it

The game's Makefile builds the 5200 client against fujinet-lib-experimental's `atari5200` target, which still carries its own start-up code:

```
git clone https://github.com/FujiNetWIFI/fujinet-battleship.git
cd fujinet-battleship
make PLATFORMS=atari5200 atari5200 FUJINET_LIB=<fujinet-lib-experimental>
```

`make atari5200-smoke` and `make atari5200-play` run it in MAME, headless or in a window, through the game's own `support/atari5200/smoke.lua`, which this handbook used for the pictures.

22. 5 Card Stud

5 Card Stud is FujiNet's poker table: up to eight players, people and bots, on any computer or console a FujiNet serves, dealt and refereed by a server. Like Battleship, it is one program for every client, and the 5200's part is the screen, the controls, the sound and the network calls.

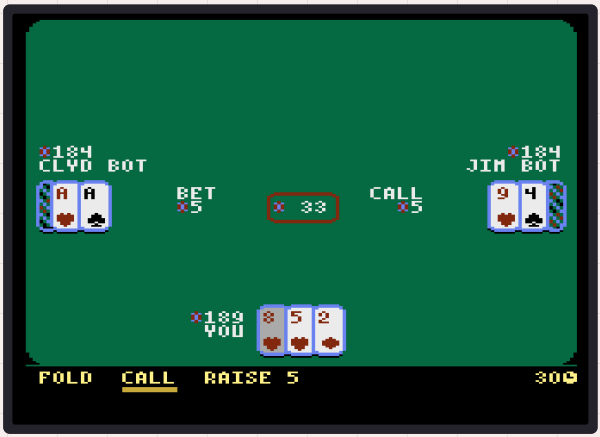


Figure 17. A hand against two bots on the bench's server: the stakes over each player, the pot in the middle, your three cards showing and one face down, and your moves along the bottom.

The 5200 layer

File	What it does
`antic.s`, `graphics.c`	The Atari 8-bit client's table: ANTIC mode 4, 40 × 26, two screens in RAM for double buffering, a display-list interrupt that colours the felt and the status rows, players for the black sidebars, the hole-card mask and the moving underline.
`input.c`	The stick picks a move, the fire button makes it, the keypad answers.
`osk.c`	An on-screen keyboard for the player's name.
`sound.c`	POKEY for the deal, the chips and the turn.
`network.c`	The server: OPEN, READ, CLOSE.

File	What it does
`appkey.c`	The player's name and the server's address.

The server's address comes from appkey creator 1, app 1, key 1, the same way as Battleship's; the bench points it at a local server's `ai2` table, two bots.

Talking to the server

One request per poll, and the whole answer in one READ:

C (cc65 + fujinet-lib) · network.c

```
uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_len)
{
    int16_t count;

    if (!initialized) {
        if (network_init() != FN_ERR_OK)
            return 0;
        initialized = 1;
    }

    // Don't read from a channel that never opened - network_read() would
    // otherwise poll status on a dead unit and hand back whatever it got.
    if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 0;

    count = network_read(url, buffer, max_len);
    network_close(url);

    return count > 0;
}
```

`network_read()` keeps reading until it has `max_len` bytes or the server has sent everything, so the buffer is the size of the largest state the server sends.

Two screens and an interrupt

The table is redrawn whenever the server's state changes, and a half-drawn table would flicker. So the client keeps two screens in the reserved top 4K of RAM: it draws into the work screen while ANTIC shows a copy of the last finished frame, and switches display lists when the new frame is complete:

6502 ASSEMBLY (ca65) · antic.s

```
_scrWork  = $3000               ; 1040 bytes each, inside one 4K block
_scrFront = $3410
_pmBase   = $3800               ; single-line PMG: players from $3C00
```

The felt and the status bar are different colours on the same mode-4 screen. A display-list interrupt at the top of the table and two more at the status rows change them: each loads its background colour from a table first, stores to `WSYNC`, and only then writes `COLBK` and `COLPF1`, so the change lands between lines (Section 16). The font stays in ROM, at `$B800`, clear of the mailbox.

Building it

```
git clone https://github.com/FujiNetWIFI/fujinet-5cardstud.git
cd fujinet-5cardstud
make PLATFORMS=atari5200 atari5200 FUJINET_LIB=<fujinet-lib-experimental>
```

The image is `r2r/atari5200/fcs.bin`; the game's `support/atari5200/smoke.lua` drives it in MAME, and took the picture above.

23. CONFIG

CONFIG is the program the cartridge serves at power-on: the FujiNet's own menu, where you join a WiFi network, pick a host, browse it, and boot what you find. It is a 32K FujiNet APP in C, like this handbook's programs, baked into the cartridge's flash. Its source is `atari5200/` in the fujinet-config repository.



Figure 18. The host slots, and the files on host 1, the SD card, with `battleship.a52` picked.



Figure 19. The adapter's information (START on the host slots), and a game being pushed and booted.

The controls

Control	Does
The stick	Moves the bar up and down a list; left and right page through a long directory.
# or the fire button	Opens: a host, a directory, a file to boot.
* or the other button	Goes back; on the host slots, * opens the WiFi screen.
PAUSE	Renames a host slot; on a directory, sets a filter.
START	The adapter's information; on a file, copies it to another host.

How it is built

File	What it does
`config.c`	The entry point and the state dispatcher: one state byte, one module per screen.
`st_wifi.c`, `st_hosts.c`, `st_files.c`, `st_copy.c`, `st_info.c`, `st_lobby.c`, `st_boot.c`	The screens.
`antic.c`, `antic_dl.s`	The text screen this handbook's C programs use (Section 5).
`fujidisp.c`	Frames, the bar, the progress bar, the cursor, on top of it.
`fujiin.c`	The stick, the buttons and the keypad, with repeat.
`fujiedit.c`	The on-screen keyboard for passwords and names.
`fujiraw.c`	The transactions fujinet-lib's calls do not fit (below).
`sfx.c`	POKEY's beeps.

Lists are drawn straight out of the cartridge's reply window, never copied: a directory page is one READ_DIR_ENTRY after another, each name drawn the moment it arrives.

Streaming a payload

fujinet-lib's calls take a payload as one buffer. A few of CONFIG's commands want a payload made of pieces, such as a directory path in

RAM followed by a file name that exists only in the reply window. For those, `fujiraw.c` uses the library's own bus layer and streams the bytes with `fn_tx()`, padding as it goes:

C (cc65 + fujinet-lib) · fujiraw.c

```
static void raw_begin(unsigned char cmd)
{
    fn_regwr(FNR_DATA_RST, 0);
    fn_regwr(FNR_DEVICE, FUJI_DEVICEID_FUJINET);
    fn_regwr(FNR_CMD, cmd);
    fn_regwr(FNR_NPARAM, 0);
```

Booting with a progress bar

MOUNT_IMAGE is answered only after the whole image has been pushed, on some FujiNets, so a program that waits for the answer shows nothing moving. CONFIG launches the transaction and does not wait: it writes the sequence number, then watches `ACKSEQ` and the boot registers together, drawing the bar from BOOTPCT as it goes.

C (cc65 + fujinet-lib) · st_boot.c

```
    want = fnraw_mount_start(DEVICE_SLOT, MODE_READ);
    for (;;) {
        unsigned char now;
        bool replied = !acked && FN_ACKSEQ == want;

        st = fuji_a5200_boot_state();
        if (replied) {
            acked = true;
            acked_at = frames;
        }
        if (acked && st == FUJI_A5200_BOOT_FAILED) {
            load_error(fuji_a5200_boot_error());
            return;
        }
        if (replied && !fnraw_reply_ok()) {
            fail("MOUNT");
            return;
        }
        if (acked && st == FUJI_A5200_BOOT_READY)
            break;
        if (acked && st == FUJI_A5200_BOOT_IDLE && frames - acked_at > IDLE_FRAMES) {
            status_line("?NOT A CARTRIDGE IMAGE");
            sfx_beep();
            return;
        }
        if (++frames > MOUNT_FRAMES) {
            load_error(st);
            return;
        }
```

An answer of yes with nothing arriving means the FujiNet did not take the file for a cartridge image, and CONFIG says so after three seconds.

Building and baking

```
cd fujinet-config/atari5200
make FNLIB=<fujinet-lib-experimental> PICO_5200=<fujinet-firmware>/pico/atari-5200
make cart          # copy build/config.bin into the bring-up and build the firmware
```

`make` builds `build/config.bin` and runs both checkers on it. The firmware build turns it into a header, `fujiconfigrom.h`, compiled into the cartridge's flash; the firmware keeps a copy in `firmware/baked/` for builds that have no fresh one. The CONFIG built from fujinet-config's `main` for this handbook is byte for byte the one baked into the firmware it describes.

NOTE: CONFIG builds against fujinet-lib-experimental's `atari5200` target, which still carries its own start-up code and linker configuration. The calls are the same as the ones this handbook's programs make through fujinet-lib and MekkoGX.

24. Things That Cost Real Time

Every one of these cost somebody an afternoon, in the cartridge's bring-up or in the writing of this handbook. Each is written down so it costs you nothing.

On the console

- **A held key counts many times.** The BIOS scans the keypad with POKEY's debouncing off, so POKEY interrupts on every pass over a held key, and the key-down bit drops between passes. A handler that counts interrupts sees one press as several; a held # once booted a random file in CONFIG. Count a key, then ignore it until it has been gone for four frames (Section 9).
- **The stick needs CONSOL bit 2.** The pots are switched off until a program writes $04 (or more) to CONSOL, and neither the BIOS nor either start-up file does it. MAME does not model the switch, so a program that forgets runs in the emulator and not on a console. cc65's joystick driver and the M.U.L.E. client set it; check your own.
- **Which button is which.** MAME and the Atari800 emulator call GTIA's TRIG the first button and POKEY's SKSTAT bit 3 the second; the Owner's Manual says the bottom buttons are the fire buttons for every game. Some FujiNet ports call TRIG0 the top button. Until someone checks on a controller, write programs that do not care.
- **MAME's CHBASE comes once a mode line.** A display-list interrupt that changes the font late on a busy line shows the old font on the next row, in MAME only (Section 16).

On the cartridge

- **There is no read/write line.** Every byte to the cartridge travels in an address. A store into the window is a fight for the bus, and an INC is two events (Section 4).
- **Two reads in a row can be one.** On the four-port console there is no clock on the edge, and the cartridge decides an access has happened when the address holds still; two cycles at the same address merge. Nothing in the mailbox ever reads one hotspot twice in a row.
- **Bounty Bob's banks are where the hotspot is, not what it reads.** The bank a read of $4FF6–$4FF9 picks is the address minus $4FF6. The

cartridge's first emulation of the board got it wrong in the same way the test that checked it did; an independent test from the documentation caught it.

- **An old fujinet-pc pushes nothing.** One built before the `.a52` extension was known mounts a 5200 image as a disk, says yes, and the cartridge waits for a push that never comes. CONFIG now says `?NOT A CARTRIDGE IMAGE` after three seconds instead of waiting a minute (Section 23).
- **MAME runs a throttled cartridge on its own thread.** Under `-nothrottle` the model does each transaction inside the CPU's read, so nothing moves during a push; throttled, it works as the real cartridge does, and the progress bar moves. Drive a program that talks to a live server with MAME throttled (`THROTTLE=1`), or it burns emulated seconds waiting.

In this handbook's code

- **`fn_txb` takes its byte in Y.** The assembly library carries the byte in the address of `LDA FN_TXDATA,Y`, so the 7800 programs this handbook's were ported from, which counted their strings in Y, sent garbage, and assembled perfectly. Every loop round `fn_txb` counts in X, and `snipcheck.py` refuses one that does not.
- **The library's timeout is in seconds.** The 7800's counted loops; the 5200's counts frames by the VBI, so its `FN_TMO` values are seconds: 66 for a mount, 96 for the network.
- **cc65 reads `'\n'` as `$9B`.** On the Atari targets, C's newline is the Atari's end-of-line. `netget.c` ignored every line ending until they were spelled `0x0A` and `0x0D`.
- **The assembly zero page starts on top of `KEY`.** `a52.cfg` gives the linker `$19` up, and `$19` is the keypad's byte (Section 6).
- **cc65's console lives where the C stack goes.** Its screen is at `$3E20`; reserve at least `$1E0` bytes or the stack writes on it (Section 5).
- **cc65 drops a read it thinks is useless**, even an inline `lda` that switches a bank. Bank switches go in assembly files (Section 15).
- **The 5200 header moved.** fujinet-lib's pull request for the 5200 keeps `fujinet-atari5200.h` in `bus/atari5200/`, and MekkoGX puts only `include/` on the path, so a program that includes it must add the other directory itself. This handbook's build does; the games' do not yet, and build against fujinet-lib-experimental.

- **The count starts over at every program.** The 5200 cartridge watches the console's power and repaints its status page at every hand-over, so `ACKSEQ` is 0 whenever a program starts. Take the next number from it all the same (Section 4).
- **The bench's `fnconfig.ini` has Windows line endings.** An edit that anchored on the end of the line changed nothing, and the private adapter came up on another bench's port.

25. Troubleshooting Checklist

The following list of symptoms, causes, and remedies is supplied to help you find the trouble when a FujiNet program does not do what you meant. Always check the status page first: `ERR` at `$B402` and `REPLYCMD` at `$B403` say most of what went wrong.

Symptom

- The program never gets past its first transaction.

Probable Cause and Remedy

- `'F','N'` is not at `$B409`: the image is not running from a FujiNet cartridge, or MAME was started without `-cartslot fujinet`.
- The image has no claim at `$BFE0`, so it runs as a GAME and the mailbox is switched off.
- Interrupts are off. The library's wait counts frames by the VBI, and without it never runs out.
- `fujinet-pc` is not listening on the bus-over-IP port MAME was given, or another MAME is already connected to it.

Symptom

- Every transaction answers with ERR 1.

Probable Cause and Remedy

- There is no link to the FujiNet: the USB cable, or on the bench the adapter is not running. The cartridge waits three seconds for the link before each transaction.

Symptom

- A transaction answers with ERR 2.

Probable Cause and Remedy

- The FujiNet took longer than the cartridge's budget: five seconds for most commands, sixty for a mount, ninety for the network device. Check the devicespec, the host and the server.

Symptom	• A transaction answers with ERR 3.
Probable Cause and Remedy	• A parameter's size byte is not 1, 2 or 4, or NPARAM says more parameters than the TX stream holds. Count them against the command card.
Symptom	• `fn_go` returns `$FF`, or C reports `FN_EWAIT`.
Probable Cause and Remedy	• The cartridge never copied the sequence number: the hotspot reads went wrong. Look for a store, an `INC` or a `(zp),Y` on `$B500`–`$B7FF`, an index that crosses a page, or a loop that counts in Y round `fn_txb`. • The sequence number repeated the last one. Take it from `ACKSEQ` plus one, every time.
Symptom	• The FujiNet says no (`REPLYCMD $15`; `FNENAK $EE` in assembly, 144 in C).
Probable Cause and Remedy	• The command was understood and refused: a host not mounted, a path that does not exist, a full appkey, a creator of 0. The card for the command lists its refusals.
Symptom	• Booting another image, BOOTSTATE stays 0.
Probable Cause and Remedy	• The FujiNet did not take the file for a cartridge image: it must end in `.a52` or `.bin`. • The `fujinet-pc` was built before it knew `.a52`.

Symptom	• Booting another image, BOOTSTATE reads `$80`.
Probable Cause and Remedy	• BOOTERR 1: the image is bigger than 128K. • BOOTERR 2: the file is shorter than its size said; copy it again. • BOOTERR 3: no board fits it. Give it a `.cfg` file with a `mapper=` line.
Symptom	• The booted program cannot find the mailbox.
Probable Cause and Remedy	• It has no claim, or it is a two-chip or Bounty Bob image, which cannot be an APP (Section 13).
Symptom	• The screen is black, or garbage.
Probable Cause and Remedy	• `SDMCTL` is 0, or the display list never set: in assembly, `disp_on`; in C, `mt_init`. • The display list, a screen or a font is in `$B000`–`$B7FF`, where ANTIC reads the mailbox. • A screen crosses a 4K boundary, or a display list a 1K one.
Symptom	• A key on the keypad acts twice.
Probable Cause and Remedy	• The keypad interrupt fires on every scan of a held key. Count a key once until the hold timer has run out, as `a52init.s` and the C start-up code do.

Symptom	• The stick does nothing on a real console, though it works in MAME.
Probable Cause and Remedy	• `CONSOL` bit 2 is clear, so the pots are off. Write `$04` to `$C01F` (Section 9).
Symptom	• No sound.
Probable Cause and Remedy	• `SKCTL` is 0, which holds POKEY in reset; the start-up code leaves it at `$02`. • The voice's volume, the low four bits of `AUDC1`, is 0.
Symptom	• A C program draws stray marks low on cc65's console, or crashes.
Probable Cause and Remedy	• The C stack is on the console's screen: link with `__RESERVED_MEMORY__` of at least `0x1E0`.
Symptom	• Text from the network comes out on one line, in C.
Probable Cause and Remedy	• The code compares with `'\n'`, which cc65's Atari targets make `$9B`. Use `0x0A` and `0x0D`.

Symptom	• A Super Cart program crashes when it switches banks.
Probable Cause and Remedy	• The code running at the switch, an interrupt handler, the display list or the font is not the same in every bank. • Something read `$BFE0` (the claim) and selected the last bank. • cc65 dropped the read that was meant to switch: do it with `BIT` in an assembly file.
Symptom	• `checkrom.py` or the romstamp refuses the image.
Probable Cause and Remedy	• It says what it found and where: a store or read-modify-write in the window, a hotspot read from a base that is not page-aligned, `JMP ($xxFF)`, or a missing claim or header.

26. Parts List

Every address, register, command and listing in this handbook was transcribed from these, at these revisions, and every program was built and run against them. When ordering repair parts, always include the part number.

The cartridge

Mailbox, stub, boards, power-on.	`fujinet-firmware pico/atari-5200 @ 7a53805c8`
Shared mailbox protocol and fujimail.	`fujinet-firmware pico/common @ 7a53805c8`
Client library, start-up, text, test programs.	`pico/atari-5200/testrom @ 7a53805c8`
Checkers: checkrom.py, mkfont.py.	`pico/atari-5200/tools @ 7a53805c8`
MAME cartridge model.	`pico/atari-5200/emu @ 7a53805c8`
Baked CONFIG.	`pico/atari-5200/firmware/baked @ 7a53805c8`
Board.	`fujinet-hardware ATARI-5200/FujiNet-5200-Rev0`

The adapter

Command dispatch: FUJI device and mixins.	`lib/device/rs232, fujiDevice @ f0b62cb09`
Network device.	`lib/device/NDevice @ f0b62cb09`
Clock.	`lib/device/fujiClock @ f0b62cb09`
fujinet-pc on the bench.	`v1.6-f0b62cb09, RS232 build`
Bus-over-IP.	`127.0.0.1:9952`

The programs

fujinet-lib, atari5200 target.	`tschak909/fujinet-lib add-atari5200 @ 7cf47dc (FujiNetWIFI/fujinet-lib PR 139)`
Start-up, linker configuration, romstamp.	`tschak909/MekkoGX add-atari5200 @ faeb3fd (FozzTexx/MekkoGX PR 27)`
fujinet-lib for CONFIG and the games.	`fujinet-lib-experimental add-atari5200 @ 7101377`

CONFIG, and the C text engine	`fujinet-config main @ 1bd6ec8 (atari5200/ @ cb44218)`
Battleship	`fujinet-battleship main @ 3f2a548`
5 Card Stud	`fujinet-5cardstud main @ d5db857`
Game servers	`fujinet-game-system @ 19770c3`
This handbook's programs	`listings/asm, listings/c`

The tools

cc65: ca65, ld65, cc65, cl65	`V2.19, 92fa7cb23`
MAME, single-driver a5200 build	`mamedev @ 533800328a2, with the cartridge model grafted`
5200 BIOS	`MAME a5200.zip, co19156.u8`
Typesetting	`Typst 0.14.2`
Type	`Helvetica; ITC Avant Garde Gothic; Source Code Pro`
House style	`ATARI 5200 Owner's Manual, 1982, CO 19277`

This handbook

Part number	`FN5200-PH REV. A`
Source	`fujinet-manuals atari-5200/ programmers-handbook`

A. Mailbox Quick Reference

The map

Address	Access	What
`$B000`–`$B3FF`	read	Reply window, 1K
`$B400`–`$B4FB`	read	Status page
`$B500`+*n*	read	Select register *n* (*n* < `$80`)
`$B5F0`–`$B5F2`	`JMP`	The stub: `JMP ($FFFC)`, swaps the armed image in
`$B600`+*v*	read	Set the selected register to *v*
`$B700`+*b*	read	TX stream: append the byte *b*
`$BFC0`–`$BFFF`	read	Super Cart bank switch
`$BFE0`–`$BFE6`	—	The claim: `"FUJI"`, version, kind + 1, flags

A transaction

1. DEVICE, CMD, NPARAM; DATA_RST.
2. Parameters (size, value, low first), then the payload.
3. SEQ = `ACKSEQ` + 1 (255 wraps to 1).
4. Wait for `ACKSEQ` = SEQ.
5. `ERR` = 0, `REPLYCMD` = `$06`.
6. Read `RXLEN` bytes from `$B000`.

Registers

Register	Read	Meaning
`$00`	`$B500`	DEVICE
`$01`	`$B501`	CMD
`$02`	`$B502`	NPARAM, at most 8
`$05`	`$B505`	DATA_RST: any value
`$06`	`$B506`	RXSLICE: 0
`$10`	`$B510`	SEQ: launch
`$11`	`$B511`	BOOTLOCK: `$B5` arms the pushed image
`$12`, `$13`	`$B512`, `$B513`	BOOTSEL: `$B5` then `$4A`
`$14`	`$B514`	CONFIG: `$C5` arms CONFIG

Rules

- Hotspot pages: `LDA`/`LDX`/`LDY`/`BIT`, absolute or absolute-indexed, from a page-aligned base.
- No store, no read-modify-write, no `(zp),Y` in `$4000`–`$BFFF`; no `JMP ($xxFF)`.
- Never the same hotspot on two cycles in a row.
- No display list, screen or font in `$B000`–`$B7FF`.
- The claim at `$BFE0`, `$02` at `$BFE7`, `$FF` at `$BFFD`, the start at `$BFFE`.

Budgets

Seconds	For
5	Most commands
60	MOUNT_IMAGE, COPY_FILE
90	Network OPEN, CLOSE, STATUS, READ

B. Error and Status Codes

ERR, at $B402

Value	Meaning
0	None.
1	No link to the FujiNet.
2	The FujiNet did not answer within the cartridge's budget.
3	A bad frame: a parameter whose size is not 1, 2 or 4.
4	Too big.

REPLYCMD, at $B403

Value	Meaning
$06	ACK: the FujiNet did it.
$15	NAK: the FujiNet refused.
0	The transaction never reached the FujiNet (see ERR).

BOOTSTATE and BOOTERR, at $B406 and $B408

Field	Value	Meaning
BOOTSTATE	0	Idle.
	1	An image is arriving.
	2	Ready: arm it with BOOTLOCK.
	$80	Failed: see BOOTERR.
BOOTERR	1	Bigger than 128K.
	2	Shorter than its declared size.
	3	No board fits it.
	4	The only room that fits it is in use.

MODE, at $B412

Value	Meaning
0	BOOT: CONFIG, at power-on.
2	GAME: an image without the claim; no mailbox.
3	APP: an image with the claim.

The assembly library's own

Value	Name	Meaning
0	FNEOK	Success.
$EE	FNENAK	The FujiNet said no.
$FC	FNENARM	The swap was never armed.
$FD	FNENOC	No cartridge: 'F','N' missing.
$FF	FNEWAIT	The cartridge never answered.

fujinet-lib's

Value	Name	Meaning
0–4	FN_OK … FN_ETOOBIG	The cartridge's ERR, in fn_device_error.
$FF	FN_EWAIT	The cartridge never answered.
144		The FujiNet said no, in fn_device_error.

C. Program Listings

Every program in this handbook, as built and run. The assembly library and its programs from `listings/common/` are the cartridge bring-up's, unchanged; `antic.c` and its display list are CONFIG's.

The assembly library

fujinet.inc

```
; fujinet.inc -- the cart's mailbox and the 5200's registers, for ca65.
; Hand-mirrored from firmware/include/fuji_mailbox.h; tools/checkdefs.py
; compares them.
;
; Console -> cart is reads only: `lda FN_REGSEL+n` then `lda FN_REGDATA,y`
; with the value in Y. Hotspot pages take LDA/LDY/BIT abs or abs,X/Y
; from a page-aligned base, never a store, an RMW or (zp),Y.

FN_REPLY        = $B000         ; 1K reply window
FN_STATUS       = $B400         ; status page
FN_REGSEL       = $B500         ; read +n: select register n
FN_REGDATA      = $B600         ; read +v: the selected register = v
FN_TXDATA       = $B700         ; read +b: append one TX byte
FN_STUB         = $B5F0         ; cart-served JMP ($FFFC); swaps when armed

; status page (FN_R_* - $400)
FN_ACKSEQ       = FN_STATUS + $00
FN_STAT         = FN_STATUS + $01
FN_ERR          = FN_STATUS + $02
FN_REPLYCMD     = FN_STATUS + $03
FN_RXLEN        = FN_STATUS + $04
FN_BOOTSTATE    = FN_STATUS + $06
FN_BOOTPCT      = FN_STATUS + $07
FN_BOOTERR      = FN_STATUS + $08
FN_MAGIC        = FN_STATUS + $09   ; 'F','N'
FN_PROTOVER     = FN_STATUS + $0B
FN_MODE         = FN_STATUS + $12
FN_MAPPER       = FN_STATUS + $14
FN_LINK         = FN_STATUS + $15
FN_BOOTGOT      = FN_STATUS + $16
FN_BOOTTOT      = FN_STATUS + $19
FN_ARMED        = FN_STATUS + $1C
FN_STAGED       = FN_STATUS + $1D
FN_STAGEDKIND   = FN_STATUS + $1E
FN_STAGEDCRC    = FN_STATUS + $1F
FN_SWAPS        = FN_STATUS + $23

; registers (read FN_REGSEL + n, then FN_REGDATA + value)
FN_REG_DEVICE   = $00
FN_REG_CMD      = $01
FN_REG_NPARAM   = $02
FN_REG_DATA_RST = $05
FN_REG_RXSLICE  = $06
FN_REG_SEQ      = $10
FN_REG_BOOTLOCK = $11
FN_REG_BOOTSEL_1 = $12
FN_REG_BOOTSEL_2 = $13
FN_REG_CONFIG   = $14

FN_BOOTLOCK_MAGIC = $B5
FN_CONFIG_MAGIC = $C5

FN_BOOT_READY   = 2
FN_BOOT_FAILED  = $80
FN_MODE_BOOT    = 0
FN_MODE_GAME    = 2
FN_MODE_APP     = 3

; the claim, at $BFE0 in every FujiNet app
FN_CLAIM        = $BFE0

; 5200 hardware
GTIA            = $C000
COLPM0          = $C012
COLPF0          = $C016
COLPF1          = $C017
COLPF2          = $C018
COLPF3          = $C019
COLBK           = $C01A
PRIOR           = $C01B
GRACTL          = $C01D
CONSOL          = $C01F
TRIG0           = $C010
PAL             = $C014
ANTIC           = $D400
DMACTL          = $D400
CHACTL          = $D401
DLISTL          = $D402
DLISTH          = $D403
PMBASE          = $D407
CHBASE          = $D409
WSYNC           = $D40A
VCOUNT          = $D40B
NMIEN           = $D40E
NMIST           = $D40F
NMIRES          = $D40F
POKEY           = $E800
AUDF1           = $E800
AUDC1           = $E801
AUDCTL          = $E808
POT0            = $E800
KBCODE          = $E809
RANDOM          = $E80A
POTGO           = $E80B
IRQEN           = $E80E
IRQST           = $E80E
SKCTL           = $E80F
SKSTAT          = $E80F

; the BIOS's zero page, which a52init.s keeps the same
POKMSK          = $00
RTCLOKH         = $01
RTCLOKL         = $02
CRITIC          = $03
ATRACT          = $04
SDLSTL          = $05
SDLSTH          = $06
SDMCTL          = $07
PCOLR0          = $08
COLOR0          = $0C
COLOR1          = $0D
COLOR2          = $0E
COLOR3          = $0F
COLOR4          = $10
PADDL0          = $11

; ...and its vectors
VIMIRQ          = $0200
VVBLKI          = $0202
VVBLKD          = $0204
VDSLST          = $0206
VKYBDI          = $0208
VKYBDF          = $020A
VTRIGR          = $020C
VBRKOP          = $020E
VSERIN          = $0210
VTIMR4          = $021A

; client error codes (fujilib.s)
FNEOK     = 0                   ; FN_ERR_OK
```

```
131 FNEWAIT   = $FF                ; the cart
    never answered at all
132 FNENAK    = $EE                ; the server
    said NAK
133 FNENOC    = $FD                ; no cartridge
    answered ('F','N' missing)
134 FNENARM   = $FC                ; the swap was
    never armed
135
136 ; FujiNet devices and commands
137 FNDEVF    = $70                ; the FUJI
    device
138 FNCADPX   = $C4                ;
    GET_ADAPTERCONFIG_EXTENDED
139 FNCMHST   = $F9                ; MOUNT_HOST
140 FNCMIMG   = $F8                ; MOUNT_IMAGE
141 FNCSDFP   = $E2                ;
    SET_DEVICE_FULLPATH
142 FMREAD    = 1                  ; disk access
    mode: read
143
144 ; the clients' zero page, clear of the BIOS's
    $00-$18
145 KEY       = $19                ; last keypad
    code + 1, 0 = none (a52init.s)
146 FN_SEQ    = $E0                ; the sequence
    we are waiting for
147 FN_CNT    = $E1                ; frames left
    before giving up (2 bytes)
148 FN_DEV    = $E3                ; transaction
    setup, read by fn_beg
149 FN_CMD    = $E4
150 FN_NPR    = $E5
151 FN_TMO    = $E6                ; timeout, in
    seconds
152 FN_LAST   = $E7                ; RTCLOKL as
    fn_wait last saw it
153 FN_PTR    = $E8                ; a string
    pointer (2 bytes)
154 DISP_PTR  = $EA                ; the text
    cursor (2 bytes)
155 DISP_TMP  = $EC
156
157 SCREEN    = $1000              ; 40 x 24,
    mode 2
158
```

header.inc

```
1  ; header.inc -- the claim and the BIOS's
   cartridge header, for every client.
2  ; $BFFD = $FF sends the BIOS straight to the
   entry vector, no title screen.
3
4          .import start
5
6          .segment "CLAIM"
7          .byte   "FUJI"
8          .byte   1               ; claim
   version
9          .byte   0               ; mapper: from
   the size
10         .byte   0               ; flags
11         .byte   $02             ; $BFE7: PAL-
   compatible, for the 2-port BIOS
12
13         .segment "HEADER"
14         .byte   "FUJINET             "      ;
   $BFE8: 20 characters, unseen
15         .byte   $FF, $FF                    ;
   $BFFC: year; $BFFD = $FF: no title
16         .word   start                       ;
   $BFFE: entry
17
```

fujilib.s

```
1  ; fujilib.s -- the 6502 side of the FujiNet
   cartridge mailbox: the code.
2  ;
3  ; The 5200 edge has no R/W line, so every byte
   reaches the cart in the
4  ; address of a read: a register write is `lda
   FN_REGSEL+n` then
5  ; `lda FN_REGDATA,y` with the value in Y. Both
   bases are page-aligned, so
6  ; an index never carries into another page.
7
8          .include "fujinet.inc"
9
10         .export fn_chk, fn_rw, fn_beg, fn_txb,
   fn_pb, fn_pw, fn_path
```

```
        .export fn_go, fn_ack, fn_blk, fn_boot, fn_config, fn_frames, fn_wait

        .segment "CODE"

; fn_rw -- register X = A. Clobbers A, Y.
.proc fn_rw
        tay
        lda     FN_REGSEL,x
        lda     FN_REGDATA,y
        rts
.endproc

; fn_chk -- Z set if a FujiNet cartridge is answering.
.proc fn_chk
        lda     FN_MAGIC
        cmp     #'F'
        bne     done
        lda     FN_MAGIC+1
        cmp     #'N'
done:   rts
.endproc

; fn_beg -- begin the transaction described by FN_DEV / FN_CMD / FN_NPR.
; Rewinds the TX stream last, so parameters may be appended at once. Resets
; FN_TMO, so a slow command raises it after this.
.proc fn_beg
        lda     #12             ; seconds: past the cart's 5 s window for ordinary commands
        sta     FN_TMO
        lda     FN_DEV
        ldx     #FN_REG_DEVICE
        jsr     fn_rw
        lda     FN_CMD
        ldx     #FN_REG_CMD
        jsr     fn_rw
        lda     FN_NPR
        ldx     #FN_REG_NPARAM
        jsr     fn_rw
        lda     #0
        ldx     #FN_REG_DATA_RST
        jmp     fn_rw
.endproc

; fn_txb -- append A to the TX stream. Clobbers Y.
.proc fn_txb
        tay
        lda     FN_TXDATA,y
        rts
.endproc

; fn_pb -- append a one-byte parameter with value A.
.proc fn_pb
        pha
        ldy     #1
        lda     FN_TXDATA,y
        pla
        tay
        lda     FN_TXDATA,y
        rts
.endproc

; fn_pw -- append a two-byte little-endian parameter. A = low, X = high.
.proc fn_pw
        pha
        ldy     #2
        lda     FN_TXDATA,y
        pla
        tay
        lda     FN_TXDATA,y
        txa
        tay
        lda     FN_TXDATA,y
        rts
.endproc

; fn_path -- append the NUL-terminated string at FN_PTR to the TX stream,
; padded with NULs to EXACTLY 256 bytes: OPEN_DIRECTORY and
; SET_DEVICE_FULLPATH both read exactly 256.
.proc fn_path
        ldy     #0
loop:   lda     (FN_PTR),y
        beq     pad
        tax
        lda     FN_TXDATA,x
        iny
        bne     loop
        rts
pad:    lda     FN_TXDATA
        iny
        bne     pad
        rts
.endproc

; fn_wait -- count down FN_CNT by the frames that have passed. C clear once
; it runs out.
.proc fn_wait
        lda     RTCLOKL
        cmp     FN_LAST
        beq     still
        sta     FN_LAST
        lda     FN_CNT
        bne     :+
        dec     FN_CNT+1
:       dec     FN_CNT
        lda     FN_CNT
        ora     FN_CNT+1
        bne     still
        clc
        rts
still:  sec
        rts
.endproc

; fn_frames -- FN_CNT = A seconds of frames (x 64, near enough to 60).
.proc fn_frames
        sta     FN_CNT+1
        lda     #0
        lsr     FN_CNT+1
        ror     a
        lsr     FN_CNT+1
        ror     a
        sta     FN_CNT
        lda     RTCLOKL
        sta     FN_LAST
        rts
.endproc

; fn_go -- commit and wait. Returns A = FNE* (0 = the transaction completed).
; The next sequence number comes from the CART's ACKSEQ + 1, never a counter
; in RAM: a power cycle restarts the client, not the cart.
.proc fn_go
        lda     FN_ACKSEQ
        clc
        adc     #1
        bne     have
        lda     #1              ; 0 is reserved as "never used"
have:   sta     FN_SEQ
        ldx     #FN_REG_SEQ
        jsr     fn_rw           ; launching it is this one pair
        lda     FN_TMO
        jsr     fn_frames
wait:   lda     FN_ACKSEQ       ; published LAST, after the whole reply
        cmp     FN_SEQ
        beq     got
        jsr     fn_wait
        bcs     wait
        lda     #FNEWAIT
        rts
got:    lda     FN_ERR
        rts
.endproc

; fn_ack -- 0 on success, FNENAK if the server said no. Call after fn_go
; returned 0.
.proc fn_ack
        lda     FN_REPLYCMD
        cmp     #$06
        beq     ok
        lda     #FNENAK
        rts
ok:     lda     #FNEOK
        rts
.endproc

; fn_blk -- arm the staged image (FN_REG_BOOTLOCK).
```

```
.proc fn_blk
        lda     #FN_BOOTLOCK_MAGIC
        ldx     #FN_REG_BOOTLOCK
        jmp     fn_rw
.endproc

; fn_boot -- arm the staged image and hand the console to it. Returns only
; if the cart never armed the swap, with A = FNENARM.
.proc fn_boot
        jsr     fn_blk
        jmp     handover
.endproc

; fn_config -- back to CONFIG. Returns only if the cart never armed it.
.proc fn_config
        lda     #FN_CONFIG_MAGIC
        ldx     #FN_REG_CONFIG
        jsr     fn_rw
        jmp     handover
.endproc

; Wait for FN_ARMED, quiet the machine as a power-on leaves it, and jump to
; the cart's stub: its JMP ($FFFC) is the last cart read before the BIOS's
; reset code, and the read of its last byte swaps the image in.
.proc handover
        lda     #2
        jsr     fn_frames
wait:   lda     FN_ARMED
        bne     armed
        jsr     fn_wait
        bcs     wait
        lda     #FNENARM
        rts
armed:  sei
        lda     #0
        sta     NMIEN           ; first: a pending NMI would run through $0202
        sta     NMIRES
        sta     DMACTL          ; no display-list fetch may reach the new image
        sta     GRACTL
        sta     IRQEN
vbl:    lda     VCOUNT          ; let the frame's DMA finish
        cmp     #124
        bcc     vbl
        lda     #0
        tax
chips:  sta     POKEY,x
        sta     GTIA,x
        sta     ANTIC,x
        inx
        bne     chips
ramlo:  .repeat 32, I                   ; two halves: one is past a branch's reach
        sta     I * $100,x
        .endrepeat
        inx
        bne     ramlo
ramhi:  .repeat 32, I
        sta     $2000 + I * $100,x
        .endrepeat
        inx
        bne     ramhi
        clc                     ; the flags as the BIOS's own start leaves them
        jmp     FN_STUB
.endproc

```

fujidisp.s

```
; fujidisp.s -- the bring-up clients' text: ANTIC mode 2, 40 x 24, from a
; font in ROM and a screen in RAM. Characters are ASCII on the way in.

        .include "fujinet.inc"

        .export disp_init, disp_at, disp_putc, disp_str, disp_hex, disp_dec
        .export disp_rpl, disp_on

        .segment "CODE"

; disp_init -- clear the screen, point ANTIC at it; the display is off
; until disp_on.
.proc disp_init
        lda     #<SCREEN
        sta     DISP_PTR
        lda     #>SCREEN
        sta     DISP_PTR+1
        lda     #0
```

```
        ldx     #4              ; 4 x 256 covers 960
        ldy     #0
clr:    sta     (DISP_PTR),y
        iny
        bne     clr
        inc     DISP_PTR+1
        dex
        bne     clr
        lda     #<dlist
        sta     SDLSTL
        lda     #>dlist
        sta     SDLSTH
        lda     #>font
        sta     CHBASE
        lda     #2
        sta     CHACTL
        lda     #$0E            ; text luminance
        sta     COLOR1
        lda     #$84            ; blue background
        sta     COLOR2
        lda     #$00
        sta     COLOR4
        lda     #0
        tax
        jmp     disp_at
.endproc

.proc disp_on
        lda     #$22            ; normal playfield, display list DMA
        sta     SDMCTL
        rts
.endproc

; disp_at -- cursor to row A, column X.
.proc disp_at
        stx     DISP_TMP
        tax
        lda     #<SCREEN
        sta     DISP_PTR
        lda     #>SCREEN
        sta     DISP_PTR+1
        cpx     #0
        beq     col
row:    clc
        lda     DISP_PTR
        adc     #40
        sta     DISP_PTR
        bcc     :+
        inc     DISP_PTR+1
:       dex
        bne     row
col:    clc
        lda     DISP_PTR
        adc     DISP_TMP
        sta     DISP_PTR
        bcc     :+
        inc     DISP_PTR+1
:       rts
.endproc

; disp_putc -- ASCII A at the cursor, which advances. Y is kept.
.proc disp_putc
        cmp     #$20
        bcc     ctl
        cmp     #$60
        bcs     put
        sbc     #$1F            ; $20-$5F -> $00-$3F (carry was clear: -$20)
        bcs     put
ctl:    adc     #$40            ; $00-$1F -> $40-$5F
put:    sty     DISP_TMP
        ldy     #0
        sta     (DISP_PTR),y
        ldy     DISP_TMP
        inc     DISP_PTR
        bne     :+
        inc     DISP_PTR+1
:       rts
.endproc

; disp_str -- the NUL-terminated string at FN_PTR.
.proc disp_str
        ldy     #0
loop:   lda     (FN_PTR),y
        beq     done
        jsr     disp_putc
        iny
        bne     loop
done:   rts
.endproc

.proc disp_hex
        pha
        lsr     a
        lsr     a
        lsr     a
        lsr     a
        jsr     nib
        pla
        and     #$0F
nib:    cmp     #10
        bcc     :+
        adc     #6
:       adc     #'0'
        jmp     disp_putc
.endproc

; disp_dec -- A as up to three decimal digits.
.proc disp_dec
        ldx     #0
        ldy     #100
        jsr     digit
        ldy     #10
        jsr     digit
        ora     #'0'
        jmp     disp_putc
digit:  sty     DISP_TMP+1
        ldy     #0
:       cmp     DISP_TMP+1
        bcc     :+
        sbc     DISP_TMP+1
        iny
        bne     :-
:       pha
        tya
        bne     show
        cpx     #0
        beq     skip
show:   ora     #'0'
        jsr     disp_putc
        ldx     #1
skip:   pla
        rts
.endproc

; disp_rpl -- up to Y characters of the reply from offset X, to a NUL.
.proc disp_rpl
        sty     DISP_TMP+1
loop:   lda     FN_REPLY,x
        beq     done
        jsr     disp_putc
        inx
        dec     DISP_TMP+1
        bne     loop
done:   rts
.endproc

        .segment "DLIST"

dlist:  .byte   $70, $70, $70
        .byte   $42, <SCREEN, >SCREEN
        .repeat 23
        .byte   $02
        .endrepeat
        .byte   $41, <dlist, >dlist

        .segment "FONT"

font:
        .include "font.inc"

```

```
; a52init.s -- start-up and interrupts for a FujiNet 5200 client.
;
; The header's $BFFD is $FF, so the BIOS jumps straight here with nothing
; set up. This does what the BIOS would have, without depending on which
; BIOS it is: the chips cleared, the RAM vectors at $0200-$021B, and a VBI
; and keypad handler that keep the BIOS's zero-page layout, so cc65's
; atari5200 library works unchanged on top of them.
;
; The console's NMI and IRQ vectors are the BIOS's, and both revisions
; enter through $FCA2 / $FC00 to JMP (VVBLKI) / JMP (VIMIRQ) with nothing
; pushed: these handlers save what they use.

        .include "fujinet.inc"

        .import main
        .export start

HOLD_FRAMES = 4

        .segment "STARTUP"

.proc start
        sei
        cld
        ldx     #$FF
        txs
        lda     #0
        sta     NMIEN
        sta     DMACTL
        sta     IRQEN
        tax
clr:    sta     POKEY,x         ; every chip register to 0, as the BIOS does
        sta     GTIA,x
        sta     ANTIC,x
        sta     $00,x           ; and the zero page
        sta     $0200,x
        inx
        bne     clr
        ldx     #vecs_end - vecs - 1
vec:    lda     vecs,x
        sta     VIMIRQ,x
        dex
        bpl     vec
        lda     #0              ; no key held
        sta     hold
        lda     #$02            ; keypad scan on
        sta     SKCTL
        lda     #$40            ; its IRQ, alone
        sta     POKMSK
        sta     IRQEN
        sta     NMIEN           ; the VBI, no DLIs
        cli
        jmp     main
.endproc

        .segment "CODE"

vecs:   .word   irq, vbi, vbi_exit, dli, kbd, kbd_done
        .word   pla_rti, pla_rti, pla_rti, pla_rti, pla_rti, pla_rti, pla_rti
vecs_end:

; The BIOS's $FCB8, register for register.
.proc vbi
        pha
        txa
        pha
        tya
        pha
        inc     RTCLOKL
        bne     :+
        inc     ATRACT
        inc     RTCLOKH
:       lda     hold            ; a held key's repeats keep this above zero
        beq     :+
        dec     hold
:       lda     CRITIC
        bne     vbi_exit
        lda     SDLSTH
        sta     DLISTH
        lda     SDLSTL
        sta     DLISTL
        lda     SDMCTL
        sta     DMACTL
        ldy     ATRACT
        bpl     :+
        ldy     #$80
        sty     ATRACT
:       ldx     #8
col:    lda     PCOLR0,x
        cpy     #$80
        bcc     :+
        eor     RTCLOKH
        and     #$F6
:       sta     COLPM0,x
        dex
        bpl     col
        ldx     #7
pot:    lda     POT0,x
        sta     PADDL0,x
        dex
        bpl     pot
        sta     POTGO
        jmp     (VVBLKD)
.endproc

.proc vbi_exit
        pla
        tay
        pla
        tax
        pla
        rti
.endproc

dli:    rti

; Only the keypad's IRQ is ever enabled; anything else is acknowledged.
.proc irq
        pha
        lda     IRQST
        and     #$40
        bne     other
        lda     #$BF
        sta     IRQEN
        lda     POKMSK
        sta     IRQEN
        jmp     (VKYBDI)
other:  lda     #0
        sta     IRQEN
        lda     POKMSK
        sta     IRQEN
        pla
        rti
.endproc

pla_rti:
        pla
        rti

; The BIOS's $FD02: KBCODE through its table, then VKYBDF.
.proc kbd
        txa
        pha
        tya
        pha
        lda     KBCODE
        lsr     a
        and     #$0F
        tax
        lda     keys,x
        jmp     (VKYBDF)
.endproc

; 0-9, $0A '*', $0B '#', $0C START, $0D PAUSE, $0E RESET; KEY = code + 1.
; Without debounce (SKCTL $02, the BIOS's) POKEY interrupts on every scan of
; a held key, and SKSTAT's key-down bit drops between scans: a repeat of the
```

```
157 ; key while `hold` runs only restarts it, so
    each press counts once.
158 .proc kbd_done
159         ldx     #HOLD_FRAMES
160         cmp     held
161         bne     @new
162         ldy     hold
163         stx     hold
164         cpy     #0
165         bne     @same
166 @new:   stx     hold
167         sta     held
168         clc
169         adc     #1
170         sta     KEY
171         lda     #0
172         sta     ATRACT
173 @same:  jmp     vbi_exit
174 .endproc
175
176         .segment "BSS"
177 held:   .res    1               ; the last key
178 hold:   .res    1               ; frames until
    it may count again
179
180         .segment "CODE"
181
182 keys:   .byte   $FF, $0B, $00, $0A, $0E, $09,
    $08, $07
183         .byte   $0D, $06, $05, $04, $0C, $03,
    $02, $01
184
```

a52.cfg

```
1  # a52.cfg -- a 32K FujiNet client: ROM at
     $4000-$BFFF around the cart's
2  # mailbox arena ($B000-$B7FF, served by the
     cart), the claim at $BFE0 and
3  # the BIOS header at $BFE8. RAM: the zero page
     above the BIOS's shadows,
4  # $0300-$0FFF, the screen at $1000.
5  MEMORY {
6      ZP:    file = "", start = $0019, size =
     $00E7;
7      RAM:   file = "", start = $0300, size =
     $0D00;
8      ROM:   file = %O, start = $4000, size =
     $7000, fill = yes, fillval = $FF;
9      ARENA: file = %O, start = $B000, size =
     $0800, fill = yes, fillval = $FF;
10     ROM2:  file = %O, start = $B800, size =
     $07E0, fill = yes, fillval = $FF;
11     CLAIM: file = %O, start = $BFE0, size =
     $0008, fill = yes, fillval = $00;
12     HDR:   file = %O, start = $BFE8, size =
     $0018;
13 }
14 SEGMENTS {
15     ZEROPAGE: load = ZP,    type = zp,
     optional = yes;
16     BSS:      load = RAM,   type = bss,
     optional = yes;
17     STARTUP:  load = ROM,   type = ro;
18     CODE:     load = ROM,   type = ro;
19     RODATA:   load = ROM,   type = ro,
     optional = yes;
20     DLIST:    load = ROM,   type = ro,  align
     = $100;
21     FONT:     load = ROM,   type = ro,  align
     = $400;
22     CLAIM:    load = CLAIM, type = ro;
23     HEADER:   load = HDR,   type = ro;
24 }
25
```

a52bank.cfg

```
1  # a52bank.cfg -- bank 1 (the last, the power-
     on bank) of a 64K Super Cart
2  # FujiNet client. Every bank is this image
     except $A000-$AFFF, which build.sh
3  # fills from bank0.s, so code anywhere else is
     common to both and a bank
4  # switch can happen mid-routine. $BFC0-$BFFF
     are the Super Cart's hotspots:
5  # nothing but the claim and the header lives
     there.
6  MEMORY {
```

```
    ZP:    file = "", start = $0019, size = $00E7;
    RAM:   file = "", start = $0300, size = $0D00;
    ROM:   file = %O, start = $4000, size = $6000, fill = yes, fillval = $FF;
    BANKA: file = %O, start = $A000, size = $1000, fill = yes, fillval = $FF;
    ARENA: file = %O, start = $B000, size = $0800, fill = yes, fillval = $FF;
    ROM2:  file = %O, start = $B800, size = $07C0, fill = yes, fillval = $FF;
    HOT:   file = %O, start = $BFC0, size = $0020, fill = yes, fillval = $FF;
    CLAIM: file = %O, start = $BFE0, size = $0008, fill = yes, fillval = $00;
    HDR:   file = %O, start = $BFE8, size = $0018;
}
SEGMENTS {
    ZEROPAGE: load = ZP,    type = zp, optional = yes;
    BSS:      load = RAM,   type = bss, optional = yes;
    STARTUP:  load = ROM,   type = ro;
    CODE:     load = ROM,   type = ro;
    RODATA:   load = ROM,   type = ro, optional = yes;
    DLIST:    load = ROM,   type = ro,  align = $100;
    FONT:     load = ROM,   type = ro,  align = $400;
    BANKCODE: load = BANKA, type = ro;
    CLAIM:    load = CLAIM, type = ro;
    HEADER:   load = HDR,   type = ro;
}

```

bank0.cfg

```
# bank0.cfg -- bank 0's own 4K of a 64K Super Cart client, at $A000.
MEMORY {
    ZP:    file = "", start = $0019, size = $00E7;
    BANKA: file = %O, start = $A000, size = $1000, fill = yes, fillval = $FF;
}
SEGMENTS {
    ZEROPAGE: load = ZP,    type = zp, optional = yes;
    BANKCODE: load = BANKA, type = ro;
}

```

devdefs.inc

```
; devdefs.inc -- the other devices, and the FUJI commands the bring-up's
; fujinet.inc does not name. Values from include/fujiDeviceID.h and
; include/fujiCommandID.h; the appkey struct from lib/device/fujiDevice/
; AppKeyMixin.h. Kept apart from fujinet.inc so that file stays verbatim.

; ---------------- device ids ----------------
DSKDEV  =       $31             ; D1: (D2:-D8: are $32-$38)
PRNDEV  =       $40             ; P1:
CLKDEV  =       $45             ; the clock (APETIME)
MDMDEV  =       $50             ; R: the modem
CPMDEV  =       $5A             ; CP/M

; ---------------- FUJI ($70) commands ----------------
FNCRST  =       $FF             ; RESET
FNCGSSD =       $FE             ; GET_SSID
FNCSCAN =       $FD             ; SCAN_NETWORKS
FNCSCNR =       $FC             ; GET_SCAN_RESULT
FNCSSSD =       $FB             ; SET_SSID
FNCWIFI =       $FA             ; GET_WIFISTATUS
FNCRHST =       $F4             ; READ_HOST_SLOTS
FNCWHST =       $F3             ; WRITE_HOST_SLOTS
FNCRDEV =       $F2             ; READ_DEVICE_SLOTS
FNCWDEV =       $F1             ; WRITE_DEVICE_SLOTS
FNCWIEN =       $EA             ; GET_WIFI_ENABLED
FNCODIR =       $F7             ; OPEN_DIRECTORY
FNCRDIR =       $F6             ; READ_DIR_ENTRY
FNCCDIR =       $F5             ; CLOSE_DIRECTORY
FNCSDPS =       $E4             ; SET_DIRECTORY_POSITION
FNCUIMG =       $E9             ; UNMOUNT_IMAGE
FNCADPC =       $E8             ; GET_ADAPTERCONFIG
FNCNEWD =       $E7             ; NEW_DISK
FNCUHST =       $E6             ; UNMOUNT_HOST
```

```
FNCGDPS =       $E5             ;
  GET_DIRECTORY_POSITION
FNCSHPX =       $E1             ;
  SET_HOST_PREFIX
FNCGHPX =       $E0             ;
  GET_HOST_PREFIX
FNCAKWR =       $DE             ; WRITE_APPKEY
FNCAKRD =       $DD             ; READ_APPKEY
FNCAKOP =       $DC             ; OPEN_APPKEY
FNCAKCL =       $DB             ; CLOSE_APPKEY
FNCGDFP =       $DA             ;
  GET_DEVICE_FULLPATH
FNCCBOT =       $D9             ; CONFIG_BOOT
FNCCOPY =       $D8             ; COPY_FILE
FNCMALL =       $D7             ; MOUNT_ALL
FNCSBM  =       $D6             ;
  SET_BOOT_MODE
FNCRAND =       $D3             ;
  RANDOM_NUMBER
FNCB64EI =      $D0             ;
  BASE64_ENCODE_INPUT
FNCB64EC =      $CF             ;
  BASE64_ENCODE_COMPUTE
FNCB64EL =      $CE             ;
  BASE64_ENCODE_LENGTH
FNCB64EO =      $CD             ;
  BASE64_ENCODE_OUTPUT
FNCB64DI =      $CC             ;
  BASE64_DECODE_INPUT
FNCB64DC =      $CB             ;
  BASE64_DECODE_COMPUTE
FNCB64DL =      $CA             ;
  BASE64_DECODE_LENGTH
FNCB64DO =      $C9             ;
  BASE64_DECODE_OUTPUT
FNCHSHI =       $C8             ; HASH_INPUT
FNCHSHC =       $C7             ; HASH_COMPUTE
FNCHSHL =       $C6             ; HASH_LENGTH
FNCHSHO =       $C5             ; HASH_OUTPUT
FNCHSHN =       $C3             ;
  HASH_COMPUTE_NO_CLEAR
FNCHSHX =       $C2             ; HASH_CLEAR
FNCQRO  =       $BF             ;
  QRCODE_OUTPUT
FNCQRL  =       $BE             ;
  QRCODE_LENGTH
FNCQRE  =       $BD             ;
  QRCODE_ENCODE
FNCQRI  =       $BC             ; QRCODE_INPUT
FNCGUID =       $BB             ;
  GENERATE_GUID
FNCSTAT =       $53             ; STATUS
FNCRDY  =       $00             ; DEVICE_READY

; ---------------- appkeys ----------------
; OPEN_APPKEY's payload is ONE six-byte packed
  struct: creator (u16, little-
; endian), app, key, mode, and a reserved byte
  that is not optional.
AKMRD   =       0               ; mode: read
AKMWR   =       1               ; mode: write
AKCREAT =       1               ; the shared
  FujiNet username slot...
AKAPP   =       1               ; ...creator
  1, app 1, key 0, which every
AKKEY   =       0               ; game in the
  family reads

; ---------------- the clock ($45)
  ----------------
CKGTIME =       $93             ; GETTIME: 7
  bytes, binary
CKISOL  =       $49             ; 'I'
  GET_ISO_LOCAL: "YYYY-MM-DDTHH:MM:SS"...
CKISOU  =       $5A             ; 'Z'
  GET_ISO_UTC
CKGEN   =       $47             ; 'G'
  GET_GENERAL
CKTZLEN =       $4C             ; 'L'
  GETTZ_LEN
CKSETTZ =       $99             ; SETTZ:
  payload = the POSIX TZ string
CKTZTIM =       $9A             ; GETTZTIME

; ---------------- hash algorithms
  (HASH_COMPUTE's parameter) ----------------
HAMD5   =       0
HASHA1  =       1
HASHA256 =      3
HASHA512 =      4


; ---------------- FNBEGIN: the four stores
  before fn_beg ----------------
; FNBEGIN dev, cmd, nparam -- set up and begin
  a transaction (fn_beg).
.macro  FNBEGIN dev, cmd, npr
        lda     #dev
        sta     FN_DEV
        lda     #cmd
        sta     FN_CMD
        lda     #npr
        sta     FN_NPR
        jsr     fn_beg
.endmacro


```

```
; netdefs.inc -- the N: (network) device, for the client library.
;
; Device ids from include/fujiDeviceID.h, command bytes from
; include/fujiCommandID.h, open modes from lib/network-protocol/HTTP.cpp and
; the dispatch table and STATUS reply from lib/device/NDevice/NDevice.cpp.
; The bring-up's fujinet.inc covers the FUJI device only; this is the rest of
; what a client that talks to the network needs, kept separate so that file
; stays verbatim.
NETDEV  =       $71             ; N1: (N2:-N8: are $72-$78)
NCOPEN  =       $4F             ; 'O' open: params mode, translation; payload = devicespec
NCCLOSE =       $43             ; 'C' close
NCREAD  =       $52             ; 'R' read: ONE two-byte param, the count
NCWRITE =       $57             ; 'W' write: ONE two-byte param, then the payload
NCSTAT  =       $53             ; 'S' status: two one-byte params, both 0
NCPARSE =       $50             ; 'P' parse the reply as JSON
NCQUERY =       $51             ; 'Q' query a JSON path; payload = the path
NCPRSR  =       $FC             ; SET_PARSER: second param 0 none, 1 JSON, 2 HTML, 3 XML
NCJSON  =       NCPRSR
NCSETP  =       $FB             ; SET_PARAMETER: params type (0 query flags, 1 EOL), value
NCTRANS =       $54             ; 'T' translation: second param the code
NCEOL   =       $4C             ; 'L' set the EOL: payload the string
NCRATE  =       $5A             ; 'Z' interrupt rate: param the rate
NCSEEK  =       $25             ; '%' seek: ONE four-byte param, the offset
NCTELL  =       $26             ; '&' tell: reply 4 bytes
NCREN   =       $20             ; rename: params mode, 0; payload "N:...old,new"
NCDEL   =       $21             ; '!' delete \
NCLOCK  =       $23             ; '#' lock |
NCUNLK  =       $24             ; '$' unlock | params mode, 0;
NCMKDIR =       $2A             ; '*' mkdir | payload the devicespec
NCRMDIR =       $2B             ; '+' rmdir       /
NCCHDIR =       $2C             ; ',' chdir: payload the path
NCGETCW =       $30             ; '0' getcwd: reply the prefix
NCACCPT =       $41             ; 'A' accept a TCP client
NCCCLI  =       $63             ; 'c' close the TCP client
NCRMT   =       $72             ; 'r' UDP: who sent the last datagram
NCDEST  =       $44             ; 'D' UDP: payload host:port
NCUSER  =       $FD             ; payload the user name
NCPASS  =       $FE             ; payload the password
NPNONE  =       0               ; parsers
NPJSON  =       1
NPHTML  =       2
NPXML   =       3
NCHTTPM =       $4D             ; 'M' HTTP channel mode: second param 0 body, 1 collect
                                ;   headers, 2 get headers, 3 set headers, 4 POST data
NMREAD  =       4               ; open mode: read (HTTP GET)
NMWRITE =       8               ; write (HTTP PUT)
NMHTTPG =       12              ; read/write (HTTP GET, headers allowed)
NMPOST  =       13              ; HTTP POST
NTRNONE =       0               ; translation: none
NTRCR   =       1               ; CR to/from the host's EOL
NTRLF   =       2               ; LF
NTRCRLF =       3               ; CR LF
NSAVLO  =       0               ; STATUS reply: bytes waiting, low
NSAVHI  =       1               ;   high
NSCONN  =       2               ;   connected
NSDEVST =       3               ;   nDevStatus_t: 1 = success

```

book.inc

```
; book.inc -- what the handbook's own assembly programs share beyond the
; bring-up's fujinet.inc: their zero page, and the PUTS macro.
;
; a52.cfg starts the ZEROPAGE segment at $19, which is also KEY, the byte
; a52init.s's keypad handler writes. These programs leave the segment
; empty: variables go in BSS ($0300 up), and the few pointers that must be
; in zero page sit here, above fujilib's $E0-$ED.
PTR1    = $F0                   ; a pointer (2 bytes)
PTR2    = $F2                   ; another (2 bytes)
ZTMP    = $F4                   ; scratch (2 bytes)

; PUTS row, col, str -- the NUL-terminated string at str, from (row, col).
.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

; WAITFRAME -- until the VBI next counts a frame. Clobbers A.
.macro  WAITFRAME
        .local  w
        lda     RTCLOKL
w:      cmp     RTCLOKL
        beq     w
.endmacro

```

The assembly programs

hello.s

```
; hello.s -- M0: the cart serves a client the BIOS hands straight to, and it
; runs: text from a font in ROM, the VBI ticking, the keypad answering, and
; the cart's status page answering 'F','N'.

        .include "fujinet.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_on
        .import fn_chk
        .export main

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        jsr     disp_init
        PUTS    2, 2, title
        PUTS    4, 2, tcart
        jsr     fn_chk
        bne     nocart
        PUTS    4, 8, tyes
        jmp     on
nocart: PUTS    4, 8, tno
on:     PUTS    6, 2, tframe
        PUTS    8, 2, tkey
        jsr     disp_on
loop:   lda     #6
        ldx     #9
        jsr     disp_at
        lda     RTCLOKH
        jsr     disp_hex
        lda     RTCLOKL
        jsr     disp_hex
        lda     #8
        ldx     #9
        jsr     disp_at
        lda     KEY
        jsr     disp_hex
        jmp     loop
.endproc

        .segment "RODATA"
title:  .byte "FUJINET ATARI 5200 M0", 0
tcart:  .byte "CART", 0
tyes:   .byte "FN OK", 0
tno:    .byte "NO MAILBOX", 0
tframe: .byte "FRAME", 0
tkey:   .byte "KEY", 0

        .include "header.inc"

```

fujitest.s

```
; fujitest.s -- M1: one real transaction, and proof the cart survives a
; power cycle.
;
; Shows GET_ADAPTERCONFIG_EXTENDED's SSID, IP address and firmware version.
; emu/powertest.lua power-cycles the console: the cart starts the interlock
; over (ACKSEQ 0), and the next transaction must still go through.

        .include "fujinet.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_rpl, disp_on
        .import fn_chk, fn_beg, fn_go, fn_ack
        .export main

; Field offsets in the packed GET_ADAPTERCONFIG_EXTENDED reply, all below 256.
AC_SSID = 0                     ; char[33]
AC_VER  = 125                   ; char[15]
AC_SIP  = 140                   ; char[16], the IP already formatted as text

        .segment "ZEROPAGE"
aperr:  .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        jsr     disp_init
        lda     #0
        sta     aperr
        jsr     fn_chk
        beq     have
        lda     #FNENOC
        sta     aperr
        jmp     show

have:   lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        bne     show
        jsr     fn_ack
        sta     aperr

show:   PUTS    2, 2, title
        lda     aperr
        beq     ok

        PUTS    5, 2, tfail
        lda     #5
        ldx     #12
        jsr     disp_at
        lda     aperr
        jsr     disp_hex
        jmp     seq

ok:     PUTS    5, 2, tssid
        lda     #5
        ldx     #8
        jsr     disp_at
        ldx     #AC_SSID
        ldy     #30
        jsr     disp_rpl

        PUTS    7, 2, tip
        lda     #7
        ldx     #8
        jsr     disp_at
        ldx     #AC_SIP
        ldy     #16
        jsr     disp_rpl

        PUTS    9, 2, tver
        lda     #9
        ldx     #8
        jsr     disp_at
        ldx     #AC_VER
        ldy     #15
        jsr     disp_rpl

seq:    PUTS    12, 2, tseq
        lda     #12
        ldx     #10
        jsr     disp_at
        lda     FN_ACKSEQ
        jsr     disp_hex

        PUTS    14, 2, terr
        lda     #14
        ldx     #10
        jsr     disp_at
        lda     FN_ERR
        jsr     disp_hex

        jsr     disp_on
loop:   jmp     loop
.endproc

        .segment "RODATA"
title:  .byte "FUJINET ATARI 5200 M1", 0
tfail:  .byte "FAILED: E", 0
tssid:  .byte "SSID", 0
tip:    .byte "IP  ", 0
tver:   .byte "FW  ", 0
tseq:   .byte "ACKSEQ: ", 0
terr:   .byte "ERR:   ", 0

        .include "header.inc"
```

pads.s

```
; pads.s -- controller 1: the stick, both buttons and the keypad, and a
; tone from POKEY while the bottom button is held.
;
; The stick is two pots, which the VBI copies to PADDL0 (X) and PADDL1 (Y)
; every frame; they read nothing until CONSOL bit 2 turns them on. The
; bottom button is GTIA's TRIG0 (0 while held). The top button is POKEY's
; SKSTAT bit 3, clear while the top button of the controller CONSOL's bits
; 0-1 pick is held. The keypad is a52init.s's: KEY holds the last key + 1.

        .include "fujinet.inc"
        .include "book.inc"

        .import disp_init, disp_at, disp_str, disp_putc, disp_on
        .export main

TONE    = $50                   ; AUDF1: the pitch
VOICE   = $A8                   ; AUDC1: a pure tone at volume 8

        .segment "CODE"

.proc main
        jsr     disp_init
        lda     #$04            ; the pots on; controller 1's keypad and top button
        sta     CONSOL
        PUTS    2, 2, title
        PUTS    4, 2, tstx
        PUTS    5, 2, tsty
        PUTS    7, 2, tbot
        PUTS    8, 2, ttop
        PUTS    10, 2, tkey
        PUTS    12, 2, tsnd
        jsr     disp_on

loop:   WAITFRAME
        lda     #4              ; the stick, as the VBI last read it
        ldx     #16
        jsr     disp_at
        lda     PADDL0
        jsr     dec3
        lda     #5
        ldx     #16
        jsr     disp_at
        lda     PADDL0+1
        jsr     dec3

        lda     #7              ; the bottom button: TRIG0 is 0 while held
        ldx     #16
        jsr     disp_at
        lda     TRIG0
        jsr     updown
        lda     #8              ; the top button: SKSTAT bit 3 clear while held
        ldx     #16
        jsr     disp_at
        lda     SKSTAT
        and     #$08
        jsr     updown

        lda     #10             ; the last key
        ldx     #16
        jsr     disp_at
        ldx     KEY
        lda     knlo,x
        sta     FN_PTR
        lda     knhi,x
        sta     FN_PTR+1
        jsr     disp_str

        lda     #12             ; a tone while the bottom button is held
        ldx     #16
        jsr     disp_at
        lda     TRIG0
        bne     quiet
        lda     #TONE
        sta     AUDF1
        lda     #VOICE
        sta     AUDC1
        lda     #<ton
        ldx     #>ton
        jmp     said
quiet:  lda     #0
        sta     AUDC1
        lda     #<toff
        ldx     #>toff
said:   sta     FN_PTR
        stx     FN_PTR+1
        jsr     disp_str
        jmp     loop
.endproc

; updown -- "DOWN" if A is 0 (held), else "UP".
.proc updown
        cmp     #0
        bne     up
        lda     #<tdown
        ldx     #>tdown
        jmp     show
up:     lda     #<tup
        ldx     #>tup
show:   sta     FN_PTR
        stx     FN_PTR+1
        jmp     disp_str
.endproc

; dec3 -- A as three decimal digits, leading zeros as spaces, so a shorter
; number overwrites a longer one.
.proc dec3
        ldx     #0              ; hundreds
:       cmp     #100
        bcc     :+
        sbc     #100
        inx
        bne     :-
:       sta     ZTMP
        txa
        sta     ZTMP+1          ; nonzero once a digit has been shown
        jsr     digit
        lda     ZTMP
        ldx     #0              ; tens
:       cmp     #10
        bcc     :+
        sbc     #10
        inx
        bne     :-
:       sta     ZTMP
        txa
        ora     ZTMP+1
        sta     ZTMP+1
        txa
        jsr     digit
        lda     ZTMP            ; ones: always shown
        ora     #'0'
        jmp     disp_putc
digit:  ldy     ZTMP+1
        bne     :+
        lda     #' '
        jmp     disp_putc
:       ora     #'0'
        jmp     disp_putc
.endproc

        .segment "RODATA"
title:  .byte "CONTROLLER 1", 0
tstx:   .byte "STICK X", 0
tsty:   .byte "STICK Y", 0
tbot:   .byte "BOTTOM BUTTON", 0
ttop:   .byte "TOP BUTTON", 0
tkey:   .byte "KEYPAD", 0
tsnd:   .byte "SOUND", 0
tup:    .byte "UP ", 0
tdown:  .byte "DOWN", 0
ton:    .byte "ON ", 0
toff:   .byte "OFF", 0

; KEY is the key's code + 1: 0 none, then 0-9, *, #, START, PAUSE, RESET.
.define KNAMES kn0, kd0, kd1, kd2, kd3, kd4, kd5, kd6, kd7, kd8, kd9, kstar, khash, kstart, kpause, kreset
knlo:   .lobytes KNAMES
```

```
knhi:   .hibytes KNAMES
kn0:    .byte "-    ", 0
kd0:    .byte "0    ", 0
kd1:    .byte "1    ", 0
kd2:    .byte "2    ", 0
kd3:    .byte "3    ", 0
kd4:    .byte "4    ", 0
kd5:    .byte "5    ", 0
kd6:    .byte "6    ", 0
kd7:    .byte "7    ", 0
kd8:    .byte "8    ", 0
kd9:    .byte "9    ", 0
kstar:  .byte "*    ", 0
khash:  .byte "#    ", 0
kstart: .byte "START", 0
kpause: .byte "PAUSE", 0
kreset: .byte "RESET", 0

        .include "header.inc"
```

netget.s

```
; netget.s -- fetch a URL through the N: device and show the text.
;
; OPEN the devicespec as an HTTP GET, STATUS until the byte count settles,
; READ what is waiting (at most 512 bytes) into the reply window, show it a
; line at a time -- LF ends a row, CR is skipped, a row is at most 36
; characters -- then CLOSE.

        .include "fujinet.inc"
        .include "netdefs.inc"
        .include "book.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_on, disp_putc
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_pw, fn_txb
        .export main

NMAX    = 512                   ; the most we read
NROW0   = 4                     ; the first text row of the reply
NROWS   = 18                    ; rows 4-21
NCOLS   = 36
NTRIES  = 30                    ; STATUS readings before giving up

        .segment "BSS"
aperr:  .res 1                  ; what went wrong, if anything
navail: .res 2                  ; bytes waiting, from STATUS
nprev:  .res 2                  ; the previous STATUS reading
ntry:   .res 1
nleft:  .res 2                  ; reply bytes still to show
nrow:   .res 1
ncol:   .res 1

nsrc    = PTR1                  ; the reply cursor: read through, so zero page

        .segment "CODE"

.proc main
        jsr     disp_init
        PUTS    2, 2, title
        jsr     disp_on
        lda     #0
        sta     aperr
        jsr     fn_chk
        beq     open
        lda     #FNENOC
        sta     aperr
        jmp     fail

        ; OPEN: two one-byte parameters, the mode and the translation, then
        ; the devicespec as the payload
open:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCOPEN
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NMHTTPG
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldx     #0                  ; X: fn_txb takes its byte in Y
@url:   lda     url,x
        beq     @urle
        jsr     fn_txb
        inx
        bne     @url
@urle:  jsr     go_ack

        ; STATUS until two readings agree and are not zero: the adapter
        ; answers as soon as SOME of the response has arrived
        lda     #$FF
        sta     nprev
        sta     nprev+1
        lda     #0
        sta     ntry
stat:   lda     #NETDEV
        sta     FN_DEV
        lda     #NCSTAT
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #0
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY+NSAVLO
        sta     navail
        lda     FN_REPLY+NSAVHI
        sta     navail+1
        cmp     nprev+1
        bne     again
        lda     navail
        cmp     nprev
        bne     again
        ora     navail+1
        bne     read
again:  lda     navail
        sta     nprev
        lda     navail+1
        sta     nprev+1
        inc     ntry
        lda     ntry
        cmp     #NTRIES
        bcc     @wait
        lda     #FNEWAIT
        sta     aperr
        jmp     fail
@wait:  ldx     #3                  ; three frames between readings
@w:     lda     RTCLOKL
@f:     cmp     RTCLOKL
        beq     @f
```

```
113         dex
114         bne     @w
115         jmp     stat
116
117         ; READ: ONE two-byte parameter, the
    count, capped at NMAX
118 read:   lda     navail+1
119         cmp     #>NMAX
120         bcc     @len
121         lda     #<NMAX
122         sta     navail
123         lda     #>NMAX
124         sta     navail+1
125 @len:   sta     #NETDEV
126         sta     FN_DEV
127         lda     #NCREAD
128         sta     FN_CMD
129         lda     #1
130         sta     FN_NPR
131         jsr     fn_beg
132         lda     navail
133         ldx     navail+1
134         jsr     fn_pw
135         jsr     go_ack
136
137         ; show it, a row per line
138         lda     #<FN_REPLY
139         sta     nsrc
140         lda     #>FN_REPLY
141         sta     nsrc+1
142         lda     FN_RXLEN
143         sta     nleft
144         lda     FN_RXLEN+1
145         sta     nleft+1
146         lda     #NROW0
147         sta     nrow
148         jsr     newrow
149 next:   lda     nleft
150         ora     nleft+1
151         beq     close
152         ldy     #0
153         lda     (nsrc),y
154         inc     nsrc
155         bne     @s
156         inc     nsrc+1
157 @s:     ldx     nleft
158         bne     @n
159         dec     nleft+1
160 @n:     dec     nleft
161         cmp     #13             ; CR: skip
162         beq     next
163         cmp     #10             ; LF: next row
164         bne     @c
165         inc     nrow
166         lda     nrow
167         cmp     #NROW0+NROWS
168         beq     close
169         jsr     newrow
170         jmp     next
171 @c:     ldx     ncol
172         cpx     #NCOLS
173         beq     next            ; past the
    edge: drop it
174         inc     ncol
175         jsr     disp_putc
176         jmp     next
177
178         ; CLOSE: no parameters
179 close:  lda     #NETDEV
180         sta     FN_DEV
181         lda     #NCCLOSE
182         sta     FN_CMD
183         lda     #0
184         sta     FN_NPR
185         jsr     fn_beg
186         jsr     go_ack
187         PUTS    22, 2, tdone
188 loop:   jmp     loop
189 .endproc
190
191 ; newrow -- the cursor to column 2 of row
    nrow.
192 .proc newrow
193         lda     #0
194         sta     ncol
195         lda     nrow
196         ldx     #2
197         jmp     disp_at
198 .endproc
199
200 ; go_ack -- fn_go, then fn_ack; on any failure
    the code goes in aperr and
201 ; the failure screen follows, so callers need
    no branch.
202 .proc go_ack
203         jsr     fn_go
204         sta     aperr
205         bne     @bad
206         jsr     fn_ack
207         sta     aperr
208         bne     @bad
209         rts
210 @bad:   pla                     ; drop our
    caller's return
211         pla
212         jmp     fail
213 .endproc
214
215 ; fail -- FAILED and the code. Does not
    return.
216 .proc fail
217         PUTS    22, 2, tfail
218         lda     aperr
219         jsr     disp_hex
220 @h:     jmp     @h
221 .endproc
222
223         .segment "RODATA"
224 url:    .byte "N:http://127.0.0.1:8768/
    hello.txt", 0
225 title:  .byte "NETGET", 0
226 tdone:  .byte "CLOSED", 0
227 tfail:  .byte "FAILED: E", 0
228
229         .include "header.inc"
230
```

```
; netcat.s -- talk to a TCP server a line at a time.
;
; OPEN the devicespec read/write, then every frame: the stick moves round an
; on-screen keyboard and the bottom button types its key; the keypad's
; digits type themselves, * rubs out, # sends the line with CR LF (a
; WRITE), and START closes the connection. Every eighth frame a STATUS
; asks what is waiting, and a READ fetches it (at most 128 bytes) into
; rows 3-14, which scroll.

        .include "fujinet.inc"
        .include "netdefs.inc"
        .include "devdefs.inc"
        .include "book.inc"

        .import disp_init, disp_at, disp_str, disp_putc, disp_hex, disp_on
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_pw, fn_txb
        .export main

ROW0    = 3                     ; the received text: rows 3-14
NROWS   = 12
NCOLS   = 36
LROW    = 16                    ; the line being typed, after "> "
LMAX    = 34
KROW    = 18                    ; the keyboard: rows 18-20, 13 keys a row
KCOLS   = 13
NMAX    = 128                   ; the most one READ asks for
LO      = $40                   ; the stick's dead zone: $40-$A4
HI      = $A4

        .segment "BSS"
aperr:  .res 1
rrow:   .res 1                  ; where received text goes next
rcol:   .res 1
llen:   .res 1                  ; the typed line
line:   .res LMAX + 2           ; ...with room for CR LF
cx:     .res 1                  ; the keyboard's cursor
cy:     .res 1
latch:  .res 1                  ; the stick moved; wait for the centre
fired:  .res 1                  ; the bottom button was down last frame
navail: .res 2
nleft:  .res 2

        .segment "CODE"

.proc main
        jsr     disp_init
        lda     #$04            ; the pots on; controller 1
        sta     CONSOL
        lda     #0
        sta     rcol
        sta     llen
        sta     cx
        sta     cy
        sta     latch
        sta     fired
        sta     KEY
        lda     #ROW0
        sta     rrow
        PUTS    1, 2, title
        PUTS    LROW, 2, tprompt
        PUTS    22, 1, thelp
        jsr     keyboard
        jsr     cursor          ; the cursor on A
        jsr     disp_on
        jsr     fn_chk
        beq     open
        lda     #FNENOC
        sta     aperr
        jmp     fail

        ; OPEN: mode read/write, no translation; the devicespec follows
open:   FNBEGIN NETDEV, NCOPEN, 2
        lda     #NMHTTPG        ; 12: read and write
        jsr     fn_pb
        lda     #NTRNONE
        jsr     fn_pb
        ldx     #0              ; X: fn_txb takes its byte in Y
@url:   lda     url,x
        beq     @go
        jsr     fn_txb
        inx
        bne     @url
@go:    jsr     go_ack

loop:   WAITFRAME
        jsr     stick
        jsr     button
        jsr     keypad
        lda     RTCLOKL         ; every eighth frame, see what has come
        and     #7
        bne     loop
        jsr     receive
        jmp     loop
.endproc

; ---- the keyboard ----------------------------------------------------

; keyboard -- draw the three rows, a key every other column from column 2.
.proc keyboard
        lda     #0
        sta     ZTMP            ; the row
@row:   lda     ZTMP
        clc
        adc     #KROW
        ldx     #2
        jsr     disp_at
        lda     ZTMP
        lda     krow,x
        tax                     ; X: the row's first key in keys
        ldy     #KCOLS
@key:   lda     keys,x
        jsr     disp_putc
        lda     #' '
        jsr     disp_putc
        inx
        dey
        bne     @key
        inc     ZTMP
        lda     ZTMP
        cmp     #3
        bne     @row
        rts
.endproc

; cursor -- turn the key under the cursor to reverse video, or back.
.proc cursor
        lda     cy
        clc
        adc     #KROW
        pha
        lda     cx
        asl     a               ; a key every other column
        adc     #2
        tax
        pla
        jsr     disp_at
        ldy     #0
        lda     (DISP_PTR),y
        eor     #$80
        sta     (DISP_PTR),y
        rts
.endproc

; stick -- one step per push: move, then wait for the stick to come back
; to the centre.
.proc stick
        lda     PADDL0
```

```
148         ldx     PADDL0+1
149         ldy     latch
150         beq     free
151         cmp     #LO             ; latched:
    free again once both are centred
152         bcc     done
153         cmp     #HI+1
154         bcs     done
155         cpx     #LO
156         bcc     done
157         cpx     #HI+1
158         bcs     done
159         lda     #0
160         sta     latch
161 done:   rts
162 free:   cmp     #LO
163         bcc     left
164         cmp     #HI+1
165         bcs     right
166         cpx     #LO
167         bcc     up
168         cpx     #HI+1
169         bcs     down
170         rts
171 left:   jsr     cursor
172         dec     cx
173         bpl     moved
174         lda     #KCOLS-1
175         sta     cx
176         bne     moved
177 right:  jsr     cursor
178         inc     cx
179         lda     cx
180         cmp     #KCOLS
181         bne     moved
182         lda     #0
183         sta     cx
184         beq     moved
185 up:     jsr     cursor
186         dec     cy
187         bpl     moved
188         lda     #2
189         sta     cy
190         bne     moved
191 down:   jsr     cursor
192         inc     cy
193         lda     cy
194         cmp     #3
195         bne     moved
196         lda     #0
197         sta     cy
198 moved:  lda     #1
199         sta     latch
200         jmp     cursor
201 .endproc
202
203 ; button -- the bottom button types the key
    under the cursor, once a press.
204 .proc button
205         lda     TRIG0           ; 0 while held
206         bne     up
207         lda     fired
208         bne     done
209         inc     fired
210         ldx     cy
211         lda     krow,x
212         clc
213         adc     cx
214         tax
215         lda     keys,x
216         cmp     #'_'            ; the key that
    types a space
217         bne     :+
218         lda     #' '
219 :       jmp     type
220 up:     lda     #0
221         sta     fired
222 done:   rts
223 .endproc
224
225 ; keypad -- digits type, * rubs out, # sends,
    START hangs up.
226 .proc keypad
227         lda     KEY             ; the last key
    + 1, 0 = none
228         beq     done
229         ldx     #0
230         stx     KEY             ; taken
231         sec
232         sbc     #1
233         cmp     #10
234         bcs     :+
235         ora     #'0'            ; 0-9
236         jmp     type
237 :       cmp     #$0A            ; *
238         beq     rub
239         cmp     #$0B            ; #
240         bne     :+
241         jmp     send
242 :       cmp     #$0C            ; START
243         bne     done
244         jmp     hangup
245 done:   rts
246 .endproc
247
248 ; type -- ASCII A onto the end of the line.
249 .proc type
250         ldx     llen
251         cpx     #LMAX
252         beq     full
253         sta     line,x
254         inc     llen
255         pha
256         txa
257         clc
258         adc     #4              ; after "> "
259         tax
260         lda     #LROW
261         jsr     disp_at
262         pla
263         jmp     disp_putc
264 full:   rts
265 .endproc
266
267 .proc rub
268         ldx     llen
269         beq     done
270         dex
271         stx     llen
272         txa
273         clc
274         adc     #4
275         tax
276         lda     #LROW
277         jsr     disp_at
278         lda     #' '
279         jmp     disp_putc
280 done:   rts
281 .endproc
282
283 ; ---- the network
    ------------------------------------------------------------

284
285 ; send -- WRITE the line and CR LF: one two-
    byte parameter, the length,
286 ; then the bytes. Then clear the line.
287 .proc send
288         ldx     llen
289         lda     #13
290         sta     line,x
291         lda     #10
292         sta     line+1,x
293         inx
294         inx
295         stx     ZTMP
296         FNBEGIN NETDEV, NCWRITE, 1
297         lda     ZTMP
298         ldx     #0
299         jsr     fn_pw
300         ldx     #0
301 @b:     lda     line,x
302         jsr     fn_txb
303         inx
304         cpx     ZTMP
305         bne     @b
306         jsr     go_ack
307         lda     #0
308         sta     llen
309         lda     #LROW
310         ldx     #4
311         jsr     disp_at
312         ldx     #LMAX
313 @clr:   lda     #' '
314         jsr     disp_putc
315         dex
316         bne     @clr
317         rts
318 .endproc
319
320 ; receive -- STATUS; if anything is waiting,
    READ it (at most NMAX) and
321 ; show it.
322 .proc receive
323         FNBEGIN NETDEV, NCSTAT, 2
324         lda     #0
325         jsr     fn_pb
```

```
326         lda     #0
327         jsr     fn_pb
328         jsr     go_ack
329         lda     FN_REPLY+NSAVLO
330         sta     navail
331         lda     FN_REPLY+NSAVHI
332         sta     navail+1
333         ora     navail
334         beq     done
335         lda     navail+1        ; at most NMAX
336         bne     cap
337         lda     navail
338         cmp     #NMAX+1
339         bcc     read
340 cap:    lda     #NMAX
341         sta     navail
342         lda     #0
343         sta     navail+1
344 read:   FNBEGIN NETDEV, NCREAD, 1
345         lda     navail
346         ldx     navail+1
347         jsr     fn_pw
348         jsr     go_ack
349         lda     FN_RXLEN        ; the reply is
    at most NMAX: one byte
350         sta     nleft
351         lda     #<FN_REPLY
352         sta     PTR1
353         lda     #>FN_REPLY
354         sta     PTR1+1
355 next:   lda     nleft
356         beq     done
357         dec     nleft
358         ldy     #0
359         lda     (PTR1),y
360         inc     PTR1
361         bne     :+
362         inc     PTR1+1
363 :       jsr     show
364         jmp     next
365 done:   rts
366 .endproc
367
368 ; show -- one received byte: CR skipped, LF a
    new row, the rest printed.
369 .proc show
370         cmp     #13
371         beq     done
372         cmp     #10
373         beq     newrow
374         pha
375         lda     rcol
376         clc
377         adc     #2
378         tax
379         lda     rrow
380         jsr     disp_at
381         pla
382         jsr     disp_putc
383         inc     rcol
384         lda     rcol
385         cmp     #NCOLS
386         beq     newrow
387 done:   rts
388 .endproc
389
390 ; newrow -- the next row; past the last,
    scroll rows 4-14 up one.
391 .proc newrow
392         lda     #0
393         sta     rcol
394         inc     rrow
395         lda     rrow
396         cmp     #ROW0+NROWS
397         bne     done
398         dec     rrow
399         lda     #<(SCREEN+(ROW0+1)*40)
400         sta     PTR1
401         lda     #>(SCREEN+(ROW0+1)*40)
402         sta     PTR1+1
403         lda     #<(SCREEN+ROW0*40)
404         sta     PTR2
405         lda     #>(SCREEN+ROW0*40)
406         sta     PTR2+1
407         ldx     #NROWS-1
408 @row:   ldy     #39
409 @b:     lda     (PTR1),y
410         sta     (PTR2),y
411         dey
412         bpl     @b
413         lda     PTR1
414         sta     PTR2
415         lda     PTR1+1
416         sta     PTR2+1
417         lda     PTR1
418         clc
419         adc     #40
420         sta     PTR1
421         bcc     :+
422         inc     PTR1+1
423 :       dex
424         bne     @row
425         ldy     #39             ; PTR2 is the
    last row now: blank it
426         lda     #0              ; ANTIC's code
    for a space
427 @clr:   sta     (PTR2),y
428         dey
429         bpl     @clr
430 done:   rts
431 .endproc
432
433 ; hangup -- CLOSE, say so, and stop.
434 .proc hangup
435         FNBEGIN NETDEV, NCCLOSE, 0
436         jsr     go_ack
437         PUTS    LROW, 2, tclosed
438 @h:     jmp     @h
439 .endproc
440
441 ; go_ack -- fn_go, then fn_ack; on any failure
    the code goes in aperr and
442 ; the failure screen follows, so callers need
    no branch.
443 .proc go_ack
444         jsr     fn_go
445         sta     aperr
446         bne     @bad
447         jsr     fn_ack
448         sta     aperr
449         bne     @bad
450         rts
451 @bad:   pla                     ; drop our
    caller's return
452         pla
453         jmp     fail
454 .endproc
455
456 ; fail -- FAILED and the code. Does not
    return.
457 .proc fail
458         PUTS    LROW, 2, tfail
459         lda     aperr
460         jsr     disp_hex
461 @h:     jmp     @h
462 .endproc
463
464         .segment "RODATA"
465 url:    .byte "N:TCP://127.0.0.1:8769/", 0
466 title:  .byte "NETCAT  127.0.0.1:8769", 0
467 tprompt: .byte "> ", 0
468 thelp:  .byte "FIRE TYPES  # SENDS  * RUBS
    START ENDS", 0
469 tclosed: .byte "CLOSED
    ", 0
470 tfail:  .byte "FAILED: E", 0
471 keys:   .byte "ABCDEFGHIJKLM"
472         .byte "NOPQRSTUVWXYZ"
473         .byte "_.,:/?!-'@()="       ; _ types
    a space
474 krow:   .byte 0, KCOLS, 2*KCOLS
475
476         .include "header.inc"
477
```

fujidir.s

```
; fujidir.s -- list a directory on a FujiNet host.
;
; MOUNT_HOST, then OPEN_DIRECTORY with the path, then READ_DIR_ENTRY until
; the $7F,$7F marker says there are no more (or the screen is full), each
; name shown straight from the reply window, then CLOSE_DIRECTORY.

        .include "fujinet.inc"
        .include "devdefs.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_on, disp_rpl
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_path
        .export main

HOSTSL  = 0                     ; host slot 0: the SD card
NAMELN  = 36                    ;
READ_DIR_ENTRY's maxlen (never 31: at 31 the
                                ;   firmware puts icon bytes in front)
ROW0    = 4
NROWS   = 18

        .segment "BSS"
aperr:  .res 1
row:    .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        jsr     disp_init
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     mount
        lda     #FNENOC
        sta     aperr
        jmp     fail

        ; MOUNT_HOST: one parameter, the host slot
mount:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMHST
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #HOSTSL
        jsr     fn_pb
        jsr     go_ack

        ; OPEN_DIRECTORY: the host slot, then exactly 256 bytes of path
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCODIR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #HOSTSL
        jsr     fn_pb
        lda     #<path
        sta     FN_PTR
        lda     #>path
        sta     FN_PTR+1
        jsr     fn_path
        jsr     go_ack

        ; READ_DIR_ENTRY: two one-byte parameters, the longest name wanted
        ; and the flags; one name a call
        lda     #ROW0
        sta     row
entry:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCRDIR
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #NAMELN
        jsr     fn_pb
        lda     #0
        jsr     fn_pb
        jsr     go_ack
        lda     FN_REPLY        ; $7F,$7F: the end of the directory
        cmp     #$7F
        bne     @show
        lda     FN_REPLY+1
        cmp     #$7F
        beq     close
@show:  lda     row
        ldx     #2
        jsr     disp_at
        ldx     #0
        ldy     #NAMELN
        jsr     disp_rpl
        inc     row
        lda     row
        cmp     #ROW0+NROWS
        bne     entry

        ; CLOSE_DIRECTORY: no parameters
close:  lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCCDIR
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
loop:   jmp     loop
.endproc

; go_ack -- fn_go, then fn_ack; on any failure the code goes in aperr and
; the failure screen follows, so callers need no branch.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla
        pla
        jmp     fail
.endproc

.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

        .segment "RODATA"
path:   .byte "/a5200book/", 0
title:  .byte "DIRECTORY OF SD:/A5200BOOK/", 0
tfail:  .byte "FAILED: E", 0

        .include "header.inc"

```

fujiboot.s

```
; fujiboot.s -- M2: mount a cartridge image over the network and boot it.
;
; MOUNT_HOST -> SET_DEVICE_FULLPATH -> MOUNT_IMAGE. During MOUNT_IMAGE the
; ESP32 streams the file to the DBC device on the same link; the cart stages
; it and reports BOOT_STATE. Then BOOTLOCK, and the cart's stub hands the
; console to the image through the BIOS's own reset code.
;
; Target host slot and path come from bootcfg.inc, written by build.sh:
;   BOOT_HOST=0 BOOT_PATH=/game.a52 ./build.sh fujiboot

        .include "fujinet.inc"
        .include "bootcfg.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_on, disp_putc
        .import fn_chk, fn_beg, fn_go, fn_ack, fn_pb, fn_path, fn_boot
        .import fn_frames, fn_wait
        .export main

DEVSLOT = 0

        .segment "ZEROPAGE"
aperr:  .res 1
apstep: .res 1
lastpct: .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

.proc main
        jsr     disp_init
        lda     #0
        sta     aperr
        sta     apstep
        PUTS    2, 2, title
        PUTS    4, 2, tpath
        lda     #4
        ldx     #8
        jsr     disp_at
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     disp_str
        PUTS    6, 2, tmount
        jsr     disp_on

        jsr     fn_chk
        beq     have
        lda     #FNENOC
        sta     aperr
        jmp     show

; ---- MOUNT_HOST(host) ----
have:   lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMHST
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #BOOTHST
        jsr     fn_pb
        lda     #1
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:

; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ----
; Three one-byte parameters and then a FULL 256-byte NUL-padded path.
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCSDFP
        sta     FN_CMD
        lda     #3
        sta     FN_NPR
        jsr     fn_beg
        lda     #DEVSLOT
        jsr     fn_pb
        lda     #BOOTHST
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #<bootpath
        sta     FN_PTR
        lda     #>bootpath
        sta     FN_PTR+1
        jsr     fn_path
        lda     #2
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:

; ---- MOUNT_IMAGE(dev, mode) ----
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCMIMG
        sta     FN_CMD
        lda     #2
        sta     FN_NPR
        jsr     fn_beg
        lda     #DEVSLOT
        jsr     fn_pb
        lda     #FMREAD
        jsr     fn_pb
        lda     #66             ; seconds: past the cart's 60 s for MOUNT_IMAGE
        sta     FN_TMO
        lda     #3
        sta     apstep
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:

; ---- wait for the staged image ----
        lda     #4
        sta     apstep
        lda     #10
        jsr     fn_frames
wait:   lda     FN_BOOTSTATE
        cmp     #FN_BOOT_READY
        beq     boot
        cmp     #FN_BOOT_FAILED
        beq     bfail
        jsr     fn_wait
        bcs     wait
        lda     #FNEWAIT
        sta     aperr
```

```
166         jmp     show
167 bfail:  lda     FN_BOOTERR
168         sta     aperr
169         lda     #5
170         sta     apstep
171         jmp     show
172
173 ; ---- boot it: returns only if the cart never
    armed the swap ----
174 boot:   jsr     fn_boot
175         sta     aperr
176         lda     #6
177         sta     apstep
178
179 ; ---- the failure screen: step and error ----
180 show:   PUTS    8, 2, tfail
181         lda     #8
182         ldx     #14
183         jsr     disp_at
184         lda     apstep
185         jsr     disp_hex
186         lda     #' '
187         jsr     disp_putc
188         lda     #'E'
189         jsr     disp_putc
190         lda     aperr
191         jsr     disp_hex
192         jsr     disp_on
193 hang:   jmp     hang
194 .endproc
195
196         .segment "RODATA"
197 title:  .byte "FUJINET ATARI 5200 M2", 0
198 tpath:  .byte "BOOT: ", 0
199 tmount: .byte "MOUNTING...", 0
200 tfail:  .byte "FAILED STEP ", 0
201 bootpath: .byte BOOTPATH, 0
202
203         .include "header.inc"
204
```

appkey.s

```
  1 ; appkey.s -- read the shared FujiNet
    username, then write a key of our own
  2 ; and read it back.
  3 ;
  4 ; An appkey is a small file on the adapter's
    SD card, named by a creator id,
  5 ; an application id and a key number. Creator
    1, app 1, key 0 is the slot
  6 ; every FujiNet game reads the player's name
    from. This program shows it,
  7 ; then writes "HELLO 5200" into creator $5200,
    app $FF, key 0 and reads that
  8 ; back. (Creator $5200, app $FF is this
    handbook's scratch key.)
  9 ;
 10 ; OPEN_APPKEY's payload is ONE six-byte packed
    struct -- creator (u16, little-
 11 ; endian), app, key, mode, and a reserved byte
    that is not optional -- not
 12 ; four parameters. READ_APPKEY's reply is a
    two-byte length and then the
 13 ; value. WRITE_APPKEY takes one parameter, the
    length, then the bytes.
 14
 15         .include "fujinet.inc"
 16         .include "devdefs.inc"
 17
 18         .import disp_init, disp_at, disp_str,
    disp_hex, disp_on, disp_rpl
 19         .import fn_chk, fn_beg, fn_go, fn_ack,
    fn_pb, fn_txb
 20         .export main
 21
 22 OURCRE  = $5200
 23 OURAPP  = $FF
 24 OURKEY  = 0
 25
 26         .segment "BSS"
 27 aperr:  .res 1
 28 akcre:  .res 2                  ; the key
    being opened: creator...
 29 akapp:  .res 1                  ; ...app...
 30 akkey:  .res 1                  ; ...key
    number...
 31 akmode: .res 1                  ; ...and mode
    (AKMRD / AKMWR)
 32
 33         .segment "CODE"
 34
 35 .macro  PUTS    row, col, str
 36         lda     #row
 37         ldx     #col
 38         jsr     disp_at
 39         lda     #<str
 40         sta     FN_PTR
 41         lda     #>str
 42         sta     FN_PTR+1
 43         jsr     disp_str
 44 .endmacro
 45
 46 .macro  KEY     cre, app, key, mode
 47         lda     #<cre
 48         sta     akcre
 49         lda     #>cre
 50         sta     akcre+1
 51         lda     #app
 52         sta     akapp
 53         lda     #key
 54         sta     akkey
 55         lda     #mode
 56         sta     akmode
 57 .endmacro
 58
 59 .proc main
 60         jsr     disp_init
 61         PUTS    2, 2, title
 62         jsr     disp_on
 63         jsr     fn_chk
 64         beq     @go
 65         lda     #FNENOC
 66         sta     aperr
 67         jmp     fail
 68
 69 @go:    PUTS    5, 2, tname     ; the shared
    username
 70         KEY     AKCREAT, AKAPP, AKKEY, AKMRD
 71         jsr     akopen
 72         jsr     akread
 73         lda     #6
 74         jsr     akshow
 75
```

```
        PUTS    9, 2, tours     ; ours: write it, then read it back
        KEY     OURCRE, OURAPP, OURKEY, AKMWR
        jsr     akopen
        jsr     akwrite
        lda     #AKMRD
        sta     akmode
        jsr     akopen
        jsr     akread
        lda     #10
        jsr     akshow
loop:   jmp     loop
.endproc

; akopen -- OPEN_APPKEY for the key in akcre/akapp/akkey, mode akmode: no
; parameters, the six-byte struct as the payload.
.proc akopen
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKOP
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        lda     akcre
        jsr     fn_txb
        lda     akcre+1
        jsr     fn_txb
        lda     akapp
        jsr     fn_txb
        lda     akkey
        jsr     fn_txb
        lda     akmode
        jsr     fn_txb
        lda     #0              ; reserved, and not optional
        jsr     fn_txb
        jmp     go_ack
.endproc

; akread -- READ_APPKEY: the value is left in the reply window, two bytes of
; length, then the bytes.
.proc akread
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKRD
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jmp     go_ack
.endproc

; akwrite -- WRITE_APPKEY: one parameter, the length, then the value.
.proc akwrite
        lda     #FNDEVF
        sta     FN_DEV
        lda     #FNCAKWR
        sta     FN_CMD
        lda     #1
        sta     FN_NPR
        jsr     fn_beg
        lda     #valend-val
        jsr     fn_pb
        ldx     #0              ; X: fn_txb takes its byte in Y
@w:     lda     val,x
        jsr     fn_txb
        inx
        cpx     #valend-val
        bne     @w
        jmp     go_ack
.endproc

; akshow -- the value just read, on row A; "(EMPTY)" if there is none.
.proc akshow
        ldx     #4
        jsr     disp_at
        lda     FN_REPLY
        ora     FN_REPLY+1
        bne     @val
        lda     #<tempty
        sta     FN_PTR
        lda     #>tempty
        sta     FN_PTR+1
        jmp     disp_str
@val:   ldx     #2              ; the value starts after the length word
        ldy     FN_REPLY        ; at most its length (all are under 64)
        jmp     disp_rpl
.endproc

; go_ack -- fn_go, then fn_ack; on any failure, the failure screen.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla
        pla
        jmp     fail
.endproc

.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

        .segment "RODATA"
title:  .byte "APPKEYS", 0
tname:  .byte "USERNAME (CREATOR 1, APP 1, KEY 0)", 0
tours:  .byte "OURS (CREATOR $5200, APP $FF, KEY 0)", 0
tempty: .byte "(EMPTY)", 0
tfail:  .byte "FAILED: E", 0
val:    .byte "HELLO 5200"
valend:

        .include "header.inc"

```

clock.s

```
; clock.s -- the time of day, from the adapter's clock.
;
; The clock is a device of its own ($45). GET_ISO_LOCAL answers with the
; local time as text -- "2026-10-20T10:45:31" and a NUL -- in the adapter's
; configured time zone, and GET_ISO_UTC the same in UTC. Each goes on the
; screen straight out of the reply window.

        .include "fujinet.inc"
        .include "devdefs.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_on, disp_rpl
        .import fn_chk, fn_beg, fn_go, fn_ack
        .export main

        .segment "BSS"
aperr:  .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

; ASK cmd, row -- a clock command with no parameters, its text on the row.
.macro  ASK     cmd, row
        lda     #CLKDEV
        sta     FN_DEV
        lda     #cmd
        sta     FN_CMD
        lda     #0
        sta     FN_NPR
        jsr     fn_beg
        jsr     go_ack
        lda     #row
        ldx     #10
        jsr     disp_at
        ldx     #0
        ldy     #25
        jsr     disp_rpl
.endmacro

.proc main
        jsr     disp_init
        PUTS    2, 2, title
        jsr     disp_on
        jsr     fn_chk
        beq     @go
        lda     #FNENOC
        sta     aperr
        jmp     fail
@go:    PUTS    5, 2, tlocal
        ASK     CKISOL, 5
        PUTS    7, 2, tutc
        ASK     CKISOU, 7
loop:   jmp     loop
.endproc

; go_ack -- fn_go, then fn_ack; on any failure, the failure screen.
.proc go_ack
        jsr     fn_go
        sta     aperr
        bne     @bad
        jsr     fn_ack
        sta     aperr
        bne     @bad
        rts
@bad:   pla
        pla
        jmp     fail
.endproc

.proc fail
        PUTS    22, 2, tfail
        lda     aperr
        jsr     disp_hex
@h:     jmp     @h
.endproc

        .segment "RODATA"
title:  .byte "CLOCK", 0
tlocal: .byte "LOCAL", 0
tutc:   .byte "UTC", 0
tfail:  .byte "FAILED: E", 0

        .include "header.inc"

```

bank.s

```
; bank.s -- a 64K Super Cart FujiNet app, after the bring-up's fujibank.s
; without its last step: bank 1 (the power-on bank) runs a transaction,
; calls into bank 0 -- whose own code runs another one through the common
; fujilib -- and comes back. The arena is served in every bank.
;
; Banks switch on reads: $BFD0 selects bank 0, $BFE0-$BFFF the last bank.

        .macpack longbranch
        .include "fujinet.inc"

        .import disp_init, disp_at, disp_str, disp_hex, disp_rpl, disp_on, disp_putc
        .import fn_chk, fn_beg, fn_go, fn_ack
        .export main, far0

BANK0_ENTRY = $A000
AC_SIP  = 140

        .segment "BSS"
aperr:  .res 1
apstep: .res 1

        .segment "CODE"

.macro  PUTS    row, col, str
        lda     #row
        ldx     #col
        jsr     disp_at
        lda     #<str
        sta     FN_PTR
        lda     #>str
        sta     FN_PTR+1
        jsr     disp_str
.endmacro

; far0 -- call bank 0's entry and come back to bank 1. Common code: the
; same bytes are at this address in both banks.
.proc far0
        bit     $BFD0           ; bank 0 from the next fetch on
        jsr     BANK0_ENTRY
        bit     $BFE0           ; the last bank again; A is bank 0's answer
        rts
.endproc

.proc main
        jsr     disp_init
        lda     #0
        sta     aperr
        sta     apstep
        PUTS    2, 2, title
        PUTS    4, 2, tb1
        lda     #4
        ldx     #9
        jsr     disp_at
        lda     marker
        jsr     disp_putc
        jsr     disp_on
        jsr     fn_chk
        beq     :+
        lda     #FNENOC
        sta     aperr
        jmp     show
:       jsr     adapter
        jne     show
        PUTS    5, 2, tip
        lda     #5
        ldx     #9
        jsr     disp_at
        ldx     #AC_SIP
        ldy     #16
        jsr     disp_rpl

        lda     #1
        sta     apstep
        jsr     far0            ; bank 0 runs a transaction of its own
        sta     aperr
        cmp     #FNEOK
        beq     :+
        jmp     show
:       PUTS    8, 2, tback
        lda     #8
        ldx     #25
        jsr     disp_at
        lda     marker          ; bank 1's byte again: the switch came back
        jsr     disp_putc

        PUTS    10, 2, tdone
here:   jmp     here

show:   PUTS    12, 2, tfail
        lda     #12
        ldx     #14
        jsr     disp_at
        lda     apstep
        jsr     disp_hex
        lda     #' '
        jsr     disp_putc
        lda     #'E'
        jsr     disp_putc
        lda     aperr
        jsr     disp_hex
hang:   jmp     hang
.endproc

; begin -- FUJI device, command FN_CMD, A parameters.
.proc begin
        sta     FN_NPR
        lda     #FNDEVF
        sta     FN_DEV
        jmp     fn_beg
.endproc

; go -- commit; Z set and aperr 0 when the server said yes.
.proc go
        jsr     fn_go
        sta     aperr
        cmp     #FNEOK
        bne     done
        jsr     fn_ack
        sta     aperr
        cmp     #FNEOK
done:   rts
.endproc

; adapter -- GET_ADAPTERCONFIG_EXTENDED; Z set on success.
.proc adapter
        lda     #FNCADPX
        sta     FN_CMD
        lda     #0
        jsr     begin
        jmp     go
.endproc
        .export adapter

        .segment "RODATA"
title:  .byte "BANKS ON A SUPER CART", 0
tb1:    .byte "BANK 1", 0
tip:    .byte "IP", 0
tback:  .byte "BACK FROM BANK 0: BANK", 0
tfail:  .byte "FAILED STEP ", 0
tdone:  .byte "DONE", 0

        .segment "BANKCODE"
        .res    3               ; bank 0's entry jump
marker: .byte "1"               ; $A003: bank 0 has '0' here

        .segment "CLAIM"
        .byte   "FUJI"
        .byte   1               ; claim version
        .byte   4               ; mapper: A52MAP_SUPERCART + 1
        .byte   0
        .byte   $02             ; $BFE7: PAL-compatible

        .segment "HEADER"
        .byte   "FUJINET        "
        .byte   $FF, $FF
        .import start
        .word   start

```

bank0.s

```
; bank0.s -- bank 0's own code in fujibank: a transaction through the
; common fujilib, its result on screen, and a byte only this bank has.
; bank1syms.inc (build.sh) gives the common routines' addresses.

        .include "fujinet.inc"
        .include "bank1syms.inc"

        .segment "BANKCODE"

        jmp     entry           ; $A000: far0 calls here
marker: .byte "0"               ; $A003: bank 1 has '1' here

; Returns A = FNE*.
entry:  lda     #6
        ldx     #2
        jsr     disp_at
        lda     #<tb0
        sta     FN_PTR
        lda     #>tb0
        sta     FN_PTR+1
        jsr     disp_str
        lda     marker
        jsr     disp_putc
        jsr     adapter         ; a transaction from banked code
        bne     done
        lda     #6
        ldx     #12
        jsr     disp_at
        ldx     #0              ; the SSID
        ldy     #20
        jsr     disp_rpl
        lda     #FNEOK
done:   rts

tb0:    .byte "BANK ", 0

```

build.sh

```
#!/usr/bin/env bash
# build.sh -- build the assembly programs into build/<name>.bin with ca65/ld65.
#
#   ./build.sh hello fujitest ...          the named programs
#   BOOT_HOST=0 BOOT_PATH=/a5200book/hello.a52 ./build.sh fujiboot
#
# A program is <name>.s here or in ../common (the bring-up's, verbatim),
# linked with the bring-up's library (a52init, fujidisp, fujilib) and
# checked by tools/checkrom.py. fujibank and bank also build their bank 0
# (bank0.s), as the bring-up's build.sh does.
set -euo pipefail
cd "$(dirname "$0")"
BOOK=$(cd ../.. && pwd)
COMMON=../common

mkdir -p build/gen build/obj
python3 "$BOOK/tools/mkfont.py" > build/gen/font.inc
cat > build/gen/bootcfg.inc <<EOC
; GENERATED by build.sh -- do not edit.
BOOTHST = ${BOOT_HOST:-0}
.define BOOTPATH "${BOOT_PATH:-/a5200book/hello.a52}"
EOC

src() { if [ -f "$1.s" ]; then echo "$1.s"; else echo "$COMMON/$1.s"; fi; }
asm() { ca65 -I . -I "$COMMON" -I build/gen -o "build/obj/$1.o" "$(src "$1")"; }

build_one() {
    local name=$1 objs=() cfg=$COMMON/a52.cfg
    case $name in fujibank|bank) cfg=$COMMON/a52bank.cfg ;; esac
    for f in a52init fujidisp fujilib $name; do
        asm "$f"
        objs+=("build/obj/$f.o")
    done
    ld65 -C "$cfg" -m "build/obj/$name.map" -Ln "build/obj/$name.lbl" \
         -o "build/$name.bin" "${objs[@]}"
    case $name in fujibank|bank) build_bank0 "$name" ;; esac
    python3 "$BOOK/tools/checkrom.py" --claim --map "build/obj/$name.map" "build/$name.bin"
}

# A banked program's bank 0 (bank0.s): its own $A000-$AFFF, linked against
# the common code's addresses from bank 1's label file, laid over a copy of
# bank 1.
build_bank0() {
    local name=$1
    python3 - "build/obj/$name.lbl" > build/gen/bank1syms.inc <<'EOP'
import re, sys
syms = dict((n, a) for a, n in re.findall(r"al ([0-9A-F]{6}) \.(\w+)", open(sys.argv[1]).read()))
print("; GENERATED by build.sh from bank 1's label file -- do not edit.")
for w in ("disp_at", "disp_str", "disp_putc", "disp_rpl", "adapter"):
    print("%s = $%s" % (w, syms[w][2:]))
EOP
    asm bank0
    ld65 -C "$COMMON/bank0.cfg" -o build/obj/bank0.bin build/obj/bank0.o
    python3 - "build/$name.bin" <<'EOP'
import sys
b1 = open(sys.argv[1], "rb").read()
b0 = bytearray(b1)
```

```
b0[0x6000:0x7000] = open("build/obj/bank0.bin", "rb").read()
open(sys.argv[1], "wb").write(bytes(b0) + b1)
EOP
}

for t in "$@"; do
    build_one "$t"
done

```

The C programs

antic.h

```
/* antic.h -- CONFIG's text screen on ANTIC mode 2: 40 x 24 cells of 8 x 8
 * pixels from a font in ROM (font.txt, tools/mkfont.py), the screen at
 * $3C00 in the RAM the linker reserves. The calls are the 7800 MARIA
 * engine's, so the screens above port unchanged: "palette" 1 is reverse
 * video, which is the selection bar.
 *
 * Characters go in as the 7800's tile numbers -- ASCII, with the frame,
 * cursor and progress-bar tiles below $20 -- and are stored as ANTIC's
 * internal codes.
 */

#ifndef ANTIC_H
#define ANTIC_H

#define MT_COLS 40
#define MT_ROWS 24

/* Frames since start-up, wrapping at 256: the VBI's RTCLOK. */
#define mt_frames (*(volatile unsigned char *)0x02)

void mt_init(unsigned char fill);
void mt_clear(unsigned char fill);
/* The background's colour and the text's luminance: mode 2 has no more. */
void mt_colours(unsigned char background, unsigned char text);

/* Move the write position. */
void mt_at(unsigned char col, unsigned char row);
/* One tile at the write position, which advances. */
void mt_put(unsigned char t);
/* `n` copies of a tile. */
void mt_fill(unsigned char t, unsigned char n);
/* A NUL-terminated string, at most `max` tiles; returns how many went. */
unsigned char mt_text(const char *s, unsigned char max);
/* The tile at a cell. */
unsigned char mt_get(unsigned char col, unsigned char row);
/* The `n` cells from the write position in reverse video (pal 1) or not;
 * the write position does not move. */
void mt_setpal(unsigned char pal, unsigned char n);

/* Wait for the next frame. */
void mt_sync(void);

#endif /* ANTIC_H */

```

antic.c

```
/* antic.c -- vendored from fujinet-config atari5200/ (main cb44218): CONFIG's
 * text engine, used here as it is. */
#include "antic.h"

```

```
#define REG(a)   (*(volatile unsigned char *)(a))
#define SDLSTL   REG(0x05)
#define SDLSTH   REG(0x06)
#define SDMCTL   REG(0x07)
#define COLOR1   REG(0x0D)       /* mode 2: the text's luminance */
#define COLOR2   REG(0x0E)       /* mode 2: the background */
#define COLOR4   REG(0x10)       /* the border */
#define CHACTL   REG(0xD401)
#define CHBASE   REG(0xD409)

/* The top 1K of RAM, which the Makefile reserves: 1K-aligned, so the
 * screen never crosses the 4K boundary ANTIC's address counter wraps at. */
#define SCREEN   ((unsigned char *)0x3C00)

extern const unsigned char mt_dlist[];  /* antic_dl.s */
extern const unsigned char mt_font[];

static unsigned char *wp;

/* ASCII -> ANTIC internal code. */
static unsigned char code(unsigned char t)
{
    t &= 0x7F;
    if (t < 0x20)
        return (unsigned char)(t + 0x40);
    if (t < 0x60)
        return (unsigned char)(t - 0x20);
    return t;
}

static unsigned char ascii(unsigned char c)
{
    c &= 0x7F;
    if (c < 0x40)
        return (unsigned char)(c + 0x20);
    if (c < 0x60)
        return (unsigned char)(c - 0x40);
    return c;
}

void mt_clear(unsigned char fill)
{
    unsigned char *p = SCREEN;
    unsigned int n = MT_COLS * MT_ROWS;
    unsigned char c = code(fill);

    while (n--)
        *p++ = c;
    wp = SCREEN;
}

void mt_colours(unsigned char background, unsigned char text)
{
    COLOR2 = background;
    COLOR4 = background;
    COLOR1 = text;
}

void mt_init(unsigned char fill)
{
    mt_clear(fill);
    CHBASE = (unsigned char)((unsigned int)mt_font >> 8);
    CHACTL = 2;                  /* code + $80 is reverse video */
    SDLSTL = (unsigned char)(unsigned int)mt_dlist;
    SDLSTH = (unsigned char)((unsigned int)mt_dlist >> 8);
    SDMCTL = 0x22;               /* normal playfield, display-list DMA */
}

void mt_at(unsigned char col, unsigned char row)
{
    wp = SCREEN + (unsigned int)row * MT_COLS + col;
}

void mt_put(unsigned char t)
{
    *wp++ = code(t);
}

void mt_fill(unsigned char t, unsigned char n)
{
    unsigned char c = code(t);

    while (n--)
        *wp++ = c;
}

unsigned char mt_text(const char *s, unsigned char max)
{
    unsigned char n = 0;

    while (n < max && s[n]) {
        unsigned char t = (unsigned char)s[n];

        *wp++ = code(t < 32 || t > 126 ? ' ' : t);
        n++;
    }
    return n;
}

unsigned char mt_get(unsigned char col, unsigned char row)
{
    return ascii(SCREEN[(unsigned int)row * MT_COLS + col]);
}

void mt_setpal(unsigned char pal, unsigned char n)
{
    unsigned char *p = wp;

    while (n--) {
        if (pal)
            *p++ |= 0x80;
        else
            *p++ &= 0x7F;
    }
}

void mt_sync(void)
{
    unsigned char f = mt_frames;

    while (mt_frames == f)
        ;
}
```

antic_dl.s

```
; antic_dl.s -- vendored from fujinet-config atari5200/ (main cb44218). The
; font is the bring-up clients' (tools/mkfont.py), so both languages' screens match.

        .export _mt_dlist, _mt_font

SCREEN = $3C00

        .segment "DLIST"
_mt_dlist:
        .byte   $70, $70, $70                   ; 24 blank lines
        .byte   $42, <SCREEN, >SCREEN           ; mode 2, from the screen
        .repeat 23
        .byte   $02
        .endrepeat
        .byte   $41, <_mt_dlist, >_mt_dlist     ; wait for the next frame

        .segment "FONT"
_mt_font:
        .include "font.inc"

```

book.h

```
/* book.h -- what the handbook's C programs share beyond antic.h. */
#ifndef BOOK_H
#define BOOK_H

/* A NUL-terminated string at the write position, up to the row's end. */
#define mt_puts(s) mt_text((s), MT_COLS)

#endif /* BOOK_H */

```

hello.c

```
/* hello.c -- the C road to the first screen: text through CONFIG's ANTIC
 * engine, the frame counter ticking, the keypad answering, and the cart's
 * status page answering 'F','N'. The same screen as hello.s.
 */
#include <stdint.h>
#include <fujinet-atari5200.h>
#include "antic.h"

#define RTCLOKH (*(volatile uint8_t *) 0x01)
#define RTCLOKL (*(volatile uint8_t *) 0x02)

static void puts_at(uint8_t row, uint8_t col, const char *s)
{
    mt_at(col, row);
    mt_text(s, MT_COLS);
}

static void hex(uint8_t v)
{
    static const char digits[] = "0123456789ABCDEF";

    mt_put(digits[v >> 4]);
    mt_put(digits[v & 15]);
}

void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    puts_at(2, 2, "FUJINET ATARI 5200 C");
    puts_at(4, 2, "CART");
    puts_at(4, 8, fuji_a5200_present() ? "FN OK" : "NO MAILBOX");
    puts_at(6, 2, "FRAME");
    puts_at(8, 2, "KEY");
    for (;;) {
        mt_at(9, 6);
        hex(RTCLOKH);
        hex(RTCLOKL);
        mt_at(9, 8);
        hex(fuji_a5200_key);
    }
}

```

conio.c

```
/* conio.c -- the third road: cc65's own console, a 20 x 24 screen in ANTIC
 * mode 6 that cc65 keeps at $3E20-$3FFF. Link with __RESERVED_MEMORY__ of
 * at least 0x1E0, or the C stack is built on top of it (build.sh: RESERVE). */
#include <conio.h>
#include <fujinet-atari5200.h>

void main(void)
{
    clrscr();
    gotoxy(2, 2);
    cputs("HELLO FROM CONIO");
    gotoxy(2, 4);
    cputs(fuji_a5200_present() ? "CART FN OK" : "NO MAILBOX");
    for (;;)
        ;
}

```

acfg.c

```
/* acfg.c -- First Contact in C: one GET_ADAPTERCONFIG_EXTENDED, and the
 * SSID, IP address and firmware version on the screen. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

static AdapterConfigExtended ac;

static void hex(uint8_t v)
{
    static const char digit[] = "0123456789ABCDEF";
    mt_put(digit[v >> 4]);
    mt_put(digit[v & 15]);
}

static void field(uint8_t row, const char *label, const char *s, uint8_t max)
{
    mt_at(2, row);
    mt_puts(label);
    mt_at(8, row);
    mt_text(s, max);
}

void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    mt_at(2, 2);
    mt_puts("FUJINET 5200 C FIRST CONTACT");

    if (!fuji_a5200_present()) {
        mt_at(2, 5);
        mt_puts("NO FUJINET CARTRIDGE");
    } else if (!fuji_get_adapter_config_extended(&ac)) {
        mt_at(2, 5);
        mt_puts("FAILED: E");
        hex(fn_device_error);
    } else {
        field(5, "SSID", ac.ssid, 24);
        field(7, "IP", ac.sLocalIP, 16);
        field(9, "FW", ac.fn_version, 15);
    }
    for (;;)
        mt_sync();
}

```

pads.c

```
/* pads.c -- controller 1: the stick, both buttons and the keypad, and a
 * tone from POKEY while the bottom button is held. The same screen as
 * pads.s.
 *
 * The stick is two pots, which the VBI copies to PADDL0 (X) and PADDL1 (Y)
 * every frame; they read nothing until CONSOL bit 2 turns them on. The
 * bottom button is GTIA's TRIG0 (0 while held), the top one POKEY's SKSTAT
 * bit 3 (clear while held, for the controller CONSOL picks). The keypad is
 * the crt0's: fuji_a5200_key holds the last key + 1. */
#include <stdint.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

#define REG(a)  (*(volatile uint8_t *) (a))
#define PADDL0  REG(0x11)
#define PADDL1  REG(0x12)
#define TRIG0   REG(0xC010)
#define CONSOL  REG(0xC01F)
#define AUDF1   REG(0xE800)
#define AUDC1   REG(0xE801)
#define SKSTAT  REG(0xE80F)

#define TONE    0x50            /* AUDF1: the pitch */
#define VOICE   0xA8            /* AUDC1: a pure tone at volume 8 */

/* fuji_a5200_key is the key's code + 1: 0 none, then 0-9, *, #, START,
 * PAUSE, RESET. */
static const char *const names[16] = {
    "-    ", "0    ", "1    ", "2    ", "3    ", "4    ", "5    ", "6    ",
    "7    ", "8    ", "9    ", "*    ", "#    ", "START", "PAUSE", "RESET"
};

static void label(uint8_t row, const char *s)
{
    mt_at(2, row);
    mt_puts(s);
}

/* Three digits, leading zeros as spaces. */
static void dec3(uint8_t v)
{
    uint8_t h = v / 100, t = (v / 10) % 10;

    mt_put(h ? '0' + h : ' ');
    mt_put(h || t ? '0' + t : ' ');
    mt_put('0' + v % 10);
}

void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    CONSOL = 0x04;                      /* the pots on; controller 1 */
    label(2, "CONTROLLER 1");
    label(4, "STICK X");
    label(5, "STICK Y");
    label(7, "BOTTOM BUTTON");
    label(8, "TOP BUTTON");
    label(10, "KEYPAD");
    label(12, "SOUND");

    for (;;) {
        mt_sync();
        mt_at(16, 4);
        dec3(PADDL0);
        mt_at(16, 5);
        dec3(PADDL1);
        mt_at(16, 7);
        mt_puts(TRIG0 ? "UP  " : "DOWN");
        mt_at(16, 8);
        mt_puts(SKSTAT & 0x08 ? "UP  " : "DOWN");
        mt_at(16, 10);
        mt_puts(names[fuji_a5200_key & 15]);
        mt_at(16, 12);
        if (TRIG0 == 0) {
            AUDF1 = TONE;
            AUDC1 = VOICE;
            mt_puts("ON ");
        } else {
            AUDC1 = 0;
            mt_puts("OFF");
        }
    }
}

```

netget.c

```
/* netget.c -- fetch a URL through the N: device and show the text.
 *
 * network_open() as an HTTP GET, network_read() what arrives, then
 * network_close(). The text goes up a line at a time: LF ends a row, CR is
 * skipped, a row is at most 36 characters. cc65's Atari targets read '\n'
 * as ATASCII's EOL ($9B), so the codes are spelled out. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-network.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

#define URL   "N:http://127.0.0.1:8768/hello.txt"
#define ROW0  4
#define ROWS  18
#define COLS  36
#define CR    0x0D
#define LF    0x0A

static char buf[512];

static void hex(uint8_t v)
{
    static const char digit[] = "0123456789ABCDEF";
    mt_put(digit[v >> 4]);
    mt_put(digit[v & 15]);
}

static void fail(const char *what, uint8_t code)
{
    mt_at(2, 22);
    mt_puts(what);
    mt_puts(" FAILED: E");
    hex(code);
    for (;;)
        mt_sync();
}

void main(void)
{
    int16_t n, i;
    uint8_t row = ROW0, col = 0, err;

    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    mt_at(2, 2);
    mt_puts("NETGET");

    network_init();
    err = network_open(URL, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE);
    if (err)
        fail("OPEN", err);
    n = network_read(URL, buf, sizeof buf);
    if (n < 0)
        fail("READ", fn_device_error);

    mt_at(2, row);
    for (i = 0; i < n; ++i) {
        if (buf[i] == CR)
            continue;
        if (buf[i] == LF) {
            if (++row == ROW0 + ROWS)
                break;
            mt_at(2, row);
            col = 0;
        } else if (col < COLS) {
            mt_put(buf[i]);
            ++col;
        }
    }
    network_close(URL);
    mt_at(2, 22);
    mt_puts("CLOSED");
    for (;;)
        mt_sync();
}

```

```c
/* netcat.c -- talk to a TCP server a line at a time. The same screen and
 * keys as netcat.s.
 *
 * network_open() read/write, then every frame: the stick moves round an
 * on-screen keyboard and the bottom button types its key; the keypad's
 * digits type themselves, * rubs out, # sends the line with CR LF
 * (network_write()), and START closes the connection. Every eighth frame
 * network_status() asks what is waiting, and network_read() fetches it (at
 * most 128 bytes) into rows 3-14, which scroll. */
#include <stdint.h>
#include <string.h>
#include <fujinet-network.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

#define URL     "N:TCP://127.0.0.1:8769/"
#define ROW0    3               /* the received text: rows 3-14 */
#define NROWS   12
#define NCOLS   36
#define LROW    16              /* the line being typed, after "> " */
#define LMAX    34
#define KROW    18              /* the keyboard: rows 18-20 */
#define KCOLS   13
#define NMAX    128
#define LO      0x40            /* the stick's dead zone: $40-$A4 */
#define HI      0xA4
#define CR      0x0D            /* cc65's Atari targets read '\n' as $9B */
#define LF      0x0A

#define REG(a)  (*(volatile uint8_t *) (a))
#define PADDL0  REG(0x11)
#define PADDL1  REG(0x12)
#define TRIG0   REG(0xC010)
#define CONSOL  REG(0xC01F)
#define SCREEN  ((uint8_t *) 0x3C00)    /* antic.c's */

static const char keys[] = "ABCDEFGHIJKLM"
    "NOPQRSTUVWXYZ" "_.,;/?!-'@()=";

static uint8_t rrow = ROW0, rcol, llen, cx, cy, latch, fired;
static char line[LMAX + 2];
static uint8_t buf[NMAX];

static void fail(const char *what, uint8_t code)
{
    static const char digit[] = "0123456789ABCDEF";

    mt_at(2, LROW);
    mt_puts(what);
    mt_puts(" FAILED: E");
    mt_put(digit[code >> 4]);
    mt_put(digit[code & 15]);
    for (;;)
        mt_sync();
}

/* The key under the cursor in reverse video, or back. */
static void cursor(uint8_t on)
{
    mt_at(2 + 2 * cx, KROW + cy);
    mt_setpal(on, 1);
}

static void keyboard(void)
{
    uint8_t r, c;

    for (r = 0; r < 3; ++r) {
        mt_at(2, KROW + r);
        for (c = 0; c < KCOLS; ++c) {
            mt_put(keys[r * KCOLS + c]);
            mt_put(' ');
        }
    }
}

static void type(char c)
{
    if (llen == LMAX)
        return;
    mt_at(4 + llen, LROW);
    mt_put(c);
    line[llen++] = c;
}

static void rub(void)
{
    if (llen == 0)
        return;
    --llen;
    mt_at(4 + llen, LROW);
    mt_put(' ');
}

static void send(void)
{
    line[llen] = CR;
    line[llen + 1] = LF;
    if (network_write(URL, line, llen + 2))
        fail("WRITE", fn_device_error);
    llen = 0;
    mt_at(4, LROW);
    mt_fill(' ', LMAX);
}

/* One step per push: move, then wait for the stick to come back to the
 * centre. */
static void stick(void)
{
    uint8_t x = PADDL0, y = PADDL1;

    if (latch) {
        if (x >= LO && x <= HI && y >= LO && y <= HI)
            latch = 0;
        return;
    }
    if (x >= LO && x <= HI && y >= LO && y <= HI)
        return;
    cursor(0);
    if (x < LO)
        cx = cx ? cx - 1 : KCOLS - 1;
    else if (x > HI)
        cx = cx == KCOLS - 1 ? 0 : cx + 1;
    else if (y < LO)
        cy = cy ? cy - 1 : 2;
    else
        cy = cy == 2 ? 0 : cy + 1;
    cursor(1);
    latch = 1;
}

/* The bottom button types the key under the cursor, once a press. */
static void button(void)
{
    char c;

    if (TRIG0) {
        fired = 0;
        return;
    }
    if (fired)
        return;
    fired = 1;
    c = keys[cy * KCOLS + cx];
    type(c == '_' ? ' ' : c);
}

/* A new row; past the last, scroll the rows up one. */
static void newrow(void)
{
    rcol = 0;
    if (++rrow < ROW0 + NROWS)
        return;
    --rrow;
```

```
    memmove(SCREEN + ROW0 * 40, SCREEN + (ROW0 + 1) * 40, (NROWS - 1) * 40);
    mt_at(0, ROW0 + NROWS - 1);
    mt_fill(' ', 40);
}

static void show(uint8_t c)
{
    if (c == CR)
        return;
    if (c == LF) {
        newrow();
        return;
    }
    mt_at(2 + rcol, rrow);
    mt_put(c);
    if (++rcol == NCOLS)
        newrow();
}

static void receive(void)
{
    uint16_t bw;
    uint8_t connected, err;
    int16_t n, i;

    if (network_status(URL, &bw, &connected, &err))
        fail("STATUS", fn_device_error);
    if (bw == 0)
        return;
    n = network_read(URL, buf, bw > NMAX ? NMAX : bw);
    if (n < 0)
        fail("READ", fn_device_error);
    for (i = 0; i < n; ++i)
        show(buf[i]);
}

void main(void)
{
    uint8_t k;

    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    CONSOL = 0x04;                      /* the pots on; controller 1 */
    mt_at(2, 1);
    mt_puts("NETCAT  127.0.0.1:8769");
    mt_at(2, LROW);
    mt_puts("> ");
    mt_at(1, 22);
    mt_puts("FIRE TYPES  # SENDS  * RUBS START ENDS");
    keyboard();
    cursor(1);
    fuji_a5200_key = 0;

    network_init();
    if (network_open(URL, OPEN_MODE_RW, OPEN_TRANS_NONE))
        fail("OPEN", fn_device_error);

    for (;;) {
        mt_sync();
        stick();
        button();
        k = fuji_a5200_key;             /* the last key + 1, 0 = none */
        if (k) {
            fuji_a5200_key = 0;
            --k;
            if (k < 10)
                type('0' + k);
            else if (k == 0x0A)         /* * */
                rub();
            else if (k == 0x0B)         /* # */
                send();
            else if (k == 0x0C) {       /* START */
                network_close(URL);
                mt_at(2, LROW);
                mt_puts("CLOSED");
                mt_fill(' ', 30);
                for (;;)
                    mt_sync();
            }
        }
        if ((mt_frames & 7) == 0)
            receive();
    }
}
```

dir.c

```
/* dir.c -- list a directory on a FujiNet
host.
 *
 * fuji_mount_host_slot(),
fuji_open_directory() with the path, then
 * fuji_read_directory() until the 0x7F,0x7F
marker (or the screen is
 * full), then fuji_close_directory(). */
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

#define HOSTSL  0                       /*
host slot 0: the SD card */
#define NAMELN  36                      /*
never 31: icon bytes come first */
#define ROW0    4
#define ROWS    18

static char path[256];                  /*
OPEN_DIRECTORY takes exactly 256 */
static char name[NAMELN + 1];

static void fail(const char *what)
{
    mt_at(2, 22);
    mt_puts(what);
    mt_puts(" FAILED");
    for (;;)
        mt_sync();
}

void main(void)
{
    uint8_t row;

    mt_init(' ');
    mt_colours(0x84, 0x0E);             /*
blue background, white text */
    mt_at(2, 2);
    mt_puts("DIRECTORY OF SD:/A5200BOOK/");

    strcpy(path, "/a5200book/");
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_open_directory(HOSTSL, path))
        fail("OPEN_DIRECTORY");
    for (row = ROW0; row < ROW0 + ROWS; ++row)
{
        if (!fuji_read_directory(NAMELN, 0,
name))
            fail("READ_DIR_ENTRY");
        if (name[0] == 0x7F && name[1] ==
0x7F)
            break;                      /* the
end of the directory */
        mt_at(2, row);
        mt_text(name, NAMELN);
    }
    fuji_close_directory();
    for (;;)
        mt_sync();
}

```

boot.c

```
/* boot.c -- push a program from the SD card
into the cartridge and run it.
 *
 * MOUNT_HOST, SET_DEVICE_FULLPATH, then
MOUNT_IMAGE, which makes the
 * FujiNet push the image into the cartridge.
When the cartridge says the
 * image is READY, fuji_a5200_boot() quiets
the console and jumps to the
 * cartridge's stub, which swaps the image in
as the BIOS starts it. A bar
 * shows the push. */
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

#define HOSTSL   0                      /*
host slot 0: the SD card */
#define DEVSL    0                      /*
device slot 0 */
#define FMREAD   1                      /*
mount for reading */
#define BOOTPATH "/a5200book/helloc.a52"

```

```
static char path[256];                  /*
SET_DEVICE_FULLPATH takes 256 */

static void fail(const char *what)
{
    mt_at(2, 22);
    mt_puts(what);
    mt_puts(" FAILED");
    for (;;)
        mt_sync();
}

void main(void)
{
    uint8_t state, pct, shown = 0;

    mt_init(' ');
    mt_colours(0x84, 0x0E);             /*
blue background, white text */
    mt_at(2, 2);
    mt_puts("BOOT " BOOTPATH);

    strcpy(path, BOOTPATH);
    if (!fuji_mount_host_slot(HOSTSL))
        fail("MOUNT_HOST");
    if (!fuji_set_device_filename(FMREAD,
HOSTSL, DEVSL, path))
        fail("SET_DEVICE_FULLPATH");
    if (!fuji_mount_disk_image(DEVSL, FMREAD))
        fail("MOUNT_IMAGE");

    /* the push may finish before or after
MOUNT_IMAGE returns */
    mt_at(2, 4);
    for (;;) {
        state = fuji_a5200_boot_state();
        pct = fuji_a5200_boot_percent();
        while (shown < pct / 4) {       /* 25
cells for 100 per cent */
            mt_put('#');
            ++shown;
        }
        if (state == FUJI_A5200_BOOT_READY)
            break;
        if (state == FUJI_A5200_BOOT_FAILED)
            fail("PUSH");
        mt_sync();
    }
    fuji_a5200_boot();                  /*
does not return */
}

```

appkey.c

```
/* appkey.c -- read the shared FujiNet
username, then write a key of our own
 * and read it back.
 *
 * Creator 1, app 1, key 0 is the slot every
FujiNet game reads the player's
 * name from. Ours is creator 0x5200, app 0xFF
(this handbook's scratch
 * key), key 0. fuji_read_appkey() and
 * fuji_write_appkey() each do the OPEN_APPKEY
themselves. */
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

static uint8_t value[64 + 2];           /* a
key, plus 2: the lib's rule */
static const char ours[] = "HELLO 5200";

static void show(uint8_t row)
{
    uint16_t n;

    mt_at(4, row);
    if (!fuji_read_appkey(0, &n, value))
        mt_puts("(READ FAILED)");
    else if (n == 0)
        mt_puts("(EMPTY)");
    else
        mt_text((char *)value, n);
}

void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /*
blue background, white text */
    mt_at(2, 2);
    mt_puts("APPKEYS");

    mt_at(2, 5);
    mt_puts("USERNAME: KEY 1:1:0");
    fuji_set_appkey_details(1, 1, DEFAULT);
    show(6);

    mt_at(2, 9);
    mt_puts("OURS: KEY $5200:$FF:0");
    fuji_set_appkey_details(0x5200, 0xFF,
DEFAULT);
    if (!fuji_write_appkey(0, sizeof ours - 1,
(uint8_t *)ours)) {
        mt_at(4, 10);
        mt_puts("(WRITE FAILED)");
    } else
        show(10);
    for (;;)
        mt_sync();
}

```

clock.c

```
/* clock.c -- the time of day, from the adapter's clock.
 *
 * clock_get_time() with TZ_ISO_STRING answers with the local time as text,
 * "2026-10-20T10:45:31" and more, in the adapter's configured time zone;
 * UTC_ISO_STRING the same in UTC. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-clock.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

static uint8_t when[32];

static void line(uint8_t row, const char *label, TimeFormat f)
{
    mt_at(2, row);
    mt_puts(label);
    mt_at(10, row);
    if (clock_get_time(when, f))
        mt_puts("FAILED");
    else
        mt_text((char *)when, 25);
}

void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    mt_at(2, 2);
    mt_puts("CLOCK");
    line(5, "LOCAL", TZ_ISO_STRING);
    line(7, "UTC", UTC_ISO_STRING);
    for (;;)
        mt_sync();
}

```

bank.c

```
/* bank.c -- a 64K Super Cart FujiNet app in C: bank 1 (the power-on
 * bank) runs a transaction, calls into bank 0 -- whose own code runs
 * another -- and comes back. The same screen as bank.s.
 *
 * Built with bank.cfg, supercart.s and bank0.c (build.sh's "bank"). */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include "antic.h"
#include "book.h"

#define MARK (*(const char *) 0xA000)   /* '1' in bank 1, '0' in bank 0 */

#pragma rodata-name (push, "MARK1")
const char mark1 = '1';
#pragma rodata-name (pop)

extern uint8_t far_bank0(void);         /* supercart.s */

static AdapterConfigExtended ac;

static void fail(uint8_t step)
{
    mt_at(2, 12);
    mt_puts("FAILED STEP ");
    mt_put('0' + step);
    for (;;)
        mt_sync();
}

void main(void)
{
    mt_init(' ');
    mt_colours(0x84, 0x0E);             /* blue background, white text */
    mt_at(2, 2);
    mt_puts("BANKS ON A SUPER CART");
    mt_at(2, 4);
    mt_puts("BANK 1 ");
    mt_put(MARK);

    if (!fuji_a5200_present() || !fuji_get_adapter_config_extended(&ac))
        fail(0);
    mt_at(2, 5);
    mt_puts("IP");
    mt_at(9, 5);
    mt_text(ac.sLocalIP, 16);

    if (!far_bank0())                   /* bank 0 runs a transaction of its own */
        fail(1);
    mt_at(2, 8);
    mt_puts("BACK FROM BANK 0: BANK ");
    mt_put(MARK);                       /* bank 1's byte again */
    mt_at(2, 10);
    mt_puts("DONE");
    for (;;)
        mt_sync();
}

```

bank0.c

```
/* bank0.c -- bank 0's own code in bank.c's Super Cart: a transaction
 * through fujinet-lib, which lives in the common part, its result on the
 * screen, and a byte only this bank has. */
#include <stdint.h>
#include <stdbool.h>
#include <fujinet-fuji.h>
#include "antic.h"
#include "book.h"

#pragma rodata-name (push, "MARK0")
const char mark0 = '0';                 /* $A000: bank 1 has '1' here */
#pragma rodata-name (pop)

#pragma code-name (push, "BANK0")
#pragma rodata-name (push, "BANK0")

static AdapterConfigExtended ac;

uint8_t bank0_main(void)
{
    mt_at(2, 6);
    mt_puts("BANK ");
    mt_put(*(const char *) 0xA000);
    if (!fuji_get_adapter_config_extended(&ac))
        return 0;
    mt_at(12, 6);
    mt_text(ac.ssid, 20);
    return 1;
}

#pragma rodata-name (pop)
#pragma code-name (pop)

```

supercart.s

```
; supercart.s -- the Super Cart's bank switch, for C. A bank is chosen by
; READING a hotspot, which C cannot be trusted to do: cc65 drops a load
; whose value is unused. BIT reads without touching A or X, so a far call
; keeps its return value.

        .export _far_bank0
        .import _bank0_main

; uint8_t far_bank0 (void) -- run bank 0's bank0_main() and come back to the
; last bank (bank 1, the power-on bank). Common code: the same bytes are at
; this address in both banks.
.proc _far_bank0
        bit     $BFD0           ; bank 0 from the next fetch on
        jsr     _bank0_main
        bit     $BFE0           ; the last bank again
        rts
.endproc

```

bank.cfg

```
# bank.cfg -- a 64K Super Cart FujiNet client in C: MekkoGX's
# atari5200-fujinet.cfg with $A000-$AFFF made a bank window.
#
# The Super Cart switches the whole 32K, so everything outside $A000-$AFFF
# must be the same in both banks: build.sh lays bank 0's 4K (BANKA0, its
# own file) over a copy of bank 1 and puts bank 0 first. $BFC0-$BFFF are
# the Super Cart's hotspots: nothing but the claim and the header there.
SYMBOLS {
    __STACKSIZE__:       type = weak, value = $0400;
    __RESERVED_MEMORY__: type = weak, value = $0000;
}
MEMORY {
    ZP:     file = "", define = yes, start = $001D, size = $00E3, type = rw;
```

```
    RAM:    file = "", define = yes, start = $021C, size = $4000 - $021C - __STACKSIZE__ - __RESERVED_MEMORY__, type = rw;
    STACK:  file = "", define = yes, start = $4000 - __STACKSIZE__ - __RESERVED_MEMORY__, size = __STACKSIZE__, type = rw;
    ROM:    file = %O, define = yes, start = $4000, size = $6000, type = ro, fill = yes, fillval = $FF;
    BANKA1: file = %O, start = $A000, size = $1000, type = ro, fill = yes, fillval = $FF;
    ARENA:  file = %O, start = $B000, size = $0800, type = ro, fill = yes, fillval = $FF;
    ROM2:   file = %O, define = yes, start = $B800, size = $07C0, type = ro, fill = yes, fillval = $FF;
    HOT:    file = %O, start = $BFC0, size = $0020, type = ro, fill = yes, fillval = $FF;
    CLAIM:  file = %O, start = $BFE0, size = $0008, type = ro, fill = yes, fillval = $00;
    HDR:    file = %O, start = $BFE8, size = $0018, type = ro;
    BANKA0: file = "build/obj/bank0.part", start = $A000, size = $1000, type = ro, fill = yes, fillval = $FF;
}
SEGMENTS {
    ZEROPAGE: load = ZP,              type = zp;
    EXTZP:    load = ZP,              type = zp,  optional = yes;
    STARTUP:  load = ROM,             type = ro,  define = yes;
    LOWCODE:  load = ROM,             type = ro,  optional = yes;
    CODE:     load = ROM,             type = ro,  define = yes;
    RODATA:   load = ROM,             type = ro,  define = yes;
    DLIST:    load = ROM,             type = ro,  optional = yes, align = $400;
    DATA:     load = ROM, run = RAM,  type = rw,  define = yes;
    MARK1:    load = BANKA1,          type = ro;
    BANK1:    load = BANKA1,          type = ro,  optional = yes;
    FONT:     load = ROM2,            type = ro,  optional = yes, align = $400;
    ONCE:     load = ROM2,            type = ro,  optional = yes;
    ROM2:     load = ROM2,            type = ro,  optional = yes;
    BSS:      load = RAM,             type = bss, define = yes;
    CLAIM:    load = CLAIM,           type = ro;
    CARTHDR:  load = HDR,             type = ro;
    MARK0:    load = BANKA0,          type = ro;
    BANK0:    load = BANKA0,          type = ro;
}
FEATURES {
    CONDES: type    = constructor,
            label   = __CONSTRUCTOR_TABLE__,
            count   = __CONSTRUCTOR_COUNT__,
            segment = ONCE;
    CONDES: type    = destructor,
            label   = __DESTRUCTOR_TABLE__,
            count   = __DESTRUCTOR_COUNT__,
            segment = RODATA;
    CONDES: type    = interruptor,
            label   = __INTERRUPTOR_TABLE__,
            count   = __INTERRUPTOR_COUNT__,
            segment = RODATA,
            import  = __CALLIRQ__;
}

```

build.sh

```
#!/usr/bin/env bash
# build.sh -- build the C programs into build/<name>.bin with cc65,
# fujinet-lib's atari5200 target and MekkoGX's crt0 and linker config.
#
#   ./build.sh hello acfg ...     the named programs
#   FNLIB=... MEKKO=... ./build.sh hello
#
```

```
# FNLIB is a fujinet-lib tree with the atari5200 target built in it
# (make atari5200); MEKKO is MekkoGX's mekkogx/ directory.
set -euo pipefail
cd "$(dirname "$0")"
BOOK=$(cd ../.. && pwd)

FNLIB=${FNLIB:-$HOME/Workspace/a5book-fw/fujinet-lib}
MEKKO=${MEKKO:-$HOME/Workspace/a5book-fw/MekkoGX/mekkogx}
LIB=$FNLIB/r2r/atari5200/fujinet.atari5200.lib
[ -f "$LIB" ] || { echo "build.sh: no $LIB (make atari5200 in $FNLIB)" >&2; exit 1; }

CFLAGS=(-t atari5200 -O --cpu 6502 -I "$FNLIB/include" -I "$FNLIB/bus/atari5200" -I .)
# The top 1K of RAM is the screen (antic.c). The crt0 comes first on the
# link line, so it, not cc65's, is __STARTUP__.
LDFLAGS=(-t atari5200 -Wl "-D,__RESERVED_MEMORY__=${RESERVE:-0x400}")

mkdir -p build/obj
python3 "$BOOK/tools/mkfont.py" > build/obj/font.inc
cl65 -t atari5200 --asm-include-dir build/obj -c -o build/obj/antic_dl.o antic_dl.s
cl65 "${CFLAGS[@]}" -c -o build/obj/antic.o antic.c

# bank: a 64K Super Cart. bank.cfg writes bank 0's own 4K to its own file;
# it goes over a copy of bank 1, and bank 0 goes first.
compose_bank() {
    python3 - "$1" build/obj/bank0.part <<'EOP'
import sys
b1 = open(sys.argv[1], "rb").read()
b0 = bytearray(b1)
b0[0x6000:0x7000] = open(sys.argv[2], "rb").read()
open(sys.argv[1], "wb").write(bytes(b0) + b1)
EOP
}

for name in "$@"; do
    objs=("build/obj/$name.o")
    cfg=$MEKKO/atari5200-fujinet.cfg
    if [ "$name" = bank ]; then
        cfg=bank.cfg
        cl65 "${CFLAGS[@]}" -c -o build/obj/bank0.o bank0.c
        cl65 -t atari5200 -c -o build/obj/supercart.o supercart.s
        objs+=(build/obj/bank0.o build/obj/supercart.o)
    fi
    cl65 "${CFLAGS[@]}" -c -o "build/obj/$name.o" "$name.c"
    cl65 "${LDFLAGS[@]}" -C "$cfg" -m "build/obj/$name.map" \
        -o "build/$name.bin" "$MEKKO/atari5200-crt0.s" "${objs[@]}" \
        build/obj/antic.o build/obj/antic_dl.o "$LIB"
    [ "$name" = bank ] && compose_bank "build/$name.bin"
    python3 "$MEKKO/atari5200-romstamp.py" --map "build/obj/$name.map" "build/$name.bin"
    python3 "$BOOK/tools/checkrom.py" --claim --map "build/obj/$name.map" "build/$name.bin"
done

```

D. Game and CONFIG Listings

The 5200 layers of Battleship (`src/atari5200/` at `3f2a548`) and 5 Card Stud (`src/atari5200/` at `d5db857`), and CONFIG's screens and calls (`atari5200/` at fujinet-config `1bd6ec8`). The shared game code is every port's and is not printed; CONFIG's text engine is in Appendix C.

Battleship

fujinet.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief   FujiNet calls - Atari 5200 (CUSTOM_FUJINET_CALLS, and the lobby boot)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 *
 * src/atari7800/fujinet.c on the 5200's cartridge. Everything that talks to
 * the cartridge through fujinet-lib is in this one file, because it has to
 * see the LIBRARY's <fujinet-fuji.h>: the atari5200 target implements most
 * fuji_* calls as macros over its bus call, so the prototypes in
 * src/fujinet-fuji.h link against nothing. This file therefore includes none
 * of the shared headers (src/misc.h, src/platform-specific/), and the shared
 * code reaches the cartridge through the CUSTOM_FUJINET_CALLS hooks instead.
 */

#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include <fujinet-fuji.h>
#include <fujinet-network.h>
#include <fujinet-atari5200.h>
#include "vars.h"

// graphics.c - declared here rather than through ../platform-specific, which
// would bring in src/fujinet-fuji.h
void drawText(uint8_t x, uint8_t y, const char *s);
void drawSpace(uint8_t x, uint8_t y, uint8_t w);
void waitvsync(void);

#define FUJI_HOST_SLOT_COUNT 8
#define LOBBY_DEVICE_SLOT 0
#define LOBBY_MODE_READ 1

int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    int16_t read;

    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;

    read = network_read(url, buffer, max_len);
    network_close(url);
    return read;
}

uint16_t custom_read_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, char *destination)
{
    uint16_t read = 0;

    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    if (!fuji_read_appkey(key_id, &read, (uint8_t *)destination))
        read = 0;
    return read;
}

void custom_write_appkey(uint16_t creator_id, uint8_t app_id, uint8_t key_id, uint16_t count, char *data)
{
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    fuji_write_appkey(key_id, count, (uint8_t *)data);
}

// The charset has letters, digits and : - . / ! , only
static void quitStatus(const char *s)
{
    drawSpace(0, HEIGHT - 1, WIDTH);
    drawText((WIDTH - (unsigned char)strlen(s)) / 2, HEIGHT - 1, s);
}

static bool sameHost(const char *a, const char *b)
{
    while (*a && *b)
    {
        if ((*a | 0x20) != (*b | 0x20))
            return false;
        a++;
        b++;
    }
    return *a == *b;
}

/*
  SET_DEVICE_FULLPATH is a fixed 256-byte payload, so the path is padded
  here rather than at the call site. The path is CONFIG's lobby.
*/
static const char lobbyPath[MAX_FILENAME_LEN] = "/atari5200/lobby.a52";
static const char lobbyHost[] = "ec.tnfs.io";
static HostSlot slots[FUJI_HOST_SLOT_COUNT];

static uint8_t findLobbyHost(void)
{
    uint8_t i;

    if (!fuji_get_host_slots(slots, FUJI_HOST_SLOT_COUNT))
        return FUJI_HOST_SLOT_COUNT;

    for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
        if (sameHost(lobbyHost, (const char *)slots[i]))
            return i;

    return FUJI_HOST_SLOT_COUNT;
}

// CONFIG's wait: the cart gives a push 60 seconds.
```

```
#define LOBBY_FRAMES 3900u

void quit(void)
{
    uint8_t slot, state, pct = 0xFF;
    uint16_t frames = 0;
    char pctText[4];

    quitStatus("LOADING LOBBY...");

    if (!fuji_a5200_present())
    {
        quitStatus("NO FUJINET CARTRIDGE");
        return;
    }

    // The slot is used as found, never created: a player who came through
    // the FujiNet Lobby already has the host.
    slot = findLobbyHost();
    if (slot == FUJI_HOST_SLOT_COUNT)
    {
        quitStatus("ADD EC.TNFS.IO IN CONFIG");
        return;
    }

    if (!fuji_mount_host_slot(slot)
        || !fuji_set_device_filename(LOBBY_MODE_READ, slot, LOBBY_DEVICE_SLOT, (char *)lobbyPath)
        || !fuji_mount_disk_image(LOBBY_DEVICE_SLOT, LOBBY_MODE_READ))
    {
        quitStatus("LOBBY NOT AVAILABLE");
        return;
    }

    // Some FujiNets push the image after answering MOUNT_IMAGE, so watch the
    // cart's own progress rather than trust the reply.
    for (;;)
    {
        state = fuji_a5200_boot_state();
        if (state == FUJI_A5200_BOOT_READY)
            break;
        if (state == FUJI_A5200_BOOT_FAILED || ++frames > LOBBY_FRAMES)
        {
            quitStatus("LOBBY LOAD FAILED");
            return;
        }
        if (fuji_a5200_boot_percent() != pct)
        {
            pct = fuji_a5200_boot_percent();
            utoa(pct, pctText, 10);
            drawSpace(30, HEIGHT - 1, 3);
            drawText(30, HEIGHT - 1, pctText);
        }
        waitvsync();
    }

    // Hand the console to the cartridge's loader. It returns only if the
    // cart never armed the swap.
    if (!fuji_a5200_boot())
        quitStatus("LOBBY BOOT FAILED");
}

#endif /* BUILD_ATARI5200 */

```

vars.h

```
#ifdef BUILD_ATARI5200

#ifndef KEYMAP_H
#define KEYMAP_H

#include <joystick.h>

// Screen dimensions for platform: the Atari 8-bit's ANTIC mode 4 screen
#define WIDTH 40
#define HEIGHT 26

#define GAMEOVER_PROMPT_Y HEIGHT - 3

// Icons - codes in support/atari/charset.fnt, as on the Atari 8-bit
#define ICON_TEXT_CURSOR 0x3A
#define ICON_MARK 0x2B
#define ICON_MARK_ALT 0
#define ICON_PLAYER 0x2A

/* readJoystick() (src/atari5200/input.c) returns the stick only, in cc65's
   joystick bits. The buttons and the keypad never appear in the joystick
   byte: input.c reports them as key codes behind kbhit()/cgetc(), as on the
```

```
   7800, so the "press any key" screens see the fire buttons too. */
#undef JOY_BTN_1
#undef JOY_BTN_2
#define JOY_BTN_1(v) ((v) & 0)
#define JOY_BTN_2(v) ((v) & 0)

/**
 * Platform specific key map for common input
 */

// No keyboard: an analog stick, two fire buttons and a twelve-key keypad.
// Direction comes from the stick, so the arrow codes are unreachable
// placeholders for readCommonInput()'s switch.
#define KEY_LEFT_ARROW      0xF1
#define KEY_LEFT_ARROW_2    0xF2
#define KEY_LEFT_ARROW_3    0xF3

#define KEY_RIGHT_ARROW     0xF4
#define KEY_RIGHT_ARROW_2   0xF5
#define KEY_RIGHT_ARROW_3   0xF6

#define KEY_UP_ARROW        0xF7
#define KEY_UP_ARROW_2      0xF8
#define KEY_UP_ARROW_3      0xF9

#define KEY_DOWN_ARROW      0xFA
#define KEY_DOWN_ARROW_2    0xFB
#define KEY_DOWN_ARROW_3    0xFC

#define KEY_RETURN 0x0D
#define KEY_ESCAPE 0x1B
#define KEY_ESCAPE_ALT 0x03
#define KEY_SPACEBAR 0x20
#define KEY_BACKSPACE 0x08

// What the shared screens call the escape key
#define ESCAPE "PAUSE"
#define ESC "PAUSE"

// Name entry happens on src/atari5200/osk.c's on-screen keyboard.
#define USE_PLATFORM_NAME_ENTRY 1

// The keypad stands in for the letter shortcuts the other ports type, with
// the ColecoVision's digits. src/atari5200/input.c maps them; keep the two
// in step.
//   top fire, #, START    RETURN   join / fire / place / ready
//   lower fire, 1         'r'      refresh the table list, rotate a ship
//   PAUSE, *              ESCAPE   the in-game menu
//   2 'h' how to play     3 's' sound     4 'n' name     5 'q' quit
// The charset has letters, digits and : - . / ! , only.
#define TABLE_STATUS_TEXT "1:Refresh  2:Help  4:Name  5:Quit"
#define MENU_QUIT_TEXT "  5: quit game"
#define MENU_HELP_TEXT "  2: how to play"
#define MENU_SOUND_ON_TEXT "  3: sound ON "
#define MENU_SOUND_OFF_TEXT "  3: sound OFF"
#define ROTATE_PROMPT_TEXT "press LOWER FIRE to rotate"
#define READY_PROMPT_TEXT "press FIRE when ready"
#define MENU_CLOSE_TEXT "press FIRE to close"

#endif /* KEYMAP_H */

#endif /* BUILD_ATARI5200 */

```

graphics.c

```
#ifdef BUILD_ATARI5200

/*
  Graphics functionality - Atari 5200

  The Atari 8-bit's screen (src/atari/graphics.c): ANTIC mode 4 at 40x26 in
  support/atari/charset.fnt, single-line player graphics for the target
  cursors. There is no OS: the display list (charset.s) is our own, in ROM,
  and the chips are written directly or through the crt0 VBI's zero-page
  shadows. The cart window has no R/W line, so everything rewritten lives
  in the top 4K of RAM, which the Makefile reserves:

```

```
    $3000  the charset, copied from ROM (1K-aligned for CHBASE)
    $3400  the screen, 40x26 (inside one 4K block, ending in the PMG
           block's first 768 bytes, which single-line PMG never reads)
    $3800  player/missile graphics (2K-aligned for PMBASE)
*/

#include "../misc.h"
#include <peekpoke.h>

extern unsigned char charset[];
extern unsigned char displayList[];

#define CHARSET_LOC 0x3000
#define SCREEN_LOC ((uint8_t *)0x3400)
#define PM_BASE 0x3800

#define xypos(x, y) (SCREEN_LOC + x + (y) * WIDTH)

// The crt0 VBI copies these shadows to the chips every frame
#define SDLSTL 0x05
#define SDMCTL 0x07
#define PCOLR0 0x08

// No shadows: written directly
#define HPOSP0 0xC000
#define PAL 0xC014
#define PRIOR 0xC01B
#define GRACTL 0xC01D
#define PMBASE 0xD407
#define CHBASE 0xD409

#define TILE_SEA 0x38
#define TILE_MISS 0xE1
#define TILE_HIT 0x39
#define TILE_HIT2 0x1B
#define TILE_HIT_LEGOND 0x1C

static uint8_t colIndex = 0, fieldX = 0, playerCount;
static bool inGameCharSet = false;
static uint16_t lastCursor[] = {0, PM_BASE + 1024, PM_BASE + 1024, PM_BASE + 1024};

static uint16_t quadrant_offset[] = {
    WIDTH * 14 + 8,
    WIDTH * 2 + 8,
    WIDTH * 2 + 21,
    WIDTH * 14 + 21};
uint8_t legendShipOffset[] = {2, 1, 0, 40 * 5, 40 * 6 + 1};

// Color scheme:
// Player/Missle: Target cursor, UNUSED, UNUSED, UNUSED,
// White, Blue, Red, Green, Black background
const unsigned char colors[] = {
    0xff, 0xff, 0xff, 0xff, 0x00, 0x0F, 0x84, 0x46, 0xC8, 0x02, // NTSC
    0xff, 0xff, 0xff, 0xff, 0x00, 0x0F, 0x74, 0x26, 0xB8, 0x02  // PAL TBD
};

const uint8_t cursor_pmg[] = {
    0b11110000,
    0b10010000,
    0b10010000,
    0b00000000,
    0b00000000,
    0b10010000,
    0b10010000,
    0b11110000};

// Defined in this file
void drawTextAdd(uint8_t *dest, const char *s, uint8_t add);
void drawShipInternal(uint8_t *dest, uint8_t size, uint8_t delta);

unsigned char cycleNextColor()
{
    return 0;
}

void initGraphics()
{
    // RAM is not cleared at power-on: the screen by resetScreen(), the
    // missiles and players here
    memset((void *)(PM_BASE + 768), 0, 0x500);
    memcpy((void *)CHARSET_LOC, charset, 1024);

    resetScreen();

    // Just past the VBI, so it never copies a half-written pointer
    waitvsync();
    POKEW(SDLSTL, (uint16_t)displayList);
    POKE(CHBASE, CHARSET_LOC / 256);

    // Initialize player+missle graphics, single line
    POKE(SDMCTL, PEEK(SDMCTL) | (12 + 16));

    POKE(PRIOR, 1); // PM graphics above playfield

    POKE(PMBASE, PM_BASE / 256);

    // PM
    // Turn on P+M
    POKE(GRACTL, 3);

    // PAL colors
    if (PEEK(PAL) == 1)
    {
        colIndex = 10;
    }

    // Set color palette
    memcpy((void *)PCOLR0, &colors[colIndex + 1], 9);
}

bool saveScreenBuffer()
{
    return false;
}

void restoreScreenBuffer()
{
}

void drawText(unsigned char x, unsigned char y, const char *s)
{
    static unsigned char c;
    static unsigned char *pos;

    pos = xypos(x, y);

    while (c = *s++)
    {
        if (c > 90 || (c < 65 && c > 32))
            c -= 32;
        else if (c == 0x20)
            c = 0x40;
        *pos++ = c;
    }
}

void drawTextAlt(unsigned char x, unsigned char y, const char *s)
{
    static uint8_t c;
    static uint8_t *pos;

    pos = xypos(x, y);

    while (c = *s++)
    {
        if (c > 90 || (c < 65 && c >= 32))
        {
            // Use alternate numbers if showing the clock
            if (inGameCharSet && y == HEIGHT - 1 && c >= 0x30 && c <= 0x39)
                c += 0x60;
            c += 96;
        }

        *pos++ = c;
    }
}

void resetScreen()
{
    waitvsync();
    memset((void *)SCREEN_LOC, 0, WIDTH * HEIGHT);
    if (inGameCharSet)
    {
```

```
        // Restore normal charset
        memcpy((void *)CHARSET_LOC, charset, 1024);
        inGameCharSet = false;

        // Clear any cursors
        memset((void *)(PM_BASE + 1024), 0, 768);
    }
}

void drawIcon(unsigned char x, unsigned char y, unsigned char icon)
{
    POKE(xypos(x, y), icon);
}

void drawBlank(unsigned char x, unsigned char y)
{
    POKE(xypos(x, y), 0);
}

void drawSpace(unsigned char x, unsigned char y, unsigned char w)
{
    memset(xypos(x, y), 0, w);
}

void drawClock()
{
    POKE(xypos(WIDTH - 1, HEIGHT - 1), 0x1D);
}

void drawConnectionIcon(bool show)
{
    POKEW(xypos(0, HEIGHT - 1), show ? 0x1f1e : 0);
}

void drawTextAdd(uint8_t *dest, const char *s, uint8_t add)
{
    char c;

    while ((c = *s++))
    {
        // Digits too, as drawText() maps them: names can have them here
        if ((c >= 97 && c <= 122) || (c >= '0' && c <= '9'))
            c -= 32;
        else if (c == 0x20)
            c = 0x40;
        *dest++ = c + add;
    }
}

void drawPlayerName(uint8_t player, const char *name, bool active)
{
    static uint8_t i, add;
    uint8_t *dest = SCREEN_LOC + fieldX + quadrant_offset[player] - WIDTH - 1;
    add = active ? 0 : 128;

    // Draw top and bottom borders and name label
    if (player == 0 || player == 3)
    {
        // Bottom player boards

        // Thin horizontal border
        dest[0] = 0x08 + add;
        dest[11] = 0x09 + add;
        memset(dest + 1, 0x27 + add, 10);

        // Name Label
        dest[WIDTH * 11] = 0x5E + add;
        dest[WIDTH * 11 + 11] = 0x5F + add;
        memset(WIDTH * 11 + 1 + dest, 0x60 + add, 10);
        drawTextAdd(dest + WIDTH * 11 + 2, name, add);

        // Active indicator
        if (active)
        {
            dest[WIDTH * 11 + 1] = 0x5b;
        }

        // Bottom border below name label
        dest[WIDTH * 12] = 0x20 + add;
        dest[WIDTH * 12 + 11] = 0x21 + add;
        memset(WIDTH * 12 + 1 + dest, 0x28 + add, 10);
    }
    else
    {
        // Top player boards

        // Top border above name label
        *(dest - WIDTH) = 0x5;
        *(dest - WIDTH + 11) = 0x6;
        memset(dest - WIDTH + 1, 0x26, 10);

        // Name Label
        dest[0] = 0x5C + add;
        dest[11] = 0x5D + add;
        memset(dest + 1, 0x60 + add, 10);
        drawTextAdd(dest + 2, name, add);

        // Active indicator
        if (active)
        {
            dest[1] = 0x5b;
        }

        // Thin horizontal border
        dest[WIDTH * 11] = 0x0A + add;
        dest[WIDTH * 11 + 11] = 0x0B + add;
        memset(WIDTH * 11 + 1 + dest, 0x29 + add, 10);
    }

    // Draw left/right borders and drawers
    if (player > 1 || playerCount == 2 && player > 0)
    {
        // Right drawer
        dest += WIDTH;
        memset(dest + 12, 0x31 + add, 3);
        dest[11] = 0x25 + add;
        dest[15] = 0x2D + add;
        dest[0] = 0x22 + add; // Left edge

        for (i = 0; i < 8; i++)
        {
            dest += WIDTH;
            dest[11] = 0x03 + add;
            dest[15] = 0x02 + add;
            dest[0] = 0x22 + add; // Left edge
        }

        dest += WIDTH;
        memset(dest + 12, 0x31 + add, 3);
        dest[11] = 0x25 + add;
        dest[15] = 0x2F + add;
        dest[0] = 0x22 + add; // Left edge
    }
    else
    {
        // Left drawer
        dest += WIDTH - 4;
        dest[0] = 0x2C + add;
        memset(dest + 1, 0x31 + add, 3);
        dest[4] = 0x24 + add;
        dest[15] = 0x23 + add; // Right edge

        for (i = 0; i < 8; i++)
        {
            dest += WIDTH;
            dest[0] = dest[4] = 0x02 + add;
            dest[15] = 0x23 + add; // Right edge
        }

        dest += WIDTH;
        dest[0] = 0x2E + add;
        memset(dest + 1, 0x31 + add, 3);
        dest[4] = 0x24 + add;
        dest[15] = 0x23 + add; // Right edge
    }
}

void drawBoard(uint8_t currentPlayerCount)
{
    static uint8_t i, y;
    static uint8_t *dest;
    playerCount = currentPlayerCount;
    fieldX = playerCount > 2 ? 0 : 7;

    if (playerCount > 1 && !inGameCharSet)
    {
        // Invert 0-9 & A-Z characters for in-game charset
        inGameCharSet = true;

```

```
        dest = CHARSET_LOC + 0x01 * 8;
        while (dest < CHARSET_LOC + 0x5b * 8)
        {
            *dest = *dest ^ 0xff | 0b01010101;
            dest++;
            if (dest == CHARSET_LOC + 0x1A * 8)
                dest = CHARSET_LOC + 0x40 * 8;
            if (dest == CHARSET_LOC + 0x02 * 8)
                dest = CHARSET_LOC + 0x10 * 8;
        }
    }

    for (i = 0; i < playerCount; i++)
    {
        dest = SCREEN_LOC + fieldX + quadrant_offset[i];

        // Draw player border
        drawPlayerName(i, "", false);

        // Blue gamefield
        for (y = 0; y < 10; y++)
        {
            memset(dest + y * WIDTH, TILE_SEA, 10);
        }

        // Blue drawer
        if (i > 1 || (i > 0 && fieldX > 0))
        {
            dest += WIDTH + 11;
        }
        else
        {
            dest += WIDTH - 4;
        }
        for (y = 0; y < 8; y++)
        {
            memset(dest, TILE_SEA, 3);
            dest += WIDTH;
        }
    }

    dest = xypos(16, 0);
}

void drawLine(unsigned char x, unsigned char y, unsigned char w)
{
    memset(xypos(x, y), 0x3F, w);
}

void drawShipInternal(uint8_t *dest, uint8_t size, uint8_t delta)
{
    static uint8_t i;
    uint8_t c = 0x32;
    uint8_t *src;
    if (delta)
        c = 0x37;

    for (i = 0; i < size; i++)
    {
        *dest = c;
        if (delta)
        {
            c = 0x36;
            if (i == size - 2)
                c = 0x35;
            dest += WIDTH;
        }
        else
        {
            dest++;
            c = 0x33;
            if (i == size - 2)
                c = 0x34;
        }
    }
}

void drawShip(uint8_t quadrant, uint8_t size, uint8_t pos, bool hide)
{
    uint8_t i, delta = 0;
    uint8_t *dest;

    if (pos > 99)
    {
        delta = 1; // 1=vertical, 0=horizontal
        pos -= 100;
    }

    dest = xypos((pos % 10), (pos / 10)) + fieldX + quadrant_offset[quadrant];

    if (hide)
    {
        if (!delta)
            memset(dest, 0x38, size);
        else
            for (i = 0; i < size; i++)
                *(dest + i * WIDTH) = 0x38;
        return;
    }

    drawShipInternal(dest, size, delta);
}

void drawLegendShip(uint8_t player, uint8_t index, uint8_t size, uint8_t status)
{
    static uint8_t i;
    uint8_t *dest = SCREEN_LOC + fieldX + quadrant_offset[player] + legendShipOffset[index];

    if (player > 1 || (player > 0 && fieldX > 0))
    {
        dest += WIDTH + 11;
    }
    else
    {
        dest += WIDTH - 4;
    }

    if (status)
    {
        drawShipInternal(dest, size, 1);
    }
    else
    {
        for (i = 0; i < size; i++)
            *(dest + i * WIDTH) = TILE_HIT_LEGOND;
    }
}

void drawGamefield(uint8_t quadrant, uint8_t *field)
{
    static uint8_t y, x;
    uint8_t *dest = SCREEN_LOC + quadrant_offset[quadrant] + fieldX;

    for (y = 0; y < 10; ++y)
    {
        for (x = 0; x < 10; ++x)
        {
            if (*field)
            {
                *dest = *field == 1 ? TILE_HIT : TILE_MISS;
            }
            field++;
            dest++;
        }

        dest += WIDTH - 10;
    }
}

static bool cursorVisible = false;
void drawGamefieldUpdate(uint8_t quadrant, uint8_t *gamefield, uint8_t attackPos, uint8_t anim)
{
    uint8_t *dest = SCREEN_LOC + quadrant_offset[quadrant] + fieldX + (uint16_t)(attackPos / 10) * WIDTH + (attackPos % 10);
    uint8_t c = gamefield[attackPos];

    if (cursorVisible)
    {
        cursorVisible = false;
        memset((void *)(PM_BASE + 1024), 0, 768);
    }

    // Animate attack only
    if (anim > 9)
    {
        *dest = 217 + anim;
        return;
```

```
    }

    if (c == FIELD_ATTACK)
    {
        *dest = anim ? TILE_HIT2 : TILE_HIT;
    }
    else if (c == FIELD_MISS)
    {
        *dest = TILE_MISS;
    }
}
void drawGamefieldCursor(uint8_t quadrant,
uint8_t x, uint8_t y, uint8_t *gamefield,
uint8_t blink)
{
    uint16_t pos = quadrant_offset[quadrant] +
y * WIDTH + fieldX + x;
    uint16_t lc = lastCursor[quadrant];

    // Only opponents (1-3) have a cursor, on
players 0-2: quadrant 0 would
    // clear zero page (lastCursor[0] is 0)
and store to HPOSP0 - 1, in the
    // cart window
    if (!quadrant)
        return;

    lastCursor[quadrant] = PM_BASE + 768 + 24
+ quadrant * 256 + (pos / WIDTH) * 8;

    memset((void *)lc, 0, 8);
    memcpy((void *)lastCursor[quadrant],
&cursor_pmg, 8);

    POKE(HPOSP0 - 1 + quadrant, (pos % WIDTH)
* 4 + 48); // horiz loc
    cursorVisible = true;

    (void)gamefield;
}

void drawEndgameMessage(const char *message)
{
    uint8_t i, x;
    i = (uint8_t)strlen(message);
    x = WIDTH / 2 - i / 2;

    memset(xypos(0, HEIGHT - 2), 0x62, WIDTH);
    memset(xypos(0, HEIGHT - 1), 0x40, x);
    memset(xypos(x + i, HEIGHT - 1), 0x40,
WIDTH - x - i);
    drawText(x, HEIGHT - 1, message);
}

void drawBox(unsigned char x, unsigned char y,
unsigned char w, unsigned char h)
{
    uint8_t *pos = xypos(x, y);
    // Draw corners
    *pos = 0x3b;
    pos[w + 1] = 0x3c;
    pos[h * WIDTH + WIDTH] = 0x3d;
    pos[w + h * WIDTH + WIDTH + 1] = 0x3e;
}

void resetGraphics()
{
    waitvsync();
}

#endif /* BUILD_ATARI5200 */
```

charset.s

```
; The Atari 8-bit's custom charset and display
list, in ROM. graphics.c copies
; the charset into RAM, where it is rewritten
for the game board, and ANTIC
; reads the display list from here. Keep
SCREEN in step with graphics.c.

        .export _charset, _displayList

SCREEN = $3400

        .segment "FONT"
_charset:
        .incbin "support/atari/charset.fnt"

; 26 lines of mode 4 under 16 blank ones, as
the 8-bit's
        .segment "DLIST"
_displayList:
        .byte   $70, $70
        .byte   $44, <SCREEN, >SCREEN
        .repeat 25
        .byte   $04
        .endrepeat
        .byte   $41, <_displayList,
>_displayList
```

input.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief Atari 5200 input routines
 * @author Thomas Cherryhomes
 * @license gpl v.3
 */

#include <joystick.h>
#include <fujinet-atari5200.h>
#include "vars.h"
#include "pad.h"
#include "../platform-specific/input.h"
#include "../platform-specific/graphics.h"

/*
  Controller 1: the analog stick from the crt0 VBI's POT shadows, the top
  fire on TRIG0, the lower fire on SKSTAT, and the keypad from the lib's
  keypad IRQ (fuji_a5200_key, one event per press).

  Only the stick travels in readJoystick(). The buttons and the keypad are
  synthesized as key codes behind kbhit()/cgetc(), as on the 7800; vars.h
  lists the mapping.
*/

#define REG(a)  (*(volatile unsigned char *)(a))
#define PADDL0  REG(0x11)   /* stick X, 0-228, 114 at rest */
#define PADDL1  REG(0x12)   /* stick Y */
#define TRIG0   REG(0xC010) /* top fire: 0 while held */
#define SKSTAT  REG(0xE80F) /* bit 3 clear: the lower fire */
#define ATRACT  REG(0x04)   /* the crt0 clears it for the keypad only */

/* Past these the stick is pushed; between them it is at rest. */
#define STICK_LO 0x40
#define STICK_HI 0xA4

static unsigned char lastButtons;
static unsigned char pendingKey;

unsigned char readPad(void)
{
    unsigned char b = 0, x = PADDL0, y = PADDL1;

    if (y < STICK_LO)
        b |= JOY_UP_MASK;
    else if (y > STICK_HI)
        b |= JOY_DOWN_MASK;
    if (x < STICK_LO)
        b |= JOY_LEFT_MASK;
    else if (x > STICK_HI)
        b |= JOY_RIGHT_MASK;

    if (TRIG0 == 0)
        b |= PAD_FIRE;
    if (!(SKSTAT & 0x08))
        b |= PAD_BACK;
    if (b)
        ATRACT = 0;
    return b;
}

unsigned char readKeypad(void)
{
    unsigned char k = fuji_a5200_key;

    if (k)
        fuji_a5200_key = 0;
    return k;
}

unsigned char readJoystick()
{
    return readPad() & PAD_DIRS;
}

static unsigned char keyFor(unsigned char k)
{
    switch (k)
    {
    case FUJI_A5200_KEY_HASH:
    case FUJI_A5200_KEY_START:
        return KEY_RETURN;
    case FUJI_A5200_KEY_DIGIT(1):
        return 'r';
    case FUJI_A5200_KEY_STAR:
    case FUJI_A5200_KEY_PAUSE:
        return KEY_ESCAPE;
    case FUJI_A5200_KEY_DIGIT(2):
        return 'h';
    case FUJI_A5200_KEY_DIGIT(3):
        return 's';
    case FUJI_A5200_KEY_DIGIT(4):
        return 'n';
    case FUJI_A5200_KEY_DIGIT(5):
        return 'q';
    }
    return 0;
}

static unsigned char poll(void)
{
    unsigned char buttons = readPad() & ~PAD_DIRS;
    unsigned char pressed = buttons & ~lastButtons;
    unsigned char k = keyFor(readKeypad());

    lastButtons = buttons;

    // The keypad wins: it is the deliberate press
    if (k)
        return k;
    if (pressed & PAD_FIRE)
        return KEY_RETURN;
    if (pressed & PAD_BACK)
        return 'r';
    return 0;
}

unsigned char kbhit(void)
{
    if (!pendingKey)
        pendingKey = poll();

    return pendingKey != 0;
}

unsigned char cgetc(void)
{
    unsigned char k;

    while (!kbhit())
        waitvsync();

    k = pendingKey;
    pendingKey = 0;
    return k;
}

#endif /* BUILD_ATARI5200 */

```

pad.h

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Atari 5200 controller - shared by input.c and osk.c
 * @license gpl v. 3, see LICENSE for details
 */

#ifndef PAD_H
#define PAD_H

// The stick in cc65's JOY_*_MASK bits (0x0F), the fire buttons above them.
#define PAD_FIRE 0x10 /* top fire */
#define PAD_BACK 0x20 /* lower fire */

#define PAD_DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK | JOY_RIGHT_MASK)

unsigned char readPad(void);

/* The keypad key pressed since the last call, as FUJI_A5200_KEY_*; 0 none. */
unsigned char readKeypad(void);

#endif /* PAD_H */

#endif /* BUILD_ATARI5200 */

```

osk.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Atari 5200 on-screen keyboard
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <joystick.h>
#include <string.h>
#include <fujinet-atari5200.h>
#include "vars.h"
#include "pad.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/sound.h"

/*
  The 7800 port's grid (src/atari7800/osk.c), on the 5200's controller. The
  stick moves, the top fire types the cell under the cursor, the lower fire
  or * backspaces, and START or # accepts; the keypad's digits type
  themselves. There is no cancel: the caller only reaches this screen when
  a name is required.

  Entry is uppercase-only (the server lowercases names anyway - see
  welcomeActionVerifyPlayerName()). The charset has no arrows or '<', so the
  cursor is the table list's mark and the specials are words.
*/

#define OSK_COLS 10
#define OSK_ROWS 4
#define OSK_PITCH 4 /* marker column + up to three label columns */
#define OSK_X 1
#define OSK_Y 19    /* below the name box showPlayerNameScreen draws at 16-18 */
#define OSK_ROW(r) (OSK_Y + (r) * 2) /* a blank row between: mode 4 has no gaps */

/* The last row is one short: its tenth cell does not exist. */
#define OSK_LAST_ROW_COLS 9

#define CELL_SPACE (OSK_COLS * 3 + 6) /* index 36 */
#define CELL_DEL (CELL_SPACE + 1)
#define CELL_OK (CELL_SPACE + 2)

static const char cells[] =
    "ABCDEFGHIJ"
    "KLMNOPQRST"
    "UVWXYZ0123"
    "456789";

#define REPEAT_FIRST 18 /* vblanks before a held stick repeats */
#define REPEAT_NEXT 5

static unsigned char curX, curY;

static unsigned char cellCols(unsigned char row)
{
    return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS : OSK_COLS;
}

static void drawCell(unsigned char col, unsigned char row, unsigned char marked)
{
    unsigned char idx = row * OSK_COLS + col;
    unsigned char x = OSK_X + col * OSK_PITCH, y = OSK_ROW(row);
    char glyph[2];

    if (marked)
        drawIcon(x, y, ICON_MARK);
    else
        drawBlank(x, y);

    if (idx == CELL_SPACE)
        drawText(x + 1, y, "SPC");
    else if (idx == CELL_DEL)
        drawText(x + 1, y, "DEL");
    else if (idx == CELL_OK)
        drawText(x + 1, y, "OK");
    else
    {
        glyph[0] = cells[idx];
        glyph[1] = 0;
        drawText(x + 1, y, glyph);
    }
}

static void drawGrid(void)
{
    unsigned char row, col;

    for (row = 0; row < OSK_ROWS; row++)
```

```
90          for (col = 0; col < cellCols(row); col++)
91              drawCell(col, row, row == curY && col == curX);
92      }
93
94      static void clearGrid(void)
95      {
96          unsigned char row;
97
98          for (row = 0; row < OSK_ROWS; row++)
99              drawSpace(0, OSK_ROW(row), WIDTH);
100     }
101
102     /* Repaint the field and its trailing cursor chip, the same shape
103        inputFieldCycle() draws on the platforms that have a keyboard. */
104     static void drawField(unsigned char x, unsigned char y, unsigned char max,
105                           const char *buffer)
106     {
107         unsigned char len = (unsigned char)strlen(buffer);
108
109         drawSpace(x, y, max);
110         drawText(x, y, buffer);
111         if (len < max)
112             drawIcon(x + len, y, ICON_TEXT_CURSOR);
113     }
114
115     static void appendChar(char *buffer, unsigned char *len, unsigned char max,
116                            char ch)
117     {
118         if (*len >= max)
119         {
120             soundInvalid();
121             return;
122         }
123         buffer[(*len)++] = ch;
124         buffer[*len] = 0;
125         soundCursor();
126     }
127
128     /* What a keypad key does here: a cell index, or 0xFF for nothing. */
129     static unsigned char keypadCell(unsigned char k)
130     {
131         switch (k)
132         {
133         case FUJI_A5200_KEY_HASH:
134         case FUJI_A5200_KEY_START:
135             return CELL_OK;
136         case FUJI_A5200_KEY_STAR:
137             return CELL_DEL;
138         }
139         if (k >= FUJI_A5200_KEY_DIGIT(0) && k <= FUJI_A5200_KEY_DIGIT(9))
140             return (unsigned char)(strchr(cells, '0' + k - FUJI_A5200_KEY_DIGIT(0)) - cells);
141         return 0xFF;
142     }
143
144     void platformNameEntry(unsigned char x, unsigned char y, unsigned char max,
145                            char *buffer)
146     {
147         unsigned char len = (unsigned char)strlen(buffer);
148         unsigned char hold = 0, lastDir = 0, lastButtons;
149         unsigned char pad, dir, move, pressed, idx;
150
151         curX = 0;
152         curY = 0;
153         drawGrid();
154         drawField(x, y, max, buffer);
155
156         // Whatever is already held (the button that opened this screen) is not a press.
157         lastButtons = readPad() & ~PAD_DIRS;
158         readKeypad();
159
160         for (;;)
161         {
162             waitvsync();
163
164             pad = readPad();
165             dir = pad & PAD_DIRS;
166             pressed = pad & ~PAD_DIRS & ~lastButtons;
167             lastButtons = pad & ~PAD_DIRS;
168
169             /* Auto-repeat is counted in vblanks. This loop runs once per frame,
170                so `hold` is already a frame count. */
171             move = 0;
172             if (dir != lastDir)
173             {
174                 lastDir = dir;
175                 hold = REPEAT_FIRST;
176                 move = dir;
177             }
178             else if (dir && --hold == 0)
179             {
180                 hold = REPEAT_NEXT;
181                 move = dir;
182             }
183
184             if (move)
185             {
186                 drawCell(curX, curY, 0);
187
188                 if ((move & JOY_LEFT_MASK) && curX)
189                     curX--;
190                 else if ((move & JOY_RIGHT_MASK) && curX + 1 < cellCols(curY))
191                     curX++;
192                 else if ((move & JOY_UP_MASK) && curY)
193                     curY--;
194                 else if ((move & JOY_DOWN_MASK) && curY + 1 < OSK_ROWS)
195                     curY++;
196
197                 if (curX >= cellCols(curY))
198                     curX = cellCols(curY) - 1;
199
200                 drawCell(curX, curY, 1);
201                 soundCursor();
202             }
203
204             /* The top fire takes whatever the cursor is sitting on; the lower
205                fire and the keypad are shortcuts. */
206             idx = keypadCell(readKeypad());
207             if (idx == 0xFF)
208             {
209                 if (pressed & PAD_BACK)
210                     idx = CELL_DEL;
211                 else if (pressed & PAD_FIRE)
212                     idx = curY * OSK_COLS + curX;
213                 else
214                     continue;
215             }
216
217             if (idx == CELL_OK)
218             {
219                 if (!len)
220                 {
221                     soundInvalid();
222                     continue;
223                 }
224                 break;
225             }
226
227             if (idx == CELL_DEL)
228             {
229                 if (!len)
230                 {
231                     soundInvalid();
232                     continue;
233                 }
234                 buffer[--len] = 0;
235                 soundCursor();
236             }
237             else
238                 appendChar(buffer, &len, max, idx == CELL_SPACE ? ' ' : cells[idx]);
239
240             drawField(x, y, max, buffer);
241         }
242
243         clearGrid();
244     }
245
246     #endif /* BUILD_ATARI5200 */
247
```

sound.c

```
#ifdef BUILD_ATARI5200

/*
  Platform specific sound functions - Atari 5200

  The Atari 8-bit's cues, unchanged. There is no OS, so no _sound(): voice()
  stores AUDF/AUDC as it did. SKCTL stays as the crt0 set it (scanning the
  keypad, which also keeps POKEY out of initialisation, so the counters run).
*/

#include <stdlib.h>
#include "../misc.h"

#define POKEY(r) (*(volatile uint8_t *)(0xE800 + (r)))
#define AUDF(v) POKEY((v) * 2)
#define AUDC(v) POKEY((v) * 2 + 1)
#define AUDCTL POKEY(0x8)

static void voice(uint8_t v, uint8_t frequency, uint8_t distortion, uint8_t volume)
{
    AUDF(v) = frequency;
    AUDC(v) = (uint8_t)(distortion << 4) + volume;
}

void initSound()
{
    AUDCTL = 0;
    soundStop();
}

void sound(unsigned char v, unsigned char frequency, unsigned char distortion, unsigned char volume)
{
    if (!prefs.disableSound)
        voice(v, frequency, distortion, volume);
}

void note(uint8_t n, uint8_t n2, uint8_t n3, uint8_t d, uint8_t f, uint8_t p)
{
    static uint8_t i;
    if (prefs.disableSound)
        return;

    voice(0, n, 10, 8);
    if (n2)
        voice(1, n2, 10, 6);
    if (n3)
        voice(2, n3, 10, 4);

    pause(d);

    for (i = 7; i < 255; i--)
    {
        voice(0, n, 10, i);
        if (n2 && i > 1)
            voice(1, n2, 10, i - 2);
        if (n3 && i > 3)
            voice(2, n3, 10, i - 4);
        pause(f);
    }
    pause(p);
}

void soundJoinGame()
{
    static uint8_t j;
    for (j = 0; j < 2; j++)
    {
        note(81, 0, 0, 0, 1, 0);
        if (j == 0)
            note(96, 0, 0, 0, 1, 0);
    }
}

void soundMyTurn()
{
    static uint8_t i, j;
    for (j = 0; j < 2; j++)
    {
        sound(0, 81, 10, 5);
        pause(2);
        for (i = 6; i < 255; i--)
        {
            sound(0, 81, 10, i);
            waitvsync();
        }
        waitvsync();
    }
}

void soundGameDone()
{
    note(128, 204, 64, 6, 2, 0);
    note(96, 153, 193, 25, 2, 3);
    note(76, 128, 153, 6, 2, 0);
    note(96, 153, 193, 25, 2, 3);
}

void soundCursor()
{
    sound(0, 91, 10, 7);
    pause(1);
    sound(0, 91, 10, 3);
    pause(1);
    soundStop();
}

void soundPlaceShip()
{
    sound(0, 96, 10, 5);
    pause(2);
    sound(0, 81, 10, 4);
    pause(2);
    soundStop();
}

void soundTick()
{
    sound(0, 200, 8, 7);
    waitvsync();
    soundStop();
}

void soundSelect()
{
    sound(0, 96, 10, 5);
    pause(2);
    sound(0, 81, 10, 4);
    pause(2);
    soundStop();
}

void soundMiss()
{
    uint8_t i;
    for (i = 0; i < 10; i++)
    {
        sound(0, 0, 8, 4 - i / 2);
        pause(2);
    }
}

void soundInvalid()
{
    static uint8_t i;
    for (i = 6; i < 255; i--)
    {
        sound(0, 255 - i * 5, 10, i);
        waitvsync();
    }
}

void soundAttack()
{
    uint8_t i;
    for (i = 1; i < 8; i++)
    {
        sound(0, 200 + i, 2, 7 - i);
        pause(2);
    }
}

void soundHit()
{
    uint8_t i;
    for (i = 0; i < 10; i++)
    {
        sound(0, 80 + i, 0, 9 - i);
        pause(2);
```

```
    }
    soundStop();
}

void soundSink()
{
    uint8_t i;
    for (i = 0; i < 10; i++)
    {
        sound(0, 50 + i, 0, 9 - i);
        pause(1);
        sound(0, 80 + i, 0, 9 - i);
        pause(1);
    }
    soundStop();
}

void soundStop()
{
    voice(0, 0, 0, 0);
}

// No key click to silence: the keypad is read through the IRQ, not the OS
void disableKeySounds()
{
}

void enableKeySounds()
{
}

#endif /* BUILD_ATARI5200 */
```

util.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Utility Functions - Atari 5200
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdbool.h>
#include <stdint.h>
#include <peekpoke.h>
#include "vars.h"
#include "../platform-specific/util.h"

// The crt0's VBI keeps the BIOS's zero page: RTCLOK is two bytes, high then
// low, and ATRACT starts colour cycling once it passes $80.
#define RTCLOKH 0x01
#define RTCLOKL 0x02
#define ATRACT  0x04
#define PAL     0xC014
#define RANDOM  0xE80A

void resetTimer()
{
    POKEW(RTCLOKH, 0);
}

uint16_t getTime()
{
    uint8_t h, l;

    // The VBI carries into the high byte between the two reads
    do
    {
        h = PEEK(RTCLOKH);
        l = PEEK(RTCLOKL);
    } while (h != PEEK(RTCLOKH));

    return h * 256 + l;
}

void housekeeping()
{
    // Clear attract mode
    POKE(ATRACT, 0);
}

uint8_t getJiffiesPerSecond()
{
    return PEEK(PAL) == 1 ? 50 : 60;
}

uint8_t getRandomNumber(uint8_t maxExclusive)
{
    return PEEK(RANDOM) % maxExclusive;
}

// quit() is in fujinet.c, with the rest of the cartridge calls.

#endif /* BUILD_ATARI5200 */
```

5 Card Stud

network.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Network functions
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdint.h>
#include <fujinet-network.h>

static uint8_t initialized = 0;

uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_len)
{
    int16_t count;

    if (!initialized) {
        if (network_init() != FN_ERR_OK)
            return 0;
        initialized = 1;
    }

    // Don't read from a channel that never opened - network_read() would
    // otherwise poll status on a dead unit and hand back whatever it got.
    if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 0;

    count = network_read(url, buffer, max_len);
    network_close(url);

    return count > 0;
}

#endif /* BUILD_ATARI5200 */

```

appkey.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Atari 5200 App Key Functions
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdint.h>
#include <string.h>
#include <fujinet-fuji.h>
#include "../platform-specific/appkey.h"

void read_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, char *data)
{
    uint16_t read = 0;

    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    if (!fuji_read_appkey(key_id, &read, (uint8_t *) data))
        read = 0;

    // Terminate as a string. Callers hand us the shared tempBuffer scratch
    // global and then strlen() it, so this has to happen even on failure or
    // they read back whatever the previous screen left behind.
    data[read] = 0;
}

void write_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, const char *data)
{
    fuji_set_appkey_details(creator_id, app_id, DEFAULT);
    fuji_write_appkey(key_id, strlen(data), (uint8_t *) data);
}

#endif /* BUILD_ATARI5200 */

```

vars.h

```
#ifdef BUILD_ATARI5200

#ifndef KEYMAP_H
#define KEYMAP_H

// The 8-bit Atari client's screen: ANTIC mode 4, 40x26, the bottom row the
// status bar under its own colours (src/atari5200/antic.s).

#define WIDTH 40
#define HEIGHT 26

#define POT_Y_MODIFIER 1
#define STATUS_TIMER_WIDTH 1

// A controller is an analog stick, two fire buttons and a 12-key keypad with
// START, PAUSE and RESET. src/atari5200/input.c synthesises key codes from
// them and src/atari5200/osk.c types text on screen.
#define USE_PLATFORM_SPECIFIC_INPUT 1
#define USE_PLATFORM_NAME_ENTRY 1

/**
 * Platform specific key map for common input
 */

// Direction comes from the stick, never from a key, so the arrow codes are
// deliberately unreachable placeholders -- they exist because readCommonInput()
// switches on all twelve of them.
#define KEY_LEFT_ARROW      0xF1
#define KEY_LEFT_ARROW_2    0xF2
#define KEY_LEFT_ARROW_3    0xF3

#define KEY_RIGHT_ARROW     0xF4
#define KEY_RIGHT_ARROW_2   0xF5
#define KEY_RIGHT_ARROW_3   0xF6

#define KEY_UP_ARROW        0xF7
#define KEY_UP_ARROW_2      0xF8
#define KEY_UP_ARROW_3      0xF9

#define KEY_DOWN_ARROW      0xFA
#define KEY_DOWN_ARROW_2    0xFB
#define KEY_DOWN_ARROW_3    0xFC

// Fire buttons and keypad.
#define KEY_RETURN       0x0D
#define KEY_ESCAPE       0x1B
#define KEY_ESCAPE_ALT   0x03
#define KEY_SPACE        0x20
#define KEY_BACKSPACE    0x08

/*
  Mapping for converting incoming ALT letters to a standard case
*/
#define LINE_ENDING 0x0A
#define ALT_LETTER_START 0x0
#define ALT_LETTER_END 0x0
#define ALT_LETTER_AND 0x0

#define QUERY_SUFFIX ""

/*
 The controls, as readPad() reports them: the stick in cc65's JOY_*_MASK
  bits, the top button as JOY_BTN_1_MASK and the lower one as PAD_BTN_2.
*/
#define PAD_BTN_2 0x20

unsigned char readPad(void);

// Frames since start-up (the VBI's RTCLOK), and whether the console is PAL.
unsigned int fcsFrames(void);
unsigned char fcsPal(void);

// One glyph of the font, by its ANTIC internal code.
void fcsPutGlyph(unsigned char x, unsigned char y, unsigned char g);
#define G_POINT_LEFT 121
#define G_DASH       125

/*
 Screen related variables
*/

// Screen specific player/bet coordinates, in ROM.
extern const unsigned char playerXMaster[];
extern const unsigned char playerYMaster[];
extern const char playerDirMaster[];
extern const char playerBetXMaster[];
extern const char playerBetYMaster[];

// Simple hard coded arrangment of players around the table based on player count.
// These refer to index positions in the Master arrays above
// Downside is new players will cause existing player positions to move.
extern const char playerCountIndex[];

#endif /* KEYMAP_H */

#endif /* BUILD_ATARI5200 */

```

antic.s

```
; antic.s -- the 8-bit client's screen (atari-fastbasic-version/5card.bas
; InitScreen) on the 5200: ANTIC mode 4, 40 x 26, the font, the display lists,
; the DLI that colours the status rows and the deferred VBI that slides the
; underline.
;
; The screens and player/missile memory sit in the top 4K of RAM, which the
; Makefile reserves. Drawing always goes to the work screen; the front screen
; is a copy shown while a frame is being built (graphics.c).

        .export _scrWork, _scrFront, _pmBase
        .export _dlFrontNorm, _dlFrontTall, _dlWorkNorm, _dlWorkTall
        .export _fcsFont, _fcsDli, _dliBg, _dliFg
        .export _fcsVbi, _barHpos, _barTarget

_scrWork  = $3000               ; 1040 bytes each, inside one 4K block
_scrFront = $3410
_pmBase   = $3800               ; single-line PMG: players from $3C00

WSYNC   = $D40A
VCOUNT  = $D40B
HPOSP2  = $C002
COLPF1  = $C017
COLBK   = $C01A

; 3 + 8 blank lines (a DLI on the second: the table's colours), rows 0-23,
; row 24 and, 3 lines below, the status row: each of the last two after a
; DLI. A tall status bar moves the gap above row 24 so both rows read as one.
.macro  dlist   scr, tall
        .local  top
top:    .byte   $20, $F0
        .byte   $44, <scr, >scr
        .repeat 22
        .byte   $04
        .endrepeat
        .byte   $84
.if tall
        .byte   $20, $84
.else
        .byte   $84, $20
.endif
        .byte   $04
        .byte   $41, <top, >top
.endmacro

        .segment "DLIST"        ; 1K-aligned: all four in one page
_dlFrontNorm:   dlist _scrFront, 0
_dlFrontTall:   dlist _scrFront, 1
_dlWorkNorm:    dlist _scrWork, 0
_dlWorkTall:    dlist _scrWork, 1
        .assert >_dlFrontNorm = >(_dlWorkTall + 33), error, "display lists span a page"

        .segment "FONT"         ; $B800, clear of the cart's mailbox
_fcsFont:
        .incbin "atari-fastbasic-version/5card-fontmaker.fnt"

        .code

; The BIOS's dispatcher pushes nothing. The first DLI is told apart by
; VCOUNT, so a frame never inherits a miscount.
.proc   _fcsDli
        pha
        txa
        pha
        ldx     #0
        lda     VCOUNT
        cmp     #32
        bcc     :+
        ldx     idx
:       lda     _dliBg,x
        sta     WSYNC
        sta     COLBK
        lda     _dliFg,x
        sta     COLPF1
        inx
        stx     idx
        pla
        tax
        pla
        rti
.endproc

; The deferred VBI: the underline (player 2) glides half the remaining way
; to its target each frame, as the BASIC's move cursor does. The crt0's VBI
; has pushed A, X and Y; this ends the way its own deferred vector does.
.proc   _fcsVbi
        lda     _barTarget
        sec
        sbc     _barHpos
        beq     set
        bcc     down
        lsr     a               ; up: half the distance, rounded up
        adc     #0
        clc
        adc     _barHpos
        sta     _barHpos
        jmp     set
down:   eor     #$FF            ; down: the distance, halved, rounded up
        clc
        adc     #1
        lsr     a
        adc     #0
        sta     step
        lda     _barHpos
        sec
        sbc     step
        sta     _barHpos
set:    lda     _barHpos
        sta     HPOSP2
        pla
        tay
        pla
        tax
        pla
        rti
.endproc

        .bss
_dliBg: .res    3               ; the table, row 24, the status row
_dliFg: .res    3
idx:    .res    1
_barHpos:   .res 1
_barTarget: .res 1
step:   .res    1
```

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Atari 5200 Graphics Routines for 5cardstud
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 * @verbose The 8-bit client's screen (atari-fastbasic-version/5card.bas):
 *          ANTIC mode 4 in its font, felt and status-bar colours from a DLI,
 *          black player sidebars round the table, player 1 darkening the
 *          hole card and player 2 underlining the active name or move.
 *          Everything draws to the work screen; with double buffering on,
 *          the front copy is shown until drawBuffer(), and the display list,
 *          the hole-card mask and the underline change with it.
 */

#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include "vars.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/sound.h"

#define REG(a)  (*(volatile unsigned char *)(a))

/* The BIOS's shadows, which the crt0's VBI copies to the chips. */
#define RTCLOKH REG(0x01)
#define RTCLOKL REG(0x02)
#define CRITIC  REG(0x03)
#define ATRACT  REG(0x04)
#define SDLSTL  REG(0x05)
#define SDLSTH  REG(0x06)
#define SDMCTL  REG(0x07)
#define PCOLR0  REG(0x08)
#define PCOLR1  REG(0x09)
#define PCOLR2  REG(0x0A)
#define PCOLR3  REG(0x0B)
#define COLOR0  REG(0x0C)
#define COLOR1  REG(0x0D)
#define COLOR2  REG(0x0E)
#define COLOR3  REG(0x0F)
#define COLOR4  REG(0x10)
#define VVBLKD  (*(void (**)(void))0x0204)
#define VDSLST  (*(void (**)(void))0x0206)

/* No shadows: written once. */
#define HPOSP0  REG(0xC000)
#define HPOSP1  REG(0xC001)
#define HPOSP2  REG(0xC002)
#define HPOSP3  REG(0xC003)
#define SIZEP0  REG(0xC008)
#define SIZEP1  REG(0xC009)
#define SIZEP2  REG(0xC00A)
#define SIZEP3  REG(0xC00B)
#define GTIAPAL REG(0xC014)
#define PRIOR   REG(0xC01B)
#define GRACTL  REG(0xC01D)
#define PMBASE  REG(0xD407)
#define CHBASE  REG(0xD409)

/* antic.s */
extern unsigned char scrWork[], scrFront[], pmBase[];
extern const unsigned char dlFrontNorm[], dlFrontTall[], dlWorkNorm[], dlWorkTall[];
extern const unsigned char fcsFont[];
extern unsigned char dliBg[3], dliFg[3];
extern unsigned char barHpos, barTarget;
void fcsDli(void);
void fcsVbi(void);

#define SCREEN_SIZE (WIDTH * HEIGHT)
#define STATUS_ROW  (HEIGHT - 1)
#define TALL_ROW    (HEIGHT - 2)

#define P0 (pmBase + 0x400)
#define P1 (pmBase + 0x500)
#define P2 (pmBase + 0x600)
#define P3 (pmBase + 0x700)

/* Horizontal position of column 0, and the scan line of row 0. */
#define LEFT_EDGE 48
#define TOP_LINE  19
#define STATUS_BAR_LINE 232     /* just under the status row's text */

/* Glyphs in the font, as ANTIC internal codes; +$80 draws "11" in red. */
#define G_BLANK   0
#define G_CHIP    0xEF
#define G_BOX_TL  27
#define G_BOX_TR  28
#define G_BOX_BL  29
#define G_BOX_BR  30
#define G_BOX_V   31
#define G_BOX_H   32
#define G_CLOCK   0x7A
#define G_RED     0x80

/* Colours. Each theme is the felt and the card edges (5card.bas). */
#define C_BLACK       0x00
#define C_WHITE       0x0E
#define C_MASK        0x0A
#define C_CURSOR      0xFA
#define C_RED_NTSC    0x34
#define C_RED_PAL     0x24
#define C_MOVE_NTSC   0xFE
#define C_MOVE_PAL    0xEE
#define THEMES        4

static const unsigned char themeNtsc[THEMES * 2] = { 0xB4,0x88, 0x84,0x08, 0x22,0x28, 0x04,0x08 };
static const unsigned char themePal[THEMES * 2]  = { 0xA4,0x78, 0x74,0x08, 0x12,0x18, 0x04,0x08 };

static const unsigned int rowOff[HEIGHT] = {
    0, 40, 80, 120, 160, 200, 240, 280, 320, 360, 400, 440, 480,
    520, 560, 600, 640, 680, 720, 760, 800, 840, 880, 920, 960, 1000
};

#define CELL(x, y) (scrWork + rowOff[y] + (x))

bool always_render_full_cards = 0;

static unsigned char theme;
static bool pal;
static bool dbl;                /* the front copy is shown while drawing */
static bool tall;               /* rows 24 and 25 are the status bar */

static unsigned char maskX, maskY;  /* the hole card under player 1 */
static bool maskWanted, maskDirty;
#define MASK_LINES 26
static unsigned char *maskBuilt;    /* the player 1 lines last written */

#define NO_BAR 0xFF
static unsigned char barRow = NO_BAR, barA, barB;  /* underline: cells [a,b) */
static unsigned char shownRow = NO_BAR, shownBits;

static unsigned char felt(void)
{
    return (pal ? themePal : themeNtsc)[theme * 2];
}

/* Frames since start-up, from the VBI's RTCLOK. */
unsigned int fcsFrames(void)
{
    unsigned char hi, lo;

    do {
        hi = RTCLOKH;
        lo = RTCLOKL;
    } while (hi != RTCLOKH);
    return ((unsigned int) hi << 8) | lo;
}
```

```

unsigned char fcsPal(void)
{
    return pal;
}

void waitvsync(void)
{
    unsigned char f = RTCLOKL;

    while (RTCLOKL == f) ;
}

/* The scan lines an underline at `row` occupies, and how many. */
static unsigned char barLine(unsigned char row)
{
    return row == STATUS_ROW ? STATUS_BAR_LINE : TOP_LINE + 1 + (row << 3);
}

static unsigned char barLines(unsigned char row)
{
    return row == STATUS_ROW ? 3 : 2;
}

static void setBarData(unsigned char row, unsigned char bits)
{
    memset(P2 + barLine(row), bits, barLines(row));
}

/* Put player 2 where barRow/barA/barB say: at most 8 cells, centred on the
   range. It slides there (antic.s), in from the left on a new row. */
static void applyBar(void)
{
    unsigned char w, n, bits, target;

    if (barRow == NO_BAR || barB <= barA)
    {
        if (shownRow != NO_BAR)
            setBarData(shownRow, 0);
        shownRow = NO_BAR;
        barTarget = barHpos = 0;
        HPOSP2 = 0;
        return;
    }

    w = barB - barA;
    n = w > 8 ? 8 : w;
    bits = (unsigned char) (0xFF00 >> n);
    target = LEFT_EDGE + ((barA + ((w - n) >> 1)) << 2);

    if (shownRow != barRow)
    {
        if (shownRow != NO_BAR)
            setBarData(shownRow, 0);
        barHpos = target - 20;
        HPOSP2 = barHpos;
        shownRow = barRow;
        shownBits = 0;
    }
    if (shownBits != bits)
    {
        setBarData(barRow, bits);
        shownBits = bits;
    }
    barTarget = target;
}

/* The two interior columns of the hole card, darkened wherever the glyph is
   not ink: CreatePlayerSecretCardMask in 5card.bas. It is rewritten in place,
   as the beam may be passing: the same card writes the same bytes. */
static void buildMask(void)
{
    unsigned char *p = P1 + TOP_LINE + (maskY << 3) + 7;
    unsigned char *cell = CELL(maskX, maskY + 1);
    const unsigned char *l, *r;
    unsigned char row, j, v, m;

    if (maskBuilt != p)
    {
        if (maskBuilt)
            memset(maskBuilt, 0, MASK_LINES);
        maskBuilt = p;
    }
    *p++ = 0x7E;
    for (row = 0; row < 3; row++, cell += WIDTH)
    {
        l = fcsFont + ((cell[0] & 0x7F) << 3);
        r = fcsFont + ((cell[1] & 0x7F) << 3);
        for (j = 0; j < 8; j++)
        {
            m = 0;
            v = l[j];
            if ((v & 0xC0) != 0xC0) m |= 0x80;
            if ((v & 0x30) != 0x30) m |= 0x40;
            if ((v & 0x0C) != 0x0C) m |= 0x20;
            if ((v & 0x03) != 0x03) m |= 0x10;
            v = r[j];
            if ((v & 0xC0) != 0xC0) m |= 0x08;
            if ((v & 0x30) != 0x30) m |= 0x04;
            if ((v & 0x0C) != 0x0C) m |= 0x02;
            *p++ = m;
        }
    }
    *p = 0x7E;
    maskDirty = false;
}

/* Show what has been drawn: the screen, the status bar's height, the mask
   and the underline. The four display lists share a page, so one store
   switches between them. */
static void present(void)
{
    const unsigned char *dl;

    if (dbl)
        dl = tall ? dlFrontTall : dlFrontNorm;
    else
        dl = tall ? dlWorkTall : dlWorkNorm;
    SDLSTL = (unsigned char) (unsigned int) dl;

    dliBg[1] = tall ? C_BLACK : felt();
    dliFg[1] = tall ? dliFg[2] : C_WHITE;

    if (maskWanted && maskDirty)
        buildMask();
    HPOSP1 = maskWanted ? LEFT_EDGE + (maskX << 2) : 0;

    applyBar();
}

/* A change shows at once while the work screen is the one shown. */
static void changed(void)
{
    if (!dbl)
        present();
}

static void put(unsigned char x, unsigned char y, unsigned char g)
{
    if (x < WIDTH && y < HEIGHT)
        *CELL(x, y) = g;
}

/* ASCII to the font, as the BASIC's Print does it: upper case, its own
   glyphs for , - . / and its occupied-table mark for *. What the font has
   turned into card or box art prints blank. */
static unsigned char textGlyph(unsigned char c)
{
    if (c >= 'a' && c <= 'z')
        c -= 'a' - 'A';
    switch (c)
    {
    case ',': return 116;
    case '-': return 125;
    case '.': return 115;
    case '/': return 124;
    case '*': return 123;
    }
    if (c < 0x20 || c > 0x5F)
        return G_BLANK;
    c -= 0x20;
```

```
    if ((c >= 1 && c <= 11) || (c >= G_BOX_TL && c <= G_BOX_H))
        return G_BLANK;
    return c;
}

/* Up to n characters of s from (x,y). */
static void putText(unsigned char x, unsigned char y, const char *s, unsigned char n)
{
    unsigned char *p;
    unsigned char c;

    if (y >= HEIGHT || x >= WIDTH)
        return;
    if (n > WIDTH - x)
        n = WIDTH - x;
    p = CELL(x, y);
    while (n-- && (c = (unsigned char) *s++) != 0)
        *p++ = textGlyph(c);
}

static void applyTheme(void)
{
    const unsigned char *t = (pal ? themePal : themeNtsc) + theme * 2;

    dliBg[0] = t[0];
    if (!tall)
        dliBg[1] = t[0];
    COLOR0 = t[1];
}

/**
 * @brief Initialize graphics mode; set palette.
 */
void initGraphics()
{
    pal = (GTIAPAL & 0x0E) == 0;

    memset(scrWork, 0, SCREEN_SIZE);
    memset(scrFront, 0, SCREEN_SIZE);

    /* Players 0 and 3 are the black sidebars, 1 the hole-card mask, 2 the
       underline; 0 and 1 in front of the playfield, 2 and 3 behind it. */
    memset(P0, 0, 0x400);
    memset(P0, 0xFF, 255);
    memset(P3, 0xFF, 255);
    SIZEP0 = SIZEP2 = SIZEP3 = 3;
    SIZEP1 = 0;
    HPOSP0 = 16;
    HPOSP3 = 208;
    HPOSP1 = HPOSP2 = 0;
    PRIOR = 2;
    PMBASE = (unsigned char) ((unsigned int) pmBase >> 8);
    GRACTL = 2;

    PCOLR0 = C_BLACK;
    PCOLR1 = C_MASK;
    PCOLR2 = C_CURSOR;
    PCOLR3 = C_BLACK;
    COLOR1 = C_WHITE;
    COLOR2 = C_BLACK;
    COLOR3 = pal ? C_RED_PAL : C_RED_NTSC;
    COLOR4 = C_BLACK;

    dliFg[0] = dliFg[1] = C_WHITE;
    dliBg[2] = C_BLACK;
    dliFg[2] = pal ? C_MOVE_PAL : C_MOVE_NTSC;
    applyTheme();

    CHBASE = (unsigned char) ((unsigned int) fcsFont >> 8);
    VDSLST = fcsDli;

    // The VBI skips its deferred vector while CRITIC is set.
    CRITIC = 1;
    VVBLKD = fcsVbi;
    SDLSTH = (unsigned char) ((unsigned int) dlWorkNorm >> 8);
    SDLSTL = (unsigned char) (unsigned int) dlWorkNorm;
    CRITIC = 0;

    SDMCTL = 0x3A;              /* normal playfield, single-line players, DL DMA */

    resetScreen();
    enableDoubleBuffer();
    waitvsync();
}

void drawChip(unsigned char x, unsigned char y)
{
    put(x, y, G_CHIP);
    changed();
}

void fcsPutGlyph(unsigned char x, unsigned char y, unsigned char g)
{
    put(x, y, g);
    changed();
}

/* Copy the work screen to the front one and show the result. */
void drawBuffer()
{
    waitvsync();
    memcpy(scrFront, scrWork, SCREEN_SIZE);
    ATRACT = 0;
    present();
}

void drawBox(unsigned char x, unsigned char y, unsigned char w, unsigned char h)
{
    unsigned char i, red;

    // The pot's box is red, as the BASIC prints it inverse.
    red = (x == WIDTH/2-3 && y == 11+POT_Y_MODIFIER) ? G_RED : 0;

    // Correct coordinates;
    w++;
    h++;

    put(x, y, G_BOX_TL | red);
    put(x + w, y, G_BOX_TR | red);
    put(x, y + h, G_BOX_BL | red);
    put(x + w, y + h, G_BOX_BR | red);

    for (i = 1; i < w; i++)
    {
        put(x + i, y, G_BOX_H | red);
        put(x + i, y + h, G_BOX_H | red);
    }

    for (i = 1; i < h; i++)
    {
        put(x, y + i, G_BOX_V | red);
        put(x + w, y + i, G_BOX_V | red);
    }
    changed();
}

/**
 * @brief Draw text s at position x,y
 * @param x Horizontal position (0-39)
 * @param y Vertical Position (0-25)
 * @param s NULL terminated string to display.
 */
void drawText(unsigned char x, unsigned char y, const char* s)
{
    putText(x, y, s, WIDTH);
    changed();
}

/* The logo of the BASIC's WelcomeScreen, in the font's own pieces. */
static const unsigned char logo0[] = { 105, 0 };
static const unsigned char logo1[] = { 108, 106, 108, 0 };
static const unsigned char logo2[] = { 110, 106, 112, 106, 104, 0 };
static const unsigned char logo3[] = { 108, 108, 106, 104, 0 };
static const unsigned char logo4[] = { 109, 108, 108, 0 };

static void putRaw(unsigned char x, unsigned char y, const unsigned char *g)
{
    unsigned char *p = CELL(x, y);

    while (*g)
        *p++ = *g++;
```

```
}

/**
 * @brief Draw the 5 Card Stud Logo
 */
void drawLogo()
{
    putRaw(19, 3, logo0);
    putRaw(18, 4, logo1);
    putText(13, 5, "FUJI", 4);
    putRaw(17, 5, logo2);
    putText(22, 5, "NET", 3);
    putRaw(18, 6, logo3);
    putRaw(19, 7, logo4);
    putText(17, 9, "5 CARD STUD", 11);
    changed();
}

void clearStatusBar()
{
    memset(CELL(0, STATUS_ROW), G_BLANK, WIDTH);
    if (tall)
    {
        tall = false;
        memset(CELL(0, TALL_ROW), G_BLANK, WIDTH);
        put(0, TALL_ROW, 91);
        put(WIDTH-1, TALL_ROW, 92);
    }
    if (barRow == STATUS_ROW)
        barRow = NO_BAR;
    changed();
}

/**
 * @brief Clear the screen to an empty table: ResetScreen in 5card.bas.
 */
void resetScreen()
{
    memset(scrWork, G_BLANK, SCREEN_SIZE);
    put(0, 0, 89);
    put(WIDTH-1, 0, 90);
    put(0, TALL_ROW, 91);
    put(WIDTH-1, TALL_ROW, 92);
    tall = false;
    maskWanted = false;
    if (barRow == STATUS_ROW)
        barRow = NO_BAR;
    changed();
}

/* Draw on the work screen, and show it as it is drawn. */
void disableDoubleBuffer()
{
    dbl = false;
    present();
}

/* Show a copy of the work screen while the next frame is drawn on it. */
void enableDoubleBuffer()
{
    if (!dbl)
    {
        memcpy(scrFront, scrWork, SCREEN_SIZE);
        dbl = true;
        present();
    }
}

/* A hand's first card goes at its seat: clear the seat first, as the BASIC
   does before the final flip, so a hand is never joined onto the one it
   replaces. */
static void clearSeat(unsigned char x, unsigned char y)
{
    unsigned char i, r, left;

    for (i = 0; i < state.playerCount && i < 8; i++)
    {
        if (playerX[i] == x && playerY[i] == y)
        {
            left = x - 1 - (playerDir[i] < 0 ? 7 : 0);
            for (r = 0; r < 5; r++)
                memset(CELL(left, y + r), G_BLANK, 11);
            return;
        }
    }
}

/**
 * @brief Draw card at position x,y: DrawCard in 5card.bas. The card is the
 *        interior at x and x+1 with its edges at x-1 and x+2, and is drawn
 *        over whatever card is already there.
 * @param x Horizontal card position (0-39)
 * @param y Vertical card position (0-25)
 * @param partial ignored: every card is drawn whole, as the BASIC does
 * @param s String indicating number and suit. (e.g. "as" for ace of spades)
 * @param isHidden is card currently overturned?
 */
void drawCard(unsigned char x, unsigned char y, unsigned char partial, const char* s, unsigned char isHidden)
{
    unsigned char *loc, *side;
    unsigned char val, suit, hi, v, i;
    signed char tbo = 0;
    bool rightUnder, isTen, back;

    (void) partial;

    if (y + 4 >= HEIGHT || x == 0 || x + 2 >= WIDTH)
        return;
    if (state.playerCount > 1)
        clearSeat(x, y);
    loc = CELL(x, y);

    back = s[0] == '?';
    switch (s[0] | 0x20)
    {
    case 't': val = 10; break;
    case 'j': val = 12; break;
    case 'q': val = 13; break;
    case 'k': val = 14; break;
    case 'a': val = 15; break;
    default:  val = (unsigned char) (s[0] - '0'); break;
    }
    if (back)
    {
        tbo = 4;
        val = 0x3C;
    }

    switch (s[1] | 0x20)
    {
    case 'h': suit = 0xC0; val += 0x80; break;
    case 'd': suit = 0xC2; val += 0x80; break;
    case 'c': suit = 0x44; break;
    default:  suit = 0x46; break;
    }

    // A card underneath to the right shows past this one's right edge.
    rightUnder = loc[WIDTH + 2] != G_BLANK;
    if (rightUnder)
    {
        tbo = -4;
        loc[-1] = 75;
        loc[4 * WIDTH - 1] = 76;
        suit += 32;
    }

    loc[0] = 0x51 + tbo;
    loc[1] = 0x52 + tbo;

    if (!back)
    {
        isTen = val == 10 || val == 138;
        hi = 1;
        if (isTen)
            hi += val;
        if (rightUnder)
            hi += isTen ? 52 : 61;
        loc[WIDTH] = val;
        loc[WIDTH + 1] = hi;
        loc[2 * WIDTH] = 1;
        loc[2 * WIDTH + 1] = rightUnder ? 62 : 1;
        loc[3 * WIDTH] = suit;
```

```
        loc[3 * WIDTH + 1] = suit + 1;
    }
    else
    {
        loc[WIDTH] = 0x3C;
        loc[WIDTH + 1] = 0xBC;
        loc[2 * WIDTH] = 0xBC;
        loc[2 * WIDTH + 1] = 0x3C;
        loc[3 * WIDTH] = 0x3C;
        loc[3 * WIDTH + 1] = 0xBC;
    }

    // Edges, joined onto any card this one lies over.
    side = loc + WIDTH - 1;
    for (i = 0; i < 3; i++, side += WIDTH)
    {
        v = *side;
        if (v == 0x3C || v == 0xBC)
            v++;
        else if (v == 1)
            v = 62;
        else if ((v > 64 && v < 72) || (v > 191 && v < 201))
        {
            v += 32;
            side[-1] += 32;
        }
        else if (v == 11 || v == 139)
            v += 52;
        else
            v = rightUnder ? 72 : 73;
        *side = v;

        if (!rightUnder)
            side[3] = 74;
    }

    loc[4 * WIDTH] = 0x53 + tbo;
    loc[4 * WIDTH + 1] = 0x54 + tbo;

    // The player's own hole card is darkened, as long as it is face up.
    if (isHidden && !back)
    {
        maskX = x;
        maskY = y;
        maskWanted = maskDirty = true;
    }
    else if (maskWanted && x == maskX && y == maskY)
        maskWanted = false;

    changed();
}

/* A message too long for one row (the round result) takes row 24 into the
   bar as well, split after its comma as the BASIC does, or at a space. */
void drawStatusTextAt(unsigned char x, const char* s)
{
    unsigned char len = (unsigned char)strlen(s);
    unsigned char brk;

    if (len <= WIDTH)
    {
        putText(x, STATUS_ROW, s, WIDTH);
        changed();
        return;
    }

    for (brk = 0; brk < len && s[brk] != ','; brk++) ;
    if (brk >= WIDTH || len - brk > WIDTH)
        for (brk = WIDTH - 1; brk > 0 && s[brk] != ' '; brk--) ;
    if (brk == 0)
        brk = WIDTH - 1;

    tall = true;
    memset(CELL(0, TALL_ROW), G_BLANK, WIDTH * 2);
    putText(0, TALL_ROW, s, brk + 1);
    s += brk + 1;
    if (*s == ' ')
        s++;
    putText(0, STATUS_ROW, s, WIDTH);
    changed();
}

void drawStatusText(const char* s)
{
    clearStatusBar();
    drawStatusTextAt(0, s);
}

/* The clock after the move timer, as the BASIC draws it. */
void drawStatusTimer()
{
    put(WIDTH - 1, STATUS_ROW, G_CLOCK);
    changed();
}

/* Underline cells [x,x+w) of row y with player 2: the active player's name
   (the row under it) or the move under the cursor (the status row). A rule
   wider than the player is drawn in the font instead, as on the C64. */
void drawLine(unsigned char x, unsigned char y, unsigned char w)
{
    if (y >= HEIGHT || w == 0)
        return;
    if (w > 8 && y != STATUS_ROW)
    {
        memset(CELL(x, y), G_BOX_H, x + w > WIDTH ? WIDTH - x : w);
    }
    else
    {
        barRow = y;
        barA = x;
        barB = x + w;
    }
    changed();
}

/* Take cells [x,x+w) of row y out of the underline: from either end it
   shrinks, over all of it it goes. w = 0 hides it wherever it is. */
void hideLine(unsigned char x, unsigned char y, unsigned char w)
{
    unsigned char e = x + w;

    if (barRow == NO_BAR)
        return;
    if (w == 0)
        barRow = NO_BAR;
    else if (y != barRow || e <= barA || x >= barB)
        return;
    else if (x <= barA && e >= barB)
        barRow = NO_BAR;
    else if (x <= barA)
        barA = e;
    else if (e >= barB)
        barB = x;
    changed();
}

/* The table colour: 5card.bas's four themes, felt and card edges. */
unsigned char cycleNextColor()
{
    theme = (theme + 1) % THEMES;
    applyTheme();
    soundCursor();
    return theme;
}

void setColorMode(unsigned char mode)
{
    theme = mode < THEMES ? mode : 0;
}

/* The four aces and the rows of red and black between them. */
static const unsigned char trim[] = { 220, 217, 90, 91, 0 };

void drawBorder()
{
    unsigned char i;

    drawCard(1, 0, FULL_CARD, "as", 0);
    drawCard(WIDTH-3, 0, FULL_CARD, "ah", 0);
    drawCard(1, 20, FULL_CARD, "ad", 0);
    drawCard(WIDTH-3, 20, FULL_CARD, "ac", 0);

    for (i = 4; i < 36; i += 4)
    {
```

```
        putRaw(i, 1, trim);
        putRaw(i, 22, trim);
    }
    changed();
}

#endif /* BUILD_ATARI5200 */

```

input.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief Atari 5200 input routines
 * @author Thomas Cherryhomes
 * @license gpl v.3
 */

#include <atari5200.h>
#include <fujinet-atari5200.h>
#include "../platform-specific/input.h"

/*
  Controller 1. The stick comes from the VBI's POT shadows, read through a
  dead zone; the keypad from the lib's keypad IRQ, which latches one press in
  fuji_a5200_key until it is read here.

  Every button and key is reported as a KEY rather than as a stick bit.
  readCommonInput() returns early whenever readJoystick()'s value changes and
  only reaches getPlatformKey() when it has not, so a button that appeared in
  both places would be seen twice. The keypad stands in for the letter
  shortcuts the other ports type, as on the ColecoVision (src/coleco/input.c);
  src/screens.c labels them.

     top fire, #, START       RETURN  (select / join / confirm)
     lower fire, *, RESET     ESCAPE  (back / the in-game menu)
     1 refresh   2 how to play (or PAUSE)   3 table colour   4 name   5 quit
*/

#define REG(a)  (*(volatile unsigned char *)(a))
#define ATRACT  REG(0x04)
#define PADDL0  REG(0x11)       /* stick X, 0-228, 114 at rest */
#define PADDL1  REG(0x12)       /* stick Y */
#define TRIG0   REG(0xC010)     /* top fire: 0 while held */
#define SKSTAT  REG(0xE80F)     /* bit 3 clear: the lower fire button */

/* Past these the stick is pushed; between them it is at rest. */
#define STICK_LO 0x40
#define STICK_HI 0xA4

#define DIRS    (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK | JOY_RIGHT_MASK)
#define BUTTONS (JOY_BTN_1_MASK | PAD_BTN_2)

static unsigned char lastButtons, lastPad;

unsigned char readPad(void)
{
  unsigned char pad = 0;
  unsigned char x = PADDL0, y = PADDL1;

  if (y < STICK_LO)
    pad |= JOY_UP_MASK;
  else if (y > STICK_HI)
    pad |= JOY_DOWN_MASK;
  if (x < STICK_LO)
    pad |= JOY_LEFT_MASK;
  else if (x > STICK_HI)
    pad |= JOY_RIGHT_MASK;
  if (TRIG0 == 0)
    pad |= JOY_BTN_1_MASK;
  if (!(SKSTAT & 0x08))
    pad |= PAD_BTN_2;

  // Any use of the controller holds off the VBI's attract-mode colours.
  if (pad != lastPad)
  {
    lastPad = pad;
    ATRACT = 0;
  }
  return pad;
}

unsigned char readJoystick()
{
  return readPad() & DIRS;
}

void initPlatformKeyboardInput(void)
{
  lastButtons = readPad() & BUTTONS;
  fuji_a5200_key = 0;
}

int getPlatformKey(void)
{
  unsigned char buttons = readPad() & BUTTONS;
  unsigned char pressed = buttons & ~lastButtons;
  unsigned char k = fuji_a5200_key;

  lastButtons = buttons;

  if (pressed & JOY_BTN_1_MASK)
    return KEY_RETURN;
  if (pressed & PAD_BTN_2)
    return KEY_ESCAPE;

  if (!k)
    return 0;
  fuji_a5200_key = 0;

  switch (k) {
    case FUJI_A5200_KEY_HASH:
    case FUJI_A5200_KEY_START:  return KEY_RETURN;
    case FUJI_A5200_KEY_STAR:
    case FUJI_A5200_KEY_RESET:  return KEY_ESCAPE;
    case FUJI_A5200_KEY_PAUSE:  return 'h';
    case FUJI_A5200_KEY_DIGIT(1): return 'r';   /* refresh     */
    case FUJI_A5200_KEY_DIGIT(2): return 'h';   /* how to play */
    case FUJI_A5200_KEY_DIGIT(3): return 'c';   /* colour      */
    case FUJI_A5200_KEY_DIGIT(4): return 'n';   /* name        */
    case FUJI_A5200_KEY_DIGIT(5): return 'q';   /* quit        */
  }
  return 0;
}

#endif /* BUILD_ATARI5200 */

```

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Atari 5200 on-screen keyboard
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <atari5200.h>
#include <string.h>
#include <fujinet-atari5200.h>
#include "vars.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/sound.h"

/*
  The grid of the ColecoVision client (src/coleco/osk.c) and the 7800 port
  (src/atari7800/osk.c): the cursor position IS the character, so there is no
  shift key and no paging. Cancel is the OK cell's absence -- the caller only
  reaches this screen when a name is required -- and the grid never runs a
  mailbox transaction. Uppercase only: drawText() uppercases everything it
  prints.

    stick  move      top fire  take the cell
    0-9  type that digit
    lower fire, * or RESET  backspace
    # or START  OK
*/

#define OSK_COLS   10
#define OSK_ROWS   4
#define OSK_PITCH  3                    /* marker column + two label columns */
#define OSK_X      5
#define OSK_Y      18

/* The last row is one short: its tenth cell does not exist. */
#define OSK_LAST_ROW_COLS 9

#define CELL_SPACE (OSK_COLS * 3 + 6)   /* index 36 */
#define CELL_DEL   (CELL_SPACE + 1)
#define CELL_OK    (CELL_SPACE + 2)
#define CELL_DIGIT (CELL_SPACE + 3)     /* a keypad digit, not a cell */
#define CELL_NONE  0xFF

static const char cells[] =
  "ABCDEFGHIJ"
  "KLMNOPQRST"
  "UVWXYZ0123"
  "456789";

#define REPEAT_FIRST 18                 /* vblanks before a held direction repeats */
#define REPEAT_NEXT  5

#define DIRS (JOY_UP_MASK | JOY_DOWN_MASK | JOY_LEFT_MASK | JOY_RIGHT_MASK)
#define BUTTONS (JOY_BTN_1_MASK | PAD_BTN_2)

static unsigned char curX, curY;

static unsigned char cellCols(unsigned char row)
{
  return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS : OSK_COLS;
}

static void drawCell(unsigned char col, unsigned char row, unsigned char marked)
{
  unsigned char idx = row * OSK_COLS + col;
  unsigned char x = OSK_X + col * OSK_PITCH;
  unsigned char y = OSK_Y + row;
  char glyph[3];

  if (marked)
    drawChip(x, y);
  else
    drawText(x, y, " ");

  if (idx == CELL_SPACE)
    drawText(x + 1, y, "SP");
  else if (idx == CELL_DEL)
  {
    // The font's "<" is box art: a pointer and a dash instead.
    fcsPutGlyph(x + 1, y, G_POINT_LEFT);
    fcsPutGlyph(x + 2, y, G_DASH);
  }
  else if (idx == CELL_OK)
    drawText(x + 1, y, "OK");
  else
  {
    glyph[0] = cells[idx];
    glyph[1] = ' ';
    glyph[2] = 0;
    drawText(x + 1, y, glyph);
  }
}

static void drawGrid(void)
{
  unsigned char row, col;

  for (row = 0; row < OSK_ROWS; row++)
    for (col = 0; col < cellCols(row); col++)
      drawCell(col, row, row == curY && col == curX);
}

static void clearGrid(void)
{
  unsigned char row;

  for (row = 0; row < OSK_ROWS; row++)
    drawText(OSK_X, OSK_Y + row, "                    ");
}

/* Repaint the field and its trailing cursor chip, the same shape
   inputFieldCycle() draws on the platforms that have a keyboard. */
static void drawField(unsigned char x, unsigned char y, unsigned char max,
                      const char *buffer)
{
  unsigned char len = (unsigned char) strlen(buffer);

  drawText(x, y, "          ");
  drawText(x, y, buffer);
  if (len < max)
    drawChip(x + len, y);
}

static void appendChar(char *buffer, unsigned char *len, unsigned char max,
                       char ch)
{
  if (*len >= max)
  {
    soundCursorInvalid();
    return;
  }
  buffer[(*len)++] = ch;
  buffer[*len] = 0;
  soundCursor();
}

/* A keypad key as a grid action: a digit is typed straight in. */
static unsigned char keypadCell(char *digit)
{
  unsigned char k = fuji_a5200_key;

  if (!k)
    return CELL_NONE;
  fuji_a5200_key = 0;
  switch (k) {
    case FUJI_A5200_KEY_HASH:
    case FUJI_A5200_KEY_START: return CELL_OK;
    case FUJI_A5200_KEY_STAR:
    case FUJI_A5200_KEY_RESET: return CELL_DEL;
  }
  if (k >= FUJI_A5200_KEY_DIGIT(0) && k <= FUJI_A5200_KEY_DIGIT(9))
```

```
  {
    *digit = (char) ('0' + k - FUJI_A5200_KEY_DIGIT(0));
    return CELL_DIGIT;
  }
  return CELL_NONE;
}

void platformNameEntry(unsigned char x, unsigned char y, unsigned char max,
                       char *buffer)
{
  unsigned char len = (unsigned char) strlen(buffer);
  unsigned char hold = 0, lastDir = 0, lastButtons = BUTTONS;
  unsigned char pad, dir, move, buttons, pressed, idx;
  char digit = 0;

  curX = 0;
  curY = 0;
  fuji_a5200_key = 0;
  drawGrid();
  drawField(x, y, max, buffer);

  for (;;)
  {
    waitvsync();

    pad = readPad();
    dir = pad & DIRS;
    buttons = pad & BUTTONS;

    /* Auto-repeat is counted in vblanks: this loop runs once per frame. */
    move = 0;
    if (dir != lastDir)
    {
      lastDir = dir;
      hold = REPEAT_FIRST;
      move = dir;
    }
    else if (dir && --hold == 0)
    {
      hold = REPEAT_NEXT;
      move = dir;
    }

    if (move)
    {
      drawCell(curX, curY, 0);

      if ((move & JOY_LEFT_MASK) && curX)
        curX--;
      else if ((move & JOY_RIGHT_MASK) && curX + 1 < cellCols(curY))
        curX++;
      else if ((move & JOY_UP_MASK) && curY)
        curY--;
      else if ((move & JOY_DOWN_MASK) && curY + 1 < OSK_ROWS)
        curY++;

      if (curX >= cellCols(curY))
        curX = cellCols(curY) - 1;

      drawCell(curX, curY, 1);
      soundCursor();
    }

    /* The lower button is backspace; the top one takes whatever the cursor
       is sitting on. */
    pressed = buttons & ~lastButtons;
    lastButtons = buttons;
    if (pressed & PAD_BTN_2)
      idx = CELL_DEL;
    else if (pressed & JOY_BTN_1_MASK)
      idx = curY * OSK_COLS + curX;
    else if ((idx = keypadCell(&digit)) == CELL_NONE)
      continue;

    if (idx == CELL_OK)
    {
      if (!len)
      {
        soundCursorInvalid();
        continue;
      }
      break;
    }

    if (idx == CELL_DEL)
    {
      if (!len)
      {
        soundCursorInvalid();
        continue;
      }
      buffer[--len] = 0;
      soundCursor();
    }
    else if (idx == CELL_SPACE)
      appendChar(buffer, &len, max, ' ');
    else if (idx == CELL_DIGIT)
      appendChar(buffer, &len, max, digit);
    else
      appendChar(buffer, &len, max, cells[idx]);

    drawField(x, y, max, buffer);
  }

  clearGrid();

  // Forget the press that said OK.
  initPlatformKeyboardInput();
}

#endif /* BUILD_ATARI5200 */
```

sound.c

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Atari 5200 Sound Routines (POKEY)
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdint.h>
#include "vars.h"
#include "../platform-specific/graphics.h"

/* The console's POKEY. SKCTL stays as the crt0 set it: scanning the keypad,
   which also keeps POKEY out of initialisation, so the counters run. */
#define POKEY(r)  (*(volatile uint8_t *)(0xE800 + (r)))
#define AUDF1     POKEY(0x0)
#define AUDC1     POKEY(0x1)
#define AUDF2     POKEY(0x2)
#define AUDC2     POKEY(0x3)
#define AUDCTL    POKEY(0x8)

/* Channels 1 and 2 joined into one 16-bit divider on the 1.79MHz clock, so
   the sub-bass beeps and the high ones are both in reach. */
#define AUDCTL_179_JOINED 0x50
#define TONE_ON   0xA8          /* pure tone, volume 8 */
#define TONE_OFF  0x00

#define MIN_GATE  3             /* shorter blips are barely audible */

/**
 * @brief Brain dead beep routine
 * @param hz Frequency in Hz
 * @param gate Delay in vertical blanks
 * @param postGate Delay after tone off in vertical blanks
 */
void beep(int hz, int gate, int postGate)
{
    // f = 1789773 / (2 * (N + 7)) in the joined mode.
    uint16_t n = (uint16_t)(894886UL / (uint16_t)hz) - 7;

    // Stretch short blips, taking the extra out of the rest so the cue
    // keeps its rhythm.
    if (gate < MIN_GATE)
    {
        postGate -= MIN_GATE - gate;
        if (postGate < 0)
            postGate = 0;
        gate = MIN_GATE;
    }

    AUDF1 = n & 0xFF;
    AUDF2 = n >> 8;
    AUDC2 = TONE_ON;

    while (gate--)
        waitvsync();

    AUDC2 = TONE_OFF;

    while (postGate--)
        waitvsync();
}

void soundDealCard()
{
    beep(150,1,5);
}

void soundJoinGame()
{
    beep(430,5,8);
    beep(340,5,0);
    beep(500,5,0);
}

void soundPlayerLeft()
{
    uint8_t i;
    for (i=80;i>=50;i-=10)
        beep(i,2,15);
}

void soundPlayerJoin()
{
    uint8_t i;
    for (i=50;i<=80;i+=10)
        beep(i,2,15);
}

void soundGameDone()
{
    beep(311,10,0);
    beep(330,20,0);
    beep(392,10,0);
    beep(415,20,0);
}

void soundMyTurn()
{
    beep(430,4,2);
    beep(430,4,2);
}

void soundTick()
{
    beep(50,2,0);
}

void soundCursor()
{
    beep(300,2,0);
}

void soundCursorInvalid()
{
    beep(100,2,0);
}

void soundSelectMove()
{
    beep(300,3,1);
    beep(350,3,0);
}

void initSound()
{
    AUDCTL = AUDCTL_179_JOINED;
    AUDC1 = TONE_OFF;           /* the joined pair sounds on channel 2 only */
    AUDC2 = TONE_OFF;
}

void soundTakeChip(uint16_t counter)
{
    beep(50+counter*20,2,2);
}

#endif /* BUILD_ATARI5200 */

```

```
#ifdef BUILD_ATARI5200

/**
 * @brief   Utility Functions
 * @author  Thomas Cherryhomes
 * @email   thom dot cherryhomes at gmail dot com
 * @license gpl v. 3, see LICENSE for details
 */

#include <stdbool.h>
#include <stdint.h>
#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include "vars.h"
#include "../platform-specific/graphics.h"
#include "../platform-specific/util.h"
#include "../misc.h"

// READ HOST SLOTS returns all 8 slots.
#define FUJI_HOST_SLOT_COUNT 8
#define LOBBY_DEVICE_SLOT 0
#define LOBBY_MODE_READ   1

// MOUNT_IMAGE allows the push 60 seconds; the extra 10 cover the round trip.
// The push can still be running when the reply comes back, hence the wait
// for READY after it.
#define LOBBY_MOUNT_SECONDS 70
#define LOBBY_READY_FRAMES  (LOBBY_MOUNT_SECONDS * 60u)

// The game timer counts in 60ths of a second: frames since the last
// resetTimer(), a PAL console's 50 a second scaled up.
static unsigned int timerBase;

void resetTimer(void)
{
  timerBase = fcsFrames();
}

int getTime(void)
{
  unsigned int t = fcsFrames() - timerBase;

  if (fcsPal())
    t += t / 5;
  return (int) t;
}

static bool sameHost(const char *a, const char *b)
{
  while (*a && *b)
  {
    if ((*a | 0x20) != (*b | 0x20))
      return false;
    a++;
    b++;
  }
  return *a == *b;
}

/*
   SET_DEVICE_FULLPATH is a fixed 256-byte payload -- a short one is rejected on
   the ESP32 side -- so the path is padded here rather than at the call site.
*/
static const char lobbyPath[MAX_FILENAME_LEN] = "/atari5200/lobby.a52";
static const char lobbyHost[] = "ec.tnfs.io";

static HostSlot slots[FUJI_HOST_SLOT_COUNT];

static uint8_t findLobbyHost(void)
{
  uint8_t i;

  if (!fuji_get_host_slots(slots, FUJI_HOST_SLOT_COUNT))
    return FUJI_HOST_SLOT_COUNT;
  for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
    if (sameHost(lobbyHost, (const char *) slots[i]))
      return i;
  return FUJI_HOST_SLOT_COUNT;
}

// The slot is used as found, never created: a player who reached this client
// through the FujiNet Lobby already has the host.
static bool mountLobby(uint8_t slot)
{
  uint8_t timeout = fn_default_timeout;
  unsigned int t0;
  uint8_t st;
  bool ok;

  if (!fuji_mount_host_slot(slot))
    return false;
  if (!fuji_set_device_filename(LOBBY_MODE_READ, slot, LOBBY_DEVICE_SLOT,
                                (char *) lobbyPath))
    return false;
  fn_default_timeout = LOBBY_MOUNT_SECONDS;
  ok = fuji_mount_disk_image(LOBBY_DEVICE_SLOT, LOBBY_MODE_READ);
  fn_default_timeout = timeout;
  if (!ok)
    return false;

  t0 = fcsFrames();
  while ((st = fuji_a5200_boot_state()) != FUJI_A5200_BOOT_READY)
  {
    if (st == FUJI_A5200_BOOT_FAILED || fcsFrames() - t0 > LOBBY_READY_FRAMES)
      return false;
    waitvsync();
  }
  return true;
}

/*
  Quitting leaves for the FujiNet Lobby, as the 7800 port does; without it,
  back to CONFIG, which is where the 8-bit client's Q goes.
*/
void quit(void)
{
  uint8_t slot;

  drawStatusText("LOADING LOBBY...");

  if (!fuji_a5200_present())
  {
    drawStatusText("NO FUJINET CARTRIDGE");
    return;
  }

  slot = findLobbyHost();
  if (slot < FUJI_HOST_SLOT_COUNT && mountLobby(slot))
    fuji_a5200_boot();          // does not return once the cart has armed it

  drawStatusText("LOBBY NOT AVAILABLE: BACK TO CONFIG");
  pause(60);
  fuji_a5200_config();          // nor does this
  drawStatusText("CONFIG NOT AVAILABLE");
}

#endif /* BUILD_ATARI5200 */
```

CONFIG

config.c

```
/* config.c -- FujiNet CONFIG for the Atari 5200: entry point, the state
 * dispatcher, and the small drawing helpers every screen shares.
 *
 * Ported from the Atari 7800 CONFIG (../atari7800), itself the NES CONFIG's
 * with the Master System's host-to-host copy: one state byte, one screen
 * module per state, each running its own event loop until it hands the state
 * to someone else. Lists are drawn straight out of the cartridge's reply
 * window at $B000 and streamed straight back into the next transaction.
 */

#include <string.h>

#include "fujidisp.h"
#include "fujiin.h"
#include "sfx.h"
#include "state.h"

unsigned char state;

unsigned char host;
unsigned char src_host;
unsigned char copy_mode;

unsigned char cur;
unsigned char nrows;
unsigned char at_end;
unsigned int  top;
unsigned char hosts_mask;

char path[PATH_MAX_LEN];
char src_spec[SPEC_MAX_LEN];

void status_line(const char *s)
{
    disp_row_clear(STATUS_ROW);
    disp_at(IN_L, STATUS_ROW, s);
}

void legend_line(const char *s)
{
    disp_row_clear(LEGEND_ROW);
    disp_at(IN_L, LEGEND_ROW, s);
}

/* "?MOUNT ERROR 02", with a BEEP. */
void fail(const char *what)
{
    unsigned char col = (unsigned char)(IN_L + 1 + strlen(what));

    disp_row_clear(STATUS_ROW);
    disp_at(IN_L, STATUS_ROW, "?");
    disp_at(IN_L + 1, STATUS_ROW, what);
    disp_at(col, STATUS_ROW, " ERROR ");
    disp_at_hex8((unsigned char)(col + 7), STATUS_ROW, FN_ERRCODE);
    sfx_beep();
}

void draw_frame(const char *title)
{
    disp_frame(title);
}

void wait_frames(unsigned char n)
{
    while (n--)
        mt_sync();
}

/* The selection bar for the lists that fit on one screen (hosts, networks).
 * The browser has its own version with page-crossing. */
void bar_move(signed char d)
{
    if (d < 0) {
        if (cur == 0)
            return;
        disp_row_invert((unsigned char)(LIST_TOP + cur), false);
        cur--;
    } else {
        if ((unsigned char)(cur + 1) >= nrows)
            return;
        disp_row_invert((unsigned char)(LIST_TOP + cur), false);
        cur++;
    }
    disp_row_invert((unsigned char)(LIST_TOP + cur), true);
}

/* The power-on screen: the name typing itself out a character at a time,
 * then OK and the blinking cursor. Any key cuts it short. */
static void splash(void)
{
    static const char name[] = "FUJINET CONFIG";
    unsigned char i;

    for (i = 0; name[i]; i++) {
        disp_char((unsigned char)(IN_L + i), 2, name[i]);
        sfx_blip();
    }
    disp_at(IN_L, 3, "FOR THE ATARI 5200");
    disp_at(IN_L, 4, "(C) FUJINET PROJECT");
    disp_at(IN_L, 5, "OK");
    disp_cursor_at(IN_L, 6);
    for (i = 0; i < 30; i++) {
        if (in_read() != IN_NONE)
            break;
        disp_cursor_tick();
    }
    disp_cursor_off();
}

void main(void)
{
    disp_init();
    sfx_init();
    in_init();

    if (!fuji_a5200_present()) {
        disp_at(IN_L, 2, "?NO FUJINET CART");
        sfx_beep();
        for (;;)
            ;
    }
    splash();

    state = ST_CHECK_WIFI;
    for (;;) {
        switch (state) {
        case ST_CHECK_WIFI:
            st_check_wifi();
            break;
        case ST_SET_WIFI:
            st_set_wifi();
            break;
        case ST_CONNECT_WIFI:
            st_connect_wifi();
            break;
        case ST_FILES:
            st_files();
            break;
        case ST_INFO:
            st_info();
            break;
        case ST_LOBBY:
            st_lobby();
            break;
        default:
            st_hosts();
            break;
        }
    }
}
```

state.h

```
/* state.h -- the shared globals and the seams between the screen modules.
 *
 * One invariant to keep when adding code: the reply window at $B000 is only
 * repainted by committing a transaction. Any flow that edits something and
 * writes it back must therefore re-read first and stream out of the fresh
 * window (the way rename does) -- never hold a window across an intervening
 * transaction. fn_edit() runs no transactions, which is what makes "fetch,
 * edit, stream back" safe.
 */

#ifndef STATE_H
#define STATE_H

#include <stdbool.h>

#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include <fujinet-bus-atari5200.h>

#include "constants.h"

extern unsigned char state;

extern unsigned char host;       /* the mounted/browsed host slot */
extern unsigned char src_host;   /* copy: the source's host slot */
extern unsigned char copy_mode;  /* nonzero while a copy source is marked */

extern unsigned char cur;        /* selection bar row within the window */
extern unsigned char nrows;      /* rows the current list actually has */
extern unsigned char at_end;     /* the visible page ends the directory */
extern unsigned int  top;        /* first visible entry (16-bit: big dirs) */
extern unsigned char hosts_mask; /* bit i set = host slot i is nonempty */

extern char path[PATH_MAX_LEN];  /* current dir: '/'-prefixed, '/'-terminated */
extern char src_spec[SPEC_MAX_LEN]; /* copy source full path; also the                                    custom-SSID scratch (never both) */

/* config.c */
void status_line(const char *s);             /* STATUS_ROW, cleared first */
void legend_line(const char *s);             /* LEGEND_ROW, cleared first */
void fail(const char *what);                 /* "?WHAT ERROR xx" + BEEP */
void draw_frame(const char *title);          /* cls + the frame, titled */
void wait_frames(unsigned char n);           /* n frames, input ignored */
void bar_move(signed char d);                /* the non-paging selection bar */

/* One screen per state; each draws itself, runs its own event loop, and
 * returns once it has set `state` to something else. */
void st_check_wifi(void);
void st_set_wifi(void);
void st_connect_wifi(void);
void st_hosts(void);
void st_files(void);
void st_info(void);
void st_lobby(void);

/* st_files.c -- shared with the copy and boot flows */
bool dir_seek(unsigned int pos);
bool dir_read_entry(unsigned char maxlen);
bool dir_open(void);
void files_draw(void);
void files_legend(void);
void leave_host(void);

/* st_copy.c */
void copy_mark(void);                        /* PAUSE on a file: mark it */
void copy_here(void);                        /* PAUSE mid-copy: copy it here */
void copy_cancel(void);                      /* back to the source browse */

/* st_boot.c */
void boot_screen(volatile unsigned char *name); /* the LOADING screen */
void boot_reply_entry(void);                 /* boot path + name-in-window;

   returns only on failure */
void boot_mount_swap(void);                  /* MOUNT_IMAGE + progress + boot;

   returns only on failure */

#endif /* STATE_H */
```

st_hosts.c

```
/* st_hosts.c -- the host slots screen, and (in copy livery) the destination
 * picker for a host-to-host copy.
 *
 * Slot names are drawn straight out of the reply window; nothing is cached
 * but a one-byte emptiness mask, gathered while drawing, that keeps button 1
 * from mounting an empty slot. Renaming re-reads the slots and streams all
 * eight back with the edited one substituted -- see fnraw_write_host_slot.
 * A ninth row under the slots boots the FujiNet Game Lobby (st_lobby.c).
 */

#include "fujidisp.h"
#include "fujiin.h"
#include "fujiedit.h"
#include "fujiraw.h"
#include "sfx.h"
#include "state.h"

#define LOBBY_ROW   HOST_SLOTS  /* the bar index of PLAY GAME LOBBY */

static void hosts_draw(void)
{
    unsigned char i;

    draw_frame(copy_mode ? "COPY TO WHICH HOST?" : "HOST SLOTS");
    if (copy_mode) {
        status_line("#--PICK DESTINATION");
        legend_line("*--CANCEL COPY");
    } else {
        status_line("#--OPEN  PAUSE--RENAME");
        legend_line("*--WIFI  START--INFO");
    }

    if (!FUJICALL(FUJICMD_READ_HOST_SLOTS)) {
        fail("HOSTS");
        nrows = 0;
        return;
    }

    hosts_mask = 0;
    for (i = 0; i < HOST_SLOTS; i++) {
        volatile unsigned char *name = FN_REPLY + (unsigned int)i * HOST_STRIDE;
        unsigned char row = (unsigned char)(LIST_TOP + i);

        disp_row_clear(row);
        disp_at_u16(3, row, (unsigned int)(i + 1));
        if (*name == 0) {
            disp_at(5, row, "(EMPTY)");
            continue;
        }
        disp_text(5, row, name, IN_R - 4);
        hosts_mask |= (unsigned char)(1 << i);
    }
    nrows = HOST_SLOTS;
    if (!copy_mode) {
        disp_row_clear(LIST_TOP + LOBBY_ROW);
        disp_at(5, LIST_TOP + LOBBY_ROW, "PLAY GAME LOBBY");
        nrows = HOST_SLOTS + 1;
    }
    at_end = 1;
    if (cur >= nrows)
        cur = 0;
    disp_row_invert((unsigned char)(LIST_TOP + cur), true);
}

static void enter_host(void)
{
    if (!(hosts_mask & (unsigned char)(1 << cur))) {
        status_line(copy_mode ? "?EMPTY SLOT" : "?EMPTY SLOT  PAUSE--RENAME");
        sfx_beep();
        return;
    }

    status_line("MOUNTING HOST...");
    if (!fuji_mount_host_slot(cur)) {
        fail("HOST");
        return;
    }

    host = cur;
    path[0] = '/';
    path[1] = 0;
    fn_entry[0] = 0;            /* the filter starts clear */
    top = 0;
    if (!dir_open()) {
        fail("OPEN");
        return;
    }
    cur = 0;
    sfx_accept();
    state = ST_FILES;
}

static void rename_host(void)
{
    volatile unsigned char *name;
    unsigned char i;

    if (cur == LOBBY_ROW) {
        sfx_beep();
        return;
    }
    if (!FUJICALL(FUJICMD_READ_HOST_SLOTS)) {
        fail("HOSTS");
        return;
    }
    name = FN_REPLY + (unsigned int)cur * HOST_STRIDE;
    for (i = 0; i < HOST_STRIDE - 1 && name[i] != 0; i++)
        fn_entry[i] = (char)name[i];
    fn_entry[i] = 0;

    if (fn_edit("HOST NAME", HOST_STRIDE - 1)) {
        if (!fnraw_write_host_slot(cur, fn_entry)) {
            fail("WRITE");
            wait_frames(90);
        }
    }
    hosts_draw();
}

void st_hosts(void)
{
    hosts_draw();
    while (state == ST_HOSTS) {
        unsigned char ev = in_read();

        switch (ev) {
        case IN_UP:
            bar_move(-1);
            break;
        case IN_DOWN:
            bar_move(1);
            break;
        case IN_FIRE:
            if (cur == LOBBY_ROW && !copy_mode)
                state = ST_LOBBY;
            else
                enter_host();
            break;
        case IN_BACK:
            if (copy_mode) {
                copy_cancel();
                if (state == ST_HOSTS)
                    hosts_draw();   /* the source could not be reopened */
            } else {
                state = ST_SET_WIFI;
            }
            break;
        case IN_START:
            if (!copy_mode)
                state = ST_INFO;
            break;
        case IN_SELECT:
            if (!copy_mode)
                rename_host();
            break;
        }
```

```
    }
}

```

st_files.c

```
/* st_files.c -- the file browser: paging, descend/devance, filter, and the
 * two ends of a host-to-host copy (st_copy.c).
 *
 * One seek per page then sequential reads (READ_DIR_ENTRY advances the
 * cursor itself), names drawn straight from the reply window at display
 * width and re-read at full width the moment one is acted on. `path` is
 * '/'-prefixed, '/'-terminated, appended to on descend and truncated at the
 * previous '/' on devance.
 */

#include <string.h>

#include "fujidisp.h"
#include "fujiin.h"
#include "fujiedit.h"
#include "fujiraw.h"
#include "sfx.h"
#include "state.h"

bool dir_seek(unsigned int pos)
{
    return fuji_set_directory_position(pos);
}

bool dir_read_entry(unsigned char maxlen)
{
    volatile unsigned char *r;
    unsigned char b0, b1, b2;

    if (!FUJICALL_A1_A2(FUJICMD_READ_DIR_ENTRY, maxlen, 0))
        return false;

    /* End of directory is two 0x7F bytes. Not every host stops there --
     * reading past the end of an SD root gives back "." forever and poisons
     * SET_DIRECTORY_POSITION for the rest of the session -- so "." and ".."
     * count as the end too, and are never read past. */
    r = FN_REPLY;
    b0 = r[0];
    b1 = r[1];
    b2 = r[2];
    if (b0 == 0x7F && b1 == 0x7F)
        return false;
    if (b0 == '.' && (b1 == 0 || (b1 == '.' && b2 == 0)))
        return false;
    return true;
}

bool dir_open(void)
{
    return fnraw_open_directory(host, path, fn_entry);
}

static bool dir_reopen(void)
{
    fuji_close_directory();     /* best effort */
    return dir_open();
}

void files_legend(void)
{
    if (copy_mode) {
        status_line("START--COPY HERE");
        legend_line("#--OPEN DIR *--BACK <>--PAGE");
    } else {
        status_line("#--OPEN *--BACK PAUSE--FILTER");
        legend_line("<>--PAGE  START--COPY");
    }
}

/* Blank the list window from row `i` down. A short page must not leave the
 * tail of the page before it on screen. */
static void list_clear_from(unsigned char i)
{
    for (; i < LIST_ROWS; i++)
        disp_row_clear((unsigned char)(LIST_TOP + i));
}

static void files_page(void)
{
    unsigned char i;

    if (!dir_seek(top)) {
        list_clear_from(0);
        fail("SEEK");
        nrows = 0;
        return;
    }

    nrows = 0;
    at_end = 0;
    for (i = 0; i < LIST_ROWS; i++) {
        unsigned char row = (unsigned char)(LIST_TOP + i);

        disp_row_clear(row);
        if (!dir_read_entry(NAMELEN)) {
            at_end = 1;
            break;
        }
        disp_text(3, row, FN_REPLY, NAMELEN);
        nrows = (unsigned char)(i + 1);
    }
    list_clear_from(nrows);

    if (nrows == 0) {
        disp_at(3, LIST_TOP, fn_entry[0] ? "(NO MATCHES)" : "(EMPTY)");
        return;
    }
    if (cur >= nrows)
        cur = (unsigned char)(nrows - 1);
    disp_row_invert((unsigned char)(LIST_TOP + cur), true);
}

void files_draw(void)
{
    unsigned char plen = (unsigned char)strlen(path);
    char title[7];

    strcpy(title, "HOST 1");
    title[5] = (char)('1' + host);
    draw_frame(title);
    disp_at(IN_L, PATH_ROW, plen > IN_W ? path + (plen - IN_W) : path);
    files_legend();
    files_page();
}

static void fmove(signed char d)
{
    if (d < 0) {
        if (nrows != 0 && cur > 0) {
            disp_row_invert((unsigned char)(LIST_TOP + cur), false);
            cur--;
            disp_row_invert((unsigned char)(LIST_TOP + cur), true);
        } else if (top >= LIST_ROWS) {
            top -= LIST_ROWS;
            cur = LIST_ROWS - 1;
            files_page();
        }
    } else {
        if (nrows == 0)
            return;
```

```
        if ((unsigned char)(cur + 1) < nrows)
{
            disp_row_invert((unsigned char)
(LIST_TOP + cur), false);
            cur++;
            disp_row_invert((unsigned char)
(LIST_TOP + cur), true);
        } else if (!at_end) {
            top += LIST_ROWS;
            cur = 0;
            files_page();
        }
    }
}

void leave_host(void)
{
    fuji_close_directory();
    cur = host;
    state = ST_HOSTS;
}

/* Up a directory. At the root, back leaves
the host -- or, mid-copy,
 * abandons the copy. */
static void devance(void)
{
    unsigned char len = (unsigned
char)strlen(path);

    if (len <= 1) {
        if (copy_mode) {
            copy_cancel();
            if (state == ST_FILES)
                files_draw();
        } else {
            leave_host();
        }
        return;
    }
    len--;
    while (len > 0 && path[len - 1] != '/')
        len--;
    path[len] = 0;
    top = 0;
    cur = 0;
    if (!dir_reopen()) {
        fail("OPEN");
        return;
    }
    files_draw();
}

static void open_or_boot(void)
{
    volatile unsigned char *name;
    unsigned char plen;
    unsigned int n, i;

    if (nrows == 0)
        return;
    status_line("READING...");
    if (!dir_seek(top + cur) || !
dir_read_entry(FULLLEN)) {
        fail("READ");
        return;
    }
    name = FN_REPLY;
    n = 0;
    while (n < FULLLEN && name[n] != 0)
        n++;
    if (n == 0) {
        fail("NAME");
        return;
    }

    if (name[n - 1] == '/') {
        plen = (unsigned char)strlen(path);
        if (plen + n > PATH_MAX_LEN - 1) {
            status_line("?PATH TOO LONG");
            sfx_beep();
            wait_frames(90);
            files_legend();
            return;
        }
        for (i = 0; i < n; i++)
            path[plen + i] = (char)name[i];
        path[plen + n] = 0;
        top = 0;
        cur = 0;
        if (!dir_reopen()) {
            fail("OPEN");
            wait_frames(90);
            path[plen] = 0;
            dir_reopen();
        }
        files_draw();
        return;
    }

    if (copy_mode) {
        status_line("START--COPY HERE");
        sfx_beep();
        return;
    }

    boot_reply_entry();         /* only
failure comes back */
    wait_frames(120);
    files_draw();
}

static void do_filter(void)
{
    fn_entry[FILTER_MAX] = 0;
    fn_edit("FILTER", FILTER_MAX);
    top = 0;
    cur = 0;
    if (!dir_reopen())
        fail("OPEN");
    files_draw();
}

void st_files(void)
{
    files_draw();
    while (state == ST_FILES) {
        unsigned char ev = in_read();

        switch (ev) {
        case IN_UP:
            fmove(-1);
            break;
        case IN_DOWN:
            fmove(1);
            break;
        case IN_LEFT:
            if (top >= LIST_ROWS) {
                top -= LIST_ROWS;
                cur = 0;
                files_page();
            }
            break;
        case IN_RIGHT:
            if (nrows != 0 && !at_end) {
                top += LIST_ROWS;
                cur = 0;
                files_page();
            }
            break;
        case IN_FIRE:
            open_or_boot();
            break;
        case IN_BACK:
            devance();
            break;
        case IN_SELECT:
            do_filter();
            break;
        case IN_START:
            if (copy_mode) {
                copy_here();
                if (state == ST_FILES)
                    files_draw();
            } else {
                copy_mark();
            }
            break;
        }
    }
}
```

st_boot.c

```
/* st_boot.c -- mount into device slot 0 and hand the console to the image.
 *
 * Not a dispatcher state: booting is called from the browser (and, for the
 * Game Lobby, from the host slots) and only ever RETURNS on failure --
 * success ends in fuji_a5200_boot(), which clears the machine as a power-on
 * leaves it and jumps through the cartridge's stub into the console's own
 * BIOS, which starts the image the cart swaps in as it looks.
 *
 * The whole time is spent on a LOADING screen of its own: the file's name,
 * the host it comes from, a bar the cartridge's own progress fills, and the
 * bytes received so far against the image's size.
 */

#include <stdlib.h>
#include <string.h>

#include "fujidisp.h"
#include "fujiin.h"
#include "fujiraw.h"
#include "sfx.h"
#include "state.h"

#define NAME_ROW   3
#define HOST_ROW   5
#define BAR_BOX_T  8
#define BAR_ROW    9
#define BAR_L      4            /* the box's left edge; the bar starts one in */
#define BAR_CELLS  30
#define PCT_ROW    12
#define PCT_COL    18
#define BYTES_ROW  14

/* The cart gives MOUNT_IMAGE 60 seconds; wait a little longer so its own
 * timeout is the one that surfaces. */
#define MOUNT_FRAMES 3900u

/* A FujiNet that does not take the file for a cartridge image mounts it as a
 * disk, ACKs, and pushes nothing. */
#define IDLE_FRAMES 180u

/* "got/total BYTES": the total and the layout are fixed once the image
 * stream opens, so each update rewrites only the right-aligned count. */
static char tot_txt[9];
static unsigned char tot_len, got_col;
static unsigned long shown_tot, shown_got;

static void boot_bytes(unsigned long got, unsigned long tot)
{
    char txt[9];
    unsigned char n;

    if (tot != shown_tot) {
        shown_tot = tot;
        shown_got = ~0ul;
        disp_row_clear(BYTES_ROW);
        if (tot == 0)
            return;
        ultoa(tot, tot_txt, 10);
        tot_len = (unsigned char)strlen(tot_txt);
        got_col = (unsigned char)((DISP_COLS - (tot_len * 2 + 7)) / 2);
        disp_at((unsigned char)(got_col + tot_len), BYTES_ROW, "/");
        disp_at((unsigned char)(got_col + tot_len + 1), BYTES_ROW, tot_txt);
        disp_at((unsigned char)(got_col + tot_len * 2 + 1), BYTES_ROW,
                " BYTES");
    }
    if (tot == 0 || got == shown_got)
        return;
    shown_got = got;
    ultoa(got, txt, 10);
    n = (unsigned char)strlen(txt);
    if (n > tot_len)
        n = tot_len;
    disp_at((unsigned char)(got_col + tot_len - n), BYTES_ROW, txt);
    while (n < tot_len)
        disp_char((unsigned char)(got_col + tot_len - 1 - n++), BYTES_ROW,
                  ' ');
}

static void boot_pct(unsigned char pct)
{
    disp_bar((unsigned char)(BAR_L + 1), BAR_ROW, BAR_CELLS, pct, 100);
    disp_at(PCT_COL, PCT_ROW, "    %");
    disp_at_u16(PCT_COL, PCT_ROW, pct);
    disp_at((unsigned char)(PCT_COL + (pct >= 100 ? 3 : pct >= 10 ? 2 : 1)),
            PCT_ROW, "%");
}

/* The screen, with the name painted out of the reply window: drawing runs no
 * transaction, so the window still holds it for the SET_DEVICE_FULLPATH
 * after. */
void boot_screen(volatile unsigned char *name)
{
    draw_frame("LOADING");
    disp_text(IN_L, NAME_ROW, name, IN_W);
    disp_at(IN_L, HOST_ROW, "FROM HOST");
    disp_at_u16(IN_L + 10, HOST_ROW, (unsigned int)(host + 1));
    disp_box(BAR_L, BAR_BOX_T, (unsigned char)(BAR_L + BAR_CELLS + 1),
             (unsigned char)(BAR_BOX_T + 2), NULL);
    disp_bar_reset();
    boot_pct(0);
    shown_tot = 0;
    disp_row_clear(BYTES_ROW);
}

static void load_error(unsigned char code)
{
    disp_row_clear(STATUS_ROW);
    disp_at(IN_L, STATUS_ROW, "?LOAD ERROR");
    disp_at_hex8(IN_L + 12, STATUS_ROW, code);
    sfx_beep();
}

void boot_mount_swap(void)
{
    unsigned char pct = 0, want, st;
    bool acked = false;
    unsigned int frames = 0, acked_at = 0;

    /* Depending on the FujiNet, the image is pushed to the cartridge before
     * MOUNT_IMAGE is answered (so progress is only visible while that reply
     * is still outstanding) or after it. Either way: start the mount, then
     * watch the cart's boot registers -- and the reply -- until the image
     * is READY. The cart clears those registers when it starts the mount,
     * which may be after our first look, so FAILED counts only once the
     * reply is in. */
    status_line("LOADING...");
    want = fnraw_mount_start(DEVICE_SLOT, MODE_READ);
    for (;;) {
        unsigned char now;
        bool replied = !acked && FN_ACKSEQ == want;

        st = fuji_a5200_boot_state();
        if (replied) {
            acked = true;
            acked_at = frames;
        }
        if (acked && st == FUJI_A5200_BOOT_FAILED) {

load_error(fuji_a5200_boot_error());
            return;
```

```
        }
        if (replied && !fnraw_reply_ok()) {
            fail("MOUNT");
            return;
        }
        if (acked && st ==
FUJI_A5200_BOOT_READY)
            break;
        if (acked && st ==
FUJI_A5200_BOOT_IDLE && frames - acked_at >
IDLE_FRAMES) {
            status_line("?NOT A CARTRIDGE
IMAGE");
            sfx_beep();
            return;
        }
        if (++frames > MOUNT_FRAMES) {
            load_error(st);
            return;
        }
        now = fuji_a5200_boot_percent();
        if (now > 100)
            now = 100;
        if (now != pct) {
            pct = now;
            boot_pct(pct);
        }
        boot_bytes(fuji_a5200_boot_got(),
fuji_a5200_boot_total());
        wait_frames(1);
    }

    boot_pct(100);
    boot_bytes(fuji_a5200_boot_total(),
fuji_a5200_boot_total());
    status_line("BOOTING");
    disp_at(IN_L, LEGEND_ROW, "OK");
    sfx_accept();
    wait_frames(10);            /* let the
screen catch up */
    fuji_a5200_boot();          /* returns
only if the cart never armed it */
    disp_row_clear(LEGEND_ROW);
    fail("BOOT");
}

/* Boot the entry whose FULL-width name is
sitting in the reply window (the
 * caller just re-read it there): the path
prefix comes from RAM, the filename
 * streams cartridge-to-cartridge without
landing in console RAM. */
void boot_reply_entry(void)
{
    boot_screen(FN_REPLY);
    status_line("SETTING PATH...");
    if (!
fnraw_set_device_path_from_reply(DEVICE_SLOT,
host, MODE_READ,

path, FN_REPLY)) {
        fail("PATH");
        return;
    }
    boot_mount_swap();
}

```

st_wifi.c

```
/* st_wifi.c -- the three WiFi states. The
scan list is drawn straight out of
 * the reply window, one GET_SCAN_RESULT per
row. Picking a network re-fetches
 * its result and holds the SSID in src_spec
across the password edit.
 */

#include <string.h>

#include "fujidisp.h"
#include "fujiin.h"
#include "fujiedit.h"
#include "fujiraw.h"
#include "sfx.h"
#include "state.h"

static unsigned char nnets;


void st_check_wifi(void)
{
    volatile unsigned char *r = FN_REPLY;
    unsigned char s = 0;

    if (!fuji_get_wifi_enabled()) {
        state = ST_HOSTS;
        return;
    }
    if (!fuji_get_wifi_status(&s)) {
        state = ST_HOSTS;
        return;
    }
    if (s == 3) {
        state = ST_HOSTS;
        return;
    }
    if (FUJICALL(FUJICMD_GET_SSID) && r[0] !=
0)
```

```
        state = ST_CONNECT_WIFI;
    else
        state = ST_SET_WIFI;
}

void st_connect_wifi(void)
{
    unsigned char tries, s;

    draw_frame("JOINING NETWORK");
    status_line("PLEASE WAIT  *--CANCEL");
    if (FUJICALL(FUJICMD_GET_SSID))
        disp_text(IN_L + 1, LIST_TOP, FN_REPLY, SSID_LEN - 1);

    for (tries = 0; tries < 20; tries++) {
        unsigned char start = in_frames();

        while ((unsigned char)(in_frames() - start) < 120) {
            unsigned char ev = in_read();

            if (ev == IN_BACK || ev == IN_FIRE) {
                state = ST_SET_WIFI;
                return;
            }
        }

        s = 0;
        if (!fuji_get_wifi_status(&s))
            continue;
        switch (s) {
        case 3:
            status_line("CONNECTED");
            disp_at(IN_L, LEGEND_ROW, "OK");
            sfx_accept();
            wait_frames(50);
            state = ST_HOSTS;
            return;
        case 1:
            status_line("?NO SSID AVAILABLE");
            sfx_beep();
            wait_frames(120);
            state = ST_SET_WIFI;
            return;
        case 4:
            status_line("?CONNECT FAILED");
            sfx_beep();
            wait_frames(120);
            state = ST_SET_WIFI;
            return;
        case 5:
            status_line("?CONNECTION LOST");
            sfx_beep();
            wait_frames(120);
            state = ST_SET_WIFI;
            return;
        }
    }
    status_line("?UNABLE TO CONNECT");
    sfx_beep();
    wait_frames(120);
    state = ST_SET_WIFI;
}

static void draw_rssi(unsigned char row, unsigned char raw)
{
    signed char dbm = (signed char)raw;
    unsigned int mag = (unsigned int)(dbm < 0 ? -dbm : dbm);

    disp_at(IN_R - 4, row, dbm < 0 ? "-" : " ");
    disp_at_u16(IN_R - 3, row, mag);
}

static void wifi_draw(void)
{
    volatile unsigned char *r = FN_REPLY;
    unsigned char i;

    draw_frame("SELECT NETWORK");
    status_line("SCANNING...");
    disp_row_clear(LEGEND_ROW);

    nnets = 0;
    if (FUJICALL(FUJICMD_SCAN_NETWORKS)) {
        nnets = r[0];
        if (nnets > WIFI_MAX)
            nnets = WIFI_MAX;
    }
    for (i = 0; i < nnets; i++) {
        unsigned char row = (unsigned char)(LIST_TOP + i);

        if (!FUJICALL_A1(FUJICMD_GET_SCAN_RESULT, i)) {
            nnets = i;
            break;
        }
        disp_text(3, row, r, IN_R - 8);
        draw_rssi(row, r[SSID_LEN]);
    }
    disp_at(3, (unsigned char)(LIST_TOP + nnets), "OTHER (TYPE AN SSID)");
    nrows = (unsigned char)(nnets + 1);

    status_line("#--SELECT  START--RESCAN");
    legend_line("*--HOSTS");
    if (cur >= nrows)
        cur = 0;
    disp_row_invert((unsigned char)(LIST_TOP + cur), true);
}

static void wifi_pick(void)
{
    volatile unsigned char *r = FN_REPLY;
    unsigned char i;

    if (cur < nnets) {
        if (!FUJICALL_A1(FUJICMD_GET_SCAN_RESULT, cur)) {
            fail("SCAN");
            return;
        }
        for (i = 0; i < SSID_LEN - 1 && r[i] != 0; i++)
            src_spec[i] = (char)r[i];
        src_spec[i] = 0;
    } else {
        fn_entry[0] = 0;
        if (!fn_edit("SSID", SSID_LEN - 1) || fn_entry[0] == 0) {
            wifi_draw();
            return;
        }
        strcpy(src_spec, fn_entry);
    }

    fn_entry[0] = 0;
    if (!fn_edit("PASSWORD", PASS_MAX)) {
        wifi_draw();
        return;
    }

    status_line("SETTING SSID...");
    if (!fnraw_set_ssid(src_spec, fn_entry)) {
        fail("SSID");
        wait_frames(120);
        wifi_draw();
        return;
    }
    state = ST_CONNECT_WIFI;
}

void st_set_wifi(void)
{
    cur = 0;
    wifi_draw();
    while (state == ST_SET_WIFI) {
        unsigned char ev = in_read();

        switch (ev) {
        case IN_UP:
            bar_move(-1);
            break;
        case IN_DOWN:
            bar_move(1);
            break;
        case IN_FIRE:
            wifi_pick();
            break;
        case IN_START:
            cur = 0;
            wifi_draw();
            break;
        case IN_BACK:
            state = ST_HOSTS;
            break;
        }
    }
}

```

st_copy.c

```
/* st_copy.c -- host-to-host copy:
 *
 *   1. PAUSE on a highlighted FILE marks it (src_spec, src_host) and bounces
 *      to ST_HOSTS, which renders itself as "COPY TO WHICH HOST?".
 *   2. picking a host there mounts it and browses it like any other.
 *   3. PAUSE anywhere in the destination tree copies the file there and
 *      puts the browser back where the copy started.
 *
 * No confirmation step, matching every other platform's CONFIG, and no
 * progress to show: COPY_FILE is one host-side transaction whose ACK arrives
 * when the copy is done.
 */

#include <string.h>

#include "fujidisp.h"
#include "fujiedit.h"
#include "fujiraw.h"
#include "sfx.h"
#include "state.h"

void copy_mark(void)
{
    volatile unsigned char *name;
    unsigned char plen;
    unsigned int n, i;

    if (nrows == 0)
        return;
    status_line("READING...");
    if (!dir_seek(top + cur) || !dir_read_entry(FULLLEN)) {
        fail("READ");
        return;
    }
    name = FN_REPLY;
    n = 0;
    while (n < FULLLEN && name[n] != 0)
        n++;
    if (n == 0) {
        fail("NAME");
        return;
    }
    if (name[n - 1] == '/') {
        status_line("?CANNOT COPY A FOLDER");
        sfx_beep();
        wait_frames(90);
        files_legend();
        return;
    }

    plen = (unsigned char)strlen(path);
    if (plen + n > SPEC_MAX_LEN - 1) {
        status_line("?PATH TOO LONG");
        sfx_beep();
        wait_frames(90);
        files_legend();
        return;
    }
    for (i = 0; i < plen; i++)
        src_spec[i] = path[i];
    for (i = 0; i < n; i++)
        src_spec[plen + i] = (char)name[i];
    src_spec[plen + n] = 0;

    src_host = host;
    copy_mode = 1;
    fuji_close_directory();
    cur = src_host;             /* the hosts bar starts on the source */
    state = ST_HOSTS;
}

/* Back to the source browse: derive the directory from src_spec, remount and
 * reopen. On success the caller finds state == ST_FILES and redraws (or the
 * dispatcher does); on failure the host list is the safe place to land. */
static void copy_return(void)
{
    unsigned char len, i;

    copy_mode = 0;
    host = src_host;
    len = (unsigned char)strlen(src_spec);
    while (len > 0 && src_spec[len - 1] != '/')
        len--;
    for (i = 0; i < len; i++)
        path[i] = src_spec[i];
    path[len] = 0;
    fn_entry[0] = 0;            /* the filter belonged to the other browse */
    top = 0;
    cur = 0;

    fuji_close_directory();
    if (!fuji_mount_host_slot(host) || !dir_open()) {
        fail("OPEN");
        wait_frames(90);
        cur = host;
        state = ST_HOSTS;
        return;
    }
    state = ST_FILES;
}

void copy_here(void)
{
    if (!fnraw_copy_fits(src_spec, path)) {
        status_line("?PATH TOO LONG");
        sfx_beep();
    } else {
        status_line("COPYING...");
        if (fnraw_copy_file((unsigned char)(src_host + 1),
                            (unsigned char)(host + 1), src_spec, path)) {
            status_line("COPIED");
            sfx_accept();
        } else {
            fail("COPY");
        }
    }
    wait_frames(120);
    copy_return();
}

void copy_cancel(void)
{
    copy_return();
}
```

st_info.c

```
/* st_info.c -- the adapter info screen: GET_ADAPTERCONFIG_EXTENDED, drawn
 * field by field straight out of the 240-byte reply in the window.
 */

#include "fujidisp.h"
#include "fujiin.h"
#include "sfx.h"
#include "state.h"

#define VAL_COL   (IN_L + 10)
#define VAL_W     (IN_R + 1 - VAL_COL)

static void draw_field(unsigned char row, const char *label,
                       unsigned int off, unsigned char max)
{
    disp_at(IN_L + 1, row, label);
    disp_text(VAL_COL, row, FN_REPLY + off, max);
}

void st_info(void)
{
    draw_frame("ADAPTER INFO");
    status_line("READING...");

    if (FUJICALL(FUJICMD_GET_ADAPTERCONFIG_EXTENDED))
    {
        draw_field(2, "SSID", ACX_SSID, VAL_W);
        draw_field(3, "HOST", ACX_HOSTNAME, VAL_W);
        draw_field(5, "IP", ACX_SLOCALIP, 15);
        draw_field(6, "GATEWAY", ACX_SGATEWAY, 15);
        draw_field(7, "NETMASK", ACX_SNETMASK, 15);
        draw_field(8, "DNS", ACX_SDNSIP, 15);
        draw_field(10, "MAC", ACX_SMAC, 17);
        draw_field(11, "BSSID", ACX_SBSSID, 17);
        draw_field(13, "VERSION", ACX_VERSION, 14);
        status_line("");
        legend_line("#,*--BACK");
    } else {
        fail("INFO");
    }

    while (state == ST_INFO) {
        unsigned char ev = in_read();

        if (ev == IN_FIRE || ev == IN_BACK)
            state = ST_HOSTS;
    }
}

```

st_lobby.c

```
/* st_lobby.c -- PLAY GAME LOBBY on the hosts screen: mount and boot the
 * FujiNet Game Lobby without walking the browser. It uses the host slot that
 * already holds LOBBY_HOST (any case); if none does, it writes it into the
 * first empty slot, or into slot 8 when all eight are taken.
 */

#include "fujidisp.h"
#include "fujiraw.h"
#include "state.h"

static const char lobby_host[] = LOBBY_HOST;

static bool slot_matches(volatile unsigned char *name)
{
    unsigned char j;

    for (j = 0; lobby_host[j] != 0; j++) {
        unsigned char c = name[j];

        if (c >= 'A' && c <= 'Z')
            c = (unsigned char)(c + 32);
        if (c != (unsigned char)lobby_host[j])
            return false;
    }
    return name[j] == 0;
}

/* On success this never comes back -- boot_mount_swap() hands over. */
static void lobby_go(void)
{
    volatile unsigned char *r;
    unsigned char slot = 0xFF, empty = 0xFF;
    unsigned char i;

    status_line("FINDING LOBBY HOST...");
    if (!FUJICALL(FUJICMD_READ_HOST_SLOTS)) {
```

```
        fail("HOSTS");
        return;
    }
    for (i = 0; i < HOST_SLOTS; i++) {
        r = FN_REPLY + (unsigned int)i *
HOST_STRIDE;
        if (slot_matches(r)) {
            slot = i;
            break;
        }
        if (*r == 0 && empty == 0xFF)
            empty = i;
    }
    if (slot == 0xFF) {
        slot = empty != 0xFF ? empty :
HOST_SLOTS - 1;
        status_line("ADDING LOBBY HOST...");
        if (!fnraw_write_host_slot(slot,
lobby_host)) {
            fail("WRITE");
            return;
        }
    }

    status_line("MOUNTING HOST...");
    if (!fuji_mount_host_slot(slot)) {
        fail("HOST");
        return;
    }
    host = slot;

    boot_screen((volatile unsigned char
*)"GAME LOBBY");
    status_line("SETTING PATH...");
    if (!fnraw_set_device_path(DEVICE_SLOT,
slot, MODE_READ, LOBBY_PATH)) {
        fail("PATH");
        return;
    }
    boot_mount_swap();
}

void st_lobby(void)
{
    lobby_go();
    wait_frames(120);           /* whatever
failed is on the status line */
    cur = HOST_SLOTS;
    state = ST_HOSTS;
}

```

fujiraw.h

```
/* fujiraw.h -- the transactions fujinet-lib's
buffer-shaped calls do not
 * fit, built on the lib's own exported
mailbox primitives.
 *
 * fuji_bus_call() takes its payload as one
contiguous buffer. These commands
 * want a payload assembled from pieces -- a
RAM path prefix plus a filename
 * that only exists in the cartridge's reply
window, or all eight host slots
 * with one replaced -- so they stream it byte
by byte through fn_tx() and
 * fn_regwr() and pad on the fly. Everything
else goes through fujinet-lib.
 */

#ifndef FUJIRAW_H
#define FUJIRAW_H

#include <stdbool.h>

/* OPEN_DIRECTORY: "path\0filter\0" NUL-padded
to exactly 256 bytes --
 * fujiDevice splits at the first NUL and
treats the rest as the pattern.
 * (fuji_open_directory_filter() in the lib
does the same job through a
 * static 256-byte buffer; do not link it
here.) */
bool fnraw_open_directory(unsigned char
host_slot,
                          const char *dirpath,
const char *pattern);

/* WRITE_HOST_SLOTS takes no parameters and
one 256-byte payload -- all eight
 * 32-byte slots, every time. Re-reads the
slots and streams them straight
 * back out of the reply window with slot
`slot` replaced by `name`. */
bool fnraw_write_host_slot(unsigned char slot,
const char *name);

/* SET_DEVICE_FULLPATH with the filename
streamed from the reply window
 * (`name` points into it), prefixed by the
RAM-resident directory path. */
bool fnraw_set_device_path_from_reply(unsigned
char dev, unsigned char host_slot,
                                      unsigned
char mode, const char *prefix,
                                      volatile
unsigned char *name);

/* SET_DEVICE_FULLPATH from a plain string
(the lobby's fixed path). */
bool fnraw_set_device_path(unsigned char dev,
unsigned char host_slot,
                           unsigned char mode,
const char *fullpath);

/* MOUNT_IMAGE without waiting for the reply;
returns its sequence number.
 * The push may finish before or after the
reply, so the caller polls the
 * boot registers until FN_ACKSEQ reads that
number and the state reads
 * READY, checking fnraw_reply_ok() once the
reply is in. */
```

```
unsigned char fnraw_mount_start(unsigned char dev, unsigned char mode);
bool fnraw_reply_ok(void);

/* SET_SSID: nparam >= 1 (value ignored) and exactly ssid[33]+password[64]. */
bool fnraw_set_ssid(const char *ssid, const char *password);

/* COPY_FILE: "sourcefullpath|destdir+basename(source)" at its EXACT length --
 * no NUL, no padding; the firmware reads dataAsString(), and padding would
 * bury NULs inside the destination filename. Slots are 1-BASED, unlike every
 * other command (the firmware rejects 0 and then decrements). Waits out the
 * cartridge's 60-second budget: the ACK arrives only when the host-side copy
 * is done. fnraw_copy_fits() says whether the payload fits the TX stream. */
bool fnraw_copy_fits(const char *source_spec, const char *dest_dir);
bool fnraw_copy_file(unsigned char src_slot1, unsigned char dst_slot1,
                     const char *source_spec, const char *dest_dir);

#endif /* FUJIRAW_H */
```

fujiraw.c

```
/* fujiraw.c -- streamed-payload transactions over fujinet-lib's mailbox
 * primitives. See fujiraw.h for why these exist at all.
 *
 * Everything here leans on what bus/atari5200 exports: fn_tx() (a read of
 * the TX page appends a byte), fn_regwr() and fn_commit(). Nothing else
 * touches the hotspot pages.
 */

#include <string.h>

#include <fujinet-fuji.h>
#include <fujinet-atari5200.h>
#include <fujinet-bus-atari5200.h>

#include "fujiraw.h"

#define PAYLOAD_LEN 256

static void raw_begin(unsigned char cmd)
{
    fn_regwr(FNR_DATA_RST, 0);
    fn_regwr(FNR_DEVICE, FUJI_DEVICEID_FUJINET);
    fn_regwr(FNR_CMD, cmd);
    fn_regwr(FNR_NPARAM, 0);
}

/* One 8-bit parameter; `count` is the running parameter count including this
 * one. Parameters ride the TX stream as {size, value}. */
static void raw_param8(unsigned char v, unsigned char count)
{
    fn_tx(1);
    fn_tx(v);
    fn_regwr(FNR_NPARAM, count);
}

static void raw_tx(unsigned char b)
{
    fn_tx(b);
}

static void raw_tx_str(const char *s)
{
    while (*s)
        fn_tx((unsigned char)*s++);
}

static void raw_tx_padded(const char *s, unsigned int total)
{
    unsigned int n = 0;

    while (*s && n < total) {
        fn_tx((unsigned char)*s++);
        n++;
    }
    while (n++ < total)
        fn_tx(0);
}

static bool raw_finish(void)
{
    return fn_commit() == FN_OK && FN_REPLYCMD == FUJICMD_ACK;
}

/* COPY_FILE's ACK arrives only when the host-side copy is done, past the
 * cart's ordinary 5 s: outlast its 60 s budget so its timeout is the one
 * that surfaces. */
static bool raw_finish_long(void)
{
    unsigned char saved = fn_default_timeout;
    bool ok;

    fn_default_timeout = 90;
    ok = raw_finish();
    fn_default_timeout = saved;
    return ok;
}

bool fnraw_open_directory(unsigned char host_slot,
                          const char *dirpath, const char *pattern)
{
    unsigned int n = 2;         /* the two NULs */
    const char *p;

    raw_begin(FUJICMD_OPEN_DIRECTORY);
    raw_param8(host_slot, 1);
    for (p = dirpath; *p; p++, n++)
        raw_tx((unsigned char)*p);
    raw_tx(0);
    for (p = pattern; *p; p++, n++)
        raw_tx((unsigned char)*p);
    raw_tx(0);
    while (n++ < PAYLOAD_LEN)
        raw_tx(0);
    return raw_finish();
}
```

```
bool fnraw_write_host_slot(unsigned char slot, const char *name)
{
    volatile unsigned char *r;
    unsigned char i, j;

    /* Fresh window first: the write streams the other seven slots straight
     * back out of it. The reply is only repainted by a commit, so it holds
     * still while the TX stream is built. */
    if (!FUJICALL(FUJICMD_READ_HOST_SLOTS))
        return false;

    raw_begin(FUJICMD_WRITE_HOST_SLOTS);
    for (i = 0; i < 8; i++) {
        if (i == slot) {
            raw_tx_padded(name, 32);
        } else {
            r = FN_REPLY + (unsigned int)i * 32;
            for (j = 0; j < 32; j++)
                fn_tx(r[j]);
        }
    }
    return raw_finish();
}

static void raw_devpath_params(unsigned char cmd, unsigned char dev,
                               unsigned char host_slot, unsigned char mode)
{
    raw_begin(cmd);
    raw_param8(dev, 1);
    raw_param8(host_slot, 2);
    raw_param8(mode, 3);
}

bool fnraw_set_device_path_from_reply(unsigned char dev, unsigned char host_slot,
                                      unsigned char mode, const char *prefix,
                                      volatile unsigned char *name)
{
    unsigned int n = 0;

    raw_devpath_params(FUJICMD_SET_DEVICE_FULLPATH, dev, host_slot, mode);
    while (*prefix && n < PAYLOAD_LEN) {
        fn_tx((unsigned char)*prefix++);
        n++;
    }
    while (n < PAYLOAD_LEN) {
        unsigned char c = *name++;

        if (c == 0)
            break;
        fn_tx(c);
        n++;
    }
    while (n++ < PAYLOAD_LEN)
        fn_tx(0);
    return raw_finish();
}

bool fnraw_set_device_path(unsigned char dev, unsigned char host_slot,
                           unsigned char mode, const char *fullpath)
{

    raw_devpath_params(FUJICMD_SET_DEVICE_FULLPATH, dev, host_slot, mode);
    raw_tx_padded(fullpath, PAYLOAD_LEN);
    return raw_finish();
}

bool fnraw_set_ssid(const char *ssid, const char *password)
{
    raw_begin(FUJICMD_SET_SSID);
    raw_param8(0, 1);           /* required, value ignored */
    raw_tx_padded(ssid, 33);
    raw_tx_padded(password, 64);
    return raw_finish();
}

static const char *base_of(const char *spec)
{
    const char *base = spec;

    for (; *spec; spec++)
        if (*spec == '/')
            base = spec + 1;
    return base;
}

bool fnraw_copy_fits(const char *source_spec, const char *dest_dir)
{
    /* two one-byte parameters cost four stream bytes */
    return strlen(source_spec) + 1 + strlen(dest_dir)
           + strlen(base_of(source_spec)) <= FN_TX_MAX - 4;
}

bool fnraw_copy_file(unsigned char src_slot1, unsigned char dst_slot1,
                     const char *source_spec, const char *dest_dir)
{
    /* Only the source's basename lands in the destination directory, as in
     * src/perform_copy.c, so the destination is never asked for the
     * source's subdirectories. */
    const char *base = base_of(source_spec);

    raw_begin(FUJICMD_COPY_FILE);
    raw_param8(src_slot1, 1);
    raw_param8(dst_slot1, 2);
    raw_tx_str(source_spec);
    raw_tx('|');
    raw_tx_str(dest_dir);       /* always ends in '/' */
    raw_tx_str(base);
    return raw_finish_long();
}

unsigned char fnraw_mount_start(unsigned char dev, unsigned char mode)
{
    unsigned char want = (unsigned char)(FN_ACKSEQ + 1);

    if (want == 0)
        want = 1;               /* 0 means "never used" */
    raw_begin(FUJICMD_MOUNT_IMAGE);
    raw_param8(dev, 1);
    raw_param8(mode, 2);
    fn_regwr(FNR_SEQ, want);
    return want;
}

bool fnraw_reply_ok(void)
{
    return FN_ERRCODE == FN_OK && FN_REPLYCMD == FUJICMD_ACK;
}
```

```c
#include <string.h>

#include "fujidisp.h"

/* GTIA colours: hue << 4 | luminance. */
#define C_BACK   0x94           /* the 8-bit CONFIG's blue */
#define C_TEXT   0x0C

static unsigned char cur_col, cur_row, cur_under;
static bool cur_on;             /* the cursor is placed */
static bool cur_lit;            /* ...and showing its block right now */
static unsigned char blink;     /* frames into the 24-frame blink */

static unsigned char bar_last[DISP_COLS];

static char clean(char c)
{
    return (c < 32 || c > 126) ? ' ' : c;
}

void disp_init(void)
{
    mt_init(' ');
    mt_colours(C_BACK, C_TEXT);
}

void disp_cls(void)
{
    cur_on = false;
    mt_clear(' ');
}

void disp_row_clear(unsigned char row)
{
    if (row >= DISP_ROWS)
        return;
    mt_at(IN_L, row);
    mt_fill(' ', IN_W);
    mt_at(IN_L, row);
    mt_setpal(0, IN_W);
}

void disp_at_hi(unsigned char col, unsigned char row, const char *s, bool hi)
{
    unsigned char n;

    if (row >= DISP_ROWS || col > IN_R)
        return;
    mt_at(col, row);
    n = mt_text(s, (unsigned char)(IN_R + 1 - col));
    mt_at(col, row);
    mt_setpal(hi, n);
}

void disp_text(unsigned char col, unsigned char row,
               volatile unsigned char *s, unsigned char max)
{
    unsigned char n;

    if (row >= DISP_ROWS || col > IN_R)
        return;
    if (max > IN_R + 1 - col)
        max = (unsigned char)(IN_R + 1 - col);
    mt_at(col, row);
    n = mt_text((const char *)s, max);
    mt_at(col, row);
    mt_setpal(0, n);
}

void disp_at(unsigned char col, unsigned char row, const char *s)
{
    disp_at_hi(col, row, s, false);
}

void disp_char_hi(unsigned char col, unsigned char row, char c, bool hi)
{
    if (row >= DISP_ROWS || col > IN_R)
        return;
    mt_at(col, row);
    mt_put(clean(c));
    mt_at(col, row);
    mt_setpal(hi, 1);
}

void disp_char(unsigned char col, unsigned char row, char c)
{
    disp_char_hi(col, row, c, false);
}

void disp_tile(unsigned char col, unsigned char row, unsigned char t)
{
    if (row >= DISP_ROWS || col >= DISP_COLS)
        return;
    mt_at(col, row);
    mt_put(t);
    mt_at(col, row);
    mt_setpal(0, 1);
}

void disp_box(unsigned char l, unsigned char t, unsigned char r,
              unsigned char b, const char *title)
{
    unsigned char i;

    mt_at(l, t);
    mt_put(T_TL);
    mt_fill(T_HLINE, (unsigned char)(r - l - 1));
    mt_put(T_TR);
    mt_at(l, b);
    mt_put(T_BL);
    mt_fill(T_HLINE, (unsigned char)(r - l - 1));
    mt_put(T_BR);
    for (i = (unsigned char)(t + 1); i < b; i++) {
        mt_at(l, i);
        mt_put(T_VLINE);
        mt_at(r, i);
        mt_put(T_VLINE);
    }
    if (title) {
        unsigned char len = (unsigned char)strlen(title);
        unsigned char span = (unsigned char)(r - l - 1);
        unsigned char at;

        if (len + 2 > span)
            len = (unsigned char)(span - 2);
        at = (unsigned char)(l + 1 + (span - len - 2) / 2);
        mt_at(at, t);
        mt_put(' ');
        for (i = 0; i < len; i++)
            mt_put(title[i]);
        mt_put(' ');
    }
}

void disp_frame(const char *title)
{
    disp_cls();
    disp_box(FRAME_L, FRAME_T, FRAME_R, FRAME_B, title);
}

void disp_row_invert(unsigned char row, bool on)
{
    if (row >= DISP_ROWS)
        return;
    mt_at(IN_L, row);
    mt_setpal(on, IN_W);
}

void disp_bar_reset(void)
{
    memset(bar_last, 0xFF, sizeof bar_last);
}

void disp_bar(unsigned char col, unsigned char row, unsigned char cells,
```

```
                unsigned char num, unsigned char
den)
{
    unsigned int px;
    unsigned char i, t;

    if (den == 0)
        den = 1;
    if (num > den)
        num = den;
    px = (unsigned int)num * (unsigned int)
(cells * 8u) / den;
    for (i = 0; i < cells && i < DISP_COLS; i++
) {
        if (px >= 8) {
            t = T_BAR0 + 8;
            px -= 8;
        } else {
            t = (unsigned char)(T_BAR0 + px);
            px = 0;
        }
        if (bar_last[i] != t) {
            bar_last[i] = t;
            disp_tile((unsigned char)(col +
i), row, t);
        }
    }
}

/* What the block covers is read back as it
lights, and put back only if
 * the block is still there: a cell redrawn
underneath it wins. */
static void cursor_paint(bool lit)
{
    unsigned char here = mt_get(cur_col,
cur_row);

    mt_at(cur_col, cur_row);
    if (lit) {
        cur_under = here == T_CURSOR ? ' ' :
here;
        mt_put(T_CURSOR);
    } else if (here == T_CURSOR) {
        mt_put(cur_under);
    }
    cur_lit = lit;
}

void disp_cursor_at(unsigned char col,
unsigned char row)
{
    if (cur_on)
        cursor_paint(false);
    cur_col = col;
    cur_row = row;
    cur_on = true;
    blink = 0;
    cursor_paint(true);
}

void disp_cursor_tick(void)
{
    bool lit;

    if (!cur_on)
        return;
    /* 12 on, 12 off, counted in calls: the
caller ticks once a frame */
    if (++blink >= 24)
        blink = 0;
    lit = (bool)(blink < 12);
    if (lit != cur_lit)
        cursor_paint(lit);
}

void disp_cursor_off(void)
{
    if (cur_on)
        cursor_paint(false);
    cur_on = false;
}

void disp_at_u16(unsigned char col, unsigned
char row, unsigned int v)
{
    char buf[6];
    unsigned char i = 5;

    buf[5] = '\0';
    do {
        buf[--i] = (char)('0' + (v % 10u));
        v /= 10u;
    } while (v != 0 && i != 0);
    disp_at(col, row, buf + i);
}

void disp_at_hex8(unsigned char col, unsigned
char row, unsigned char v)
{
    static const char hex[] =
"0123456789ABCDEF";
    char buf[3];

    buf[0] = hex[(v >> 4) & 0x0F];
    buf[1] = hex[v & 0x0F];
    buf[2] = '\0';
    disp_at(col, row, buf);
}
```

```
/* fujiin.c -- see fujiin.h. The stick comes from the VBI's POT shadows, the
 * keypad from the lib's keypad IRQ (fuji_a5200_key). */

#include <fujinet-atari5200.h>

#include "fujiin.h"
#include "antic.h"
#include "sfx.h"

#define REG(a)      (*(volatile unsigned char *)(a))
#define PADDL0      REG(0x11)    /* stick X, 0-228, 114 at rest */
#define PADDL1      REG(0x12)    /* stick Y */
#define TRIG0       REG(0xC010)  /* top fire: 0 while held */
#define SKSTAT      REG(0xE80F)  /* bit 3 clear: the lower fire button */

/* Past these the stick is pushed; between them it is at rest. */
#define STICK_LO   0x40
#define STICK_HI   0xA4

#define REPEAT_FIRST 20      /* frames before a held direction repeats */
#define REPEAT_NEXT  6       /* frames between repeats after that */

#define B_FIRE    0x01
#define B_BACK    0x02

static unsigned char last_dir;
static unsigned char last_btn;
static unsigned char hold;

void in_init(void)
{
    fuji_a5200_key = 0;
}

unsigned char in_frames(void)
{
    return mt_frames;
}

static unsigned char stick(void)
{
    unsigned char x = PADDL0, y = PADDL1;

    if (y < STICK_LO) return IN_UP;
    if (y > STICK_HI) return IN_DOWN;
    if (x < STICK_LO) return IN_LEFT;
    if (x > STICK_HI) return IN_RIGHT;
    return 0;
}

static unsigned char buttons(void)
{
    unsigned char b = 0;

    if (TRIG0 == 0)
        b |= B_FIRE;
    if (!(SKSTAT & 0x08))
        b |= B_BACK;
    return b;
}

/* A keypad key, as an event: each press is one, the IRQ having latched it. */
static unsigned char keypad(void)
{
    unsigned char k = fuji_a5200_key;

    if (k == 0)
        return IN_NONE;
    fuji_a5200_key = 0;
    switch (k) {
    case FUJI_A5200_KEY_DIGIT(2): return IN_UP;
    case FUJI_A5200_KEY_DIGIT(8): return IN_DOWN;
    case FUJI_A5200_KEY_DIGIT(4): return IN_LEFT;
    case FUJI_A5200_KEY_DIGIT(6): return IN_RIGHT;
    case FUJI_A5200_KEY_HASH:     return IN_FIRE;
    case FUJI_A5200_KEY_STAR:
    case FUJI_A5200_KEY_RESET:    return IN_BACK;
    case FUJI_A5200_KEY_PAUSE:    return IN_SELECT;
    case FUJI_A5200_KEY_START:    return IN_START;
    default:                      return IN_NONE;
    }
}

static unsigned char read_event(void)
{
    unsigned char dir, btn, k;

    mt_sync();
    k = keypad();
    if (k != IN_NONE)
        return k;

    dir = stick();
    if (dir != 0) {
        if (dir != last_dir) {
            last_dir = dir;
            hold = REPEAT_FIRST;
            return dir;
        }
        if (hold != 0 && --hold == 0) {
            hold = REPEAT_NEXT;
            return dir;
        }
    } else {
        last_dir = 0;
    }

    btn = buttons();
    if (btn != 0 && last_btn == 0) {
        last_btn = btn;
        return (btn & B_FIRE) ? IN_FIRE : IN_BACK;
    }
    if (btn == 0)
        last_btn = 0;

    return IN_NONE;
}

/* Every keypress clicks, auto-repeats included. */
unsigned char in_read(void)
{
    unsigned char ev = read_event();

    if (ev != IN_NONE)
        sfx_click();
    return ev;
}

```

```c
/* fujiedit.c -- the text editor: the question on one line ("HOST NAME?"),
 * the answer under it with the blinking block cursor, and below, framed, the
 * on-screen keyboard for the joystick. */

#include <string.h>

#include "fujidisp.h"
#include "fujiedit.h"
#include "fujiin.h"

#define TITLE_ROW  1
#define VALUE_ROW  2
#define VALUE_COL  IN_L
#define VALUE_W    IN_W
#define BOX_L      9
#define BOX_T      4
#define BOX_R      30
#define BOX_B      17
#define GRID_COL0  12
#define GRID_ROW0  6
#define GRID_PITCH 2
#define GRID_COLS  16
#define GRID_ROWS  4
#define ACT_ROW    15
#define ACT_CELLS  5
#define FOOT_ROW   19

char fn_entry[FN_ENTRY_MAX];

static unsigned char glen;
static unsigned char gmax;
static unsigned char gcase;
static unsigned char gx, gy;

static const char *act_text[ACT_CELLS] = { "CASE", "SPC", "DEL", "OK", "ESC" };
static const unsigned char act_col[ACT_CELLS] = { 11, 16, 20, 24, 27 };

static char cell_char(unsigned char cx, unsigned char cy)
{
    char c = (char)(0x20 + cy * GRID_COLS + cx);

    if (gcase && c >= 'A' && c <= 'Z')
        c = (char)(c + 0x20);
    return c;
}

static unsigned char grid_screen_row(unsigned char cy)
{
    return (unsigned char)(GRID_ROW0 + cy * GRID_PITCH);
}

static void draw_cell(unsigned char cx, unsigned char cy, bool sel)
{
    disp_char_hi((unsigned char)(GRID_COL0 + cx), grid_screen_row(cy),
                 cell_char(cx, cy), sel);
}

/* A grid row is one string, then the picked cell is lit. */
static void draw_grid(void)
{
    char line[GRID_COLS + 1];
    unsigned char cx, cy;

    line[GRID_COLS] = '\0';
    for (cy = 0; cy < GRID_ROWS; cy++) {
        for (cx = 0; cx < GRID_COLS; cx++)
            line[cx] = cell_char(cx, cy);
        disp_at(GRID_COL0, grid_screen_row(cy), line);
    }
    if (gy < GRID_ROWS)
        draw_cell(gx, gy, true);
}

static void draw_actions(void)
{
    unsigned char i;

    for (i = 0; i < ACT_CELLS; i++)
        disp_at_hi(act_col[i], ACT_ROW, act_text[i],
                   (bool)(gy == GRID_ROWS && gx == i));
}

static void draw_value(void)
{
    unsigned char start = 0;
    unsigned char i;

    if (glen > VALUE_W - 1)
        start = (unsigned char)(glen - (VALUE_W - 1));

    disp_row_clear(VALUE_ROW);
    for (i = 0; (unsigned char)(start + i) < glen; i++)
        disp_char((unsigned char)(VALUE_COL + i), VALUE_ROW, fn_entry[start + i]);
    disp_cursor_at((unsigned char)(VALUE_COL + i), VALUE_ROW);
}

static void move_to(unsigned char nx, unsigned char ny)
{
    if (gy == GRID_ROWS)
        disp_at_hi(act_col[gx], ACT_ROW, act_text[gx], false);
    else
        draw_cell(gx, gy, false);

    gx = nx;
    gy = ny;

    if (gy == GRID_ROWS)
        disp_at_hi(act_col[gx], ACT_ROW, act_text[gx], true);
    else
        draw_cell(gx, gy, true);
}

static void type_char(char c)
{
    if (glen >= gmax)
        return;
    fn_entry[glen++] = c;
    fn_entry[glen] = '\0';
    draw_value();
}

static void backspace(void)
{
    if (glen == 0)
        return;
    fn_entry[--glen] = '\0';
    draw_value();
}

/* The keys, until OK or cancel. */
static bool edit_loop(void)
{
    for (;;) {
        unsigned char ev = in_read();

        disp_cursor_tick();
        switch (ev) {
        case IN_NONE:
            break;

        case IN_UP:
            if (gy == GRID_ROWS)
                move_to((unsigned char)(gx * 3 + 1), GRID_ROWS - 1);
            else if (gy > 0)
                move_to(gx, (unsigned char)(gy - 1));
            break;

        case IN_DOWN:
            if (gy < GRID_ROWS - 1)
                move_to(gx, (unsigned char)(gy + 1));
            else if (gy == GRID_ROWS - 1)
                move_to((unsigned char)(gx * ACT_CELLS / GRID_COLS), GRID_ROWS);
            break;

        case IN_LEFT:
```

```
            if (gx > 0)
                move_to((unsigned char)(gx -
1), gy);
            break;

        case IN_RIGHT:
            if (gy == GRID_ROWS) {
                if (gx + 1 < ACT_CELLS)
                    move_to((unsigned char)(gx
+ 1), gy);
            } else if (gx + 1 < GRID_COLS) {
                move_to((unsigned char)(gx +
1), gy);
            }
            break;

        case IN_FIRE:
            if (gy < GRID_ROWS) {
                type_char(cell_char(gx, gy));
                break;
            }
            switch (gx) {
            case 0:
                gcase = (unsigned char)(gcase
^ 1);
                draw_grid();
                break;
            case 1:
                type_char(' ');
                break;
            case 2:
                backspace();
                break;
            case 3:
                return true;
            default:            /* ESC -- the
only way to cancel */
                return false;
            }
            break;

        case IN_BACK:
            backspace();
            break;

        case IN_SELECT:
            gcase = (unsigned char)(gcase ^
1);
            draw_grid();
            break;

        case IN_START:
            return true;
        }
    }
}

bool fn_edit(const char *title, unsigned char
maxlen)
{
    bool ok;

    if (maxlen > FN_ENTRY_MAX - 1)
        maxlen = FN_ENTRY_MAX - 1;
    gmax = maxlen;
    fn_entry[maxlen] = '\0';
    glen = (unsigned char)strlen(fn_entry);
    gcase = 0;
    gx = 0;
    gy = 2;

    disp_cls();
    disp_at(IN_L, TITLE_ROW, title);
    disp_at((unsigned char)(IN_L +
strlen(title)), TITLE_ROW, "?");
    disp_box(BOX_L, BOX_T, BOX_R, BOX_B,
"KEYBOARD");
    disp_at(IN_L, FOOT_ROW, "#--PICK  *--DEL
PAUSE--CASE");
    disp_at(IN_L, FOOT_ROW + 1, "START--OK
ESC CELL--CANCEL");
    draw_grid();
    draw_actions();
    draw_value();

    ok = edit_loop();
    disp_cursor_off();
    return ok;
}

```

sfx.c

```
/* sfx.c -- see sfx.h. Plain stores to POKEY channel 1. */

#include "antic.h"
#include "sfx.h"

#define POKEY(r)   (*(volatile unsigned char *)(0xE800 + (r)))
#define AUDF1      POKEY(0x0)
#define AUDC1      POKEY(0x1)
#define AUDCTL     POKEY(0x8)

#define PURE       0xA0         /* AUDC: pure tone; low nibble the volume */

/* AUDF for the 64 kHz clock: f = 63921 / (2 * (AUDF + 1)). */
#define F_CLICK    26           /* ~1180 Hz */
#define F_A5       35

/* The arpeggio: C5 E5 G5 C6. */
static const unsigned char accept_notes[4] =
    { 60, 47, 40, 30 };

static void frames(unsigned char n)
{
    while (n--)
        mt_sync();
}

/* SKCTL stays as the crt0 set it: scanning the keypad, which also keeps
 * POKEY out of initialisation, so the counters run. */
void sfx_init(void)
{
    AUDCTL = 0;
    AUDC1 = 0;
}

static void tone(unsigned char f, unsigned char vol)
{
    AUDF1 = f;
    AUDC1 = (unsigned char)(PURE | vol);
}

void sfx_click(void)
{
    unsigned char i;

    tone(F_CLICK, 8);
    for (i = 0; i < 100; i++)   /* about 0.7 ms */
        ;
    AUDC1 = 0;
}

void sfx_beep(void)
{
    tone(F_CLICK, 8);
    frames(10);
    AUDC1 = 0;
}

void sfx_accept(void)
{
    unsigned char i;

    for (i = 0; i < sizeof accept_notes; i++)
    {
        tone(accept_notes[i], 8);
        frames(2);
    }
    frames(2);
    AUDC1 = 0;
}

void sfx_blip(void)
{
    tone(F_A5, 6);
    frames(2);
    AUDC1 = 0;
}

```

FUJINET

A FujiNet Project Publication

The FujiNet Project
fujinet.online · github.com/FujiNetWIFI

FN5200-PH REV. A

© 2026 the FujiNet Project
CC BY-SA 4.0

Printed in the FujiNet Project